Hashtag Treinamentos

CURSO COMPLETO

SQL

IMPRESSIONADOR

MySQL®

SUMÁRIO

MÓDULO 1	Introdução	004
MÓDULO 2	Instalação e Importação do Banco de Dados	033
MÓDULO 3	Consultas Básicas	038
MÓDULO 4	Filtros no MySQL	050
MÓDULO 5	Operações Básicas e Funções de Agregação	062
MÓDULO 6	Agrupamentos no MySQL	074
MÓDULO 7	Variáveis no MySQL	085
MÓDULO 8	Funções de Texto e Data no MySQL	095
MÓDULO 9	Joins	115
MÓDULO 10	Funções Condicionais	146

Próxima Página

SUMÁRIO

MÓDULO 11 Views 164

MÓDULO 12 CRUD 184

MÓDULO 13 Functions e Procedures 219

MÓDULO 14 Subqueries 243

MÓDULO 1

INTRODUÇÃO

INTRODUÇÃO
INTRODUÇÃO

Índice Geral
Índice deste Módulo

O QUE VEREMOS NESTE MÓDULO

MÓDULO 1 INTRODUÇÃO

01	O QUE É UM BANCO DE DADOS?	006
02	DO QUE É COMPOSTO UM BANCO DE DADOS?	008
03	O QUE É UM BANCO DE DADOS?	014
04	SISTEMAS DE BANCOS DE DADOS	018
05	SQL: STRUCTURED QUERY LANGUAGE	019
06	O QUE É UMA QUERY (CONSULTA)?	024
07	GRUPOS DE COMANDO NO SQL	027
08	NÃO CONFUNDA OS SQL'S!!	028

O que é um Banco de Dados?

Um banco de dados é um conjunto de dados organizados dentro de uma ou mais tabelas, que terão alguma relação entre si.

Partimos de uma informação isolada, na sua forma mais simples, e a partir de um conjunto de diversas informações isoladas, conseguimos organizar essas informações para criar tabelas e bancos de dados.

Dado

- 1 Compra online do João

Tabela

- Diversas compras online (João, Amanda, Pedro, Carol, ...)

Banco de Dados

- Diversas compras online (João, Amanda, Pedro, Carol, ...)
- Diversas informações sobre os clientes
- Diversas informações dos produtos comprados

O que é um Banco de Dados?

> **Bancos de dados** são conjuntos de tabelas, com alguma relação entre si, com dados sobre pessoas, lugares ou coisas.
>
> Estes dados organizados permitem a compreensão de um determinado fenômeno na empresa, seja a preferência dos usuários em uma rede social, seja o perfil de consumo em um aplicativo de transações financeiras.

Do que é composto um banco de dados?

Um banco de dados é composto pelos seguintes elementos:

- Tabelas
- Campos (Atributos)
- Registros (Tuplas)

Do que é composto um banco de dados?

Um banco de dados é composto pelos seguintes elementos:

- Tabelas
- Campos (Atributos)
- Registros (Tuplas)

Tabelas armazenam dados dentro de um banco de dados.

Uma tabela é um conjunto de dados relacionados, e é composta por linhas e colunas.

Do que é composto um banco de dados?

Um banco de dados é composto pelos seguintes elementos:

- Tabelas
- Campos (Atributos)
- Registros (Tuplas)

Campos (também chamados de atributos ou simplesmente colunas) representam os atributos dos dados, como ID do Produto, Nome do Cliente, Quantidade Vendida, Preço, etc.

Um campo é uma coluna de uma tabela, contendo informações específicas sobre cada registro (linha) da tabela.

Do que é composto um banco de dados?

Um banco de dados é composto pelos seguintes elementos:

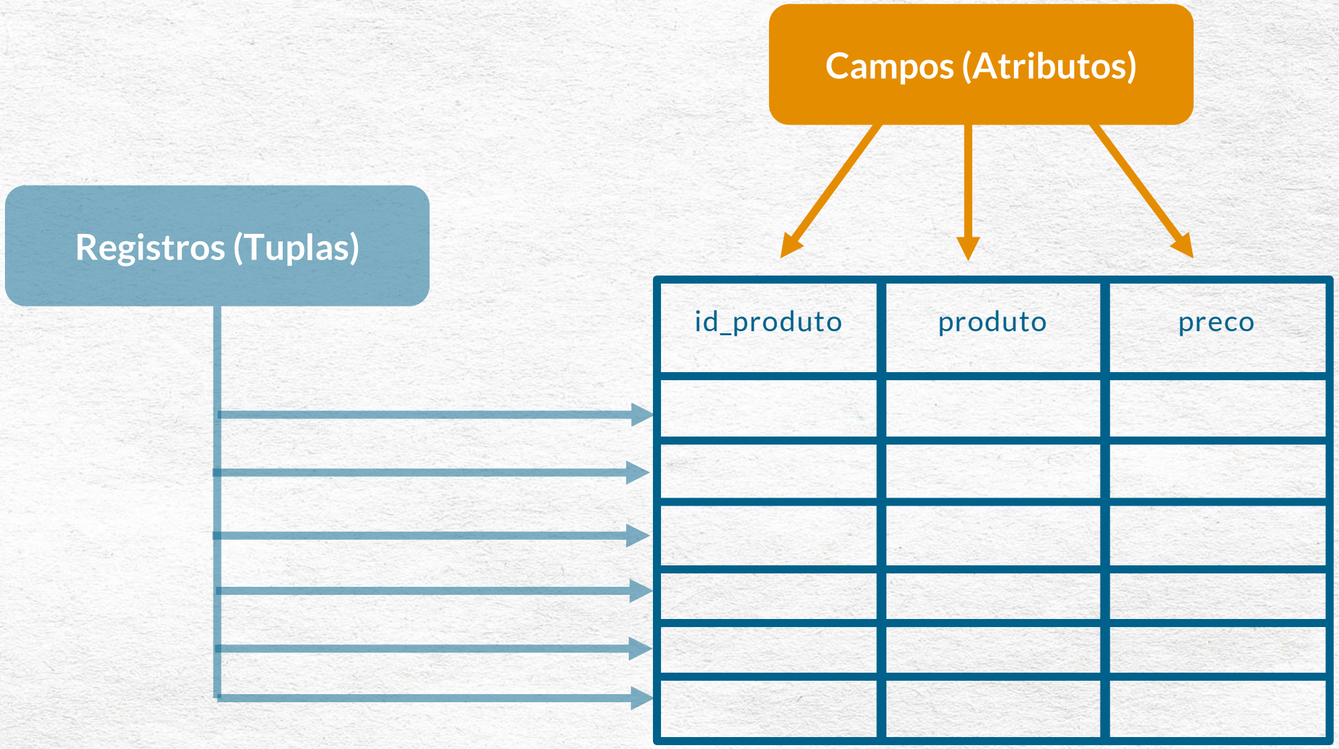
- Tabelas
- Campos (Atributos)
- Registros (Tuplas)

Registros (também chamados de tuplas ou simplesmente linhas) representam cada entrada de dados dentro de uma tabela.

Do que é composto um banco de dados?

Um banco de dados é composto pelos seguintes elementos:

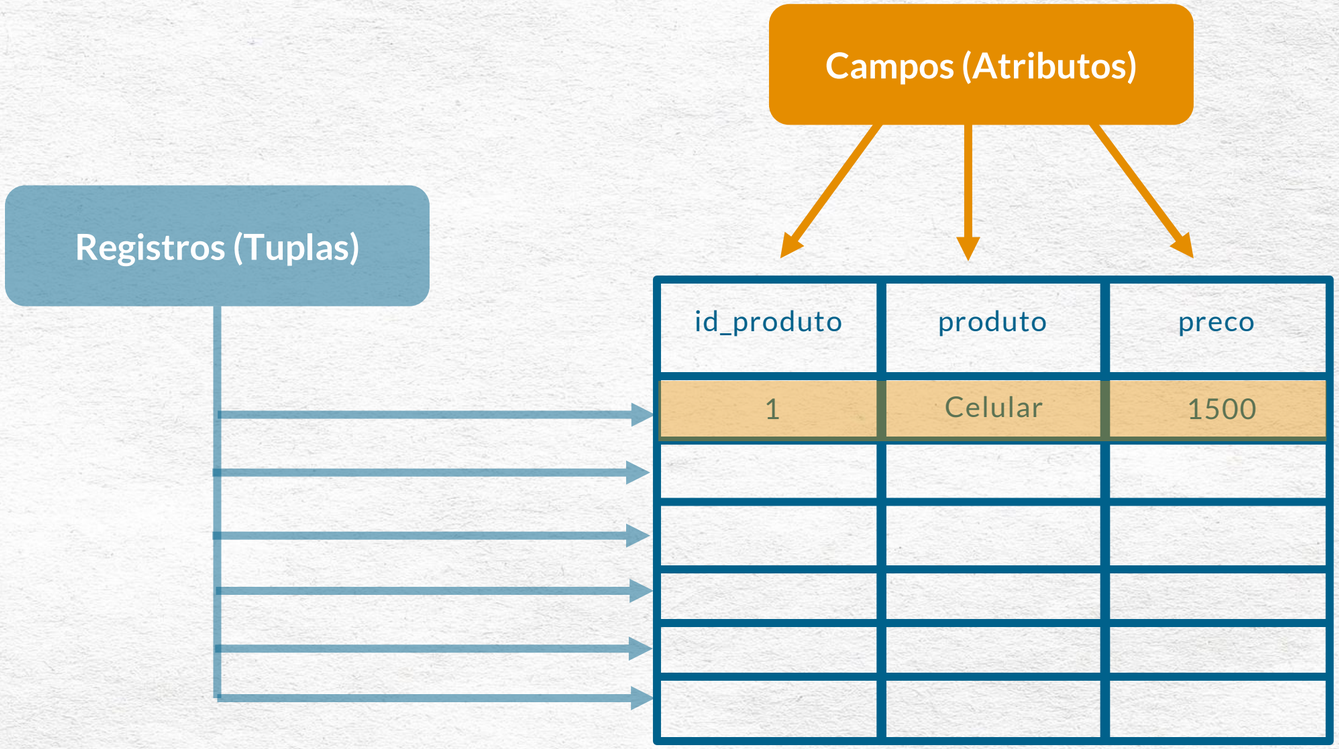
- Tabelas
- Campos (Atributos)
- Registros (Tuplas)



Do que é composto um banco de dados?

Um banco de dados é composto pelos seguintes elementos:

- Tabelas
- Campos (Atributos)
- Registros (Tuplas)

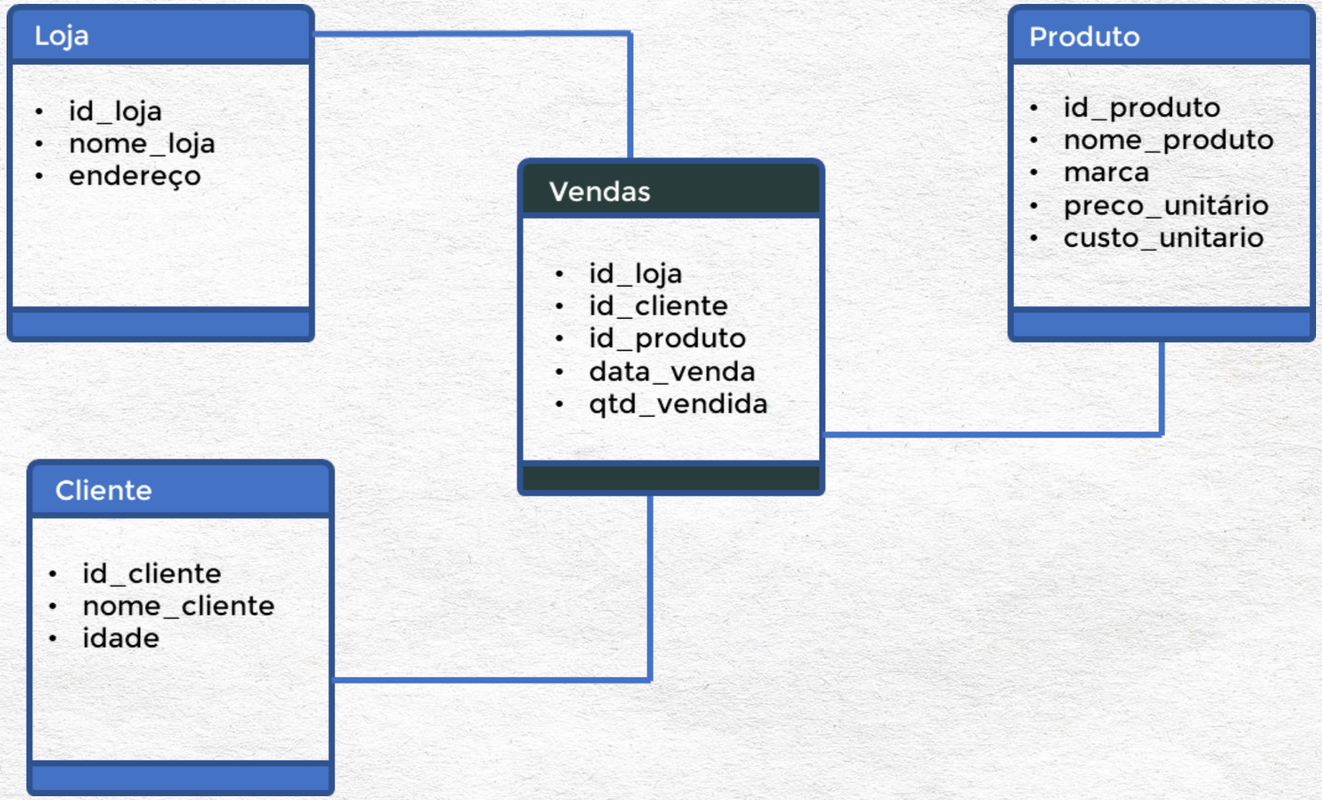


O que é um Banco de Dados Relacional?

O desenho esquemático de um banco de dados é mostrado ao lado. Diversas tabelas, com diferentes informações sobre um negócio, e que possuem algum tipo de relação entre si.

A esse banco de dados damos o nome de **RELACIONAL**.

Bancos de dados relacionais serão o foco do nosso curso, até por serem o tipo de bancos de dados mais comumente encontrados no mercado.

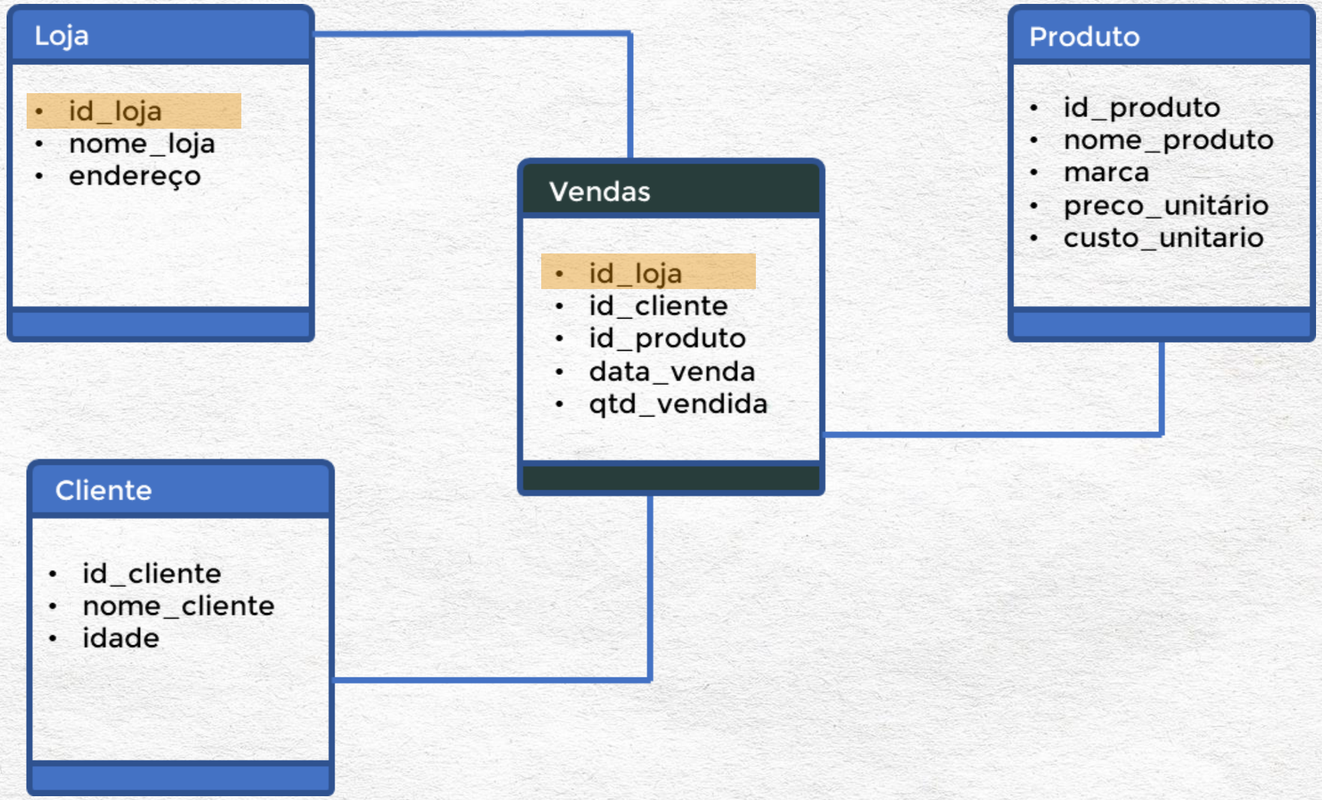


O que é um Banco de Dados Relacional?

O desenho esquemático de um banco de dados é mostrado ao lado. Diversas tabelas, com diferentes informações sobre um negócio, e que possuem algum tipo de relação entre si.

A esse banco de dados damos o nome de **RELACIONAL**.

Bancos de dados relacionais serão o foco do nosso curso, até por serem o tipo de bancos de dados mais comumente encontrados no mercado.

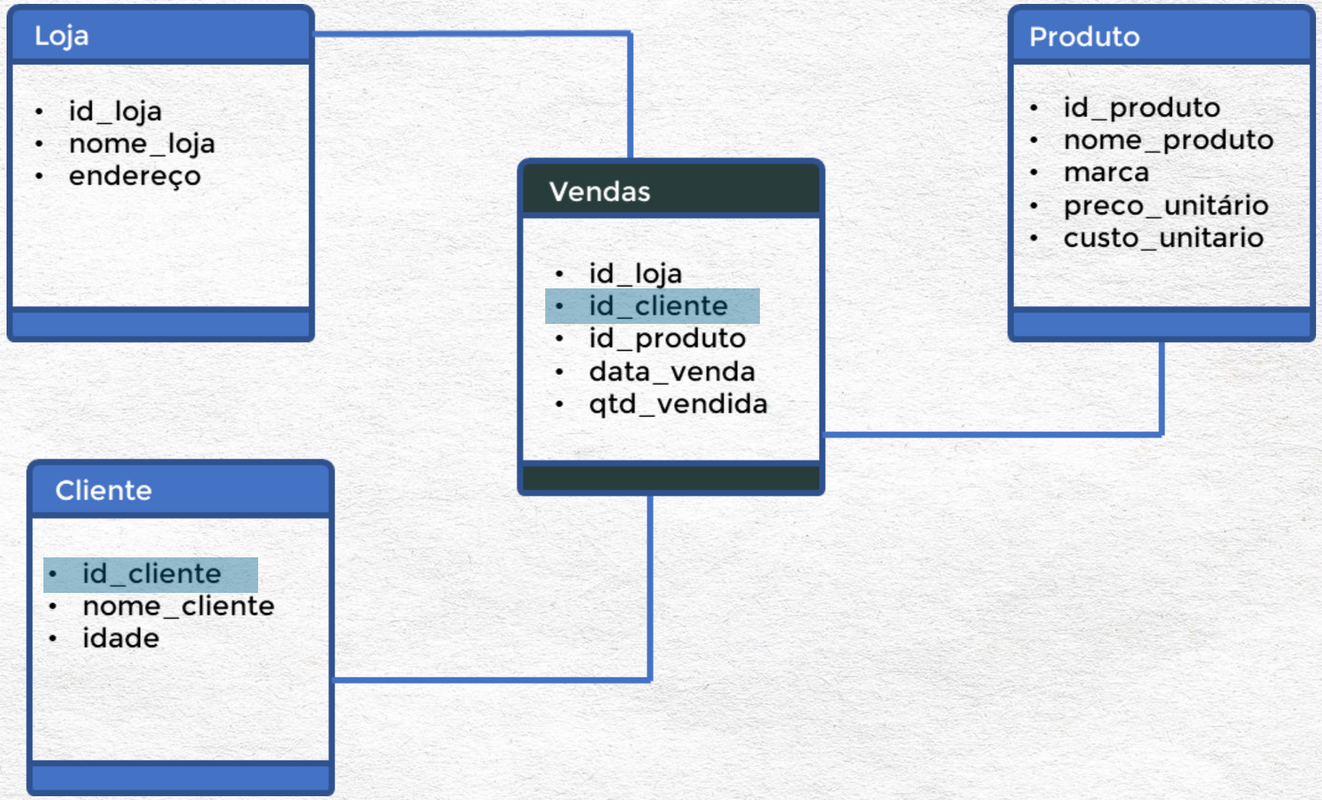


O que é um Banco de Dados?

O desenho esquemático de um banco de dados é mostrado ao lado. Diversas tabelas, com diferentes informações sobre um negócio, e que possuem algum tipo de relação entre si.

A esse banco de dados damos o nome de **RELACIONAL**.

Bancos de dados relacionais serão o foco do nosso curso, até por serem o tipo de bancos de dados mais comumente encontrados no mercado.

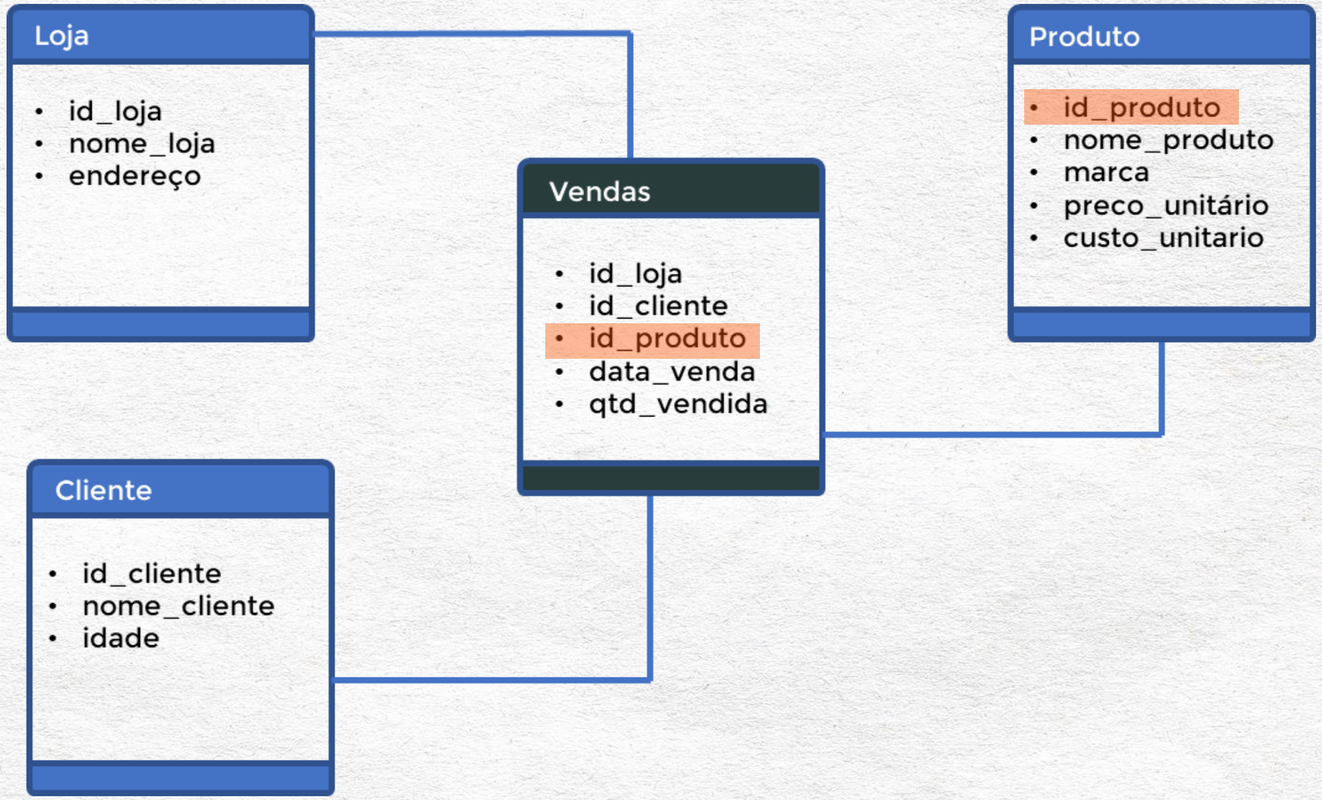


O que é um Banco de Dados?

O desenho esquemático de um banco de dados é mostrado ao lado. Diversas tabelas, com diferentes informações sobre um negócio, e que possuem algum tipo de relação entre si.

A esse banco de dados damos o nome de **RELACIONAL**.

Bancos de dados relacionais serão o foco do nosso curso, até por serem o tipo de bancos de dados mais comumente encontrados no mercado.



Sistemas de Bancos de Dados

Existem alguns SGBDs para Bancos de Dados Relacionais que são muito utilizados por grandes empresas. Abaixo, temos os 4 principais programas para SGBDs.

É importante que fique claro que **todos esses SGBDs** utilizam o SQL como linguagem de programação.

MySQL Oracle SQL Server PostgreSQL

SQL: Structured Query Language

Para acessar e consultar os dados em um banco de dados, é necessário o uso de uma série de comandos.

Esses comandos, na verdade, se tratam de uma linguagem de programação, chamada SQL: Structured Query Language.

Traduzindo para o português, a sigla SQL significa Linguagem de Consulta Estruturada.

Essa é uma linguagem de bancos de dados universal e é por dela que será possível a consulta aos dados dentro dos bancos de dados.

SQL: Structured Query Language

Para acessar e consultar os dados em um banco de dados, é necessário o uso de uma série de comandos.

Esses comandos, na verdade, se tratam de uma linguagem de programação, chamada SQL: Structured Query Language.

Traduzindo para o português, a sigla SQL significa Linguagem de Consulta Estruturada.

Essa é uma linguagem de bancos de dados universal e é por dela que será possível a consulta aos dados dentro dos bancos de dados.

> SQL significa “**Structured Query Language**”. Se trata de uma linguagem de programação utilizada para armazenar, consultar, adicionar e excluir informações em um banco de dados.

SQL: Structured Query Language

Para acessar e consultar os dados em um banco de dados, é necessário o uso de uma série de comandos.

Esses comandos, na verdade, se tratam de uma linguagem de programação, chamada SQL: Structured Query Language.

Traduzindo para o português, a sigla SQL significa Linguagem de Consulta Estruturada.

Essa é uma linguagem de bancos de dados universal e é por dela que será possível a consulta aos dados dentro dos bancos de dados.

SQL: Structured Query Language

Para acessar e consultar os dados em um banco de dados, é necessário o uso de uma série de comandos.

Esses comandos, na verdade, se tratam de uma linguagem de programação, chamada SQL: Structured Query Language.

Traduzindo para o português, a sigla SQL significa Linguagem de Consulta Estruturada.

Essa é uma linguagem de bancos de dados universal e é por dela que será possível a consulta aos dados dentro dos bancos de dados.

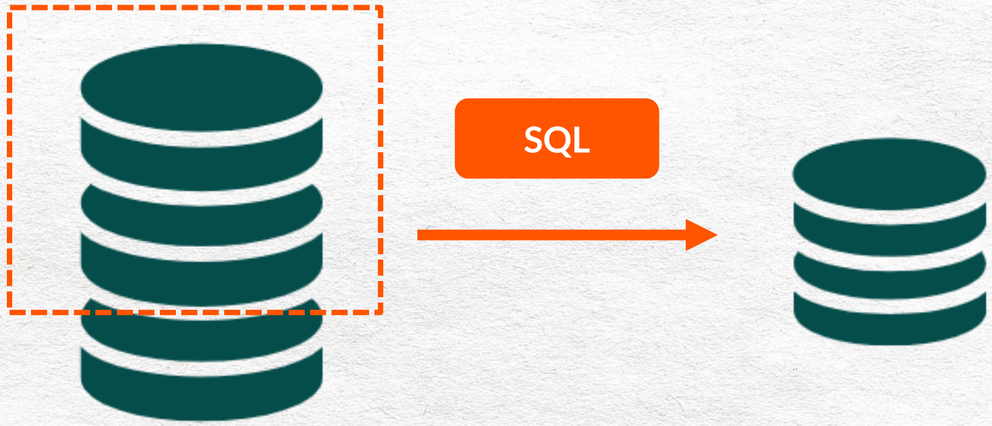
SQL: Structured Query Language

Para acessar e consultar os dados em um banco de dados, é necessário o uso de uma série de comandos.

Esses comandos, na verdade, se tratam de uma linguagem de programação, chamada SQL: Structured Query Language.

Traduzindo para o português, a sigla SQL significa Linguagem de Consulta Estruturada.

Essa é uma linguagem de bancos de dados universal e é por dela que será possível a consulta aos dados dentro dos bancos de dados.



O que é uma query (consulta)?

Uma query é um pedido de uma informação ou de um dado. Esse pedido também pode ser entendido como uma consulta, uma solicitação ou, ainda, uma requisição.

Em resumo, uma query (ou consulta) é uma leitura dos dados de uma tabela dentro de um banco de dados. Ou seja, quando queremos visualizar determinados dados de uma tabela, na prática o que queremos é fazer uma consulta aos dados do banco de dados.

Porém, a leitura desses dados não é aleatória. Ela é baseada em uma série de comandos, feitos a partir da linguagem SQL.

O que é uma query (consulta)?

Uma query é um pedido de uma informação ou de um dado. Esse pedido também pode ser entendido como uma consulta, uma solicitação ou, ainda, uma requisição.

Em resumo, uma query (ou consulta) é uma leitura dos dados de uma tabela dentro de um banco de dados. Ou seja, quando queremos visualizar determinados dados de uma tabela, na prática o que queremos é fazer uma consulta aos dados do banco de dados.

Porém, a leitura desses dados não é aleatória. Ela é baseada em uma série de comandos, feitos a partir da linguagem SQL.

O que é uma query (consulta)?

Ao lado, temos um exemplo bem simplificado de como seria essa query (consulta) aos dados em uma tabela de um banco de dados, utilizando um código em SQL.

Tabela

id	produto	data_venda	valor
1	televisão	2021-03-20	1500
2	computador	2021-03-22	2300
3	celular	2021-03-25	800
4	ps4	2021-03-28	3100
5	tablet	2021-03-28	650

SQL

```sql
SELECT *
FROM tabela_vendas
WHERE valor > 1000;
```

id	produto	data_venda	valor
1	televisão	2021-03-20	1500
2	computador	2021-03-22	2300
4	ps4	2021-03-28	3100

Consulta

Grupos de comando no SQL

Os comandos do SQL podem ser divididos em **quatro grupos**.

DDL

Data Definition Language

CREATE
Cria uma nova tabela, view ou outro objeto dentro do banco de dados.

ALTER
Modifica um objeto dentro do banco de dados (tabela, view, etc).

DROP
Exclui um objeto dentro do banco de dados (tabela, view, etc).

DML

Data Manipulation Language

INSERT
Adiciona uma nova linha em uma tabela.

UPDATE
Atualiza os valores das linhas de uma tabela.

DELETE
Exclui linhas de uma tabela.

DCL

Data Control Language

GRANT
Dá privilégios a um usuário.

REVOKE
Retira privilégios de um usuário.

DQL

Data Query Language

SELECT
Comando de seleção de linhas de uma tabela.

SQL, MySQL, SQL Server, Oracle e PostgreSQL: Não confunda os SQL's!!

O SQL é uma linguagem de consulta a bancos de dados, enquanto o MySQL, SQL Server, Oracle Database e PostgreSQL são programas utilizados para gerenciamento dos bancos de dados.

SQL, MySQL, SQL Server, Oracle e PostgreSQL: Não confunda os SQL's!!

O SQL é uma linguagem de consulta a bancos de dados, enquanto o MySQL, SQL Server, Oracle Database e PostgreSQL são programas utilizados para gerenciamento dos bancos de dados.

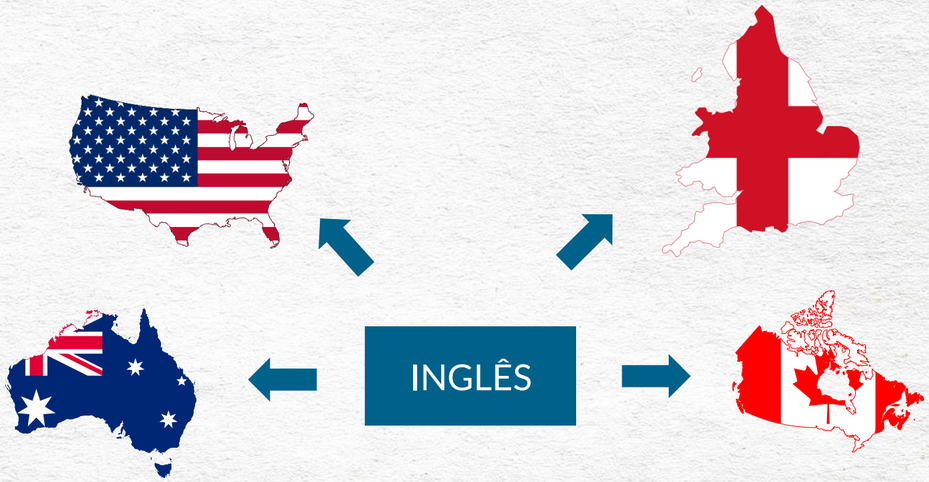
Para ficar clara essa diferença, faça um paralelo com o inglês.

SQL, MySQL, SQL Server, Oracle e PostgreSQL: Não confunda os SQL's!!

O SQL é uma linguagem de consulta a bancos de dados, enquanto o MySQL, SQL Server, Oracle Database e PostgreSQL são programas utilizados para gerenciamento dos bancos de dados.

Para ficar clara essa diferença, faça um paralelo com o inglês.

O inglês pode ser falado em diferentes países, como o EUA, Inglaterra, Austrália, dentre outros. Porém, não aprendemos inglês aplicado aos EUA, ou inglês aplicado à Inglaterra. Inglês é inglês, e pode ser falado em diferentes países.



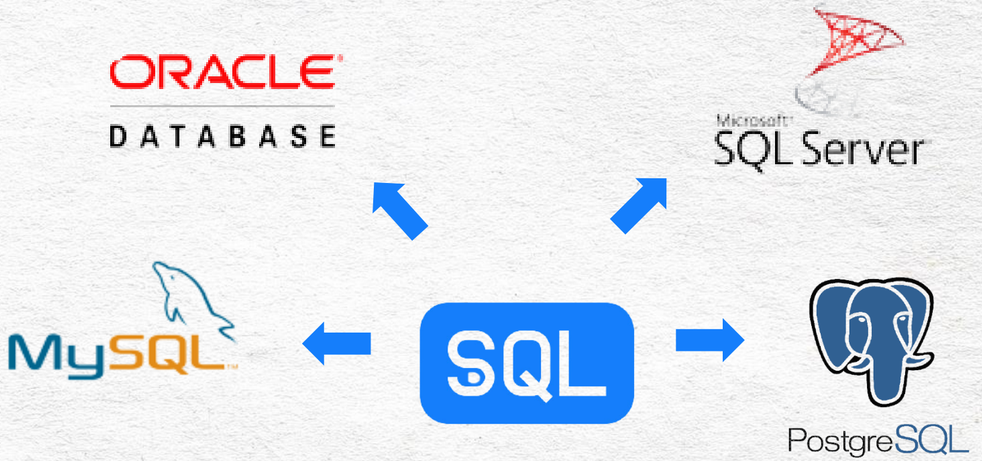
SQL, MySQL, SQL Server, Oracle e PostgreSQL: Não confunda os SQL's!!

O SQL é uma linguagem de consulta a bancos de dados, enquanto o MySQL, SQL Server, Oracle Database e PostgreSQL são programas utilizados para gerenciamento dos bancos de dados.

Para ficar clara essa diferença, faça um paralelo com o inglês.

O inglês pode ser falado em diferentes países, como o EUA, Inglaterra, Austrália, dentre outros. Porém, não aprendemos inglês aplicado aos EUA, ou inglês aplicado à Inglaterra. Inglês é inglês, e pode ser falado em diferentes países.

O mesmo vale para o SQL. O SQL é uma linguagem de consulta (idioma) que pode ser usado em diferentes programas (países): MySQL, SQL Server, Oracle e PostgreSQL.



SQL, MySQL, SQL Server, Oracle e PostgreSQL: Não confunda os SQL's!!

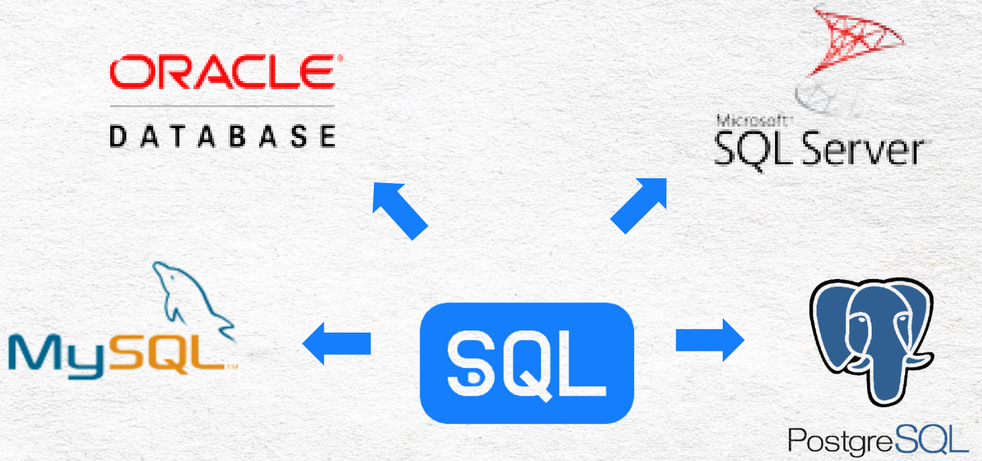
O SQL é uma linguagem de consulta a bancos de dados, enquanto o MySQL, SQL Server, Oracle Database e PostgreSQL são programas utilizados para gerenciamento dos bancos de dados.

Para ficar clara essa diferença, faça um paralelo com o inglês.

O inglês pode ser falado em diferentes países, como o EUA, Inglaterra, Austrália, dentre outros. Porém, não aprendemos inglês aplicado aos EUA, ou inglês aplicado à Inglaterra. Inglês é inglês, e pode ser falado em diferentes países.

O mesmo vale para o SQL. O SQL é uma linguagem de consulta (idioma) que pode ser usado em diferentes programas (países): MySQL, SQL Server, Oracle e PostgreSQL.

Portanto, não se assuste com tantos SQLs. Na verdade, só existe um SQL. Todos os demais são programas que utilizam o SQL para realizar as consultas aos bancos de dados.



MÓDULO 2

INSTALAÇÃO

INSTALAÇÃO
INSTALAÇÃO

Índice Geral
Índice deste Módulo

O QUE VEREMOS NESTE MÓDULO

MÓDULO 2 INSTALAÇÃO E IMPORTAÇÃO DO BANCO DE DADOS

01	INSTALAÇÃO DO MYSQL	035
02	BANCO DE DADOS UTILIZADO	036

Instalação do MySQL

O SGBD escolhido será o **MySQL**.

Lembrando que o SGBD será composto essencialmente por 2 partes: um **Servidor** e uma **Interface**.

Para o caso do MySQL, teremos que instalar o **MySQL Server** e o **MySQL Workbench**, respectivamente.

MySQL Server

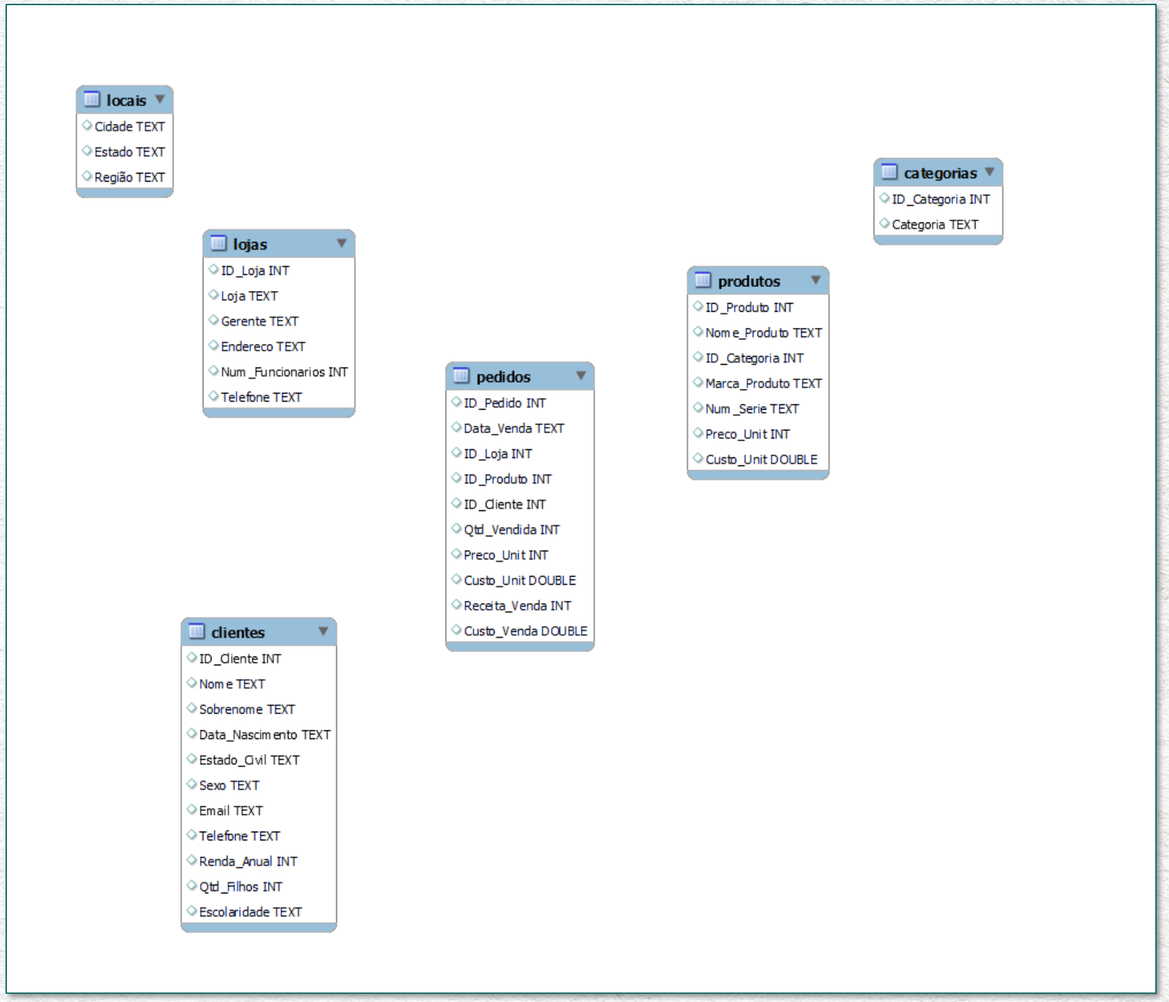
um **servidor**, onde vamos conseguir armazenar os nossos bancos de dados.

MySQL Workbench

uma **interface** amigável que nos permite escrever os códigos em SQL para acessar os bancos de dados.

Banco de Dados utilizado

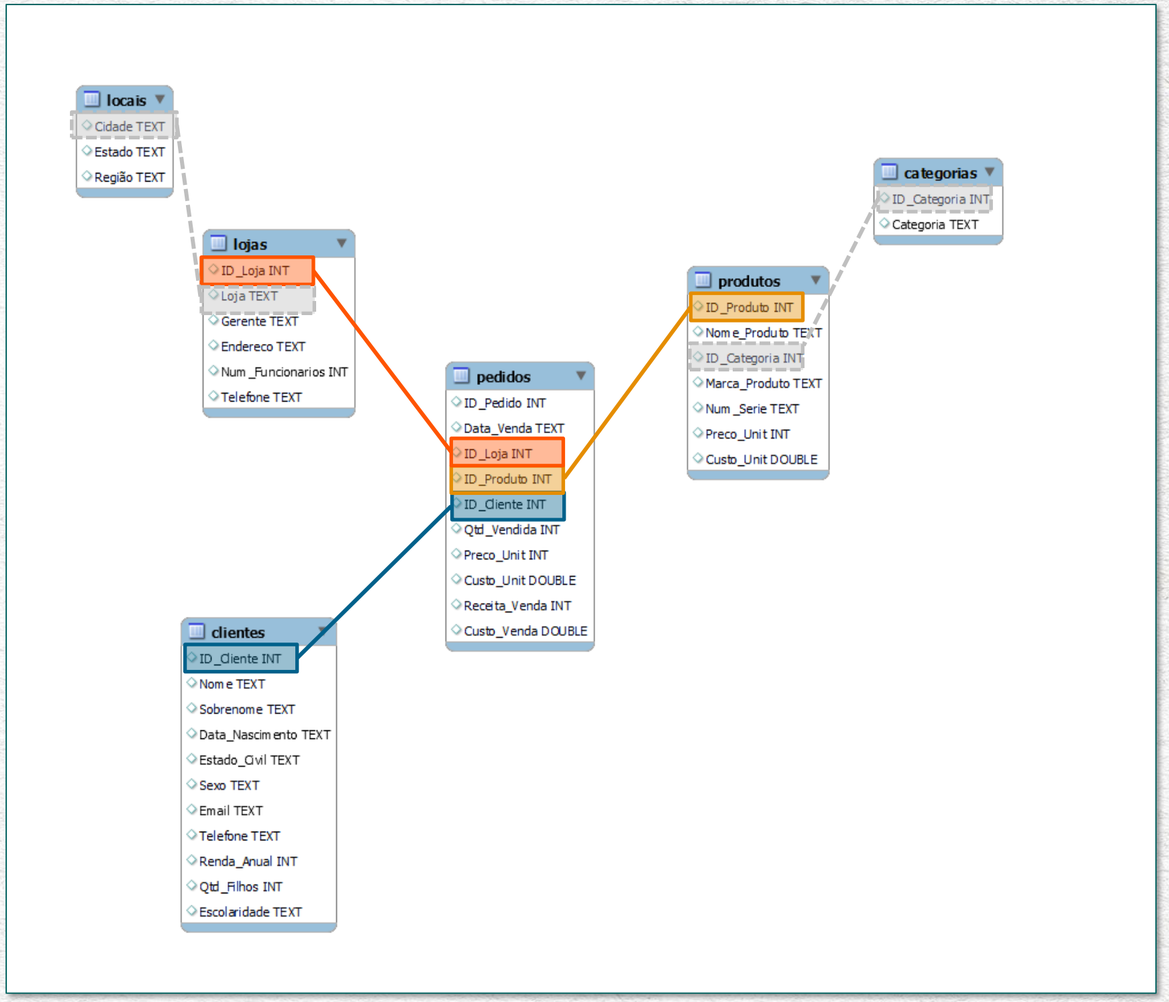
As tabelas do banco de dados estão mostradas na imagem ao lado.



Banco de Dados utilizado

As tabelas do banco de dados estão mostradas na imagem ao lado.

Existem colunas em comum entre as tabelas, para que seja possível criar um banco de dados relacional.



MÓDULO 3

CONSULTAS BÁSICAS

CONSULTAS BÁSICAS

CONSULTAS BÁSICAS

Índice Geral

Índice deste Módulo

O QUE VEREMOS NESTE MÓDULO

MÓDULO 3 CONSULTAS BÁSICAS

01	SELECT: *, FROM, AS, LIMIT e DISTINCT	040
02	ORDER BY DESC e ORDER BY ASC	046
03	COMENTÁRIOS	049
04	EXERCÍCIOS	050
05	GABARITO EXPLICATIVO	055

SELECT *, SELECT FROM, SELECT AS, SELECT LIMIT e SELECT DISTINCT

Vamos começar aprendendo sobre os comandos de seleção de dados no MySQL. São eles:

Comandos de Seleção

- SELECT FROM
- SELECT LIMIT
- SELECT AS
- SELECT DISTINCT

Índice Geral

Índice deste Módulo

SELECT *

OBJETIVO

Selecionar **todas as colunas** e todas as linhas de uma tabela.

CÓDIGO

```
1 • SELECT *
2   FROM Tabela;
```

RESULTADO

Col1	Col2	Col3
1	Nome	10.5
2	Texto	5.45
3	Carro	51.3
4	Celular	100.6
5	SQL	500

SELECT (colunas)

OBJETIVO

Selecionar **apenas colunas específicas** de uma tabela.

CÓDIGO

```
1 • SELECT Col1, Col2
2   FROM Tabela;
```

RESULTADO

Col1	Col2
1	Nome
2	Texto
3	Carro
4	Celular
5	SQL

SELECT AS

OBJETIVO

Selecionar colunas específicas e dar um nome para essas colunas.

CÓDIGO

```
1 • SELECT
2       Col1 AS "Coluna 1",
3       Col2 AS "Coluna 2"
4   FROM Tabela;
```

RESULTADO

Coluna 1	Coluna 2
1	Nome
2	Texto
3	Carro
4	Celular
5	SQL

SELECT LIMIT

OBJETIVO

Selecionar apenas as N primeiras linhas de uma determinada tabela.

CÓDIGO

```
1 • SELECT
2       *
3   FROM Tabela
4   LIMIT 2;
```

RESULTADO

Col1	Col2	Col3
1	Nome	10.5
2	Texto	5.45

SELECT DISTINCT

OBJETIVO

Selecionar apenas os valores distintos de uma coluna.

CÓDIGO

```
SELECT
    DISTINCT genero
FROM filmes;
```

RESULTADO

genero
Comédia
Drama
Ficção e Fantasia
Mistério e Suspense
Arte
Animação
Ação e Aventura

ORDER BY DESC e ORDER BY ASC

Os comandos de ordenação permitem a ordenação dos dados da nossa tabela, a partir de uma coluna. Com ele, podemos ordenar por ordem crescente, ordem alfabética, e assim vai.

Comandos de Ordenação

ORDER BY ASC

ORDER BY DESC

ORDER BY (DESC)

OBJETIVO

Permite ordenar (classificar) uma tabela a partir de uma determinada coluna em **ordem decrescente**.

CÓDIGO

```
1 • SELECT
2       *
3   FROM Tabela
4   ORDER BY Col3 DESC;
```

RESULTADO

Col1	Col2	Col3
5	SQL	500
4	Celular	100.6
3	Carro	51.3
1	Nome	10.5
2	Texto	5.45

ORDER BY (ASC)

OBJETIVO

Permite ordenar (classificar) uma tabela a partir de uma determinada coluna em **ordem crescente**.

CÓDIGO

```
1  SELECT
2      *
3  FROM Tabela
4  ORDER BY Col3;
```

RESULTADO

Col1	Col2	Col3
2	Texto	5.45
1	Nome	10.5
3	Carro	51.3
4	Celular	100.6
5	SQL	500

Comentários

Existem 3 formas de comentar códigos, assim como mostrado ao lado.

Obs.: As opções de traço duplo e barra-asterisco funcionam em qualquer SGBD que usarmos.

```
1   -- Este é um comentário
2   um codigo qualquer
3
4   # Este é outro comentário
5   outro codigo qualquer
6
7   /* Este é um bloco de codigo
8   este é outro bloco de codigo */
9   outro codigo
10  mais um codigo
```

EXERCÍCIOS

1

Questão 1

Sua empresa abrirá uma nova filial. Você faz parte da equipe responsável por decidir qual será o melhor local para abrir essa filial.

Sabendo-se que a meta de longo prazo da empresa é expandir sua marca para todas as regiões do Brasil, verifique se existe alguma região que ainda não possui uma filial em funcionamento, pois essa informação será fundamental para a tomada de decisão. **(Utilize a tabela locais)**

2

Questão 2

A empresa decidiu diversificar o seu catálogo de produtos.

Para decidir quais serão os novos produtos a comercializar, primeiramente é necessário saber exatamente quais são os produtos que a empresa já vende.

Você foi alocado para descobrir quais são esses produtos e suas respectivas marcas. **(Utilize a tabela produtos)**

EXERCÍCIOS

3

Questão 3

Você trabalha no departamento de marketing da empresa e acabou de ter uma ideia: no Dia do Consumidor, será oferecido um super desconto de 25% para os 10 clientes mais antigos, como uma forma de homenageá-los e agradecê-los. **(Utilize a tabela clientes)**

a) Descubra quem são esses 10 clientes. Liste seus IDs, nomes, sobrenomes, e-mails e telefones.

b) Você contou sua ideia para seu chefe e ele gostou tanto que resolveu incluir ainda mais clientes na oferta: além de oferecer 25% aos 10 clientes mais antigos, ele quer oferecer 10% de desconto aos outros 20 clientes mais antigos também. Ou seja: os 10 clientes mais antigos receberão 25% de desconto, e os clientes 11º a 30º mais antigos receberão 10%. Sua tarefa agora é descobrir quem são os clientes que receberão 10% de desconto no Dia do Consumidor. Novamente, liste seus IDs, nomes, sobrenomes, e-mails e telefones.

EXERCÍCIOS

4

Questão 4

Seu superior precisa de uma lista com os endereços e telefones de todas as filiais da empresa. Na tabela lojas você pode encontrar todas essas informações.

Retorne uma tabela com as colunas Loja, Endereco e Telefone renomeando-as para Filial, Endereço e Contato, respectivamente. **(Utilize a tabela lojas)**

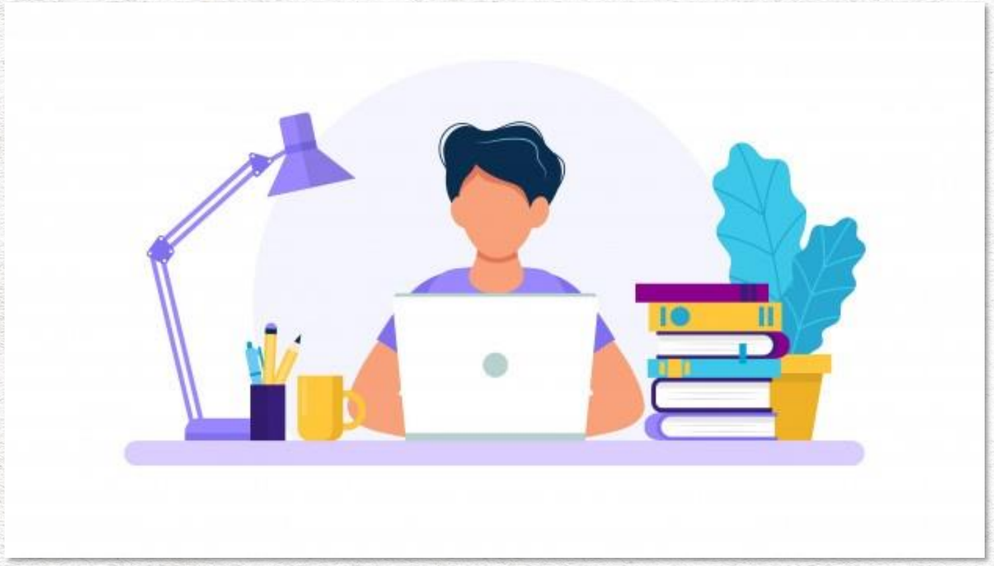
5

Questão 5

Atualmente, a empresa comercializa 16 produtos. O departamento de vendas precisa saber se existe algum produto que nunca foi vendido, pois, caso exista, será feita uma análise mais aprofundada quanto ao motivo da falta de vendas.

Sua função é descobrir se existe algum produto do catálogo de produtos da empresa que nunca foi vendido. **(Utilize a tabela pedidos)**

EXERCÍCIOS



6

Questão 6

Na semana passada, alguns colaboradores da empresa pediram demissão ou foram dispensados, o que deixou o quadro de funcionários de algumas lojas defasado. Por esse motivo, o RH iniciará um novo processo seletivo para a contratação de novos funcionários.

O diretor do RH pediu uma lista com a quantidade de funcionários em atividade em cada filial. Assim, caso verifique que outras filiais também necessitam de mais funcionários, aproveitará o processo seletivo para contratar mais colaboradores para trabalhar nessas lojas.

Crie a consulta SQL que retorna o resultado que o diretor precisa, ordenando-o pelo número de funcionários em ordem crescente. **(Utilize a tabela lojas)**

7

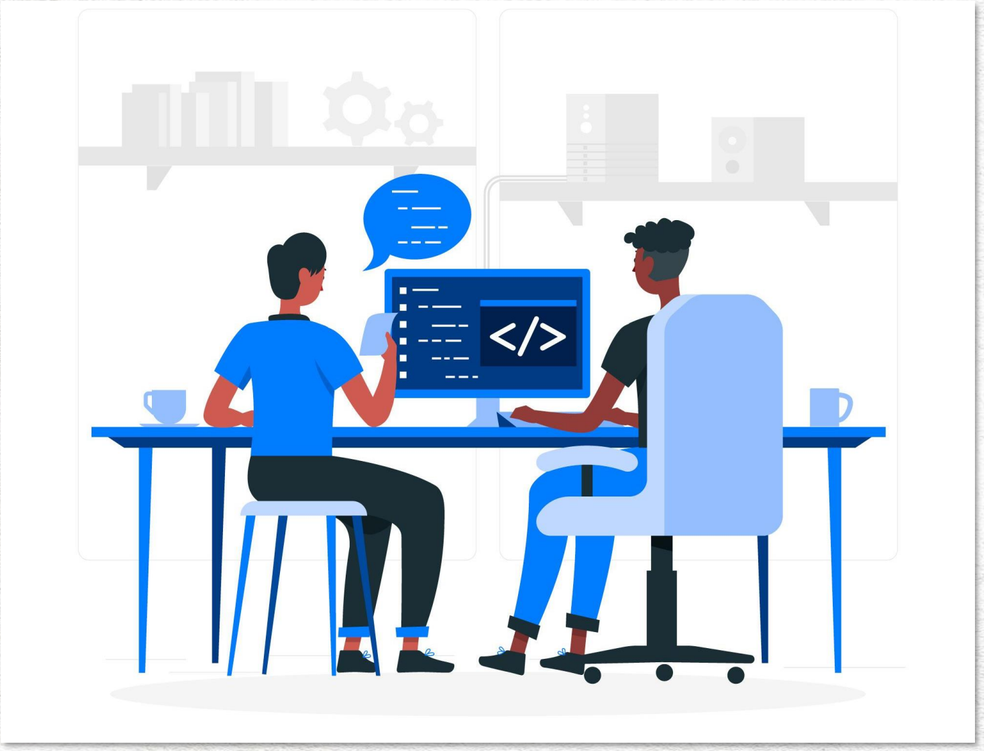
Questão 7

O diretor do setor de compras solicitou um relatório com as 100 melhores vendas efetuadas, de acordo com a receita, pois apresentará essas informações em uma reunião.

Crie a consulta SQL que retorna o resultado que o diretor precisa. **(Utilize a tabela pedidos)**

EXERCÍCIOS

8



Questão 8

O lucro gerado por um produto vendido é obtido levando-se em conta o seu preço e custo unitários. Quanto maior o preço e menor o custo, maior o lucro.

Construa uma consulta que retorne os produtos ordenados por seu preço unitário e custo unitário em ordem decrescente.

Em seguida, responda às seguintes questões: **(Utilize a tabela produtos)**

a) Existem produtos com o mesmo preço unitário? Quais?

b) Entre tais produtos, qual deles é o mais lucrativo? Por quê?

GABARITO EXPLICATIVO

1

Questão 1

Sua empresa abrirá uma nova filial. Você faz parte da equipe responsável por decidir qual será o melhor local para abrir essa filial.

Sabendo-se que a meta de longo prazo da empresa é expandir sua marca para todas as regiões do Brasil, verifique se existe alguma região que ainda não possui uma filial em funcionamento, pois essa informação será fundamental para a tomada de decisão. **(Utilize a tabela locais)**

Explicação e Resultado

Para verificarmos em quais regiões já temos uma filial aberta, podemos fazer uma consulta à tabela **LOCAIS**:

```
SELECT * FROM locais;
```

Dessa forma, verificamos que já temos filiais nas regiões Sul, Sudeste, Centro-Oeste e Nordeste:

Cidade	Estado	Região
Rio de Janeiro	RJ	Sudeste
Belo Horizonte	MG	Sudeste
Salvador	BA	Nordeste
Curitiba	PR	Sul
Fortaleza	CE	Nordeste
São Paulo	SP	Sudeste
Niterói	RJ	Sudeste
Goiânia	GO	Centro-Oeste

Portanto, temos uma região brasileira que ainda não possui uma filial: NORTE.

GABARITO EXPLICATIVO

2

Questão 2

A empresa decidiu diversificar o seu catálogo de produtos.

Para decidir quais serão os novos produtos a comercializar, primeiramente é necessário saber exatamente quais são os produtos que a empresa já vende.

Você foi alocado para descobrir quais são esses produtos e suas respectivas marcas. **(Utilize a tabela produtos)**

Explicação e Resultado

Para verificarmos quais produtos a empresa já vende, assim como suas respectivas marcas, podemos fazer uma consulta à tabela **PRODUTOS** para retornar as colunas **Nome_Produto** e **Marca_Produto**:

```
SELECT
    Nome_Produto,
    Marca_Produto
FROM produtos;
```

Assim, retornamos o seguinte resultado:

Nome_Produto	Marca_Produto
Monitor LED 19,5" Full HD HDMI	DELL
Monitor Curvo 24" 144Hz HDMI	SAMSUNG
Webcam Full HD 1080p	LOGITECH
Kit Teclado + Mouse sem fio Wireless	DELL
Kit Teclado + Mouse Slim Bluetooth	DELL
Cadeira Gamer reclinável Azul/Laranja	ALTURA
Cadeira Gamer PC Racer Vermelha	ALTURA
Headphone Bluetooth 2000	SONY
Fone de Ouvido Tune T5000	JBL
Microfone Condensador MC1000	AKG
Microfone Condensador com Tripé	BLUE
Microfone de mesa com fio condensa...	BLUE
Notebook LC2100 Intel Core i5 8GB	SAMSUNG
Notebook Inspiron 15 5000 4GB	DELL
Notebook IdeaPad RF32000	DELL
Notebook Motion Ultra 2	SAMSUNG

GABARITO EXPLICATIVO

3a

Questão 3, item A

Você trabalha no departamento de marketing da empresa e acabou de ter uma ideia: no Dia do Consumidor, será oferecido um super desconto de 25% para os 10 clientes mais antigos, como uma forma de homenageá-los e agradecê-los. **(Utilize a tabela clientes)**

a) Descubra quem são esses 10 clientes. Liste seus IDs, nomes, sobrenomes, e-mails e telefones.

b) Você contou sua ideia para seu chefe e ele gostou tanto que resolveu incluir ainda mais clientes na oferta: além de oferecer 25% aos 10 clientes mais antigos, ele quer oferecer 10% de desconto aos outros 20 clientes mais antigos também. Ou seja: os 10 clientes mais antigos receberão 25% de desconto, e os clientes 11º a 30º mais antigos receberão 10%. Sua tarefa agora é descobrir quem são os clientes que receberão 10% de desconto no Dia do Consumidor. Novamente, liste seus IDs, nomes, sobrenomes, e-mails e telefones.

Explicação e Resultado

Para verificarmos quais são os 10 clientes mais antigos, podemos fazer uma consulta à tabela **CLIENTES** limitando o resultado às 10 primeiras linhas (usando o **LIMIT 10**). Além disso, podemos retornar somente as colunas **ID_Cliente, Nome, Sobrenome, Email e Telefone**:

```
SELECT
    ID_Cliente,
    Nome,
    Sobrenome,
    Email,
    Telefone
FROM clientes
LIMIT 10;
```

Assim, temos o seguinte resultado:

ID_Cliente	Nome	Sobrenome	Email	Telefone
1	Ruben	Torres	ruben35@hotmail.com	(85) 94132-1176
2	Christy	Zhu	christy12@hotmail.com	(41) 96270-6086
3	Elizabeth	Johnson	elizabeth5@gmail.com	(31) 92039-5832
4	Julio	Ruiz	julio1@hotmail.com	(62) 93391-5891
5	Janet	Alvarez	janet9@gmail.com	(21) 93379-3743
6	Marco	Mehta	marco14@yahoo.com.br	(85) 99019-3803
7	Rob	Verhoff	rob4@hotmail.com	(11) 98162-4760
8	Shannon	Carlson	shannon38@gmail.com	(85) 96795-9950
9	Jacquelyn	Suarez	jacquelyn20@hotmail.com	(31) 98159-7254
10	Curtis	Lu	curtis9@yahoo.com.br	(71) 94509-1440

GABARITO EXPLICATIVO

3b

Questão 3, item B

Você trabalha no departamento de marketing da empresa e acabou de ter uma ideia: no Dia do Consumidor, será oferecido um super desconto de 25% para os 10 clientes mais antigos, como uma forma de homenageá-los e agradecê-los. **(Utilize a tabela clientes)**

a) Descubra quem são esses 10 clientes. Liste seus IDs, nomes, sobrenomes, e-mails e telefones.

b) Você contou sua ideia para seu chefe e ele gostou tanto que resolveu incluir ainda mais clientes na oferta: além de oferecer 25% aos 10 clientes mais antigos, ele quer oferecer 10% de desconto aos outros 20 clientes mais antigos também. Ou seja: os 10 clientes mais antigos receberão 25% de desconto, e os clientes 11º a 30º mais antigos receberão 10%. Sua tarefa agora é descobrir quem são os clientes que receberão 10% de desconto no Dia do Consumidor. Novamente, liste seus IDs, nomes, sobrenomes, e-mails e telefones.

Explicação e Resultado

Para verificarmos quais são os 20 clientes que receberão 10% de desconto, podemos fazer uma consulta à tabela **CLIENTES** limitando o resultado a 20 linhas (usando o **LIMIT 20**). Como queremos pular os 10 primeiros clientes mais antigos e retornar somente do 11º ao 30º, usamos o **OFFSET 10**. Além disso, retornamos somente as colunas **ID_Cliente, Nome, Sobrenome, Email e Telefone**:

```
SELECT
    ID_Cliente,
    Nome,
    Sobrenome,
    Email,
    Telefone
FROM clientes
LIMIT 20 OFFSET 10;
```

ID_Cliente	Nome	Sobrenome	Email	Telefone
11	Lauren	Walker	lauren41@hotmail.com	(21) 93994-2382
12	Ian	Jenkins	ian47@gmail.com	(21) 95385-3393
13	Sydney	Bennett	sydney23@gmail.com	(11) 92024-9186
14	Chloe	Young	chloe23@hotmail.com	(71) 96890-7150
15	Wyatt	Hill	wyatt32@gmail.com	(71) 98644-2695
16	Shannon	Wang	shannon1@hotmail.com	(71) 94496-1712
17	Clarence	Rai	clarence32@yahoo.com.br	(21) 98923-7805
18	Jordan	King	jordan73@gmail.com	(31) 99592-6279
19	Destiny	Wilson	destiny7@hotmail.com	(21) 99431-6061
20	Ethan	Zhang	ethan20@hotmail.com	(11) 96440-6637
21	Seth	Edwards	seth46@gmail.com	(41) 97012-3526
22	Russell	Xie	russell7@hotmail.com	NULL
23	Alejandro	Beck	alejandro45@gmail.com	(85) 94492-9582
24	Harold	Sai	harold3@hotmail.com	(31) 95052-6286
25	Jessie	Zhao	jessie16@hotmail.com	(41) 98751-8234
26	Jill	Jimenez	jill13@gmail.com	(41) 97120-5603
27	Jimmy	Moreno	jimmy9@yahoo.com.br	(85) 92931-9277
28	Bethany	Yuan	bethany10@hotmail.com	(62) 92998-6359
29	Theresa	Ramos	theresa13@hotmail.com	(71) 92187-3667
30	Denise	Stone	denise10@hotmail.com	(62) 94170-5754

Assim, obtemos o resultado ao lado:

GABARITO EXPLICATIVO

4

Questão 4

Seu superior precisa de uma lista com os endereços e telefones de todas as filiais da empresa. Na tabela lojas você pode encontrar todas essas informações.

Retorne uma tabela com as colunas Loja, Endereco e Telefone renomeando-as para Filial, Endereço e Contato, respectivamente. **(Utilize a tabela lojas)**

Explicação e Resultado

Para retornarmos uma lista com os endereços e telefones de todas as filiais da empresa, fazemos uma consulta à tabela **LOJAS** para retornar as colunas **Loja**, **Endereco** e **Telefone**. Para renomear as colunas para 'Filial', 'Endereço' e 'Contato', respectivamente, utilizamos aliases (**AS**):

```
SELECT
    Loja AS 'Filial',
    Endereco AS 'Endereço',
    Telefone AS 'Contato'
FROM lojas;
```

Assim, retornamos o seguinte resultado:

Filial	Endereço	Contato
Rio de Janeiro	R. Visc. de Pirajá, 136 - Ipanema, Rio de Janeir...	(21) 3198-1239
Belo Horizonte	Av. Barão Homem de Melo, 1389 - Nova Granad...	(31) 2891-2394
Salvador	Shopping Barra - Av. Centenário, 2992 - Barra, ...	(71) 3465-8748
Curitiba	R. Brasílio Itiberê, 3279 - Água Verde, Curitiba -...	(41) 2923-2137
Fortaleza	Av. Dom Luís, 500 - Aldeota, Fortaleza - CE, 60...	(85) 3294-1481
São Paulo	R. Teodoro Sampaio, 954 - Pinheiros, São Paulo...	(11) 3295-3945
Niterói	Rua Quinze de Novembro, 8 - Loja 301a Loja 30...	(21) 3134-7263
Goiânia	Av. Goiás Norte, 3.592 - quadra 2.1 - St. Mal. ...	NULL

GABARITO EXPLICATIVO

5

Questão 5

Atualmente, a empresa comercializa 16 produtos. O departamento de vendas precisa saber se existe algum produto que nunca foi vendido, pois, caso exista, será feita uma análise mais aprofundada quanto ao motivo da falta de vendas.

Sua função é descobrir se existe algum produto do catálogo de produtos da empresa que nunca foi vendido. **(Utilize a tabela pedidos)**

Explicação e Resultado

Sabendo que a empresa comercializa 16 produtos, podemos fazer uma consulta à tabela **PEDIDOS** para retornar somente os **ID_Produto** distintos (**DISTINCT**). Assim, dependendo da quantidade de linhas retornadas, saberemos se existem produtos que nunca foram vendidos ou não. Se forem retornadas 16 linhas (ou seja, 16 ID_Produto distintos), todos os produtos já foram vendidos ao menos uma vez. Se forem retornadas menos de 16 linhas, saberemos que algum(ns) produto(s) ainda não foi/foram vendido(s):

```
SELECT DISTINCT
    ID_Produto
FROM pedidos;
```

O SELECT à esquerda retorna o resultado à direita, com 16 linhas:

ID_Produto
11
14
9
8
13
3
15
4
16
1
12
2
6
10
7
5

Portanto, não existem produtos comercializados pela empresa que nunca foram vendidos.

GABARITO EXPLICATIVO

6

Questão 6

Na semana passada, alguns colaboradores da empresa pediram demissão ou foram dispensados, o que deixou o quadro de funcionários de algumas lojas defasado. Por esse motivo, o RH iniciará um novo processo seletivo para a contratação de novos funcionários.

O diretor do RH pediu uma lista com a quantidade de funcionários em atividade em cada filial. Assim, caso verifique que outras filiais também necessitam de mais funcionários, aproveitará o processo seletivo para contratar mais colaboradores para trabalhar nessas lojas.

Crie a consulta SQL que retorna o resultado que o diretor precisa, ordenando-o pelo número de funcionários em ordem crescente. **(Utilize a tabela lojas)**

Explicação e Resultado

Para retornarmos a quantidade de funcionários em atividade em cada filial, podemos fazer uma consulta à tabela **LOJAS**, pois nela temos a coluna **Num_Funcionarios**. Além disso, para ordenarmos o resultado em **ordem crescente** de acordo com essa quantidade, acrescentamos ao código um **ORDER BY Num_Funcionarios**. Por padrão, o ORDER BY já retorna a consulta em ordem crescente, mas, se preferir, também podemos acrescentar o **ASC** após o nome da coluna pela qual estamos ordenando o resultado:

```
SELECT * FROM lojas
ORDER BY Num_Funcionarios;
```

OU

```
SELECT * FROM lojas
ORDER BY Num_Funcionarios ASC;
```

Assim, retornamos o seguinte resultado:

ID_Loja	Loja	Gerente	Endereco	Num_Funcionarios	Telefone
8	Goiânia	Carlos Henrique	Av. Goiás Norte, 3.592 - quadra 2.1 - St. Mal. ...	11	NULL
5	Fortaleza	Letícia Gomes	Av. Dom Luís, 500 - Aldeota, Fortaleza - CE, 60...	12	(85) 3294-1481
2	Belo Horizonte	Julia Freitas	Av. Barão Homem de Melo, 1389 - Nova Granad...	13	(31) 2891-2394
7	Niterói	Matheus Leal	Rua Quinze de Novembro, 8 - Loja 301a Loja 30...	18	(21) 3134-7263
3	Salvador	Marcelo Castro	Shopping Barra - Av. Centenário, 2992 - Barra, ...	19	(71) 3465-8748
1	Rio de Janeiro	Pedro Martins	R. Visc. de Pirajá, 136 - Ipanema, Rio de Janeir...	23	(21) 3198-1239
4	Curitiba	Amanda Lima	R. Brasílio Itiberê, 3279 - Água Verde, Curitiba -...	31	(41) 2923-2137
6	São Paulo	Rogério Lopes	R. Teodoro Sampaio, 954 - Pinheiros, São Paulo...	33	(11) 3295-3945

GABARITO EXPLICATIVO

7

Questão 7

O diretor do setor de compras solicitou um relatório com as 100 melhores vendas efetuadas, de acordo com a receita, pois apresentará essas informações em uma reunião.

Crie a consulta SQL que retorna o resultado que o diretor precisa. **(Utilize a tabela pedidos)**

Explicação e Resultado

Para retornarmos as 100 melhores vendas efetuadas de acordo com a receita, podemos fazer uma consulta à tabela **PEDIDOS**, ordenando o resultado de forma decrescente (**DESC**) pela coluna **Receita_Venda** (assim, teremos um resultado da venda que gerou a maior receita para a venda que gerou a menor receita). Como queremos retornar apenas as 100 melhores vendas, acrescentamos um **LIMIT 100**:

```
SELECT * FROM pedidos
ORDER BY Receita_Venda DESC
LIMIT 100;
```

Dessa forma, retornamos o seguinte resultado:

ID_Pedido	Data_Venda	ID_Loja	ID_Produto	ID_Cliente	Qtd_Vendida	Preco_Unit	Custo_Unit	Receita_Venda	Custo_Venda
43693	2020/12/08	7	15	38	1	4200	1176	4200	1176
44116	2020/12/14	1	15	89	1	4200	1176	4200	1176
45049	2020/12/29	6	15	43	1	4200	1176	4200	1176
44640	2020/12/23	6	15	8	1	4200	1176	4200	1176
44646	2020/12/23	2	15	23	1	4200	1176	4200	1176
44356	2020/12/18	1	15	53	1	4200	1176	4200	1176
43657	2020/12/07	8	15	54	1	4200	1176	4200	1176
44621	2020/12/22	1	15	98	1	4200	1176	4200	1176
44171	2020/12/15	5	15	69	1	4200	1176	4200	1176
44147	2020/12/15	3	15	95	1	4200	1176	4200	1176
44878	2020/12/27	4	15	5	1	4200	1176	4200	1176
44876	2020/12/27	1	15	3	1	4200	1176	4200	1176
44625	2020/12/23	8	15	39	1	4200	1176	4200	1176

pedidos 11 ×

Output

Action Output

#	Time	Action	Message
1	22:12:34	SELECT * FROM pedidos ORDER BY Receita_Venda DESC LIMIT 100	100 row(s) returned

GABARITO EXPLICATIVO

8

Questão 8

O lucro gerado por um produto vendido é obtido levando-se em conta o seu preço e custo unitários. Quanto maior o preço e menor o custo, maior o lucro.

Construa uma consulta que retorne os produtos ordenados por seu preço unitário e custo unitário em ordem decrescente.

Em seguida, responda às seguintes questões: **(Utilize a tabela produtos)**

a) Existem produtos com o mesmo preço unitário? Quais?

b) Entre tais produtos, qual deles é o mais lucrativo? Por quê?

Explicação e Resultado

Para respondermos os itens A e B, primeiro criamos a consulta à tabela **PRODUTOS** que retorna o resultado ordenando as linhas tanto pela coluna **Preco_Unit** quanto pela **Custo_Unit** em ordem decrescente (**DESC**):

```
SELECT * FROM produtos
ORDER BY Preco_Unit DESC, Custo_Unit DESC;
```

Assim, obtemos o resultado abaixo (primeiro a tabela é ordenada por Preco_Unit DESC; havendo Preco_Unit repetido em algumas linhas, essas linhas são ordenadas pelo Custo_Unit DESC):

ID_Produto	Nome_Produto	ID_Categoria	Marca_Produto	Num_Serie	Preco_Unit	Custo_Unit
15	Notebook IdeaPad RF32000	6	DELL	NOT-DL-77164I	4200	1176
13	Notebook LC2100 Intel Core i5 8GB	6	SAMSUNG	NOT-SS-918457	3400	850
7	Cadeira Gamer PC Racer Vermelha	3	ALTURA	CGM-AL-0147FI	3100	1395
14	Notebook Inspiron 15 5000 4GB	6	DELL	NOT-DL-000012	3100	1209
16	Notebook Motion Ultra 2	6	SAMSUNG	NOT-SS-13139U	2900	1160
2	Monitor Curvo 24" 144Hz HDMI	1	SAMSUNG	MNT-SS-001939	2800	980
1	Monitor LED 19,5" Full HD HDMI	1	DELL	MNT-DL-831923	2300	966
6	Cadeira Gamer reclinável Azul/Laranja	3	ALTURA	CGM-AL-9N914J	1800	540
10	Microfone Condensador MC1000	5	AKG	MIC-AK-237591	1100	275
11	Microfone Condensador com Tripé	5	BLUE	MIC-BL-819455	800	344
9	Fone de Ouvido Tune T5000	4	JBL	HDP-JB-091934	780	327.6
12	Microfone de mesa com fio condensa...	5	BLUE	MIC-BL-761411	650	214.5
8	Headphone Bluetooth 2000	4	SONY	HDP-SN-194821	600	258
3	Webcam Full HD 1080p	1	LOGITECH	WBC-LT-934GG4	450	90
4	Kit Teclado + Mouse sem fio Wireless	2	DELL	KTM-DL-041039	350	129.5
5	Kit Teclado + Mouse Slim Bluetooth	2	DELL	KTM-DL-111924	280	109.2

GABARITO EXPLICATIVO

8a

Questão 8, item A

O lucro gerado por um produto vendido é obtido levando-se em conta o seu preço e custo unitários. Quanto maior o preço e menor o custo, maior o lucro.

Construa uma consulta que retorne os produtos ordenados por seu preço unitário e custo unitário em ordem decrescente.

Em seguida, responda às seguintes questões: **(Utilize a tabela produtos)**

a) Existem produtos com o mesmo preço unitário? Quais?

b) Entre tais produtos, qual deles é o mais lucrativo? Por quê?

Explicação e Resultado

ID_Produto	Nome_Produto	ID_Categoria	Marca_Produto	Num_Serie	Preco_Unit	Custo_Unit
15	Notebook IdeaPad RF32000	6	DELL	NOT-DL-77164I	4200	1176
13	Notebook LC2100 Intel Core i5 8GB	6	SAMSUNG	NOT-SS-918457	3400	850
7	Cadeira Gamer PC Racer Vermelha	3	ALTURA	CGM-AL-0147FI	3100	1395
14	Notebook Inspiron 15 5000 4GB	6	DELL	NOT-DL-000012	3100	1209
16	Notebook Motion Ultra 2	6	SAMSUNG	NOT-SS-13139U	2900	1160
2	Monitor Curvo 24" 144Hz HDMI	1	SAMSUNG	MNT-SS-001939	2800	980
1	Monitor LED 19,5" Full HD HDMI	1	DELL	MNT-DL-831923	2300	966
6	Cadeira Gamer reclinável Azul/Laranja	3	ALTURA	CGM-AL-9N914J	1800	540
10	Microfone Condensador MC1000	5	AKG	MIC-AK-237591	1100	275
11	Microfone Condensador com Tripé	5	BLUE	MIC-BL-819455	800	344
9	Fone de Ouvido Tune T5000	4	JBL	HDP-JB-091934	780	327.6
12	Microfone de mesa com fio condensa...	5	BLUE	MIC-BL-761411	650	214.5
8	Headphone Bluetooth 2000	4	SONY	HDP-SN-194821	600	258
3	Webcam Full HD 1080p	1	LOGITECH	WBC-LT-934GG4	450	90
4	Kit Teclado + Mouse sem fio Wireless	2	DELL	KTM-DL-041039	350	129.5
5	Kit Teclado + Mouse Slim Bluetooth	2	DELL	KTM-DL-111924	280	109.2

Analisando o resultado acima, vemos que **SIM, temos dois produtos com o mesmo preço unitário:** ID_Produto 7 (Cadeira Gamer PC Racer Vermelha) e ID_Produto 14 (Notebook Inspiron 15 5000 4GB). **Ambos custam R$ 3.100,00**:

ID_Produto	Nome_Produto	ID_Categoria	Marca_Produto	Num_Serie	Preco_Unit	Custo_Unit
7	Cadeira Gamer PC Racer Vermelha	3	ALTURA	CGM-AL-0147FI	3100	1395
14	Notebook Inspiron 15 5000 4GB	6	DELL	NOT-DL-000012	3100	1209

GABARITO EXPLICATIVO

8b

Questão 8, item B

O lucro gerado por um produto vendido é obtido levando-se em conta o seu preço e custo unitários. Quanto maior o preço e menor o custo, maior o lucro.

Construa uma consulta que retorne os produtos ordenados por seu preço unitário e custo unitário em ordem decrescente.

Em seguida, responda às seguintes questões: **(Utilize a tabela produtos)**

a) Existem produtos com o mesmo preço unitário? Quais?

b) Entre tais produtos, qual deles é o mais lucrativo? Por quê?

Explicação e Resultado

ID_Produto	Nome_Produto	ID_Categoria	Marca_Produto	Num_Serie	Preco_Unit	Custo_Unit
7	Cadeira Gamer PC Racer Vermelha	3	ALTURA	CGM-AL-0147FI	3100	1395
14	Notebook Inspiron 15 5000 4GB	6	DELL	NOT-DL-000012	3100	1209

Entre os dois produtos de mesmo preço, **o Notebook Inspiron 15 5000 4GB é o mais lucrativo**, pois seu custo unitário é menor em relação ao custo unitário da Cadeira Gamer PC Racer Vermelha.

MÓDULO 4

FILTROS

FILTROS

FILTROS

Índice Geral

Índice deste Módulo

O QUE VEREMOS NESTE MÓDULO

MÓDULO 4 FILTROS NO MYSQL

01	WHERE: COMANDO PARA FILTRAR DADOS	068
02	COMANDOS DE FILTRAGEM	069
03	FILTROS DE NÚMEROS	070
04	FILTROS DE TEXTOS	071
05	FILTROS DE DATA	072
06	MÚLTIPLOS FILTROS COM AND E OR	073
07	FILTRO IN	075
08	FILTRO BETWEEN	076
09	FILTRO LIKE/NOT LIKE	077
10	EXERCÍCIOS	078
11	GABARITO EXPLICATIVO	083

WHERE: Comando para filtrar dados

Os comandos de filtragem nos permitem criar filtros nas nossas tabelas dos bancos de dados.

Comandos de Filtragem

- WHERE
- WHERE AND/OR
- WHERE IN
- WHERE BETWEEN

COMANDOS DE FILTRAGEM

Essencialmente, podemos fazer filtros com colunas que contenham 3 tipos de informação.

Veremos também que é possível criar filtros em mais de 1 coluna ao mesmo tempo.

- NÚMEROS
- TEXTOS
- DATAS

FILTROS DE NÚMEROS

NÚMEROS

Podemos aplicar filtros em **colunas numéricas**.

Para isso, basta utilizar os sinais lógicos como =, <, >, <=, >=, <>.

```
1
2    -- Mostra apenas os produtos com preços iguais ou maiores que R$1.800
3 •  SELECT *
4    FROM produtos
5    WHERE Preco_Unit >= 1800;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ID_Produto	Nome_Produto	ID_Categoria	Marca_Produto	Num_Serie	Preco_Unit	Custo_Unit
1	Monitor LED 19,5" Full HD HDMI	1	DELL	MNT-DL-831923	2300	966
2	Monitor Curvo 24" 144Hz HDMI	1	SAMSUNG	MNT-SS-001939	2800	980
6	Cadeira Gamer reclinável Azul/Laranja	3	ALTURA	CGM-AL-9N914J	1800	540
7	Cadeira Gamer PC Racer Vermelha	3	ALTURA	CGM-AL-0147FI	3100	1395
13	Notebook LC2100 Intel Core i5 8GB	6	SAMSUNG	NOT-SS-918457	3400	850
14	Notebook Inspiron 15 5000 4GB	6	DELL	NOT-DL-000012	3100	1209
15	Notebook IdeaPad RF32000	6	DELL	NOT-DL-77164I	4200	1176
16	Notebook Motion Ultra 2	6	SAMSUNG	NOT-SS-13139U	2900	1160

FILTROS DE TEXTOS

TEXTOS

Podemos aplicar filtros em **colunas de texto**.

Para isso, basta utilizar o sinal de = e especificar o texto que deseja usar como critério do filtro (ou o <> caso queira apenas o que for diferente de).

```
1
2    -- Mostra apenas os produtos da marca DELL
3 •  SELECT *
4    FROM produtos
5    WHERE Marca_Produto = 'DELL';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ID_Produto	Nome_Produto	ID_Categoria	Marca_Produto	Num_Serie	Preco_Unit	Custo_Unit
1	Monitor LED 19,5" Full HD HDMI	1	DELL	MNT-DL-831923	2300	966
4	Kit Teclado + Mouse sem fio Wireless	2	DELL	KTM-DL-041039	350	129.5
5	Kit Teclado + Mouse Slim Bluetooth	2	DELL	KTM-DL-111924	280	109.2
14	Notebook Inspiron 15 5000 4GB	6	DELL	NOT-DL-000012	3100	1209
15	Notebook IdeaPad RF32000	6	DELL	NOT-DL-77164I	4200	1176

FILTROS DE DATA

DATAS

Podemos aplicar filtros em **colunas de data**.

Para isso, basta utilizar os sinais lógicos como =, <, >, <=, >=, <>.

```
1
2    -- Mostra apenas os pedidos feitos no dia 03/01/2019
3 •  SELECT *
4    FROM pedidos
5    WHERE Data_Venda = '2019-01-03';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ID_Pedido	Data_Venda	ID_Loja	ID_Produto	ID_Cliente	Qtd_Vendida	Receita_Venda	Custo_Venda	Custo_Unit	Preco_Unit
16	2019-01-03	8	4	33	1	350	129.5	129.5	350
17	2019-01-03	8	4	36	1	350	129.5	129.5	350
18	2019-01-03	1	4	37	1	350	129.5	129.5	350
19	2019-01-03	6	4	42	1	350	129.5	129.5	350
20	2019-01-03	8	4	43	1	350	129.5	129.5	350
21	2019-01-03	6	4	44	1	350	129.5	129.5	350
88	2019-01-03	6	8	29	1	600	258	258	600
89	2019-01-03	5	8	46	1	600	258	258	600
90	2019-01-03	2	8	86	1	600	258	258	600
91	2019-01-03	4	8	12	1	600	258	258	600
92	2019-01-03	4	8	82	1	600	258	258	600
93	2019-01-03	3	8	41	1	600	258	258	600

MÚLTIPLOS FILTROS COM AND E OR

OPERADOR AND

Podemos aplicar mais de um filtro usando o AND.

Com ele, todas as condições devem ser satisfeitas para que o resultado seja mostrado.

```
1
2    -- Mostra apenas os clientes SOLTEIROS do sexo MASCULINO
3 •  SELECT *
4    FROM clientes
5    WHERE Estado_Civil = 'S' AND Sexo = 'M';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ID_Cliente	Nome	Sobrenome	Data_Nascimento	Estado_Civil	Sexo	Email	Telefone	Renda_Anual	Qtd_Filhos	Escolaridade
4	Julio	Ruiz	1985-07-31	S	M	julio1@hotmail.com	(62) 93391-5891	70000	0	Pós-graduado
8	Shannon	Carlson	1984-03-27	S	M	shannon38@gmail.com	(85) 96795-9950	70000	0	Pós-graduado
17	Clarence	Rai	1964-10-04	S	M	clarence32@outlook.com	(21) 98923-7805	30000	2	Parcial
18	Jordan	King	1998-09-15	S	M	jordan73@gmail.com	(31) 99592-6279	40000	0	Ensino médio
24	Harold	Sai	1966-03-29	S	M	harold3@hotmail.com	(31) 95052-6286	30000	2	Parcial
43	Leonard	Nara	1970-05-14	S	M	leonard18@outlook.com	(31) 93245-4616	30000	3	Ensino médio
49	Daniel	Johnson	1971-07-30	S	M	daniel18@gmail.com	(31) 98422-3549	30000	3	Ensino médio
65	Caleb	Carter	1996-09-20	S	M	caleb40@gmail.com	(31) 97809-1800	60000	0	Parcial
72	Levi	Arun	1976-08-23	S	M	levi6@gmail.com	(71) 92754-9983	70000	2	Ensino médio
74	Blake	Anderson	1977-07-08	S	M	blake9@gmail.com	(11) 98232-2736	80000	2	Ensino médio
77	Donald	Gonzalez	1979-03-06	S	M	donald20@gmail.com	(31) 96897-9735	160000	0	Graduação
82	Lucas	Phillips	1977-09-07	S	M	lucas7@outlook.com	(62) 94668-3507	80000	2	Parcial
88	Trevor	Bryant	1977-12-12	S	M	trevor18@gmail.com	NULL	90000	2	Parcial
92	Cedric	Ma	1982-03-27	S	M	cedric15@outlook.com	(31) 95423-4764	70000	1	Parcial
93	Chad	Kumar	1982-08-27	S	M	chad9@gmail.com	(21) 98828-7409	70000	1	Parcial

MÚLTIPLOS FILTROS COM AND E OR

OPERADOR OR

Podemos aplicar mais de um filtro usando o **OR**.

Com ele, apenas uma condição precisa ser satisfeita para que o resultado seja mostrado.

```sql
1
2   -- Mostra apenas os produtos das marcas DELL OU SAMSUNG
3 • SELECT *
4   FROM produtos
5   WHERE Marca_Produto = 'DELL' OR Marca_Produto = 'SAMSUNG';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ID_Produto	Nome_Produto	ID_Categoria	Marca_Produto	Num_Serie	Preco_Unit	Custo_Unit
1	Monitor LED 19,5" Full HD HDMI	1	DELL	MNT-DL-831923	2300	966
2	Monitor Curvo 24" 144Hz HDMI	1	SAMSUNG	MNT-SS-001939	2800	980
4	Kit Teclado + Mouse sem fio Wireless	2	DELL	KTM-DL-041039	350	129.5
5	Kit Teclado + Mouse Slim Bluetooth	2	DELL	KTM-DL-111924	280	109.2
13	Notebook LC2100 Intel Core i5 8GB	6	SAMSUNG	NOT-SS-918457	3400	850
14	Notebook Inspiron 15 5000 4GB	6	DELL	NOT-DL-000012	3100	1209
15	Notebook IdeaPad RF32000	6	DELL	NOT-DL-77164I	4200	1176
16	Notebook Motion Ultra 2	6	SAMSUNG	NOT-SS-13139U	2900	1160

FILTRO IN

OPERADOR IN

Com o operador IN, podemos passar uma lista de valores que serão utilizados como filtro na nossa coluna.

```
86    -- Selecione apenas os clientes das seguintes nacionalidades:
87    -- Canadá, Porto Rico e Irlanda.
88
89 •  SELECT *
90    FROM atores
91    WHERE nacionalidade IN ('Canadá', 'Porto Rico', 'Irlanda');
92
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id_ator	nome_ator	ano_nascimento	nacionalidade	sexo
11	Barry Pepper	1970	Canadá	masculino
12	Benicio Del Toro	1967	Porto Rico	masculino
29	Colin Farrell	1976	Irlanda	masculino
35	Donald Sutherland	1935	Canadá	masculino
81	Keanu Reeves	1964	Canadá	masculino
112	Rachel McAdams	1978	Canadá	feminino
134	Stephen Rea	1946	Irlanda	masculino

FILTRO BETWEEN

BETWEEN

Com o operador BETWEEN, filtramos nossa tabela e retornamos apenas as linhas que estejam dentro do intervalo informado.

```
86    -- Selecione apenas os clientes que nasceram entre os anos de:
87    -- 1980 e 1983.
88
89 •  SELECT *
90    FROM atores
91    WHERE ano_nascimento BETWEEN 1980 AND 1983;
92
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id_ator	nome_ator	ano_nascimento	nacionalidade	sexo
1	Abbie Cornish	1982	Austrália	feminino
5	Andrea Riseborough	1981	Reino Unido	feminino
8	Anne Hathaway	1982	EUA	feminino
18	Bryce Dallas Howard	1981	EUA	feminino
22	Channing Tatum	1980	EUA	masculino
26	Christina Ricci	1980	EUA	feminino
37	Eddie Redmayne	1982	Reino Unido	masculino
40	Elijah Wood	1981	EUA	masculino
70	Jesse Eisenberg	1983	EUA	masculino
84	Kristen Bell	1980	EUA	feminino
94	Maryam Karimi	1980	Afeganistão	feminino
102	Natalie Portman	1981	Israel	feminino
124	Sam Riley	1980	Reino Unido	masculino
145	Zooey Deschanel	1980	EUA	feminino

FILTRO LIKE/NOT LIKE

LIKE e NOT LIKE

Com o operador LIKE, podemos filtrar textos que contenham determinada cadeia de caracteres.

```
1   -- WHERE LIKE: filtro de 'contém'
2 • SELECT *
3   FROM clientes
4   WHERE Email LIKE '%gmail%';
```

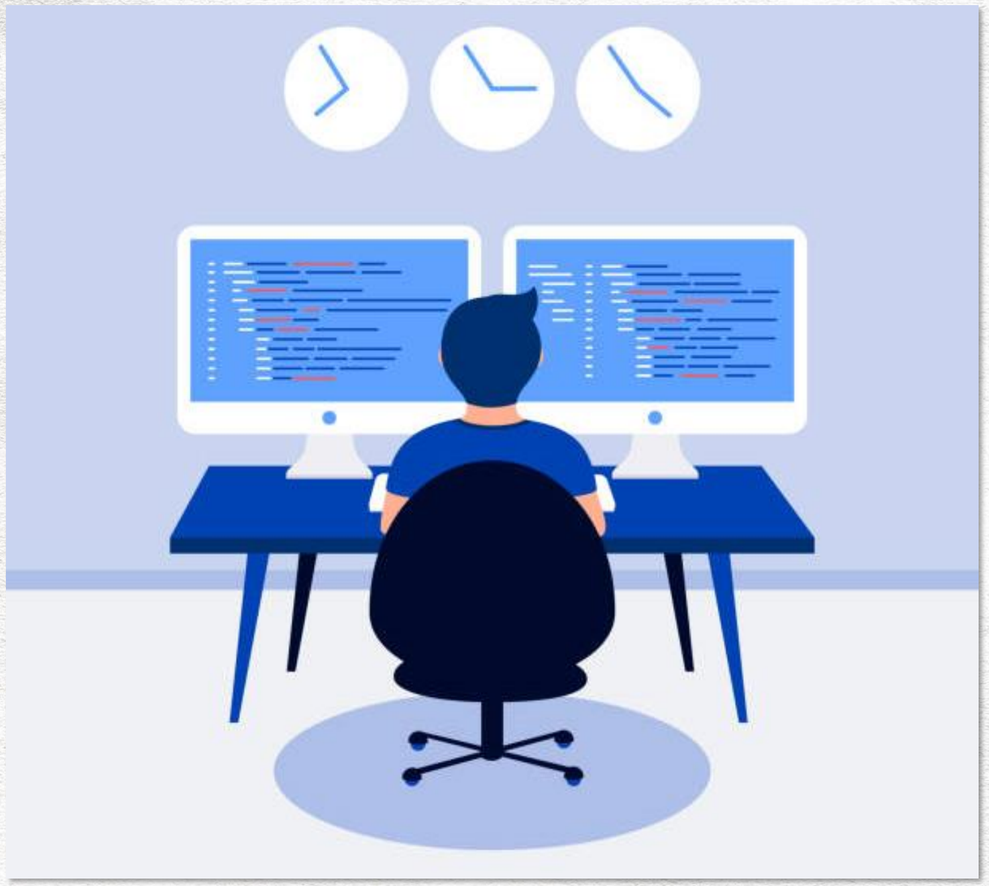
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ID_Cliente	Nome	Sobrenome	Data_Nascimento	Estado_Civil	Sexo	Email	Telefone	Renda_Anual	Qtd_Filhos	Escolaridade
3	Elizabeth	Johnson	1988-08-03	S	F	elizabeth5@gmail.com	(31) 92039-5832	80000	5	Pós-graduado
5	Janet	Alvarez	1985-12-01	S	F	janet9@gmail.com	(21) 93379-3743	70000	0	Pós-graduado
8	Shannon	Carlson	1984-03-27	S	M	shannon38@gmail.com	(85) 96795-9950	70000	0	Pós-graduado
12	Ian	Jenkins	1988-08-01	C	M	ian47@gmail.com	(21) 95385-3393	100000	2	Pós-graduado
13	Sydney	Bennett	1988-05-04	S	F	sydney23@gmail.com	(11) 92024-9186	100000	3	Pós-graduado
15	Wyatt	Hill	1999-04-23	C	M	wyatt32@gmail.com	(71) 9864		0	Parcial
18	Jordan	King	1998-09-15	S	M	jordan73@gmail.com	(31) 9959		0	Ensino médio
21	Seth	Edwards	1998-10-06	C	M	seth46@gmail.com	(41) 97012-3526	40000	0	Parcial
23	Alejandro	Beck	1965-12-18	C	M	alejandro45@gmail.com	(85) 94492-9582	10000	2	Parcial
26	Jill	Jimenez	1966-04-06	C	F	jill13@gmail.com	(41) 97120-5603	30000	2	Parcial
32	Ebony	Gonzalez	1967-06-14	C	F	ebony19@gmail.com	(71) 99664-2156	20000	4	Ensino médio
33	Wendy	Dominguez	1968-02-19	C	F	wendy12@gmail.com	(31) 97278-7135	10000	2	Parcial
35	Chloe	Garcia	1997-11-22	S	F	chloe27@gmail.com	(21) 94134-4552	40000	0	Parcial
36	Diana	Hernandez	1968-03-18	C	F	diana2@gmail.com	(85) 93094-3166	10000	2	Parcial

(71) 98644-2695

clientes 14

EXERCÍCIOS



1

Questão 1

Seu chefe precisa do nome do gerente e do telefone da loja de São Paulo.

Crie uma consulta SQL que retorne essas informações. **(Utilize a tabela lojas)**

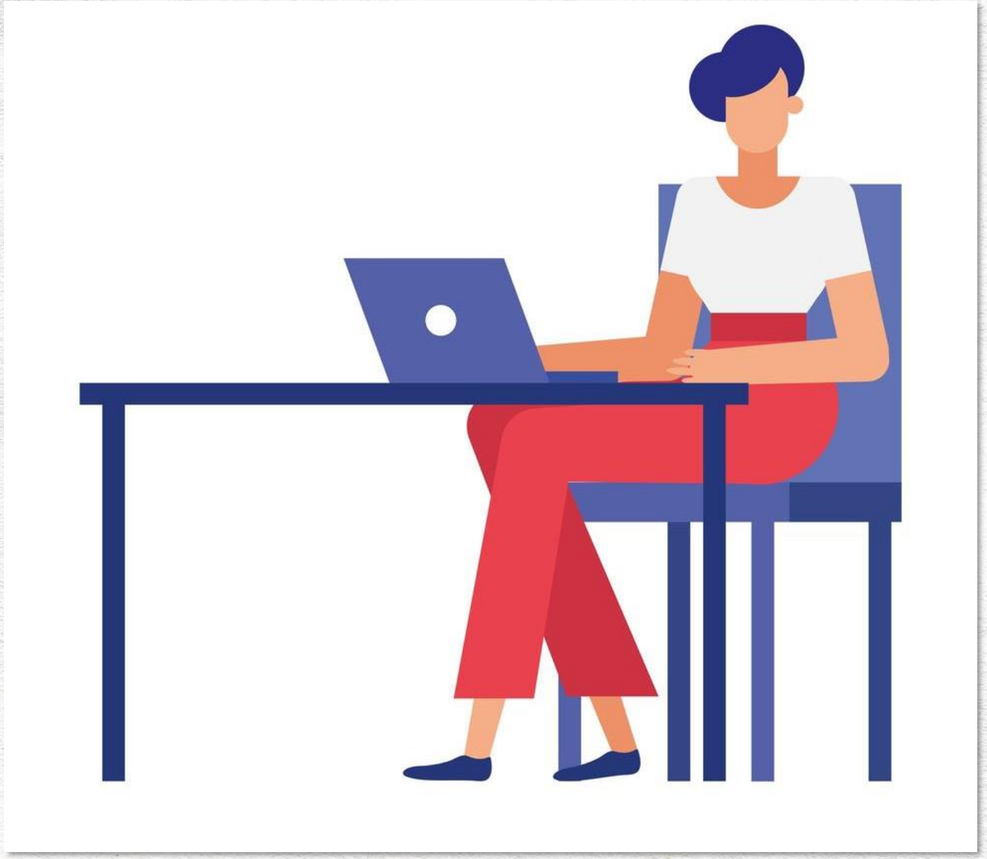
2

Questão 2

O departamento de marketing está criando uma campanha para o Dia das Crianças e deseja direcionar seus anúncios aos clientes com filhos.

Faça uma consulta que retorne as informações desses clientes, ordenando o resultado para que mostre os clientes com mais filhos no topo da lista. **(Utilize a tabela clientes)**

EXERCÍCIOS



3

Questão 3

O departamento de controle financeiro está planejando o orçamento do próximo ano e precisa da relação de produtos com alto custo em relação ao seu preço, considerando-se que tais produtos são aqueles com custo unitário acima de 1000 e preço unitário menor que 3000.

Faça uma consulta SQL que verifique quais são esses produtos. **(Utilize a tabela produtos)**

4

Questão 4

O setor de compras obteve um desconto com os fornecedores de cadeiras gamer e notebooks pelos próximos 3 meses. Sendo assim, a empresa fará uma redução temporária de 10% nos preços de todos os produtos das categorias Notebook ou Cadeira Gamer no intuito de atrair mais clientes e fechar novas vendas.

Retorne a relação desses produtos. **(Utilize as tabelas categorias e produtos)**

EXERCÍCIOS

5

Questão 5

A equipe de vendas está analisando o perfil dos clientes e deseja saber quais deles concluíram o ensino médio ou a graduação ou são pós-graduados.

Retorne as informações desses clientes, limitando o resultado a no máximo 30 clientes. **(Utilize a tabela clientes)**

6

Questão 6

O departamento de produtos quer identificar os itens cujos números de série começam com 'HDP' para planejar um aumento no estoque desses produtos.

Crie uma consulta que exiba o ID, o nome, a marca e o número de série dos produtos que atendem a esse critério. **(Utilize a tabela produtos)**

EXERCÍCIOS

7

Questão 7

O gerente geral solicitou uma lista de produtos de alto valor, especificamente aqueles com preços entre 3000 e 5000.

A lista deve estar ordenada do preço mais alto para o mais baixo. **(Utilize a tabela produtos)**

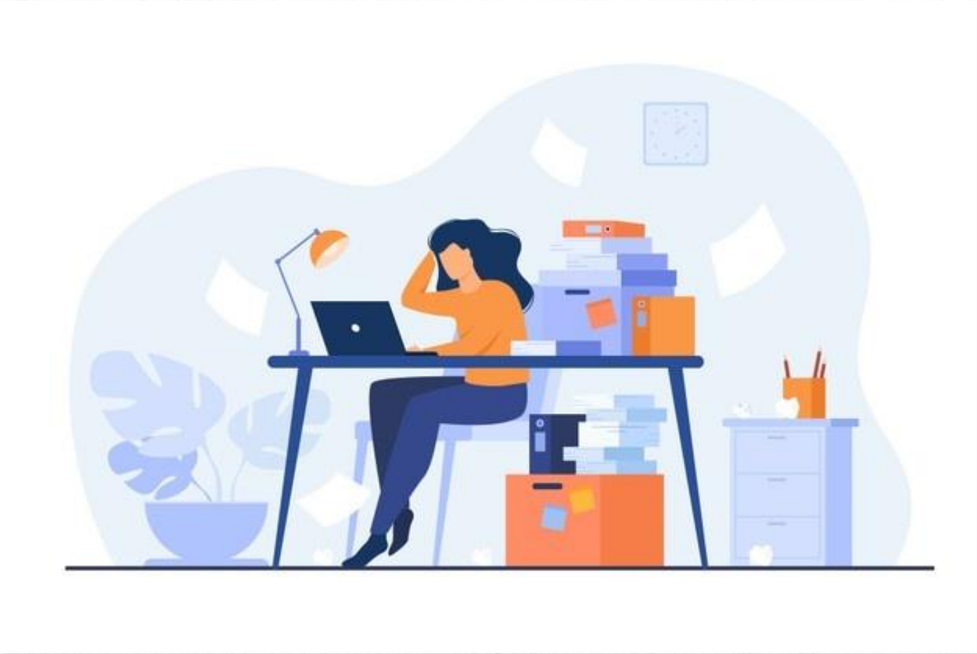
8

Questão 8

A gerência quer identificar as lojas que não estão localizadas nas regiões Sul e Sudeste para revisar suas metas de vendas.

Retorne o nome (cidade) e o endereço dessas lojas. **(Utilize as tabelas locais e lojas)**

EXERCÍCIOS



9

Questão 9

O gerente geral está solicitando as informações de todas as filiais que possuem telefone para contato.

Crie uma consulta que retorne essas lojas. **(Utilize a tabela lojas)**

10

Questão 10

O setor de marketing está planejando uma campanha especial para clientes que nasceram na década de 1980 e que são casados. Eles precisam da lista desses clientes, organizada em ordem de data de nascimento para facilitar a análise.

Crie uma consulta SQL que retorne as informações dos clientes que atendem a esses critérios. **(Utilize a tabela clientes)**

GABARITO EXPLICATIVO

1

Questão 1

Seu chefe precisa do nome do gerente e do telefone da loja de São Paulo.

Crie uma consulta SQL que retorne essas informações. **(Utilize a tabela lojas)**

Explicação e Resultado

Para retornarmos o nome do gerente e o telefone da loja de São Paulo, podemos consultar as colunas **Gerente** e **Telefone** da tabela **LOJAS**, utilizando o comando **WHERE** para filtrar somente a linha cujo campo da coluna **Loja** contém a informação **'São Paulo'**:

```
SELECT
    Gerente,
    Telefone
FROM lojas
WHERE Loja = 'São Paulo';
```

Dessa forma, retornamos o resultado abaixo:

	Gerente	Telefone
▶	Rogério Lopes	(11) 3295-3945

GABARITO EXPLICATIVO

2

Questão 2

O departamento de marketing está criando uma campanha para o Dia das Crianças e deseja direcionar seus anúncios aos clientes com filhos.

Faça uma consulta que retorne as informações desses clientes, ordenando o resultado para que mostre os clientes com mais filhos no topo da lista. **(Utilize a tabela clientes)**

Explicação e Resultado

Nesse caso, retornamos todas as colunas da tabela **CLIENTES**, filtrando todas as linhas cuja coluna **Qtd_Filhos** contenha uma valor **maior que zero**. Além disso, para retornar os clientes com mais filhos no topo da lista, ordenamos pela coluna **Qtd_Filhos** em ordem decrescente (**DESC**):

```
SELECT *
FROM clientes
WHERE Qtd_Filhos > 0
ORDER BY Qtd_Filhos DESC;
```

Dessa forma, retornamos o resultado abaixo:

ID_Cliente	Nome	Sobrenome	Data_Nascimento	Estado_Civil	Sexo	Email	Telefone	Renda_Anual	Qtd_Filhos	Escolaridade
3	Elizabeth	Johnson	1988-08-03	S	F	elizabeth5@gmail.com	(31) 92039-5832	80000	5	Pós-graduado
100	Julia	Nelson	1985-04-16	S	F	julia7@yahoo.com.br	(41) 96066-9929	80000	5	Pós-graduado
16	Shannon	Wang	1964-06-21	S	F	shannon1@hotmail.com	(71) 94496-1712	20000	4	Ensino médio
30	Denise	Stone	1967-06-06	C	F	denise10@hotmail.com	(62) 94170-5754	20000	4	Ensino médio
7	Rob	Verhoff	1984-07-02	S	F	rob4@hotmail.com	(11) 98162-4760	60000	4	Pós-graduado
10	Curtis	Lu	1983-10-30	C	M	curtis9@yahoo.com.br	(71) 94509-1440	60000	4	Pós-graduado

clientes 3

Output

Action Output

#	Time	Action	Message
1	21:55:40	SELECT * FROM clientes WHERE Qtd_Filhos > 0 ORDER BY Qtd_Filhos DESC	67 row(s) returned

GABARITO EXPLICATIVO

3

Questão 3

O departamento de controle financeiro está planejando o orçamento do próximo ano e precisa da relação de produtos com alto custo em relação ao seu preço, considerando-se que tais produtos são aqueles com custo unitário acima de 1000 e preço unitário menor que 3000.

Faça uma consulta SQL que verifique quais são esses produtos. **(Utilize a tabela produtos)**

Explicação e Resultado

Podemos retornar todas as colunas da tabela **PRODUTOS**, filtrando por todas as linhas cujo valor da coluna **Custo_Unit** seja **maior que 1000** e, ao mesmo tempo, cujo **Preco_Unit** seja **menor que 3000**. Repare que ambas as condições precisam ser verdadeiras, por isso usamos o operador **AND**:

```
SELECT *
FROM produtos
WHERE Custo_Unit > 1000 AND Preco_Unit < 3000;
```

Dessa forma, retornamos o resultado abaixo:

	ID_Produto	Nome_Produto	ID_Categoria	Marca_Produto	Num_Serie	Preco_Unit	Custo_Unit
▶	16	Notebook Motion Ultra 2	6	SAMSUNG	NOT-SS-13139U	2900	1160

GABARITO EXPLICATIVO

4

Questão 4

O setor de compras obteve um desconto com os fornecedores de cadeiras gamer e notebooks pelos próximos 3 meses. Sendo assim, a empresa fará uma redução temporária de 10% nos preços de todos os produtos das categorias Notebook ou Cadeira Gamer no intuito de atrair mais clientes e fechar novas vendas.

Retorne a relação desses produtos. **(Utilize as tabelas categorias e produtos)**

Explicação e Resultado

Primeiro, precisamos verificar os IDs das categorias **'Cadeira Gamer'** e **'Notebook'** na tabela **CATEGORIAS**. Com o SELECT abaixo, descobrimos que os **ID_Categoria** de 'Cadeira Gamer' e 'Notebook' são **3 e 6**, respectivamente:

```
SELECT * FROM categorias;
```

ID_Categoria	Categoria
1	Monitor
2	Kit Teclado e Mouse
3	Cadeira Gamer
4	Headphone
5	Microfone
6	Notebook
7	Acessórios

Feito isso, podemos usar os ID_Categoria descobertos (3 ou 6) para filtrar a tabela **PRODUTOS**. Assim, retornaremos somente os produtos que pertencem a uma dessas duas categorias. Repare que, aqui, podemos usar o operador **OR**, pois o produto deverá pertencer OU à uma categoria, OU à outra. Portanto, somente uma das duas condições filtradas precisa ser verdadeira:

```
SELECT *
FROM produtos
WHERE ID_Categoria = 3 OR ID_Categoria = 6;
```

Dessa forma, retornamos o resultado abaixo:

ID_Produto	Nome_Produto	ID_Categoria	Marca_Produto	Num_Serie	Preco_Unit	Custo_Unit
6	Cadeira Gamer reclinável A...	3	ALTURA	CGM-AL-9N914J	1800	540
7	Cadeira Gamer PC Racer V...	3	ALTURA	CGM-AL-0147FI	3100	1395
13	Notebook LC2100 Intel Cor...	6	SAMSUNG	NOT-SS-918457	3400	850
14	Notebook Inspiron 15 5000...	6	DELL	NOT-DL-000012	3100	1209
15	Notebook IdeaPad RF32000	6	DELL	NOT-DL-77164I	4200	1176
16	Notebook Motion Ultra 2	6	SAMSUNG	NOT-SS-13139U	2900	1160

GABARITO EXPLICATIVO

5

Questão 5

A equipe de vendas está analisando o perfil dos clientes e deseja saber quais deles concluíram o ensino médio ou a graduação ou são pós-graduados.

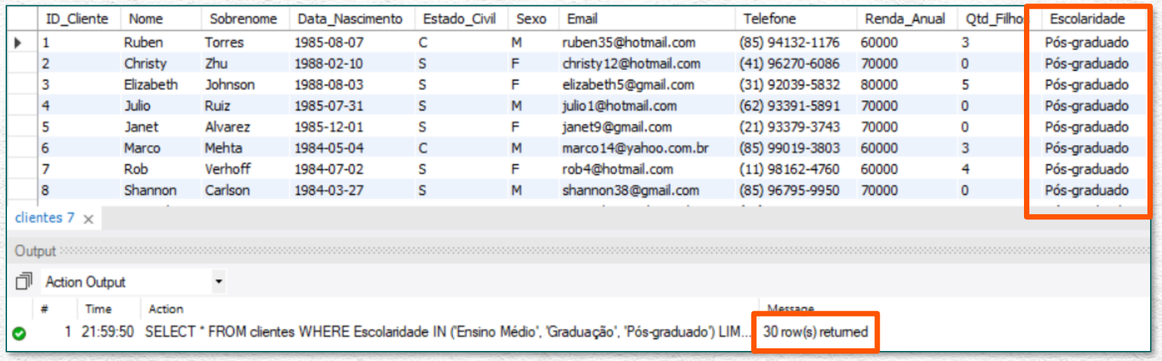
Retorne as informações desses clientes, limitando o resultado a no máximo 30 clientes. **(Utilize a tabela clientes)**

Explicação e Resultado

Precisamos filtrar a tabela **CLIENTES** para retornar somente os clientes cuja informação armazenada na coluna **Escolaridade** seja **'Ensino Médio' OU 'Graduação' OU 'Pós-Graduação'**. Podemos usar múltiplos ORs para fazer esse filtro, ou, por praticidade, o operador **IN**, passando para ele as três opções mencionadas acima. Além disso, como queremos retornar no máximo 30 clientes, usamos um **LIMIT 30** ao final do código:

```
SELECT *
FROM clientes
WHERE Escolaridade IN ('Ensino Médio', 'Graduação', 'Pós-graduado')
LIMIT 30;
```

Dessa forma, retornamos o resultado abaixo:



ID_Cliente	Nome	Sobrenome	Data_Nascimento	Estado_Civil	Sexo	Email	Telefone	Renda_Anual	Qtd_Filho	Escolaridade
1	Ruben	Torres	1985-08-07	C	M	ruben35@hotmail.com	(85) 94132-1176	60000	3	Pós-graduado
2	Christy	Zhu	1988-02-10	S	F	christy12@hotmail.com	(41) 96270-6086	70000	0	Pós-graduado
3	Elizabeth	Johnson	1988-08-03	S	F	elizabeth5@gmail.com	(31) 92039-5832	80000	5	Pós-graduado
4	Julio	Ruiz	1985-07-31	S	M	julio1@hotmail.com	(62) 93391-5891	70000	0	Pós-graduado
5	Janet	Alvarez	1985-12-01	S	F	janet9@gmail.com	(21) 93379-3743	70000	0	Pós-graduado
6	Marco	Mehta	1984-05-04	C	M	marco14@yahoo.com.br	(85) 99019-3803	60000	3	Pós-graduado
7	Rob	Verhoff	1984-07-02	S	F	rob4@hotmail.com	(11) 98162-4760	60000	4	Pós-graduado
8	Shannon	Carlson	1984-03-27	S	M	shannon38@gmail.com	(85) 96795-9950	70000	0	Pós-graduado

clientes 7 ×

Output

Action Output

#	Time	Action	Message
1	21:59:50	SELECT * FROM clientes WHERE Escolaridade IN ('Ensino Médio', 'Graduação', 'Pós-graduado') LIM...	30 row(s) returned

GABARITO EXPLICATIVO

6

Questão 6

O departamento de produtos quer identificar os itens cujos números de série começam com 'HDP' para planejar um aumento no estoque desses produtos.

Crie uma consulta que exiba o ID, o nome, a marca e o número de série dos produtos que atendem a esse critério. **(Utilize a tabela produtos)**

Explicação e Resultado

Para filtrarmos os produtos cujos números de série começam com 'HDP', podemos usar o operador **LIKE**, informando como seu parâmetro o padrão **'HDP%'**. Dessa forma, o SQL entenderá que a *string* armazenada na coluna **Num_Serie** deve iniciar com HDP e pode ter quaisquer outros caracteres após esses três primeiros. Além disso, selecionamos somente as **colunas ID_Produto, Nome_Produto, Marca_Produto e Num_Serie** da tabela **PRODUTOS**:

```
SELECT
    ID_Produto,
    Nome_Produto,
    Marca_Produto,
    Num_Serie
FROM produtos
WHERE Num_Serie LIKE 'HDP%';
```

Dessa forma, retornamos o resultado abaixo:

	ID_Produto	Nome_Produto	Marca_Produto	Num_Serie
▶	8	Headphone Bluetooth 2000	SONY	HDP-SN-194821
	9	Fone de Ouvido Tune T5000	JBL	HDP-JB-091934

GABARITO EXPLICATIVO

7

Questão 7

O gerente geral solicitou uma lista de produtos de alto valor, especificamente aqueles com preços entre 3000 e 5000.

A lista deve estar ordenada do preço mais alto para o mais baixo. **(Utilize a tabela produtos)**

Explicação e Resultado

Como queremos retornar quaisquer produtos que tenham um preço entre **3000 e 5000**, podemos filtrar a coluna **Preco_Unit** da tabela **PRODUTOS** com o operador **BETWEEN**, passando para ele os valores acima como mínimo e máximo, respectivamente. Além disso, para retornar uma lista ordenada do preço mais alto para o mais baixo, usamos o **ORDER BY** pela coluna **Preco_Unit** em ordem decrescente (**DESC**). Por opção, selecionamos somente as colunas **Nome_Produto** e **Preco_Unit**, mas se preferir, você pode retornar outras colunas também, ou todas:

```
SELECT
    Nome_Produto,
    Preco_Unit
FROM produtos
WHERE Preco_Unit BETWEEN 3000 AND 5000
ORDER BY Preco_Unit DESC;
```

Dessa forma, retornamos o resultado abaixo:

	Nome_Produto	Preco_Unit
▶	Notebook IdeaPad RF32000	4200
	Notebook LC2100 Intel Cor...	3400
	Cadeira Gamer PC Racer V...	3100
	Notebook Inspiron 15 5000...	3100

GABARITO EXPLICATIVO

8

Questão 8

A gerência quer identificar as lojas que não estão localizadas nas regiões Sul e Sudeste para revisar suas metas de vendas.

Retorne o nome (cidade) e o endereço dessas lojas. **(Utilize as tabelas locais e lojas)**

Explicação e Resultado

Primeiramente, precisamos descobrir quais são as cidades das lojas localizadas nas regiões Sul e Sudeste. Para isso, podemos consultar a tabela **LOCAIS** para que retorne somente as informações da coluna **Cidade** cujo valor da coluna **Região** seja **'Sul'** ou **'Sudeste'**:

```
SELECT Cidade
FROM locais
WHERE Região IN ('Sul', 'Sudeste');
```

Cidade
Rio de Janeiro
Belo Horizonte
Curitiba
São Paulo
Niterói

Agora, podemos usar os nomes das cidades retornadas com o SELECT acima para filtrar a coluna **Loja** da tabela **LOJAS**. O objetivo é retornar somente as linhas desta tabela que não contenham na coluna Loja os nomes dessas cidades, por isso usamos o operador **NOT IN**:

```
SELECT *
FROM lojas
WHERE Loja NOT IN ('Rio de Janeiro', 'Belo Horizonte', 'Curitiba', 'São Paulo', 'Niterói');
```

Assim, retornamos o resultado abaixo, que mostra somente as lojas das demais regiões::

ID_Loja	Loja	Gerente	Endereco	Num_Funcionarios	Telefone
3	Salvador	Marcelo Castro	Shopping Barra - Av. Centenário, 2992 - Barra, Salvador - BA, 40155-150	19	(71) 3465-8748
5	Fortaleza	Letícia Gomes	Av. Dom Luís, 500 - Aldeota, Fortaleza - CE, 60160-230	12	(85) 3294-1481
8	Goiânia	Carlos Henrique	Av. Goiás Norte, 3.592 - quadra 2.1 - St. Mal. Rondon, Goiânia - GO, 74063-010	11	NULL

GABARITO EXPLICATIVO

9

Questão 9

O gerente geral está solicitando as informações de todas as filiais que possuem telefone para contato.

Crie uma consulta que retorne essas lojas. **(Utilize a tabela lojas)**

Explicação e Resultado

Podemos retornar todas as colunas da tabela **LOJAS**, filtrando pelas linhas cujo campo da coluna **Telefone** não seja nulo (**IS NOT NULL**):

```
SELECT *
FROM lojas
WHERE Telefone IS NOT NULL;
```

Dessa forma, retornamos o resultado abaixo somente com as lojas que possuem um telefone cadastrado:

ID_Loja	Loja	Gerente	Endereco	Num_Funcionarios	Telefone
1	Rio de Janeiro	Pedro Martins	R. Visc. de Pirajá, 136 - Ipanema, Rio de Janeiro - RJ, 22410-000	23	(21) 3198-1239
2	Belo Horizonte	Julia Freitas	Av. Barão Homem de Melo, 1389 - Nova Granada, Belo Horizonte - MG, 30431-425	13	(31) 2891-2394
3	Salvador	Marcelo Castro	Shopping Barra - Av. Centenário, 2992 - Barra, Salvador - BA, 40155-150	19	(71) 3465-8748
4	Curitiba	Amanda Lima	R. Brasílio Itiberê, 3279 - Água Verde, Curitiba - PR, 80250-160	31	(41) 2923-2137
5	Fortaleza	Letícia Gomes	Av. Dom Luís, 500 - Aldeota, Fortaleza - CE, 60160-230	12	(85) 3294-1481
6	São Paulo	Rogério Lopes	R. Teodoro Sampaio, 954 - Pinheiros, São Paulo - SP, 05406-050	33	(11) 3295-3945
7	Niterói	Matheus Leal	Rua Quinze de Novembro, 8 - Loja 301a Loja 301b - Centro, Niterói - RJ, 24020-...	18	(21) 3134-7263

GABARITO EXPLICATIVO

10

Questão 10

O setor de marketing está planejando uma campanha especial para clientes que nasceram na década de 1980 e que são casados. Eles precisam da lista desses clientes, organizada em ordem de data de nascimento para facilitar a análise.

Crie uma consulta SQL que retorne as informações dos clientes que atendem a esses critérios. **(Utilize a tabela clientes)**

Explicação e Resultado

Podemos retornar todas as colunas da tabela **CLIENTES**, filtrando pelo intervalo de data que vai de **01/01/1980 a 31/12/1989** (usando o operador **BETWEEN**) e cujo valor da coluna **Estado_Civil** seja **'C'** (de 'casado'). Repare que, neste caso, ambas as condições precisam ser verdadeiras, pois queremos retornar somente os clientes que nasceram na década de 1980 **E** também sejam casados. Além disso, queremos ordenar o resultado pela data de nascimento, então informamos a coluna **Data_Nascimento** para o comando **ORDER BY**:

```
SELECT *
FROM clientes
WHERE Data_Nascimento BETWEEN '1980-01-01' AND '1989-12-31' AND Estado_Civil = 'C'
ORDER BY Data_Nascimento;
```

Dessa forma, retornamos o resultado abaixo:

ID_Cliente	Nome	Sobrenome	Data_Nascimento	Estado_Civil	Sexo	Email	Telefone	Renda_Anual	Qtd_Filhos	Escolaridade
97	Adam	Ross	1981-03-03	C	M	adam2@hotmail.com	(31) 96502-3482	60000	1	Pós-graduado
95	Edwin	Nara	1981-10-22	C	M	edwin39@yahoo.com.br	(71) 97194-6478	60000	1	Parcial
94	Andrés	Anand	1982-08-05	C	M	andrés18@gmail.com	(41) 92114-3295	60000	1	Pós-graduado
10	Curtis	Lu	1983-10-30	C	M	curtis9@yahoo.com.br	(71) 94509-1440	60000	4	Pós-graduado
6	Marco	Mehta	1984-05-04	C	M	marco14@yahoo.com.br	(85) 99019-3803	60000	3	Pós-graduado
1	Ruben	Torres	1985-08-07	C	M	ruben35@hotmail.com	(85) 94132-1176	60000	3	Pós-graduado
81	Alyssa	Cox	1986-03-10	C	F	alyssa37@gmail.com	(11) 94546-6962	130000	0	Graduação
79	Savannah	Baker	1986-07-19	C	F	savannah39@gmail.com	(11) 98574-8482	120000	2	Pós-graduado
90	Cheryl	Diaz	1987-05-01	C	F	cheryl4@gmail.com	(71) 93757-3370	90000	2	Pós-graduado
91	Aimee	He	1987-09-05	C	F	aimee13@gmail.com	(71) 94329-9879	100000	0	Graduação
11	Lauren	Walker	1988-01-13	C	F	lauren41@hotmail.com	(21) 93994-2382	100000	2	Pós-graduado
12	Ian	Jenkins	1988-08-01	C	M	ian47@gmail.com	(21) 95385-3393	100000	2	Pós-graduado

MÓDULO 5

OPERAÇÕES BÁSICAS E FUNÇÕES DE AGREGAÇÃO

OPERAÇÕES BÁSICAS E FUNÇÕES DE AGREGAÇÃO

OPERAÇÕES BÁSICAS E FUNÇÕES DE AGREGAÇÃO

O QUE VEREMOS NESTE MÓDULO

MÓDULO 5 OPERAÇÕES BÁSICAS E FUNÇÕES DE AGREGAÇÃO

01	OPERAÇÕES MATEMÁTICAS	095
02	FUNÇÕES MATEMÁTICAS	096
03	EXERCÍCIOS	105
04	GABARITO EXPLICATIVO	109

Operações matemáticas

É muito simples fazer operações matemáticas no SQL. Ao lado, temos uma sequência de exemplos que vão desde a soma até o cálculo do resto de uma divisão, usando o operador %.

```
1   -- Operações matemáticas
2 • SELECT
3       10 + 20       AS 'Soma',
4       150 - 50      AS 'Subtração',
5       300 * 3       AS 'Multiplicação',
6       450 / 5       AS 'Divisão',
7       (2 + 3) * 5   AS 'Combinação',
8       22 % 5        AS 'Resto da divisão'
```

Soma	Subtração	Multiplicação	Divisão	Combinação	Resto da divisão
30	100	900	90.0000	25	2

Funções matemáticas

As funções matemáticas são extremamente importantes para criação de **cálculos no SQL**. Elas serão a base para as análises que faremos com agrupamentos e GROUP BY!

Funções Matemáticos

- COUNT
- SUM
- MIN/MAX
- AVG

Funções matemáticas

Essas funções têm como objetivo realizar cálculos no SQL, tais como: contagem, soma, média, mínimo e máximo.

- COUNT
- COUNT(*)
- COUNT(DISTINCT)
- SUM
- AVG
- MIN/MAX

COUNT

OBJETIVO

Retorna a **quantidade total de valores** de uma coluna.

CÓDIGO

```
1    -- COUNT
2 •  SELECT
3        COUNT(Nome)
4    FROM clientes;
```

RESULTADO

	COUNT(Nome)
▶	100

COUNT

OBJETIVO

Retorna a **quantidade total de valores** de uma coluna.

CÓDIGO

```
1    -- COUNT
2 •  SELECT
3        COUNT(Telefone)
4    FROM clientes;
```

RESULTADO

	COUNT(Telefone)
▶	94

O COUNT ignora os valores nulos de uma coluna. Por isso o resultado pode mudar dependendo da coluna escolhida.

COUNT(*)

OBJETIVO

Retorna a **quantidade total de linhas** de uma tabela.

Obs: não ignora valores nulos

CÓDIGO

```
1    -- COUNT
2 •  SELECT
3        COUNT(*)
4    FROM clientes;
```

RESULTADO

	COUNT(*)
▶	100

COUNT(DISTINCT)

OBJETIVO

Retorna a **contagem distinta** de valores de uma tabela.

CÓDIGO

```
1    -- COUNT(DISTINCT)
2 •  SELECT
3        COUNT(DISTINCT Escolaridade)
4    FROM clientes;
```

RESULTADO

	COUNT(DISTINCT Escolaridade)
▶	4

SUM

OBJETIVO

Retorna a **soma total** dos valores de uma coluna.

CÓDIGO

```
1    -- SUM
2 •  SELECT
3        SUM(Receita_Venda)
4    FROM pedidos;
```

RESULTADO

	SUM(Receita_Venda)
▶	228900

AVG

OBJETIVO

Retorna a **média dos valores** de uma coluna.

CÓDIGO

```
1    -- AVG
2 •  SELECT
3        AVG(Receita_Venda)
4    FROM pedidos;
```

RESULTADO

	AVG(Receita_Venda)
▶	612.0321

MAX e MIN

OBJETIVO

Retorna o **valor máximo e mínimo** de uma coluna.

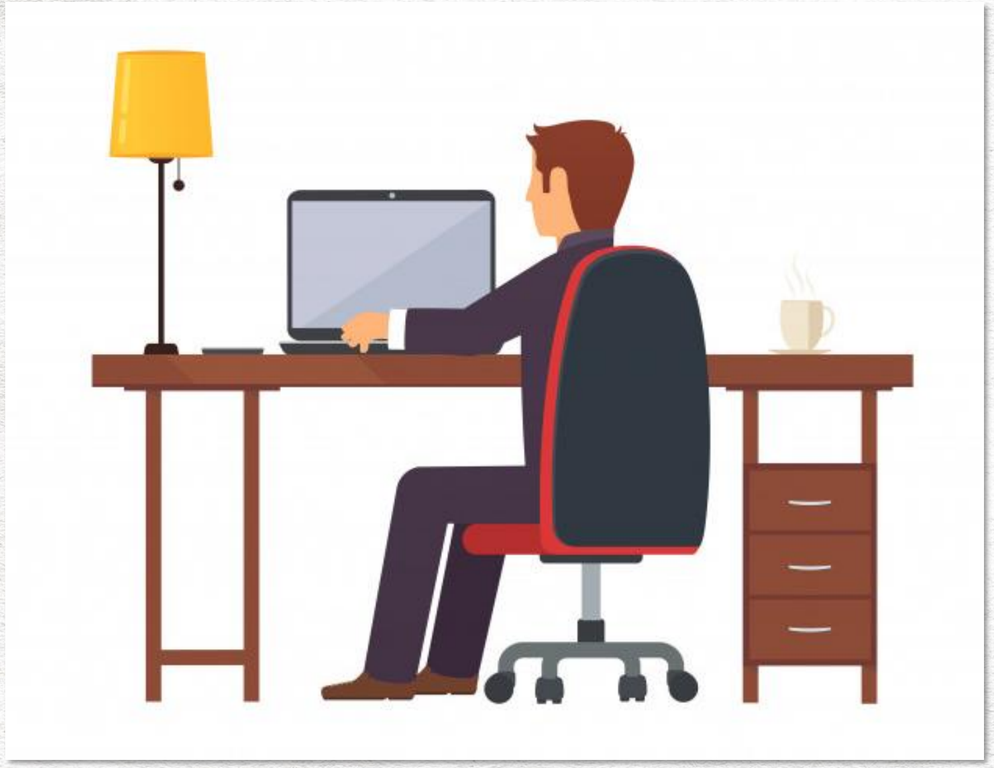
CÓDIGO

```
1    -- MAX
2 •  SELECT
3        MAX(Receita_Venda)
4    FROM pedidos;
```

RESULTADO

	MAX(Receita_Venda)
▶	1800

EXERCÍCIOS



1

Questão 1

A equipe de vendas está verificando o aumento de preços de alguns produtos.

Sabendo-se que todos os produtos da marca DELL sofrerão um reajuste de 15%, calcule o seu novo preço. **(Utilize a tabela produtos)**

2

Questão 2

O departamento financeiro precisa da relação custo-benefício dos produtos comercializados pela empresa.

Calcule a diferença entre o preço unitário e o custo unitário. Ordene o resultado do maior para o menor lucro obtido. **(Utilize a tabela produtos)**

EXERCÍCIOS

3

Questão 3

A equipe de controle de qualidade está analisando os pedidos e quer saber quantos pedidos estão cadastrados.

Crie uma consulta que conte a quantidade total de registros na tabela de pedidos. **(Utilize a tabela pedidos)**

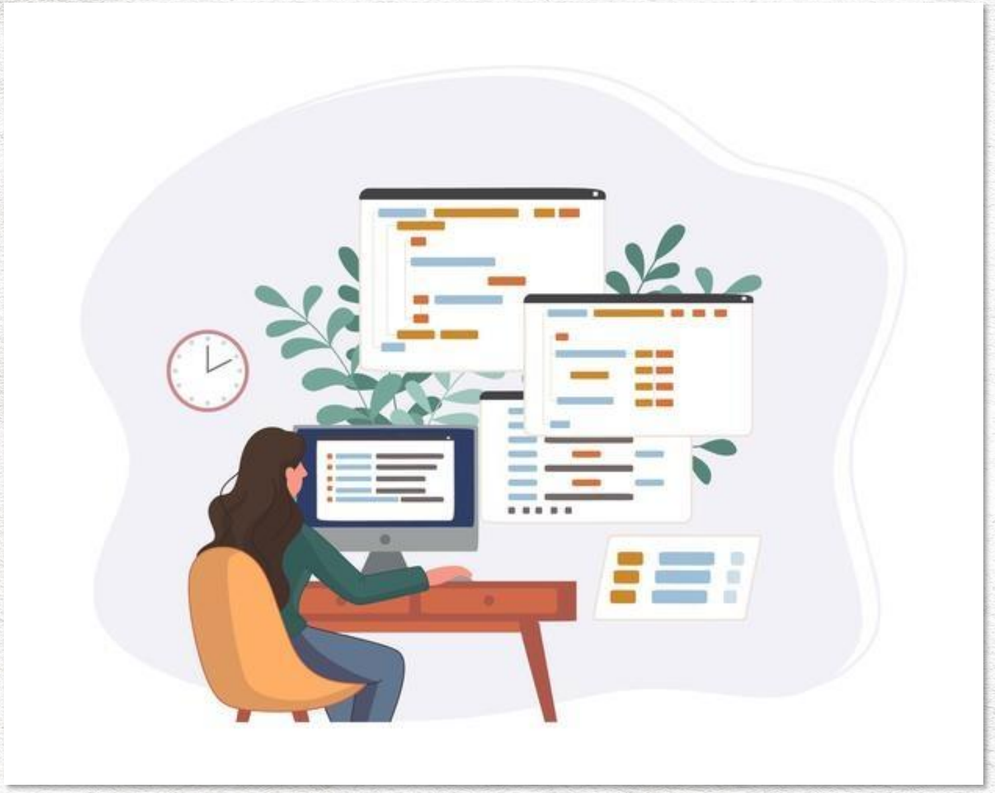
4

Questão 4

O gerente financeiro deseja saber o valor médio de receita obtido por pedido.

Faça o cálculo dessa média. **(Utilize a tabela pedidos)**

EXERCÍCIOS



5a

Questão 5-a

O setor de produtos quer saber a quantas categorias distintas os produtos cadastrados pertencem.

Crie uma consulta que faça essa contagem. **(Utilize a tabela produtos)**

5b

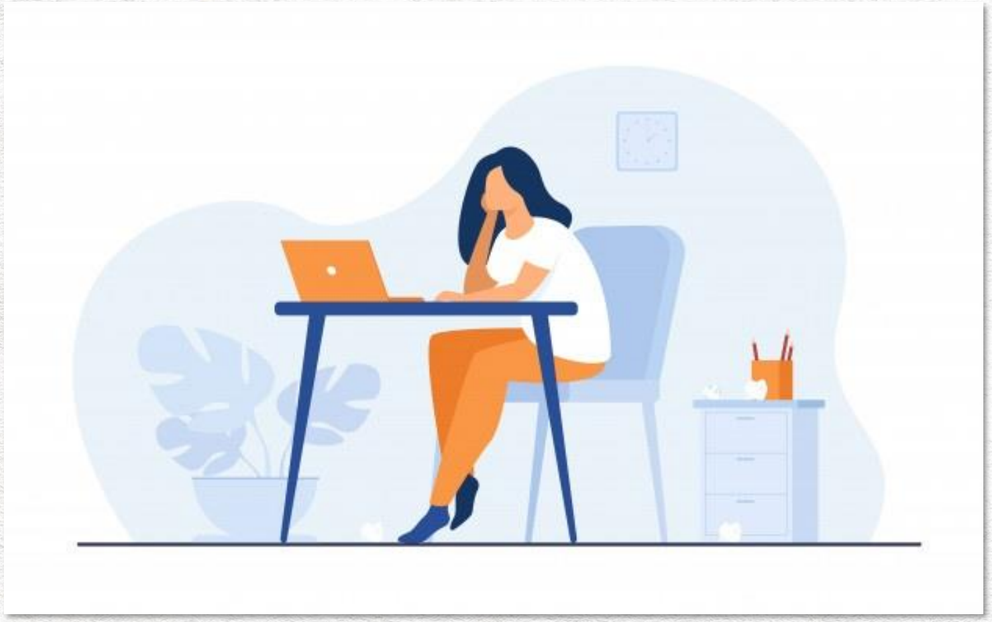
Questão 5-b

Pela análise do resultado do item A, existe alguma categoria que não possui produtos relacionados a ela? Se sim, quantas?

Obs.: Não é necessário saber qual(is) categoria(s) não possui(em) produtos cadastrados, apenas a quantidade.

Lembre-se: **utilize a tabela categorias**, sabendo-se que nesta tabela não existem categorias duplicadas.

EXERCÍCIOS



6

Questão 6

O departamento de compras está fazendo uma análise dos custos dos produtos. Ajude-os a identificar:

a) O menor custo unitário; **(Utilize a tabela produtos)**

b) O maior custo unitário. **(Utilize a tabela produtos)**

7

Questão 7

O departamento pessoal deseja saber a receita obtida por loja, pois os funcionários da loja com a maior receita receberão um bônus.

Sabendo-se que temos 8 filiais espalhadas pelo país, cujos ID_Loja vão de 1 a 8, descubra o ID da loja que mais faturou. **(Utilize a tabela pedidos)**

GABARITO EXPLICATIVO

1

Questão 1

A equipe de vendas está verificando o aumento de preços de alguns produtos.

Sabendo-se que todos os produtos da marca DELL sofrerão um reajuste de 15%, calcule o seu novo preço. **(Utilize a tabela produtos)**

Explicação e Resultado

Para calcularmos um aumento de 15% nos produtos da marca DELL, podemos **somar** a coluna **Preco_Unit** com o valor do **reajuste** (que é calculado por **Preco_Unit * 0.15**). Como somente os produtos da marca DELL sofrerão esse reajuste, filtramos a consulta para retornar apenas as linhas cuja coluna **Marca_Produto** contenha o texto '**DELL**'. Para deixar a consulta mais completa, retornamos todas as colunas da tabela **PRODUTOS**, utilizando o asterisco (*):

```
SELECT
    *,
    Preco_Unit + (Preco_Unit * 0.15) AS 'Novo_Preco_Unit'
FROM produtos
WHERE Marca_Produto = 'DELL';
```

Dessa forma, retornamos o resultado abaixo:

ID_Produto	Nome_Produto	ID_Categoria	Marca_Produto	Num_Serie	Preco_Unit	Custo_Unit	Novo_Preco_Unit
1	Monitor LED 19,5" Full HD ...	1	DELL	MNT-DL-831923	2300	966	2645.00
4	Kit Teclado + Mouse sem fi...	2	DELL	KTM-DL-041039	350	129.5	402.50
5	Kit Teclado + Mouse Slim Bl...	2	DELL	KTM-DL-111924	280	109.2	322.00
14	Notebook Inspiron 15 5000...	6	DELL	NOT-DL-000012	3100	1209	3565.00
15	Notebook IdeaPad RF32000	6	DELL	NOT-DL-77164I	4200	1176	4830.00

GABARITO EXPLICATIVO

2

Questão 2

O departamento financeiro precisa da relação custo-benefício dos produtos comercializados pela empresa.

Calcule a diferença entre o preço unitário e o custo unitário. Ordene o resultado do maior para o menor lucro obtido. **(Utilize a tabela produtos)**

Explicação e Resultado

Para calcularmos o custo-benefício (lucro) de cada produto, podemos subtrair o **Custo_Unit** do **Preco_Unit**. Para a coluna obtida com essa subtração, informamos um alias (como '**Lucro_Por_Produto**'). Para ordenar a consulta do maior para o menor lucro, passamos para o **ORDER BY** o alias da coluna de lucro por produto que criamos, ordenando de forma decrescente (**DESC**). Para deixar a consulta mais completa, retornamos todas as colunas da tabela **PRODUTOS**, utilizando o asterisco (*):

```
SELECT
    *,
    Preco_Unit - Custo_Unit AS 'Lucro_Por_Produto'
FROM produtos
ORDER BY Lucro_Por_Produto DESC;
```

Assim, retornamos o resultado abaixo:

ID_Produto	Nome_Produto	ID_Categoria	Marca_Produto	Num_Serie	Preco_Unit	Custo_Unit	Lucro_Por_Produto
15	Notebook IdeaPad RF32000	6	DELL	NOT-DL-77164I	4200	1176	3024
13	Notebook LC2100 Intel Cor…	6	SAMSUNG	NOT-SS-918457	3400	850	2550
14	Notebook Inspiron 15 5000…	6	DELL	NOT-DL-000012	3100	1209	1891
2	Monitor Curvo 24" 144Hz H…	1	SAMSUNG	MNT-SS-001939	2800	980	1820
16	Notebook Motion Ultra 2	6	SAMSUNG	NOT-SS-13139U	2900	1160	1740
7	Cadeira Gamer PC Racer V…	3	ALTURA	CGM-AL-0147FI	3100	1395	1705
1	Monitor LED 19,5" Full HD …	1	DELL	MNT-DL-831923	2300	966	1334
6	Cadeira Gamer reclinável A…	3	ALTURA	CGM-AL-9N914J	1800	540	1260
10	Microfone Condensador M…	5	AKG	MIC-AK-237591	1100	275	825
11	Microfone Condensador co…	5	BLUE	MIC-BL-819455	800	344	456
9	Fone de Ouvido Tune T5000	4	JBL	HDP-JB-091934	780	327.6	452.4
12	Microfone de mesa com fio …	5	BLUE	MIC-BL-761411	650	214.5	435.5
3	Webcam Full HD 1080p	1	LOGITECH	WBC-LT-934GG4	450	90	360
8	Headphone Bluetooth 2000	4	SONY	HDP-SN-194821	600	258	342
4	Kit Teclado + Mouse sem fi…	2	DELL	KTM-DL-041039	350	129.5	220.5
5	Kit Teclado + Mouse Slim Bl…	2	DELL	KTM-DL-111924	280	109.2	170.8

GABARITO EXPLICATIVO

3

Questão 3

A equipe de controle de qualidade está analisando os pedidos e quer saber quantos pedidos estão cadastrados.

Crie uma consulta que conte a quantidade total de registros na tabela de pedidos. **(Utilize a tabela pedidos)**

Explicação e Resultado

Sabendo-se que cada linha da tabela **PEDIDOS** contém informações sobre um pedido, podemos contar quantos pedidos estão cadastrados fazendo uma contagem das linhas dessa tabela. Para isso, usamos o **COUNT(*)**:

```
SELECT
    COUNT(*) AS 'Total_Pedidos'
FROM pedidos;
```

Dessa forma, retornamos o resultado abaixo:

Total_Pedidos
45196

GABARITO EXPLICATIVO

4

Questão 4

O gerente financeiro deseja saber o valor médio de receita obtido por pedido.

Faça o cálculo dessa média. **(Utilize a tabela pedidos)**

Explicação e Resultado

Para calcularmos o valor médio da receita, podemos aplicar a função de agregação **AVG** à coluna **Receita_Venda** da tabela **PEDIDOS**:

```
SELECT
    AVG(Receita_Venda) AS 'Media_Receita'
FROM pedidos;
```

Dessa forma, retornamos o resultado abaixo:

	Media_Receita
▶	1790.8319

GABARITO EXPLICATIVO

5a

Questão 5-a

O setor de produtos quer saber a quantas categorias distintas os produtos cadastrados pertencem.

Crie uma consulta que faça essa contagem. **(Utilize a tabela produtos)**

Explicação e Resultado

Na tabela **PRODUTOS** temos vários produtos cadastrados. Cada um deles pertence a uma categoria, sendo que uma única categoria pode ter vários produtos que pertencem à ela. Portanto, o ID de uma mesma categoria pode aparecer diversas vezes na tabela produtos.

Sendo assim, se fizermos uma contagem dos **ID_Categoria distintos** existentes na tabela **PRODUTOS**, saberemos a quantas categorias distintas os produtos cadastrados pertencem. Para isso, usamos um **COUNT(DISTINCT ID_Categoria)**:

```
SELECT
    COUNT(DISTINCT ID_Categoria) AS 'Categorias_dos_Produtos'
FROM produtos;
```

E assim, retornamos o resultado abaixo:

	Categorias_dos_Produtos
▶	6

GABARITO EXPLICATIVO

5b

Questão 5-b

Pela análise do resultado do item A, existe alguma categoria que não possui produtos relacionados a ela? Se sim, quantas?

Obs.: Não é necessário saber qual(is) categoria(s) não possui(em) produtos cadastrados, apenas a quantidade.

Lembre-se: **utilize a tabela categorias**, sabendo-se que nesta tabela não existem categorias duplicadas.

Explicação e Resultado

No resultado do item A deste exercício, descobrimos que os produtos cadastrados na tabela **PRODUTOS** pertencem a **6 categorias distintas**.

Sabemos que as categorias cadastradas na tabela **CATEGORIAS** não se repetem. Portanto, podemos fazer um **COUNT(ID_Categoria)** na tabela **CATEGORIAS**...

```
SELECT
    COUNT(ID_Categoria) AS 'Categorias_Distintas'
FROM categorias;
```

... e assim veremos que, no total, existem **7 categorias**:

	Categorias_Distintas
▶	7

Dessa forma, descobrimos que existe 1 categoria que não possui produtos relacionados a ela.

GABARITO EXPLICATIVO

6a

Questão 6-a

O departamento de compras está fazendo uma análise dos custos dos produtos. Ajude-os a identificar:

a) O menor custo unitário; **(Utilize a tabela produtos)**

b) O maior custo unitário. **(Utilize a tabela produtos)**

Explicação e Resultado

Para verificarmos o menor custo unitário cadastrado na tabela **PRODUTOS**, podemos aplicar a função de agregação **MIN** à coluna **Custo_Unit**:

```
SELECT
    MIN(Custo_Unit) AS 'Menor_Custo'
FROM produtos;
```

Dessa forma, retornamos o resultado abaixo:

	Menor_Custo
▶	90

GABARITO EXPLICATIVO

6b

Questão 6-b

O departamento de compras está fazendo uma análise dos custos dos produtos. Ajude-os a identificar:

a) O menor custo unitário; **(Utilize a tabela produtos)**

b) O maior custo unitário. **(Utilize a tabela produtos)**

Explicação e Resultado

Para verificarmos o maior custo unitário cadastrado na tabela **PRODUTOS**, podemos aplicar a função de agregação **MAX** à coluna **Custo_Unit**:

```
SELECT
    MAX(Custo_Unit) AS 'Maior_Custo'
FROM produtos;
```

Dessa forma, retornamos o resultado abaixo:

	Maior_Custo
▶	1395

GABARITO EXPLICATIVO

7

Questão 7

O departamento pessoal deseja saber a receita obtida por loja, pois os funcionários da loja com a maior receita receberão um bônus.

Sabendo-se que temos 8 filiais espalhadas pelo país, cujos ID_Loja vão de 1 a 8, descubra o ID da loja que mais faturou. **(Utilize a tabela pedidos)**

Explicação e Resultado

Para verificarmos qual loja obteve a maior receita, podemos aplicar a função de agregação **SUM** à coluna **Receita_Venda** da tabela **PEDIDOS**, filtrando pelo **ID_Loja** de cada loja.

Portanto, para descobrir a receita obtida pelo loja de **ID_Loja = 1**, executamos o código abaixo...

```
SELECT
    SUM(Receita_Venda)
FROM Pedidos
WHERE ID_Loja = 1;
```

... que retornará o seguinte resultado:

	SUM(Receita_Venda)
▶	10037160

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

7

Questão 7

O departamento pessoal deseja saber a receita obtida por loja, pois os funcionários da loja com a maior receita receberão um bônus.

Sabendo-se que temos 8 filiais espalhadas pelo país, cujos ID_Loja vão de 1 a 8, descubra o ID da loja que mais faturou. **(Utilize a tabela pedidos)**

Explicação e Resultado

Em seguida, vamos alterando no código o valor da coluna **ID_Loja** para descobrir a receita das **demais lojas**:

```
SELECT
    SUM(Receita_Venda)
FROM Pedidos
WHERE ID_Loja = 2;
```

SUM(Receita_Venda)
10061430

```
SELECT
    SUM(Receita_Venda)
FROM Pedidos
WHERE ID_Loja = 3;
```

SUM(Receita_Venda)
10107860

```
SELECT
    SUM(Receita_Venda)
FROM Pedidos
WHERE ID_Loja = 4;
```

SUM(Receita_Venda)
10139870

```
SELECT
    SUM(Receita_Venda)
FROM Pedidos
WHERE ID_Loja = 5;
```

SUM(Receita_Venda)
10217870

```
SELECT
    SUM(Receita_Venda)
FROM Pedidos
WHERE ID_Loja = 6;
```

SUM(Receita_Venda)
10443680

```
SELECT
    SUM(Receita_Venda)
FROM Pedidos
WHERE ID_Loja = 7;
```

SUM(Receita_Venda)
9986950

```
SELECT
    SUM(Receita_Venda)
FROM Pedidos
WHERE ID_Loja = 8;
```

SUM(Receita_Venda)
9943620

Assim, descobrimos que a loja que mais faturou foi a de ID_Loja = 6, com 10443680.

MÓDULO 6

AGRUPAMENTOS

AGRUPAMENTOS

AGRUPAMENTOS

Índice Geral

Índice deste Módulo

O QUE VEREMOS NESTE MÓDULO

MÓDULO 6 AGRUPAMENTOS NO MYSQL

01	GROUP BY	121
02	CRIANDO AGRUPAMENTOS	125
03	FILTRANDO AGRUPAMENTOS (WHERE)	126
04	FILTRANDO AGRUPAMENTOS (HAVING)	127
05	WHERE vs HAVING	128
06	EXERCÍCIOS	130
07	GABARITO EXPLICATIVO	135

GROUP BY

O comando GROUP BY permite agrupar os dados de uma tabela, criando uma espécie de resumo dos dados.

Data Venda	Produto	Quantidade
2019-01-01	PRODUTO A	10
2019-01-01	PRODUTO B	20
2019-01-01	PRODUTO A	5
2019-01-01	PRODUTO A	15
2019-01-01	PRODUTO B	10
2019-01-01	PRODUTO A	30

GROUP BY

Produto	Quantidade Total
PRODUTO A	60
PRODUTO B	30

GROUP BY

O comando GROUP BY permite agrupar os dados de uma tabela, criando uma espécie de resumo dos dados.

Data Venda	Produto	Quantidade
2019-01-01	PRODUTO A	10
2019-01-01	PRODUTO B	20
2019-01-01	PRODUTO A	5
2019-01-01	PRODUTO A	15
2019-01-01	PRODUTO B	10
2019-01-01	PRODUTO A	30

GROUP BY

Produto	Quantidade Total
PRODUTO A	60
PRODUTO B	30

GROUP BY

O comando GROUP BY permite agrupar os dados de uma tabela, criando uma espécie de resumo dos dados.

Data Venda	Produto	Quantidade
2019-01-01	PRODUTO A	10
2019-01-01	PRODUTO B	20
2019-01-01	PRODUTO A	5
2019-01-01	PRODUTO A	15
2019-01-01	PRODUTO B	10
2019-01-01	PRODUTO A	30

GROUP BY

Produto	Quantidade Total
PRODUTO A	60
PRODUTO B	30

GROUP BY

O comando GROUP BY permite agrupar os dados de uma tabela, criando uma espécie de resumo dos dados.

Data Venda	Produto	Quantidade
2019-01-01	PRODUTO A	10
2019-01-01	PRODUTO B	20
2019-01-01	PRODUTO A	5
2019-01-01	PRODUTO A	15
2019-01-01	PRODUTO B	10
2019-01-01	PRODUTO A	30

GROUP BY

Produto	Quantidade Total
PRODUTO A	60
PRODUTO B	30

```
SELECT
        Produto,
        SUM(Quantidade) AS 'Quantidade Total'
FROM Tabela
GROUP BY Produto;
```

CRIANDO AGRUPAMENTOS

GROUP BY

O **Group By** é o comando do SQL que vai nos permitir **criar agrupamentos**, ou seja, tabelas resumos das nossas tabelas principais.

```sql
# GROUP BY

-- Exemplo: Utilize o GROUP BY para criar uma
-- consulta e descobrir o total de filmes por gênero.
SELECT
    genero,
    COUNT(*)
FROM filmes
GROUP BY genero;
```

genero	COUNT(*)
Comédia	10
Drama	36
Ficção e Fantasia	11
Mistério e Suspense	6
Arte	1
Animação	2
Ação e Aventura	5

FILTRANDO AGRUPAMENTOS (WHERE)

GROUP BY + WHERE

Sempre que quisermos realizar um filtro **ANTES** de criar o agrupamento, usamos o WHERE.

```sql
# GROUP BY + WHERE

-- Exemplo: Utilize o GROUP BY + WHERE para criar uma
-- consulta e descobrir o total de filmes por gênero, mas
-- considerando apenas os filmes lançados em 2003.
SELECT
    genero,
    COUNT(*)
FROM filmes
WHERE ano_lancamento = 2003
GROUP BY genero;
```

Export: Wrap Cell Content:

genero	COUNT(*)
Mistério e Suspense	2
Comédia	2
Drama	3
Ficção e Fantasia	1

FILTRANDO AGRUPAMENTOS (HAVING)

GROUP BY + HAVING

Sempre que quisermos realizar um filtro **DEPOIS** que o agrupamento foi criado, usamos o HAVING.

```
86   # GROUP BY + HAVING
87
88   -- Exemplo: Utilize o GROUP BY + HAVING para filtrar a tabela resultante e mostrar
89   -- apenas os filmes dos gêneros com quantidade MAIOR OU IGUAL A 10 filmes.
90
91 • SELECT
92       genero,
93       COUNT(*) AS 'qtd_filmes'
94   FROM filmes
95   GROUP BY genero
96   HAVING COUNT(*) >= 10;
97
98
```

genero	qtd_filmes
Comédia	10
Drama	36
Ficção e Fantasia	11

WHERE vs HAVING

Como o **WHERE** funciona?

1

Partimos da tabela **original**

id_filme	titulo	genero	duracao	ano_lancamento	preco_aluguel
1	Que mulher é essa?	Comédia	93	2001	2.09
2	A Senha	Drama	99	2001	2.19
3	Do que as mulheres gostam	Comédia	127	2001	2.59
4	Dia de Treinamento	Drama	122	2001	1.79
5	O Senhor dos Anéis: A sociedade do anel	Ficção e Fantasia	178	2001	2.59
6	Harry Potter e a Pedra Filosofal	Ficção e Fantasia	152	2001	2.69
7	Os Excêntricos Tenenbaums	Comédia	110	2002	1.89
8	Seu marido e minha mulher	Comédia	91	2002	2.59
9	11 de setembro	Drama	134	2002	2.99
10	Simone	Drama	117	2002	2.69
11	É hora do show	Comédia	95	2002	1.79
12	O Senhor dos Anéis: As duas torres	Ficção e Fantasia	179	2002	2.39
13	Harry Potter e a Câmara Secreta	Ficção e Fantasia	161	2002	1.79
14	O Novato	Mistério e Suspe...	115	2003	1.69
15	Alguém tem que ceder	Comédia	128	2003	1.69
16	A última noite	Drama	135	2003	1.59
17	Revelações	Mistério e Suspe...	106	2003	1.99
18	Lições Para Toda a Vida	Drama	111	2003	1.69
19	21 gramas	Drama	124	2003	2.09
20	Simplesmente amor	Comédia	135	2003	2.29
21	O Senhor dos Anéis: O retorno do rei	Ficção e Fantasia	200	2003	1.99
22	Visões	Arte	107	2004	2.59
23	Dança comigo	Drama	106	2004	1.69
24	Uma Eleição Muito Atrapalhada	Comédia	110	2004	2.89
25	Bridget Jones: No Limite da Razão	Drama	108	2004	2.89
26	Ray	Drama	152	2004	2.59
27	Monster - Desejo Assassino	Drama	109	2004	2.09
28	Harry Potter e o Prisioneiro de Azkabam	Ficção e Fantasia	142	2004	1.69
29	Tudo por Dinheiro	Drama	122	2005	2.79
30	Capote	Drama	114	2005	2.39
31	Harry Potter e o Cálice de Fogo	Ficção e Fantasia	157	2005	2.69
32	Falsária	Drama	93	2006	2.89
33	V de Vingança	Drama	132	2006	1.59
34	Armações do Amor	Drama	97	2006	1.99
35	Happy Feet	Animação	108	2006	1.79
36	As Torres Gêmeas	Drama	129	2006	1.59

2

Usamos o GROUP BY + WHERE

```sql
SELECT
    genero,
    COUNT(*)
FROM filmes
WHERE ano_lancamento = 2003
GROUP BY genero;
```

3

Apenas os filmes do ano de 2003 são considerados

id_filme	titulo	genero	duracao	ano_lancamento	preco_aluguel
14	O Novato	Mistério e Suspe...	115	2003	1.69
15	Alguém tem que ceder	Comédia	128	2003	1.69
16	A última noite	Drama	135	2003	1.59
17	Revelações	Mistério e Suspe...	106	2003	1.99
18	Lições Para Toda a Vida	Drama	111	2003	1.69
19	21 gramas	Drama	124	2003	2.09
20	Simplesmente amor	Comédia	135	2003	2.29
21	O Senhor dos Anéis: O retorno do rei	Ficção e Fantasia	200	2003	1.99

4

E o agrupamento final de gêneros considera apenas os filmes de 2003

genero	COUNT(*)
Mistério e Suspense	2
Comédia	2
Drama	3
Ficção e Fantasia	1

Índice Geral

Índice deste Módulo

WHERE vs HAVING

Como o **HAVING** funciona?

1

Partimos da tabela **agrupada**

genero	COUNT(*)
Comédia	10
Drama	36
Ficção e Fantasia	11
Mistério e Suspense	6
Arte	1
Animação	2
Ação e Aventura	5

2

Usamos o GROUP BY + HAVING

```
SELECT
    genero,
    COUNT(*) AS 'qtd_filmes'
FROM filmes
GROUP BY genero
HAVING COUNT(*) >= 10;
```

3

Apenas os gêneros com quantidade MAIOR OU IGUAL são considerados

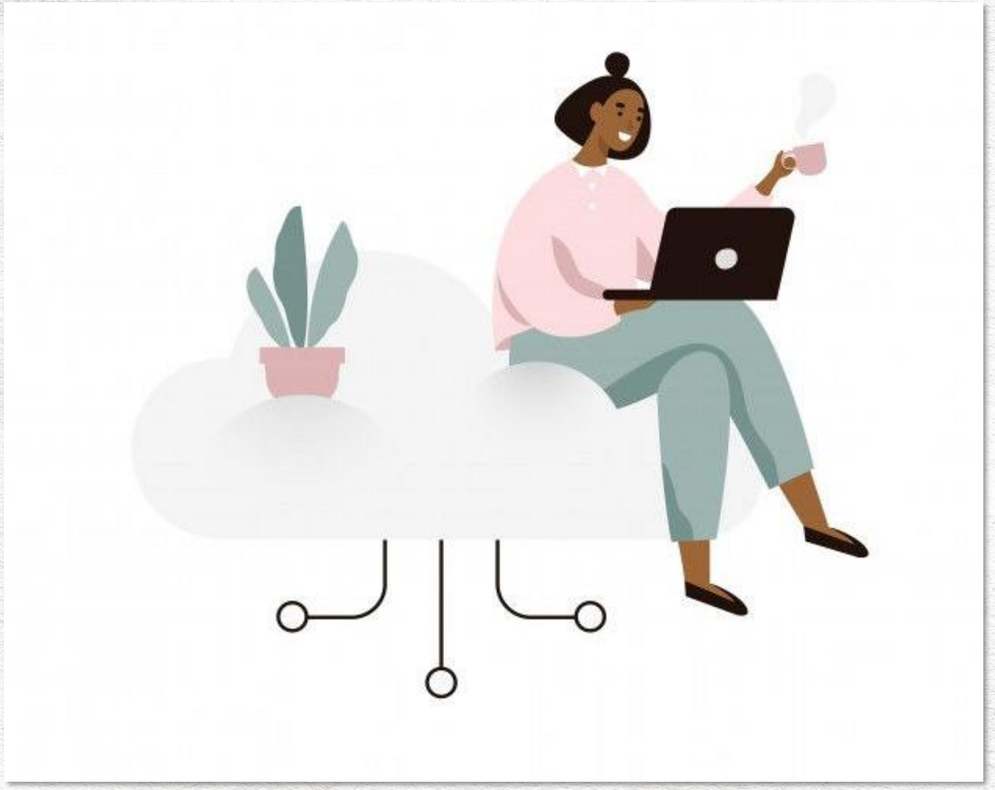
genero	COUNT(*)
Comédia	10
Drama	36
Ficção e Fantasia	11
[illegible]	[illegible]
[illegible]	[illegible]
[illegible]	[illegible]
[illegible]	[illegible]

4

E o agrupamento final de gêneros considera apenas os gêneros com mais de 10 filmes

genero	qtd_filmes
Comédia	10
Drama	36
Ficção e Fantasia	11

EXERCÍCIOS



1

Questão 1

O gerente deseja avaliar o desempenho de cada categoria em termos de produtos cadastrados, para identificar áreas onde o portfólio pode ser expandido.

Crie uma consulta que retorne o ID da categoria e a quantidade de produtos em cada uma delas. **(Utilize a tabela produtos)**

2

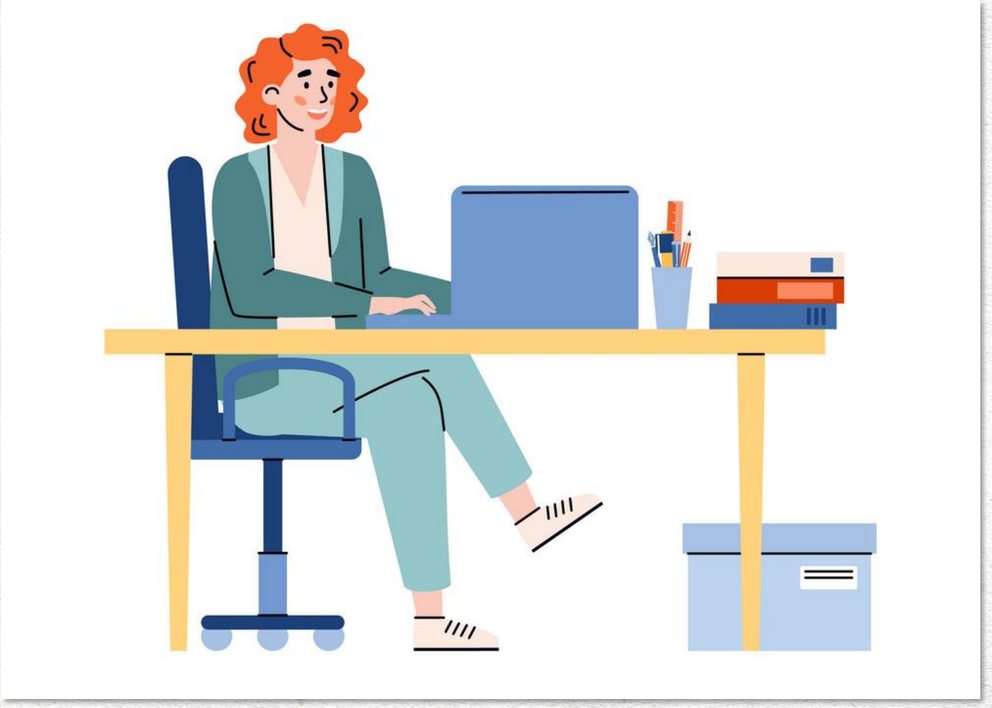
Questão 2

O departamento de marketing está desenvolvendo campanhas personalizadas para diferentes perfis de clientes com base no estado civil e no sexo.

Para planejar essas ações de forma mais eficiente, eles precisam saber a quantidade de clientes em cada combinação de estado civil e sexo.

Crie uma consulta que mostre essas informações agrupadas. **(Utilize a tabela clientes)**

EXERCÍCIOS



3

Questão 3

O setor de compras quer entender a média dos custos unitários por categoria para negociar melhores preços com os fornecedores.

Crie uma consulta que retorne o ID da categoria e a média do custo unitário dos produtos. **(Utilize a tabela produtos)**

4

Questão 4

O time financeiro está criando categorias para o programa de fidelidade baseado na renda anual dos clientes. Para isso, eles precisam compreender melhor o perfil econômico da base de clientes.

Crie uma consulta que mostre a renda anual e a quantidade de clientes em cada faixa de valor registrada no sistema. Ordene essa consulta da renda anual com maior quantidade de clientes para a menor. **(Utilize a tabela clientes)**

EXERCÍCIOS

5

Questão 5

A equipe de planejamento financeiro está revisando o desempenho das lojas e precisa identificar quais delas geraram uma receita total superior a R$ 10.000.000. Esses dados serão usados para alocar mais recursos no próximo trimestre.

Crie uma consulta para atender a essa solicitação, ordenando pela receita total. **(Utilize a tabela pedidos)**

6

Questão 6

O time de análise de vendas está avaliando os hábitos de compra dos clientes para identificar os que mais geram receita. Essa informação será utilizada para criar ações de fidelização personalizadas para esse grupo.

Crie uma consulta SQL que retorne o ID do cliente e sua média de receita ordenada de forma decrescente, exibindo apenas os 10 primeiros resultados. **(Utilize a tabela pedidos)**

EXERCÍCIOS

7

Questão 7

A equipe de gestão financeira está analisando o desempenho dos clientes em termos de lucratividade. Eles precisam identificar quais clientes geraram, em média, um lucro acima de R$ 1200. Essa análise ajudará na priorização de estratégias de retenção e no direcionamento de campanhas para esse público.

Crie uma consulta SQL que retorne o ID do cliente e a média de lucro, exibindo apenas aqueles que atendem ao critério estabelecido. Lembre-se de considerar a diferença entre receita e custo como base para o cálculo do lucro. Ordene o resultado pelo lucro médio de forma crescente. **(Utilize a tabela pedidos)**

8

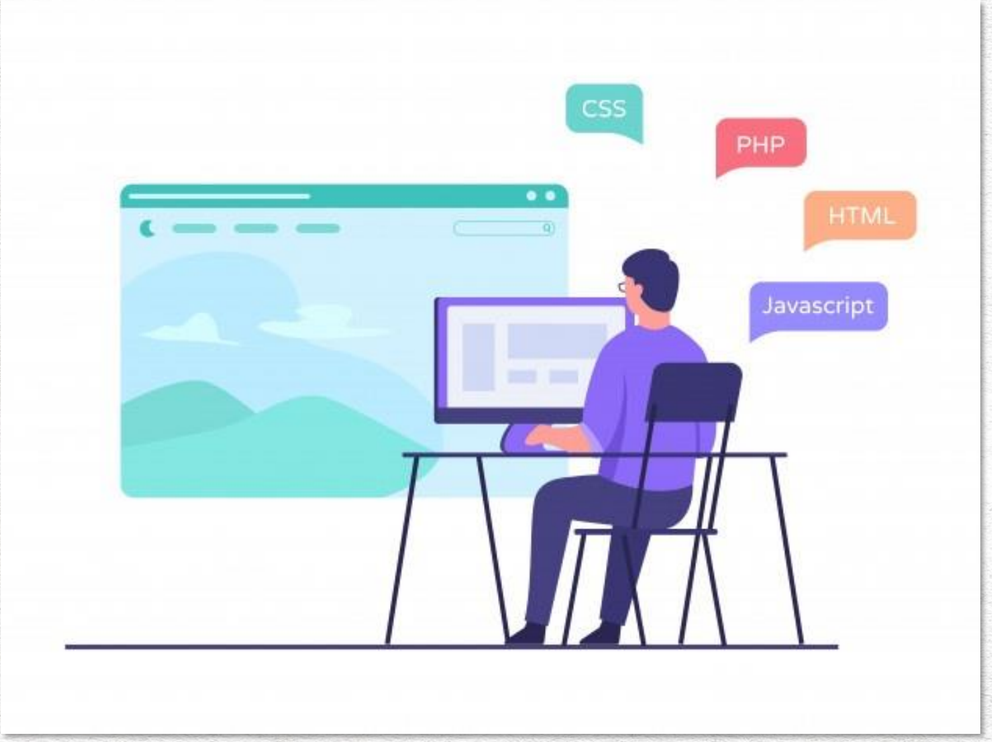
Questão 8

A diretoria quer analisar o desempenho das lojas com maior capacidade operacional (mais de 20 funcionários).

Primeiramente, eles precisam identificar quais são essas lojas para, em seguida, calcularem a receita total gerada por apenas essas lojas.

Crie as consultas SQL necessárias para atender a esses requisitos. **(Utilize as tabelas lojas e pedidos)**

EXERCÍCIOS



9

Questão 9

O time de estratégia está avaliando a lucratividade das marcas de produtos comercializados pela empresa. Para essa análise, eles levantaram a hipótese de que, caso fosse vendida uma unidade de cada produto, seria possível identificar qual marca geraria o maior lucro total. Essa informação será utilizada para decidir quais marcas merecem mais destaque em futuras campanhas de marketing.

Crie uma consulta SQL que calcule o lucro total por marca, considerando a diferença entre preço e custo unitário, e exiba o resultado agrupado por marca, ordenando da marca mais lucrativa para a menos lucrativa. **(Utilize a tabela produtos)**

10

Questão 10

O departamento de marketing está segmentando clientes para uma campanha de fidelização voltada para pessoas nascidas nos anos 1970. Eles querem entender melhor o perfil dessa faixa etária, considerando as variáveis de escolaridade, sexo e estado civil. O objetivo é direcionar a campanha de forma personalizada, identificando quais são os grupos com maior quantidade de clientes.

Crie uma consulta SQL que retorne a quantidade de clientes agrupados por escolaridade, sexo e estado civil, para clientes nascidos nos anos 1970. Ordene o resultado do grupo com a maior quantidade de clientes para a menor. **(Utilize a tabela clientes)**

GABARITO EXPLICATIVO

1

Questão 1

O gerente deseja avaliar o desempenho de cada categoria em termos de produtos cadastrados, para identificar áreas onde o portfólio pode ser expandido.

Crie uma consulta que retorne o ID da categoria e a quantidade de produtos em cada uma delas. **(Utilize a tabela produtos)**

Explicação e Resultado

Para calcularmos a quantidade de produtos pertencentes à cada categoria, podemos contar a quantidade de linhas existentes na tabela **PRODUTOS** (**COUNT(*)**), agrupando pela coluna **ID_Categoria**:

```
SELECT
    ID_Categoria,
    COUNT(*) AS Qtd_Produtos
FROM produtos
GROUP BY ID_Categoria;
```

Dessa forma, retornamos o resultado abaixo:

	ID_Categoria	Qtd_Produtos
▶	1	3
	2	2
	3	2
	4	2
	5	3
	6	4

GABARITO EXPLICATIVO

2

Questão 2

O departamento de marketing está desenvolvendo campanhas personalizadas para diferentes perfis de clientes com base no estado civil e no sexo.

Para planejar essas ações de forma mais eficiente, eles precisam saber a quantidade de clientes em cada combinação de estado civil e sexo.

Crie uma consulta que mostre essas informações agrupadas. **(Utilize a tabela clientes)**

Explicação e Resultado

Para calcularmos a quantidade de clientes cadastrados de acordo com seus estados civis e sexos, podemos contar a quantidade de linhas existentes na tabela **CLIENTES** (**COUNT(*)**), agrupando pelas colunas **Estado_Civil** e **Sexo**:

```
SELECT
    Estado_Civil,
    Sexo,
    COUNT(*) AS Qtd_Clientes
FROM clientes
GROUP BY Estado_Civil, Sexo;
```

Dessa forma, retornamos o resultado abaixo:

	Estado_Civil	Sexo	Qtd_Clientes
▶	C	M	33
	S	F	29
	S	M	15
	C	F	23

GABARITO EXPLICATIVO

3

Questão 3

O setor de compras quer entender a média dos custos unitários por categoria para negociar melhores preços com os fornecedores.

Crie uma consulta que retorne o ID da categoria e a média do custo unitário dos produtos. **(Utilize a tabela produtos)**

Explicação e Resultado

Para calcularmos a média do custo unitário de cada categoria, podemos aplicar a função de agregação **AVG** à coluna **Custo_Unit** da tabela **PRODUTOS**, agrupando pela coluna **ID_Categoria**:

```
SELECT
    ID_Categoria,
    AVG(Custo_Unit) AS Media_Custo
FROM produtos
GROUP BY ID_Categoria;
```

Dessa forma, retornamos o resultado abaixo:

ID_Categoria	Media_Custo
1	678.6666666666666
2	119.35
3	967.5
4	292.8
5	277.8333333333333
6	1098.75

GABARITO EXPLICATIVO

4

Questão 4

O time financeiro está criando categorias para o programa de fidelidade baseado na renda anual dos clientes. Para isso, eles precisam compreender melhor o perfil econômico da base de clientes.

Crie uma consulta que mostre a renda anual e a quantidade de clientes em cada faixa de valor registrada no sistema. Ordene essa consulta da renda anual com maior quantidade de clientes para a menor. **(Utilize a tabela clientes)**

Explicação e Resultado

Para calcularmos a quantidade de clientes por cada faixa de valor de renda anual registrada no sistema, podemos contar a quantidade de linhas existentes na tabela **CLIENTES** (**COUNT(*)**), agrupando pela coluna **Renda_Anual**. Para ordenar a consulta da renda anual com a maior quantidade de clientes para a menor, podemos aplicar ao código um **ORDER BY** pela coluna **Qtd_Clientes** (formada pelo COUNT(*)) de forma decrescente (**DESC**):

```
SELECT
    Renda_Anual,
    COUNT(*) AS Qtd_Clientes
FROM clientes
GROUP BY Renda_Anual
ORDER BY Qtd_Clientes DESC;
```

Dessa forma, retornamos o resultado ao lado:

	Renda_Anual	Qtd_Clientes
▶	60000	17
	80000	17
	70000	15
	30000	15
	40000	14
	20000	6
	100000	4
	10000	4
	90000	3
	130000	2
	160000	1
	170000	1
	120000	1

GABARITO EXPLICATIVO

5

Questão 5

A equipe de planejamento financeiro está revisando o desempenho das lojas e precisa identificar quais delas geraram uma receita total superior a R$ 10.000.000. Esses dados serão usados para alocar mais recursos no próximo trimestre.

Crie uma consulta para atender a essa solicitação, ordenando pela receita total. **(Utilize a tabela pedidos)**

Explicação e Resultado

Para calcularmos a receita total de cada loja, podemos aplicar a função de agregação **SUM** à coluna **Receita_Venda** da tabela **PEDIDOS**, agrupando pela coluna **ID_Loja**. Dessa forma, **obteremos um agrupamento** com a receita total de todas as lojas.

Para filtrar esse agrupamento e retornar no resultado somente as lojas que obtiveram uma receita total superior a R$ 10.000.000, utilizamos o comando **HAVING** que selecionará somente as linhas do agrupamento cuja **Receita_Total > 10000000**.

Por fim, ordenamos o resultado pela coluna **Receita_Total** (formada pela SUM(Receita_Venda)):

```sql
SELECT
    ID_Loja,
    SUM(Receita_Venda) AS Receita_Total
FROM pedidos
GROUP BY ID_Loja
HAVING Receita_Total > 10000000
ORDER BY Receita_Total;
```

Dessa forma, retornamos o resultado ao lado:

ID_Loja	Receita_Total
1	10037160
2	10061430
3	10107860
4	10139870
5	10217870
6	10443680

GABARITO EXPLICATIVO

6

Questão 6

O time de análise de vendas está avaliando os hábitos de compra dos clientes para identificar os que mais geram receita. Essa informação será utilizada para criar ações de fidelização personalizadas para esse grupo.

Crie uma consulta SQL que retorne o ID do cliente e sua média de receita ordenada de forma decrescente, exibindo apenas os 10 primeiros resultados. **(Utilize a tabela pedidos)**

Explicação e Resultado

Para calcularmos a média de receita gerada por cada cliente, podemos aplicar a função de agregação **AVG** à coluna **Receita_Venda** da tabela **PEDIDOS**, agrupando pela coluna **ID_Cliente**.

Por fim, ordenamos o resultado pela coluna **Media_Receita** (formada pela AVG(Receita_Venda)) em ordem decrescente (**DESC**), limitando a quantidade de linhas a serem retornadas para somente 10 aplicando um **LIMIT 10** ao final do código:

```sql
SELECT
    ID_Cliente,
    AVG(Receita_Venda) AS Media_Receita
FROM pedidos
GROUP BY ID_Cliente
ORDER BY Media_Receita DESC
LIMIT 10;
```

Dessa forma, retornamos o resultado ao lado:

ID_Cliente	Media_Receita
58	1929.8214
81	1902.2687
57	1894.0652
54	1885.5895
72	1874.3943
33	1871.3391
70	1865.3642
87	1864.4335
18	1863.4123
44	1858.7919

GABARITO EXPLICATIVO

7

Questão 7

A equipe de gestão financeira está analisando o desempenho dos clientes em termos de lucratividade. Eles precisam identificar quais clientes geraram, em média, um lucro acima de R$ 1200. Essa análise ajudará na priorização de estratégias de retenção e no direcionamento de campanhas para esse público.

Crie uma consulta SQL que retorne o ID do cliente e a média de lucro, exibindo apenas aqueles que atendem ao critério estabelecido. Lembre-se de considerar a diferença entre receita e custo como base para o cálculo do lucro. Ordene o resultado pelo lucro médio de forma crescente. **(Utilize a tabela pedidos)**

Explicação e Resultado

Para calcularmos a média de lucro gerado por cada cliente, podemos aplicar a função de agregação **AVG** ao **resultado da subtração entre as colunas Receita_Venda e Custo_Venda** da tabela **PEDIDOS**, agrupando pela coluna **ID_Cliente**. Dessa forma, **obteremos um agrupamento** com o lucro médio gerado por cada cliente.

Para filtrar esse agrupamento e retornar no resultado somente os clientes cujo lucro médio gerado esteja acima de R$ 1200, utilizamos o comando **HAVING** que selecionará somente as linhas do agrupamento cuja **Media_Lucro > 1200**.

Por fim, ordenamos o resultado pela coluna **Media_Lucro** (formada pela AVG(Receita_Venda – Custo_Venda)):

ID_Cliente	Media_Lucro
6	1200.0398249452958
91	1200.9884782608694
44	1206.7492170022374
78	1207.8058823529414
76	1213.4638392857141
18	1214.0715639810426
57	1217.7536956521744
72	1223.420533880904
70	1224.1401766004417
54	1226.3021834061142
87	1228.2933497536942
33	1231.0058315334777
81	1233.473788546256
58	1255.1566964285716

```sql
SELECT
    ID_Cliente,
    AVG(Receita_Venda - Custo_Venda) AS Media_Lucro
FROM pedidos
GROUP BY ID_Cliente
HAVING Media_Lucro > 1200
ORDER BY Media_Lucro;
```

Dessa forma, retornamos o resultado ao lado:

GABARITO EXPLICATIVO

8

Questão 8

A diretoria quer analisar o desempenho das lojas com maior capacidade operacional (mais de 20 funcionários).

Primeiramente, eles precisam identificar quais são essas lojas para, em seguida, calcularem a receita total gerada por apenas essas lojas.

Crie as consultas SQL necessárias para atender a esses requisitos. **(Utilize as tabelas lojas e pedidos)**

Explicação e Resultado

Para identificarmos quais são as lojas com mais de 20 funcionários, podemos retornar a coluna **ID_Loja** da tabela **LOJAS**, filtrando pela coluna **Num_Funcionarios > 20**:

```
SELECT
    ID_Loja
FROM lojas
WHERE Num_Funcionarios > 20;
```

Assim, descobrimos que os IDs das lojas com mais de 20 funcionários são **1, 4 e 6**:

	ID_Loja
▶	1
	4
	6

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

8

Questão 8

A diretoria quer analisar o desempenho das lojas com maior capacidade operacional (mais de 20 funcionários).

Primeiramente, eles precisam identificar quais são essas lojas para, em seguida, calcularem a receita total gerada por apenas essas lojas.

Crie as consultas SQL necessárias para atender a esses requisitos. **(Utilize as tabelas lojas e pedidos)**

Explicação e Resultado

Agora, sabendo os IDs das lojas com mais de 20 funcionários (**1, 4 e 6**), para calcularmos a receita total dessas lojas, podemos aplicar a função de agregação **SUM** à coluna **Receita_Venda** da tabela **PEDIDOS**, agrupando pela coluna **ID_Loja** e filtrando somente as linhas da tabela pedidos cujo ID_Loja seja ou 1, ou 4, ou 6 (**WHERE ID_Loja IN (1, 4, 6)**):

```sql
SELECT
    ID_Loja,
    SUM(Receita_Venda) AS Receita_Total
FROM pedidos
WHERE ID_Loja IN (1, 4, 6)
GROUP BY ID_Loja;
```

Dessa forma, retornamos o resultado abaixo:

	ID_Loja	Receita_Total
▶	6	10443680
	1	10037160
	4	10139870

GABARITO EXPLICATIVO

9

Questão 9

O time de estratégia está avaliando a lucratividade das marcas de produtos comercializados pela empresa.

Para essa análise, eles levantaram a hipótese de que, caso fosse vendida uma unidade de cada produto, seria possível identificar qual marca geraria o maior lucro total.

Essa informação será utilizada para decidir quais marcas merecem mais destaque em futuras campanhas de marketing.

Crie uma consulta SQL que calcule o lucro total por marca, considerando a diferença entre preço e custo unitário, e exiba o resultado agrupado por marca, ordenando da marca mais lucrativa para a menos lucrativa. **(Utilize a tabela produtos)**

Explicação e Resultado

Para calcularmos o lucro total por marca nos termos hipotéticos do enunciado, podemos aplicar a função de agregação **SUM** ao **resultado da subtração entre as colunas Preco_Unit e Custo_Unit** da tabela **PRODUTOS**, agrupando pela coluna **Marca_Produto**.

Por fim, para retornar o resultado da marca mais lucrativa para a menos lucrativa, ordenamos o resultado pela coluna **Lucro_Por_Marca** (formada pela SUM(Preco_Unit – Custo_Unit)) em ordem decrescente (**DESC**):

```sql
SELECT
   Marca_Produto,
   SUM(Preco_Unit - Custo_Unit) AS Lucro_Por_Marca
FROM produtos
GROUP BY Marca_Produto
ORDER BY Lucro_Por_Marca DESC;
```

Marca_Produto	Lucro_Por_Marca
DELL	6640.3
SAMSUNG	6110
ALTURA	2965
BLUE	891.5
AKG	825
JBL	452.4
LOGITECH	360
SONY	342

Dessa forma, retornamos o resultado ao lado:

GABARITO EXPLICATIVO

10

Questão 10

O departamento de marketing está segmentando clientes para uma campanha de fidelização voltada para pessoas nascidas nos anos 1970. Eles querem entender melhor o perfil dessa faixa etária, considerando as variáveis de escolaridade, sexo e estado civil.

O objetivo é direcionar a campanha de forma personalizada, identificando quais são os grupos com maior quantidade de clientes.

Crie uma consulta SQL que retorne a quantidade de clientes agrupados por escolaridade, sexo e estado civil, para clientes nascidos nos anos 1970. Ordene o resultado do grupo com a maior quantidade de clientes para a menor. **(Utilize a tabela clientes)**

Explicação e Resultado

Para calcularmos a quantidade de clientes nascidos nos anos 1970 por escolaridade, sexo e estado civil, podemos fazer uma contagem das linhas da tabela **CLIENTES** (**COUNT(*)**), agrupando pelas colunas **Escolaridade, Sexo e Estado_Civil**, filtrando somente as linhas da tabela clientes cuja **Data_Nascimento** esteja entre **01/01/1970 e 31/12/1979**.

Por fim, para retornarmos do grupo com a maior quantidade de clientes para a menor, ordenamos o resultado pela coluna **Qtd_Clientes** (formada pelo COUNT(*)) em ordem decrescente (**DESC**):

```sql
SELECT
    Escolaridade,
    Sexo,
    Estado_Civil,
    COUNT(*) AS 'Qtd_Clientes'
FROM clientes
WHERE Data_Nascimento BETWEEN '1970-01-01' AND '1979-12-31'
GROUP BY  Escolaridade, Sexo, Estado_Civil
ORDER BY Qtd_Clientes DESC;
```

Dessa forma, retornamos o resultado ao lado:

Escolaridade	Sexo	Estado_Civil	Qtd_Clientes
Ensino médio	F	S	8
Parcial	M	C	6
Ensino médio	M	S	4
Ensino médio	M	C	3
Parcial	F	C	3
Ensino médio	F	C	2
Parcial	F	S	2
Graduação	M	C	2
Parcial	M	S	2
Graduação	M	S	1
Pós-graduado	M	C	1
Pós-graduado	F	S	1

MÓDULO 7

VARIÁVEIS

VARIÁVEIS
VARIÁVEIS

Índice Geral
Índice deste Módulo

O QUE VEREMOS NESTE MÓDULO

MÓDULO 7 VARIÁVEIS NO MYSQL

01	O QUE É UMA VARIÁVEL	148
02	ESCOPO DAS VARIÁVEIS	150
03	TIPOS DE DADOS NO MYSQL	151
04	USER-DEFINED VARIABLES NA PRÁTICA (EXEMPLO 1)	152
05	USER-DEFINED VARIABLES NA PRÁTICA (EXEMPLO 2)	153
06	FUNÇÃO CAST	154
07	EXERCÍCIOS	156
08	GABARITO EXPLICATIVO	161

O que é uma variável

Uma variável é um local onde armazenamos um determinado valor, que pode ser usado ao longo do nosso código.

Por exemplo, observe a query abaixo:

```
SELECT *
FROM produtos
WHERE Marca_Produto = 'SAMSUNG';
```

O que é uma variável

Uma variável é um local onde armazenamos um determinado valor, que pode ser usado ao longo do nosso código.

Poderíamos utilizar variáveis para facilitar na organização e compreensão do código.

```
SET @varMarca = 'SAMSUNG';

SELECT *
FROM produtos
WHERE Marca_Produto = @varMarca;
```

Escopo das Variáveis

O escopo de uma variável se refere ao local onde essa variável existe, ou seja, o local onde ela pode ser acessada.

Temos os seguintes níveis de escopo:

1. USER-DEFINED VARIABLES
2. LOCAL VARIABLES

Tipos de Dados no MySQL

Na tabela abaixo listamos os principais tipos de dados usados para variáveis.

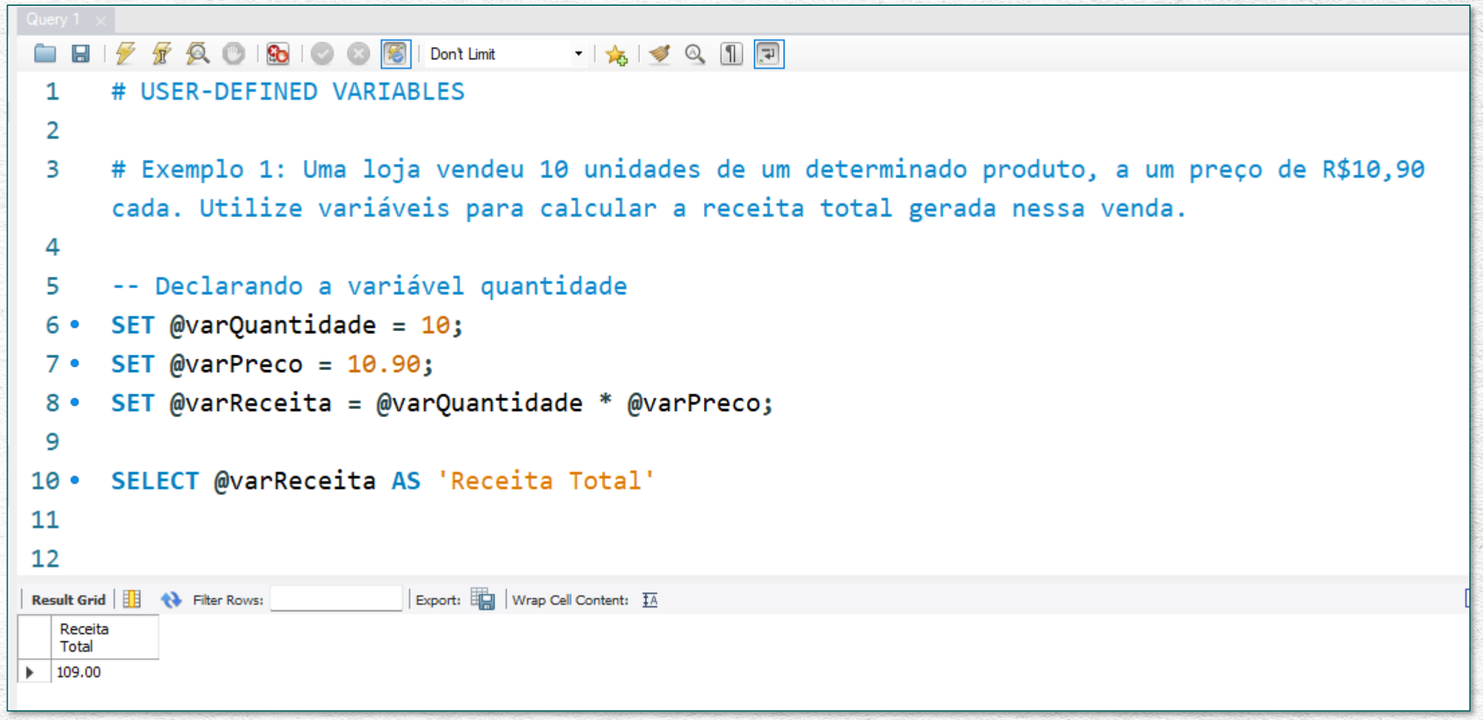
TIPO	DESCRIÇÃO
INT	Inteiros entre -2.147.483.648 e 2.147.483.647
DECIMAL(M, D)	Número decimal com M dígitos no total e D casas decimais. O padrão é (10, 2). Exemplo: o valor 100,34 poderia ser declarado como DECIMAL(5, 2).
FLOAT(M, D)	Número decimal com M dígitos no total e D casas decimais. O padrão é (10, 2). Exemplo: o valor 12345,67 poderia ser declarado como FLOAT(7, 2).
VARCHAR(N)	String de tamanho variável, até 65535 caracteres.
DATE	Uma data de 01/01/1000 a 31/12/9999, no formato YYYY-MM-DD
DATETIME	Uma combinação de data e hora de 01/01/1000 00:00:00 a 31/12/9999 23:59:59, no formato YYYY-MM-DD HH:MM:SS

User-Defined Variables na Prática (Exemplo 1)

Vamos agora ver na prática como vão funcionar as variáveis.

Ao lado, temos a seguinte situação: cálculo da receita total de uma venda, a partir da quantidade vendida e do preço do produto.

A solução final pode ser observada na imagem ao lado.



User-Defined Variables na Prática (Exemplo 2)

Podemos aplicar variáveis para facilitar consultas ao banco de dados.

Exemplo: imagine que você deseja filtrar a tabela de produtos de acordo com a Marca, mas usando uma variável para isso.

Ao lado, temos a solução para esta situação.

```
13  # Exemplo 2: Faça uma consulta na tabela de produtos de forma que você visualize apenas os
    produtos da marca DELL. Utilize variáveis para que a marca possa ser facilmente alterada
    dentro da variável.
14
15 • SET @varMarca = 'DELL';
16
17 • SELECT *
18  FROM produtos
19  WHERE Marca_Produto = @varMarca;
20
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

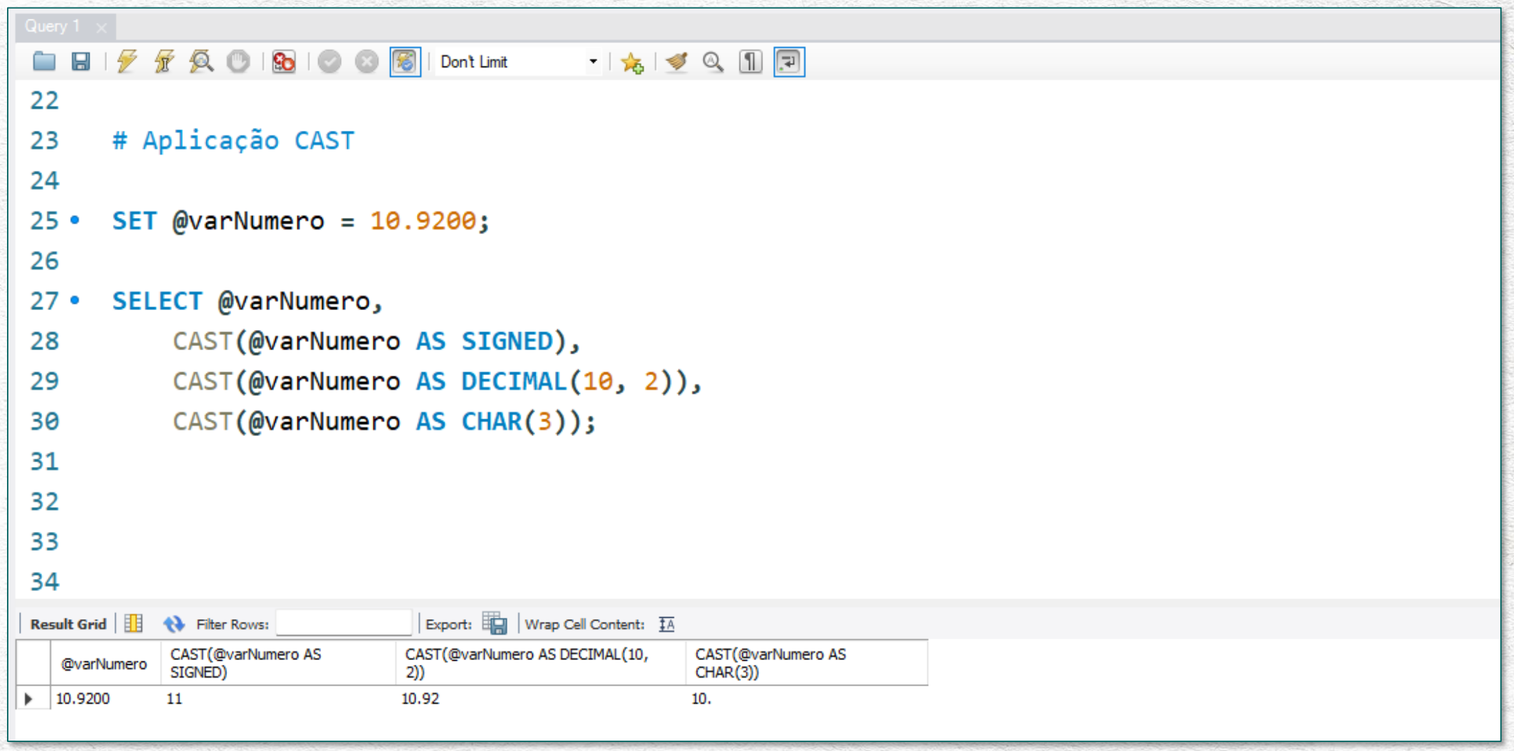
ID_Produto	Nome_Produto	ID_Categoria	Marca_Produto	Num_Serie	Preco_Unit	Custo_Unit
1	Monitor LED 19,5" Full HD HDMI	1	DELL	MNT-DL-831923	2300	966
4	Kit Teclado + Mouse sem fio Wireless	2	DELL	KTM-DL-041039	350	129.5
5	Kit Teclado + Mouse Slim Bluetooth	2	DELL	KTM-DL-111924	280	109.2
14	Notebook Inspiron 15 5000 4GB	6	DELL	NOT-DL-000012	3100	1209
15	Notebook IdeaPad RF32000	6	DELL	NOT-DL-77164I	4200	1176

Função CAST

A função CAST converte um valor de qualquer tipo em um outro tipo de dados especificado.

No CAST, especificamos o INT como SIGNED (valores positivos e negativos) ou UNISIGNED (somente valores positivos) e o VARCHAR como CHAR.

Ao lado, temos um exemplo de aplicação da função CAST sobre um número armazenado na variável @varNumero.

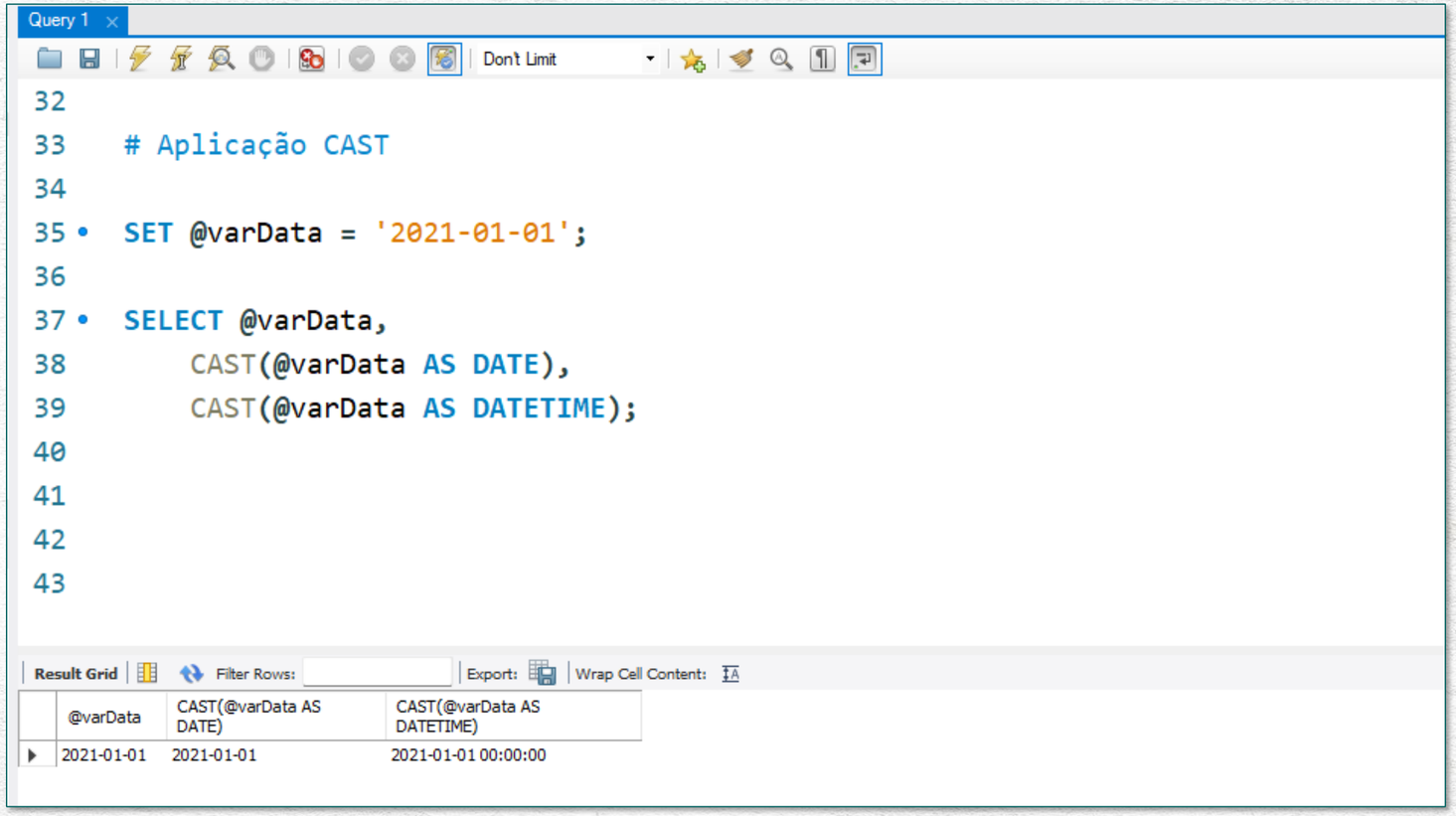


Função CAST

A função CAST converte um valor de qualquer tipo em um outro tipo de dados especificado.

No CAST, especificamos o INT como SIGNED (valores positivos e negativos) ou UNISIGNED (somente valores positivos) e o VARCHAR como CHAR.

Ao lado, temos um exemplo de aplicação da função CAST sobre um número armazenado na variável @varNumero.



EXERCÍCIOS

1

Questão 1

Uma loja vendeu 25 unidades de um produto com custo unitário de R$ 18,50. A equipe de contabilidade precisa calcular o custo total dessas vendas para registrar no balanço financeiro da loja.

Para isso, calcule o valor total do custo das unidades vendidas utilizando variáveis. **(Não é necessário utilizar uma tabela do banco de dados base)**

2

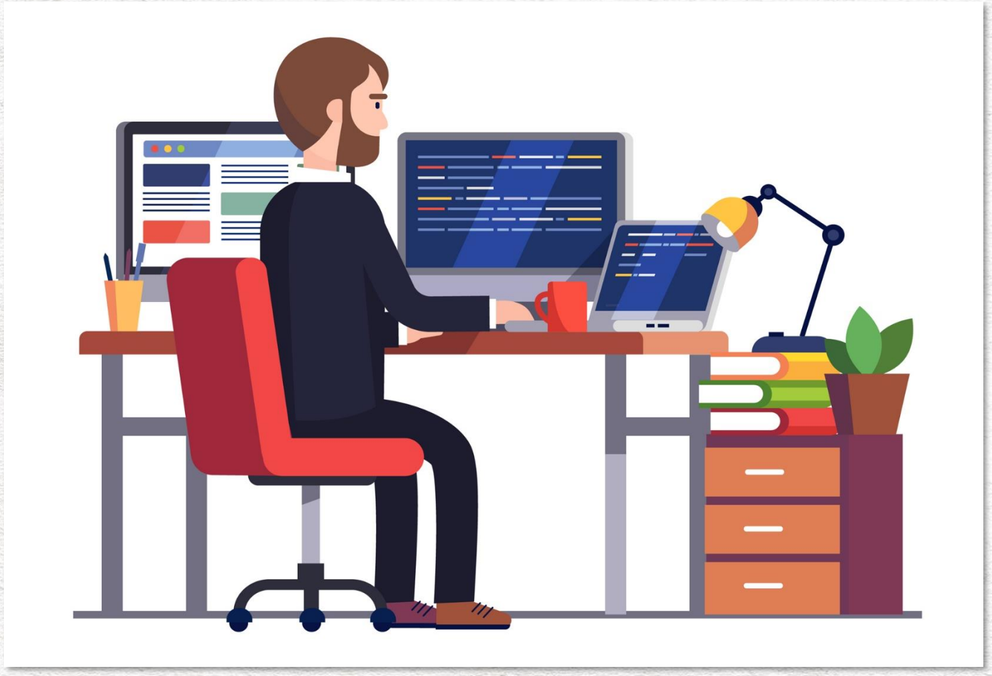
Questão 2

A equipe de marketing está criando uma promoção para os produtos de uma determinada loja. Eles decidiram que todos os produtos com preço inferior a R$ 1.500,00 serão elegíveis para a promoção.

Para isso, você precisa criar uma consulta que ajude a identificar todos os produtos com preço inferior a esse valor.

Para tanto, crie uma variável que armazene o preço unitário e utilize-a em sua *query*. **(Utilize a tabela produtos)**

EXERCÍCIOS



3

Questão 3

A equipe de marketing deseja realizar uma análise sobre a faixa etária dos clientes. Eles gostariam de identificar o cliente mais jovem para uma campanha focada em jovens consumidores.

Para isso, você deve criar uma consulta que encontre o cliente mais jovem, com base na data de nascimento, utilizando uma variável que armazene a data de nascimento mais recente. **(Utilize a tabela clientes)**

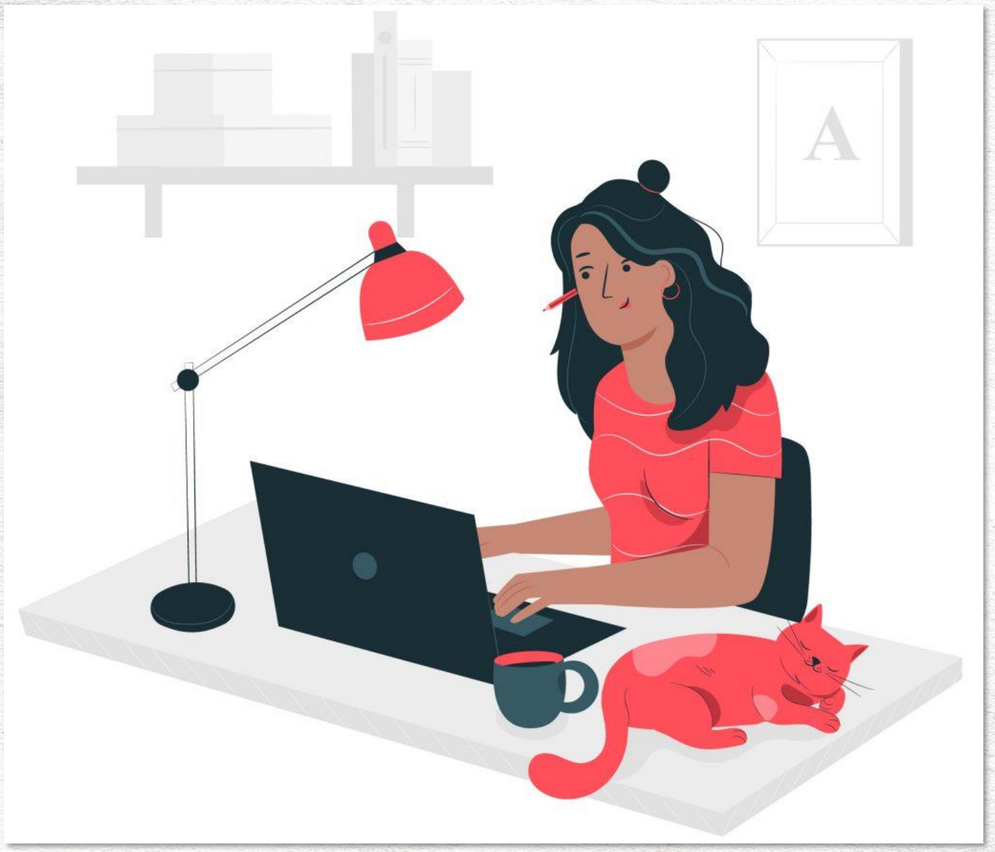
4

Questão 4

O departamento de compras está avaliando produtos da marca 'DELL' para negociar um lote com preços superiores a R$ 2.000,00.

Para isso, você precisa criar uma consulta que liste todos os produtos dessa marca com preço superior ao valor estabelecido. Utilize variáveis para armazenar a marca e o valor mínimo. **(Utilize a tabela produtos)**

EXERCÍCIOS



5

Questão 5

O departamento de marketing está criando uma segmentação de clientes para uma nova campanha. Eles estão interessados em identificar clientes que tenham mais de 2 filhos e uma renda anual superior a R$ 70.000,00.

Para isso, você precisa criar uma consulta que filtre esses clientes, utilizando variáveis para armazenar as condições de filtragem. **(Utilize a tabela clientes)**

6

Questão 6

O gerente da loja do Rio de Janeiro está interessado em identificar qual produto foi o mais vendido em termos de quantidade. Ele deseja focar em um produto específico para uma análise mais detalhada de sua popularidade.

Para isso, você deve criar uma consulta que encontre o produto mais vendido dessa filial considerando-se todas as suas vendas, utilizando o ID da loja como variável. **(Utilize as tabelas lojas e pedidos)**

EXERCÍCIOS

7

Questão 7

O departamento financeiro precisa calcular a receita das vendas realizadas em 2020 para gerar o relatório de desempenho anual.

Você deverá criar uma consulta utilizando variáveis para armazenar as datas de início e fim do período e calcular a receita das vendas durante esse intervalo. **Dica:** Atente-se ao tipo de dados da coluna de data da venda. **(Utilize a tabela pedidos)**

8

Questão 8

A equipe de finanças da empresa está analisando a rentabilidade dos produtos, buscando identificar aqueles cujo custo unitário é superior a um percentual específico do preço de venda. Eles definiram que devem ser avaliados apenas os produtos cujo custo unitário exceda 40% do preço de venda.

Para isso, você deve criar uma consulta que retorne todos os produtos que atendem a essa condição, utilizando uma variável para armazenar o percentual de comparação. **(Utilize a tabela produtos)**

EXERCÍCIOS

9

Questão 9

O time de logística da empresa deseja monitorar quais produtos estão com uma quantidade vendida superior a um valor específico. Eles precisam de uma análise para identificar rapidamente os produtos mais vendidos e tomar decisões sobre o reabastecimento do estoque.

A tarefa é criar uma consulta para listar os produtos com quantidade vendida superior a 2.850 unidades, utilizando uma variável para armazenar esse valor de comparação. O resultado deve ser ordenado pela quantidade vendida de forma decrescente. **(Utilize a tabela pedidos)**

10

Questão 10

O departamento de recursos humanos está analisando a capacidade operacional das lojas para planejar novas contratações e realocar equipes de acordo com a demanda. Para isso, eles precisam identificar quais lojas possuem mais de 20 funcionários, já que essas lojas são consideradas de grande porte.

Crie uma consulta SQL que utilize uma variável para definir o número mínimo de funcionários, e retorne as informações dessas lojas. Ordene o resultado pelo número de funcionários de forma crescente. **(Utilize a tabela lojas)**

GABARITO EXPLICATIVO

1

Questão 1

Uma loja vendeu 25 unidades de um produto com custo unitário de R$ 18,50. A equipe de contabilidade precisa calcular o custo total dessas vendas para registrar no balanço financeiro da loja.

Para isso, calcule o valor total do custo das unidades vendidas utilizando variáveis. **(Não é necessário utilizar uma tabela do banco de dados base)**

Explicação e Resultado

Para calcularmos o custo total das 25 unidades de um produto com custo unitário de R$ 18,50, podemos armazenar esses valores em duas variáveis: **@varQuantidade** e **@varCustoUnit**.

Em seguida, podemos criar outra variável para armazenar o custo total **(@varCustoTotal)**, cujo valor será calculado multiplicando-se os valores armazenados nas variáveis **@varQuantidade * @varCustoUnit**.

Feito isso, para exibir o custo total na tela, podemos fazer um **SELECT** à variável **@varCustoTotal**. Para que o resultado fique mais personalizado, podemos ainda atribuir um **alias** ao resultado retornado (**AS 'Custo Total das Vendas'**):

```
SET @varQuantidade = 25;
SET @varCustoUnit = 18.50;
SET @varCustoTotal = @varQuantidade * @varCustoUnit;

SELECT @varCustoTotal AS 'Custo Total das Vendas';
```

Dessa forma, retornamos o resultado abaixo:

	Custo Total das Vendas
▶	462.50

GABARITO EXPLICATIVO

2

Questão 2

A equipe de marketing está criando uma promoção para os produtos de uma determinada loja. Eles decidiram que todos os produtos com preço inferior a R$ 1.500,00 serão elegíveis para a promoção.

Para isso, você precisa criar uma consulta que ajude a identificar todos os produtos com preço inferior a esse valor.

Para tanto, crie uma variável que armazene o preço unitário e utilize-a em sua *query*. **(Utilize a tabela produtos)**

Explicação e Resultado

Para consultarmos todos os produtos com preço inferior ao valor máximo estipulado (R$ 1.500,00), primeiramente, podemos criar uma variável para armazenar esse valor (**@varPrecoMax**).

Em seguida, podemos fazer uma consulta à tabela **PRODUTOS** que retorne as informações de todos os produtos cujo preço unitário seja menor que o valor armazenado na variável (**WHERE Preco_Unit < @varPrecoMax**):

```
SET @varPrecoMax = 1500;

SELECT *
FROM produtos
WHERE Preco_Unit < @varPrecoMax;
```

Dessa forma, retornamos o resultado abaixo:

	ID_Produto	Nome_Produto	ID_Categoria	Marca_Produto	Num_Serie	Preco_Unit	Custo_Unit
▶	3	Webcam Full HD 1080p	1	LOGITECH	WBC-LT-934GG4	450	90
	4	Kit Teclado + Mouse sem fi...	2	DELL	KTM-DL-041039	350	129.5
	5	Kit Teclado + Mouse Slim Bl...	2	DELL	KTM-DL-111924	280	109.2
	8	Headphone Bluetooth 2000	4	SONY	HDP-SN-194821	600	258
	9	Fone de Ouvido Tune T5000	4	JBL	HDP-JB-091934	780	327.6
	10	Microfone Condensador M...	5	AKG	MIC-AK-237591	1100	275
	11	Microfone Condensador co...	5	BLUE	MIC-BL-819455	800	344
	12	Microfone de mesa com fio ...	5	BLUE	MIC-BL-761411	650	214.5

GABARITO EXPLICATIVO

3

Questão 3

A equipe de marketing deseja realizar uma análise sobre a faixa etária dos clientes. Eles gostariam de identificar o cliente mais jovem para uma campanha focada em jovens consumidores.

Para isso, você deve criar uma consulta que encontre o cliente mais jovem, com base na data de nascimento, utilizando uma variável que armazene a data de nascimento mais recente. **(Utilize a tabela clientes)**

Explicação e Resultado

Primeiramente, para descobrirmos a data de nascimento do cliente mais jovem, basta procurarmos na tabela **CLIENTES** pelo maior valor da coluna **Data_Nascimento** (**MAX(Data_Nascimento)**). Isso nos retornará a seguinte data de nascimento:

```
SELECT MAX(Data_Nascimento) FROM clientes
```

MAX(Data_Nascimento)
2000-08-15

Agora, atribuímos essa data à uma variável (**@varDataMaisRecente**). Podemos fazer isso de forma **manual** (SET @varDataMaisRecente = '2000-08-15') ou **automatizada**: em vez de atribuirmos um valor fixo à variável, **passamos entre parênteses a própria consulta que retorna a data mais recente para a variável**, conforme mostrado abaixo.

Feito isso, usamos essa variável em outra consulta à tabela **CLIENTES**, filtrando a coluna **Data_Nascimento** para retornar somente a(s) linha(s) cuja data armazenada seja **igual** à data armazenada na variável **@varDataMaisRecente**:

```
SET @varDataMaisRecente = (SELECT MAX(Data_Nascimento) FROM clientes);

SELECT *
FROM clientes
WHERE Data_Nascimento = @varDataMaisRecente;
```

Dessa forma, retornamos o resultado abaixo:

ID_Cliente	Nome	Sobrenome	Data_Nascimento	Estado_Civil	Sexo	Email	Telefone	Renda_Anual	Qtd_Filhos	Escolaridade
51	Ana	Price	2000-08-15	C	F	ana0@yahoo.com.br	(21) 99193-2686	60000	0	Parcial

GABARITO EXPLICATIVO

4

Questão 4

O departamento de compras está avaliando produtos da marca 'DELL' para negociar um lote com preços superiores a R$ 2.000,00.

Para isso, você precisa criar uma consulta que liste todos os produtos dessa marca com preço superior ao valor estabelecido. Utilize variáveis para armazenar a marca e o valor mínimo. **(Utilize a tabela produtos)**

Explicação e Resultado

Para listarmos todos os produtos da marca **DELL** com preço superior a **R$ 2.000,00**, primeiramente podemos criar duas variáveis, uma para armazenar a marca (**@varMarca**), outra para armazenar o preço mínimo limite (**@varPrecoMin**).

Em seguida, podemos fazer um **SELECT** à tabela **PRODUTOS**, filtrando todas as linhas cujos valores da coluna **Marca_Produto** corresponda à **@varMarca** e da coluna **Preco_Unit** seja maior que o valor armazenado na variável **@varPrecoMin**:

```
SET @varMarca = 'DELL';
SET @varPrecoMin = 2000;

SELECT *
FROM produtos
WHERE Marca_Produto = @varMarca
  AND Preco_Unit > @varPrecoMin;
```

Dessa forma, retornamos o resultado abaixo:

	ID_Produto	Nome_Produto	ID_Categoria	Marca_Produto	Num_Serie	Preco_Unit	Custo_Unit
▶	1	Monitor LED 19,5" Full HD ...	1	DELL	MNT-DL-831923	2300	966
	14	Notebook Inspiron 15 5000...	6	DELL	NOT-DL-000012	3100	1209
	15	Notebook IdeaPad RF32000	6	DELL	NOT-DL-77164I	4200	1176

GABARITO EXPLICATIVO

5

Questão 5

O departamento de marketing está criando uma segmentação de clientes para uma nova campanha. Eles estão interessados em identificar clientes que tenham mais de 2 filhos e uma renda anual superior a R$ 70.000,00.

Para isso, você precisa criar uma consulta que filtre esses clientes, utilizando variáveis para armazenar as condições de filtragem. **(Utilize a tabela clientes)**

Explicação e Resultado

Para retornarmos todos os clientes que tenham mais de **2 filhos** e uma renda anual superior a **R$ 70.000,00**, primeiramente podemos criar duas variáveis, uma para armazenar a quantidade de filhos (**@varQtdFilhos**), outra para armazenar a renda anual (**@varRendaMin**).

Em seguida, podemos fazer um **SELECT** à tabela **CLIENTES**, filtrando todas as linhas cujos valores da coluna **Qtd_Filhos** seja maior que a **@varQtdFilhos** e da coluna **Renda_Anual** seja maior que o valor armazenado na variável **@varRendaMin**:

```
SET @varQtdFilhos = 2;
SET @varRendaMin = 70000;

SELECT *
FROM clientes
WHERE Qtd_Filhos > @varQtdFilhos
  AND Renda_Anual > @varRendaMin;
```

Dessa forma, retornamos o resultado abaixo:

	ID_Cliente	Nome	Sobrenome	Data_Nascimento	Estado_Civil	Sexo	Email	Telefone	Renda_Anual	Qtd_Filhos	Escolaridade
▶	3	Elizabeth	Johnson	1988-08-03	S	F	elizabeth5@gmail.com	(31) 92039-5832	80000	5	Pós-graduado
	13	Sydney	Bennett	1988-05-04	S	F	sydney23@gmail.com	(11) 92024-9186	100000	3	Pós-graduado
	100	Julia	Nelson	1985-04-16	S	F	julia7@yahoo.com.br	(41) 96066-9929	80000	5	Pós-graduado

GABARITO EXPLICATIVO

6

Questão 6

O gerente da loja do Rio de Janeiro está interessado em identificar qual produto foi o mais vendido em termos de quantidade. Ele deseja focar em um produto específico para uma análise mais detalhada de sua popularidade.

Para isso, você deve criar uma consulta que encontre o produto mais vendido dessa filial considerando-se todas as suas vendas, utilizando o ID da loja como variável. **(Utilize as tabelas lojas e pedidos)**

Explicação e Resultado

Primeiramente, para descobrirmos o id da filial do Rio de Janeiro, basta procurarmos na tabela **LOJAS** pelo **ID_Loja** da loja cujo valor da coluna **Loja** seja **'Rio de Janeiro'**:

```
SELECT ID_Loja FROM lojas WHERE Loja = 'Rio de Janeiro'
```

→

	ID_Loja
▶	1

Agora, atribuímos esse ID_Loja à uma variável **(@varIDLoja)**. Podemos fazer isso de forma **manual** (SET @varIDLoja = 1) ou **automatizada**: em vez de atribuirmos um valor fixo à variável, **passamos entre parênteses a própria consulta que retorna o ID_Loja da loja do Rio de Janeiro**, conforme mostrado abaixo. Feito isso, fazemos um **SELECT** à tabela **PEDIDOS** que agrupa o total vendido (**SUM(Qtd_Vendida)**) por **ID_Produto**, filtrando somente as linhas dos pedidos cujo **ID_Loja** seja igual a **@varIDLoja**. Por fim, para retornar somente o produto com maior total vendido, ordenamos o resultado de forma **decrescente** pelo **Total_Vendido**, limitando o retorno para apenas uma linha (**LIMIT 1**):

```
SET @varIDLoja = (SELECT ID_Loja FROM lojas WHERE Loja = 'Rio de Janeiro');

SELECT ID_Produto, SUM(Qtd_Vendida) AS 'Total_Vendido'
FROM pedidos
WHERE ID_Loja = @varIDLoja
GROUP BY ID_Produto
ORDER BY Total_Vendido DESC
LIMIT 1;
```

Dessa forma, retornamos o resultado ao lado: →

	ID_Produto	Total_Vendido
▶	4	384

GABARITO EXPLICATIVO

7

Questão 7

O departamento financeiro precisa calcular a receita das vendas realizadas em 2020 para gerar o relatório de desempenho anual.

Você deverá criar uma consulta utilizando variáveis para armazenar as datas de início e fim do período e calcular a receita das vendas durante esse intervalo.

Dica: Atente-se ao tipo de dados da coluna de data da venda. **(Utilize a tabela pedidos)**

Explicação e Resultado

Para retornarmos **a receita das vendas do ano de 2020**, primeiramente podemos criar duas variáveis, uma para armazenar o primeiro dia de 2020 (**@varDataInicio**), outra para armazenar o último dia de 2020 (**@varDataFim**).

Em seguida, podemos fazer um **SELECT** à tabela **PEDIDOS** para somar a receita das vendas (**SUM(Receita_Venda)**), filtrando somente as linhas cujos valores da coluna **Data_Venda** estejam entre (**BETWEEN**) a **@varDataInicio** e a **@varDataFim**.

Como a coluna **Data_Venda** é do tipo texto (**text**), para que o filtro da cláusula **WHERE** funcione, precisamos converter seus valores para **DATE**. Para fazer isso, aplicamos a função **CAST**:

```
SET @varDataInicio = '2020-01-01';
SET @varDataFim = '2020-12-31';

SELECT SUM(Receita_Venda) AS 'Total_Vendas'
FROM pedidos
WHERE CAST(Data_Venda AS DATE) BETWEEN @varDataInicio AND @varDataFim;
```

Dessa forma, retornamos o resultado ao lado:

	Total_Vendas
▶	40451590

GABARITO EXPLICATIVO

8

Questão 8

A equipe de finanças da empresa está analisando a rentabilidade dos produtos, buscando identificar aqueles cujo custo unitário é superior a um percentual específico do preço de venda. Eles definiram que devem ser avaliados apenas os produtos cujo custo unitário exceda 40% do preço de venda.

Para isso, você deve criar uma consulta que retorne todos os produtos que atendem a essa condição, utilizando uma variável para armazenar o percentual de comparação. **(Utilize a tabela produtos)**

Explicação e Resultado

Para retornarmos os produtos cujo **custo unitário excede 40% do preço da venda**, primeiramente podemos criar uma variável para armazenar esse percentual (**@percentual**).

Em seguida, podemos fazer um **SELECT** à tabela **PRODUTOS**, filtrando todas as linhas cujos valores da coluna **Custo_Unit** seja maior que o **Preco_Unit** multiplicado pelo **@percentual**:

```
SET @percentual = 0.4;

SELECT *
FROM produtos
WHERE Custo_Unit > (Preco_Unit * @percentual);
```

Dessa forma, retornamos o resultado abaixo:

	ID_Produto	Nome_Produto	ID_Categoria	Marca_Produto	Num_Serie	Preco_Unit	Custo_Unit
▶	1	Monitor LED 19,5" Full HD ...	1	DELL	MNT-DL-831923	2300	966
	7	Cadeira Gamer PC Racer V...	3	ALTURA	CGM-AL-0147FI	3100	1395
	8	Headphone Bluetooth 2000	4	SONY	HDP-SN-194821	600	258
	9	Fone de Ouvido Tune T5000	4	JBL	HDP-JB-091934	780	327.6
	11	Microfone Condensador co...	5	BLUE	MIC-BL-819455	800	344

GABARITO EXPLICATIVO

9

Questão 9

O time de logística da empresa deseja monitorar quais produtos estão com uma quantidade vendida superior a um valor específico. Eles precisam de uma análise para identificar rapidamente os produtos mais vendidos e tomar decisões sobre o reabastecimento do estoque.

A tarefa é criar uma consulta para listar os produtos com quantidade vendida superior a 2.850 unidades, utilizando uma variável para armazenar esse valor de comparação. O resultado deve ser ordenado pela quantidade vendida de forma decrescente. **(Utilize a tabela pedidos)**

Explicação e Resultado

Para listarmos **os produtos com uma quantidade vendida superior a 2.850 unidades**, primeiramente podemos criar uma variável para armazenar essa quantidade mínima (**@varQuantidadeMinima**).

Em seguida, podemos fazer um **SELECT** à tabela **PEDIDOS** que agrupa a soma da quantidade vendida (**SUM(Qtd_Vendida)**) por **ID_Produto**, filtrando somente as linhas do agrupamento cuja **Quantidade_Vendida** seja maior que a **@varQuantidadeMinima**. Por fim, ordenamos o resultado de forma **decrescente** pela **Quantidade_Vendida**:

```sql
SET @varQuantidadeMinima = 2850;

SELECT
    ID_Produto,
    SUM(Qtd_Vendida) AS Quantidade_Vendida
FROM pedidos
GROUP BY ID_Produto
HAVING Quantidade_Vendida > @varQuantidadeMinima
ORDER BY Quantidade_Vendida DESC;
```

Dessa forma, retornamos o resultado ao lado:

ID_Produto	Quantidade_Vendida
4	2906
7	2898
8	2888
1	2875
15	2855

GABARITO EXPLICATIVO

10

Questão 10

O departamento de recursos humanos está analisando a capacidade operacional das lojas para planejar novas contratações e realocar equipes de acordo com a demanda. Para isso, eles precisam identificar quais lojas possuem mais de 20 funcionários, já que essas lojas são consideradas de grande porte.

Crie uma consulta SQL que utilize uma variável para definir o número mínimo de funcionários, e retorne as informações dessas lojas. Ordene o resultado pelo número de funcionários de forma crescente. **(Utilize a tabela lojas)**

Explicação e Resultado

Para listarmos **as lojas com mais de 20 funcionários**, primeiramente podemos criar uma variável para armazenar essa quantidade mínima (**@minFuncionarios**).

Em seguida, podemos fazer um **SELECT** à tabela **LOJAS** para retornar todas as linhas cujo valor da coluna **Num_Funcionarios** seja maior que **@minFuncionarios**. Por fim, ordenamos o resultado de forma **crescente** pela coluna **Num_Funcionarios**:

```
SET @minFuncionarios = 20;

SELECT *
FROM lojas
WHERE Num_Funcionarios > @minFuncionarios
ORDER BY Num_Funcionarios;
```

Dessa forma, retornamos o resultado abaixo:

ID_Loja	Loja	Gerente	Endereco	Num_Funcionarios	Telefone
1	Rio de Janeiro	Pedro Martins	R. Visc. de Pirajá, 136 - Ipanema, Rio de Janeiro - RJ, 22410-000	23	(21) 3198-1239
4	Curitiba	Amanda Lima	R. Brasílio Itiberê, 3279 - Água Verde, Curitiba - PR, 80250-160	31	(41) 2923-2137
6	São Paulo	Rogério Lopes	R. Teodoro Sampaio, 954 - Pinheiros, São Paulo - SP, 05406-050	33	(11) 3295-3945

MÓDULO 8

FUNÇÕES DE TEXTO E DATA

FUNÇÕES DE TEXTO E DATA

FUNÇÕES DE TEXTO E DATA

Índice Geral

Índice deste Módulo

O QUE VEREMOS NESTE MÓDULO

MÓDULO 8 FUNÇÕES DE TEXTO E DATA NO MYSQL

01	FUNÇÕES DE TEXTO E DATA	173
02	FUNÇÕES DE TEXTO	174
03	FUNÇÃO LENGTH()	175
04	FUNÇÃO CONCAT() E CONCAT_WS()	176
05	FUNÇÃO LCASE() E UCASE()	177
06	FUNÇÃO LEFT() E RIGHT()	178
07	FUNÇÃO REPLACE()	180
08	FUNÇÃO INSTR()	182
09	FUNÇÃO MID()	184
10	FUNÇÃO MID E INSTR	185
11	FUNÇÕES DE DATA	186
12	FUNÇÕES DAY(), MONTH() E YEAR()	187
12	FUNÇÕES NOW(), CURDATE() e CURTIME()	188
13	FUNÇÃO DATEDIFF()	189
14	FUNÇÃO DATE_ADD()	190
15	FUNÇÃO DATE_SUB()	191
16	EXERCÍCIOS	192
17	GABARITO EXPLICATIVO	197

Funções de Texto e Data

As funções de texto (string) e data nos permitem fazer diversos tipos de tratamentos com valores do tipo texto e do tipo data.

Abaixo, temos um resumo das principais:

STRINGS

Funções para trabalhar com **TEXTOS**.

- LENGTH
- CONCAT
- CONCAT_WS
- LCASE e UCASE
- RIGHT e LEFT
- REPLACE
- INSTR
- MID

DATAS

Funções para trabalhar com **DATAS**.

- DAY
- YEAR
- MONTH
- NOW
- CURDATE
- CURTIME
- DATE_DIFF
- DATE_ADD
- DATE_SUB

Índice Geral

Índice deste Módulo

Funções de Texto

Vamos começar com as funções de texto (string).

STRINGS

Funções para trabalhar com **TEXTOS**.

- LENGTH
- CONCAT
- CONCAT_WS
- LCASE e UCASE
- RIGHT e LEFT
- REPLACE
- INSTR
- MID

DATAS

Funções para trabalhar com **DATAS**.

- DAY
- YEAR
- MONTH
- NOW
- CURDATE
- CURTIME
- DATE_DIFF
- DATE_ADD
- DATE_SUB

Índice Geral

Índice deste Módulo

Função LENGTH()

A função LENGTH() pede apenas 1 argumento e retorna a quantidade de caracteres que um determinado texto possui.

Por exemplo, o nome Ruben possui 5 caracteres, portanto este é o retorno da função.

Query 1 | SQL File 5*

Don't Limit

```
4   -- LENGTH
5 • SELECT
6       Nome AS 'Nome',
7       LENGTH(Nome) AS 'Núm. Caract.'
8   FROM clientes;
9
10
11
```

Result Grid | Filter Rows:

Nome	Núm. Caract.
Ruben	5
Christy	7
Elizabeth	9
Julio	5
Janet	5
Marco	5
Rob	3
Shannon	7

Função CONCAT() e CONCAT_WS()

As funções CONCAT() e CONCAT_WS() retornam a concatenação de textos.

A diferença entre elas é que na CONCAT_WS() passamos no primeiro argumento o caractere que desejamos usar para separar cada texto.

No CONCAT(), cada vez que precisamos de um espaço para separar os textos, precisaremos repetir o espaço diversas vezes.

```
12    -- CONCAT e CONCAT_WS
13 •  SELECT
14        CONCAT(Nome, ' ', Sobrenome) AS 'Nome Completo (Concat)',
15        CONCAT_WS(' ', Nome, Sobrenome) AS 'Nome Completo (Concat_ws)'
16    FROM clientes;
17
```

Result Grid | Filter Rows: | Export:

Nome Completo (Concat)	Nome Completo (Concat_ws)
Ruben Torres	Ruben Torres
Christy Zhu	Christy Zhu
Elizabeth Johnson	Elizabeth Johnson
Julio Ruiz	Julio Ruiz
Janet Alvarez	Janet Alvarez
Marco Mehta	Marco Mehta
Rob Verhoff	Rob Verhoff
Shannon Carlson	Shannon Carlson
Jacquelyn Suarez	Jacquelyn Suarez
Curtis Lu	Curtis Lu

Função LCASE() e UCASE()

Estas funções permitem retornar textos em maiúscula (UCASE()) e minúscula (LCASE()).

Ao lado, temos um exemplo de aplicação dessas duas funções para retornar o nome completo de cada cliente tanto em maiúscula quanto em minúscula.

```
18    -- LCASE e UCASE
19 •  SELECT
20        UCASE(CONCAT(Nome, ' ', Sobrenome)) AS 'Nome Completo (Concat)',
21        LCASE(CONCAT_WS(' ', Nome, Sobrenome)) AS 'Nome Completo (Concat_ws)'
22    FROM clientes;
23
```

Result Grid | Filter Rows: | Export:

Nome Completo (Concat)	Nome Completo (Concat_ws)
RUBEN TORRES	ruben torres
CHRISTY ZHU	christy zhu
ELIZABETH JOHNSON	elizabeth johnson
JULIO RUIZ	julio ruiz
JANET ALVAREZ	janet alvarez
MARCO MEHTA	marco mehta
ROB VERHOFF	rob verhoff
SHANNON CARLSON	shannon carlson
JACQUELYN SUAREZ	jacquelyn suarez
CURTIS LU	curtis lu
LAUREN WALKER	lauren walker
IAN JENKINS	ian jenkins
SYDNEY BENNETT	sydney bennett
CHLOE YOUNG	chloe young
WYATT HILL	wyatt hill
SHANNON WANG	shannon wang
CLARENCE RAI	clarence rai
JORDAN KING	jordan king

Função LEFT() e RIGHT()

As funções LEFT() e RIGHT() permitem retornar parte de um texto.

A função LEFT() pede 2 argumentos: o primeiro é o texto, o segundo é a quantidade de caracteres desejada, **da esquerda para a direita**.

Já a função RIGHT() faz o oposto: permite retornar uma determinada quantidade de caracteres, **da direita para a esquerda**.

```
26    -- LEFT e RIGHT
27 •  SELECT
28        Num_Serie AS 'Serie',
29        LEFT(Num_Serie, 6) AS 'Cod 1',
30        RIGHT(Num_Serie, 6) AS 'Cod 2'
31    FROM produtos;
32
```

Serie	Cod 1	Cod 2
MNT-DL-831923	MNT-DL	831923
MNT-SS-001939	MNT-SS	001939
WBC-LT-934GG4	WBC-LT	934GG4
KTM-DL-041039	KTM-DL	041039
KTM-DL-111924	KTM-DL	111924
CGM-AL-9N914J	CGM-AL	9N914J
CGM-AL-0147FI	CGM-AL	0147FI
HDP-SN-194821	HDP-SN	194821
HDP-JB-091934	HDP-JB	091934
MIC-AK-237591	MIC-AK	237591
MIC-BL-819455	MIC-BL	819455
MIC-BL-761411	MIC-BL	761411
NOT-SS-918457	NOT-SS	918457
NOT-DL-000012	NOT-DL	000012
NOT-DL-77164I	NOT-DL	77164I
NOT-SS-13139U	NOT-SS	13139U

Função LEFT() e RIGHT()

Ao lado, aplicamos essas duas funções para dividir a número de série de cada um dos produtos em duas partes.

Por exemplo, o número de Série:

MNT-DL-831923

Pode ser dividido em 2 partes usando as funções LEFT() e RIGHT().

MNT-DL-831923

LEFT RIGHT

```
26    -- LEFT e RIGHT
27 •  SELECT
28        Num_Serie AS 'Serie',
29        LEFT(Num_Serie, 6) AS 'Cod 1',
30        RIGHT(Num_Serie, 6) AS 'Cod 2'
31    FROM produtos;
32
```

Serie	Cod 1	Cod 2
MNT-DL-831923	MNT-DL	831923
MNT-SS-001939	MNT-SS	001939
WBC-LT-934GG4	WBC-LT	934GG4
KTM-DL-041039	KTM-DL	041039
KTM-DL-111924	KTM-DL	111924
CGM-AL-9N914J	CGM-AL	9N914J
CGM-AL-0147FI	CGM-AL	0147FI
HDP-SN-194821	HDP-SN	194821
HDP-JB-091934	HDP-JB	091934
MIC-AK-237591	MIC-AK	237591
MIC-BL-819455	MIC-BL	819455
MIC-BL-761411	MIC-BL	761411
NOT-SS-918457	NOT-SS	918457
NOT-DL-000012	NOT-DL	000012
NOT-DL-77164I	NOT-DL	77164I
NOT-SS-13139U	NOT-SS	13139U

Função REPLACE()

A função REPLACE() é outra função bem comum em diversas ferramentas. Ela permite substituir um determinado texto por outro texto.

Ao lado, temos a seguinte aplicação: utilizamos a função REPLACE() para substituir as siglas do estado civil pelo texto correspondente.

Por exemplo, substituir S por Solteiro.

```
34    -- REPLACE
35 •  SELECT * FROM clientes;
36 •  SELECT
37        Nome,
38        Estado_Civil,
39        REPLACE(Estado_Civil, 'S', 'Solteiro') AS 'Estado_Civil'
40    FROM clientes;
41
```

Result Grid | Filter Rows:

Nome	Estado_Civil	Estado_Civil
Ruben	C	C
Christy	S	Solteiro
Elizabeth	S	Solteiro
Julio	S	Solteiro
Janet	S	Solteiro
Marco	C	C
Rob	S	Solteiro
Shannon	S	Solteiro
Jacquelyn	S	Solteiro
Curtis	C	C
Lauren	C	C
Ian	C	C
Sydney	S	Solteiro
Chloe	S	Solteiro
Wyatt	C	C
Shannon	S	Solteiro
Clarence	S	Solteiro

Função REPLACE()

Como queremos fazer uma nova substituição (dessa vez 'C' por 'Casado') aplicamos a função REPLACE() mais uma vez, e chegamos no resultado ao lado.

```
34    -- REPLACE
35 •  SELECT * FROM clientes;
36 •  SELECT
37        Nome,
38        Estado_Civil,
39        REPLACE(REPLACE(Estado_Civil, 'S', 'Solteiro'), 'C', 'Casado') AS 'Estado_Civil'
40    FROM clientes;
41
```

Result Grid | Filter Rows: | Exp

Nome	Estado_Civil	Estado_Civil
Ruben	C	Casado
Christy	S	Solteiro
Elizabeth	S	Solteiro
Julio	S	Solteiro
Janet	S	Solteiro
Marco	C	Casado
Rob	S	Solteiro
Shannon	S	Solteiro
Jacquelyn	S	Solteiro
Curtis	C	Casado
Lauren	C	Casado
Ian	C	Casado
Sydney	S	Solteiro
Chloe	S	Solteiro
Wyatt	C	Casado
Shannon	S	Solteiro
Clarence	S	Solteiro

Função INSTR()

Outra função de texto muito útil é a INSTR().

Ela é capaz de retornar a posição que um determinado caractere ocupa dentro de um texto.

Por exemplo, no e-mail abaixo, a posição que o caractere ‘@’ ocupa é a posição 8:

Função INSTR()

Aplicando essa função na coluna Email da tabela clientes, chegamos ao resultado para cada e-mail.

```
42    -- INSTR
43 •  SELECT
44        Email,
45        INSTR(Email, '@') FROM clientes;
```

Email	INSTR(Email, '@')
ruben35@hotmail.com	8
christy12@hotmail.com	10
elizabeth5@gmail.com	11
julio1@hotmail.com	7
janet9@gmail.com	7
marco14@yahoo.com.br	8
rob4@hotmail.com	5
shannon38@gmail.com	10

Função MID()

Podemos combinar a função INSTR() com uma nova função, chamada MID().

Essa função permite retornar uma sequência de caracteres, a partir dos seguintes argumentos:

1) Texto
2) Posição inicial
3) Quantidade de caracteres desejados a partir da posição inicial.

Por exemplo, se quisermos extrair o ID do e-mail (ou seja, tudo o que vem antes do @), podemos usar a função MID() como mostrado.

Primeiro informamos o e-mail, em seguida informamos a posição inicial em que começa o texto que queremos extrair, e depois a quantidade de caracteres que queremos a partir da posição inicial. Como o texto 'ruben35' tem 7 caracteres, passamos esse valor dentro da função.

```
42    -- MID
43 •  SELECT
44        Email,
45        MID('ruben35@hotmail.com', 1, 7)
46
47
48
49
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Email	MID('ruben35@hotmail.com', 1, 7)
ruben35@hotmail.com	ruben35

ruben35@hotmail.com

ID do e-mail

Função MID() e INSTR()

Para tornar o processo de extrair a primeira parte dos e-mails mais automática, podemos combinar a função INSTR() com a função MID().

Observe que o tamanho do ID é sempre a posição do '@' em cada um dos e-mails.

Ao lado, temos a solução final.

```
47 •  SELECT
48        Email,
49        MID(Email, 1, INSTR(Email, '@') - 1) 'ID'
50    FROM clientes;
51
52
53
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Email	ID
ruben35@hotmail.com	ruben35
christy12@hotmail.com	christy12
elizabeth5@gmail.com	elizabeth5
julio1@hotmail.com	julio1
janet9@gmail.com	janet9
marco14@yahoo.com.br	marco14
rob4@hotmail.com	rob4
shannon38@gmail.com	shannon38
jacquelyn20@hotmail.com	jacquelyn20
curtis9@yahoo.com.br	curtis9
lauren41@hotmail.com	lauren41
ian47@gmail.com	ian47

Funções de Data

Vejamos agora as funções de Data mais comuns.

STRINGS

Funções para trabalhar com **TEXTOS**.

- LENGTH
- CONCAT
- CONCAT_WS
- LCASE e UCASE
- RIGHT e LEFT
- REPLACE
- INSTR
- MID

DATAS

Funções para trabalhar com **DATAS**.

- DAY
- YEAR
- MONTH
- NOW
- CURDATE
- CURTIME
- DATE_DIFF
- DATE_ADD
- DATE_SUB

Índice Geral

Índice deste Módulo

Funções DAY(), MONTH() e YEAR()

As primeiras funções de datas que veremos serão: DAY(), MONTH() e YEAR().

Essas funções permitem retornar, respectivamente, o dia, mês e ano de uma data.

Suas aplicações são bem diretas e mostradas ao lado, onde queremos saber qual é o dia, mês e ano de cada data de nascimento dos clientes.

Query 1 | SQL File 5*

Don't Limit

```
60  -- DAY, MONTH, YEAR
61  SELECT
62      Nome,
63      Data_Nascimento,
64      DAY(Data_Nascimento) AS 'Dia',
65      MONTH(Data_Nascimento) AS 'Mês',
66      YEAR(Data_Nascimento) AS 'Ano'
67  FROM clientes;
68
```

Result Grid | Filter Rows: | Export:

Nome	Data_Nascimento	Dia	Mês	Ano
Ruben	1985-08-07	7	8	1985
Christy	1988-02-10	10	2	1988
Elizabeth	1988-08-03	3	8	1988
Julio	1985-07-31	31	7	1985
Janet	1985-12-01	1	12	1985
Marco	1984-05-04	4	5	1984
Rob	1984-07-02	2	7	1984
Shannon	1984-03-27	27	3	1984
Jacquelyn	1984-02-01	1	2	1984
Curtis	1983-10-30	30	10	1983
Lauren	1988-01-13	13	1	1988
Ian	1988-08-01	1	8	1988
Sydney	1988-05-04	4	5	1988
Chloe	1999-02-22	22	2	1999
Wyatt	1999-04-23	23	4	1999
Shannon	1964-06-21	21	6	1964

Result 24

Funções NOW(), CURDATE() e CURTIME()

As funções de datas NOW(), CURDATE() e CURTIME() servem para:

1. NOW(): Retornar a data e hora atuais;
2. CURDATE(): Retornar a data atual;
3. CURTIME(): Retornar a hora atual;

Veja o exemplo ao lado:

```
185
186 •  SELECT
187        NOW(),
188        CURDATE(),
189        CURTIME();
190
```

Result Grid | Filter Rows: | Export:

NOW()	CURDATE()	CURTIME()
2024-11-28 21:56:50	2024-11-28	21:56:50

Função DATEDIFF()

A função DATEDIFF() retorna a diferença entre duas datas.

No exemplo abaixo, calculamos o tempo de entrega (em dias) entre duas datas.

```
86    -- DATEDIFF:
87    -- Exemplo: Calcular o tempo de entrega (em dias) de um determinado projeto que teve início no dia 10/04/2021
      e terminou no dia 15/07/2021.
88
89 •  SET @varDataInicio = '2021-04-10';
90 •  SET @varDataFim = '2021-07-15';
91
92 •  SELECT DATEDIFF(@varDataFim, @varDataInicio);
93
94
95
96
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

DATEDIFF(@varDataFim, @varDataInicio)
96

Função DATE_ADD()

A função DATE_ADD() adiciona uma quantidade de dias, meses ou anos a uma determinada data.

No exemplo abaixo, adicionamos 10 dias (INTERVAL 10 DAY) à data de início.

Se quiséssemos adicionar 1 mês, por exemplo, faríamos INTERVAL 1 MONTH, e para 2 anos, por exemplo, faríamos INTERVAL 2 YEAR.

```
78    -- DATE_ADD
79    -- Exemplo: Um projeto deve ser entregue 10 dias após a data de início. Qual é a data final da entrega?
80
81 •  SET @varDataInicio = '2021-04-10';
82
83 •  SELECT DATE_ADD(@varDataInicio, INTERVAL 10 DAY);
84
85
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

DATE_ADD(@varDataInicio, INTERVAL 10 DAY)
2021-04-20

Função DATE_SUB()

A função DATE_SUB() subtrai uma quantidade de dias, meses ou anos de uma determinada data.

Observe o exemplo de aplicação abaixo:

```
 96    -- DATE_SUB
 97    -- Exemplo: Um projeto finalizou no dia 21/09/2021 e teve 10 dias de duração. Qual foi a data de início?
 98
 99 •  SET @varDataFim = '2021-09-21';
100
101 •  SELECT DATE_SUB(@varDataFim, INTERVAL 10 DAY);
102
103
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

DATE_SUB(@varDataFim, INTERVAL 10 DAY)
2021-09-11

EXERCÍCIOS

1

Questão 1

A equipe de estoque precisa identificar produtos cujo número de série inicia com 'MIC' para fazer uma verificação.

Crie uma consulta que utilize a função LEFT para listar os IDs e nomes desses produtos, filtrando os números de série que atendem a essa condição. **(Utilize a tabela produtos)**

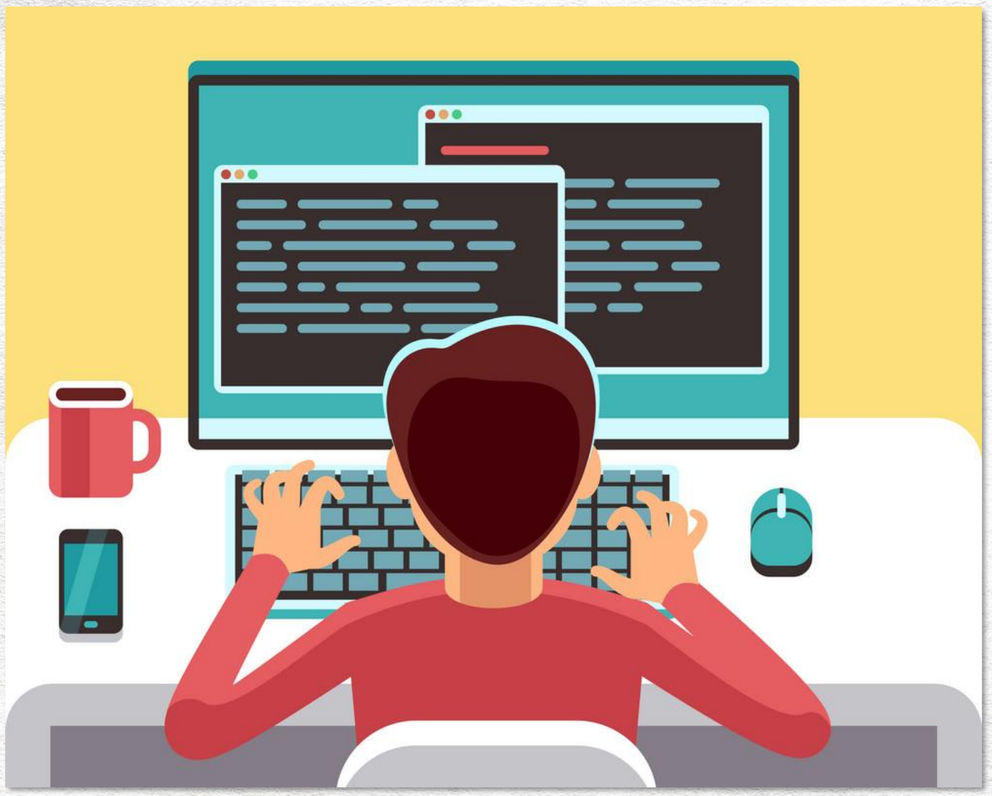
2

Questão 2

O analista de dados da empresa está verificando a integridade da base de clientes e quer garantir que não existam e-mails duplicados, já que isso pode causar problemas nas comunicações com os clientes.

Crie uma consulta que utilize a função MID para extrair os IDs dos e-mails (a parte antes do caractere '@') e liste aqueles que aparecem mais de uma vez. Caso não haja duplicatas, a consulta não deve retornar resultados. **(Utilize a tabela clientes)**

EXERCÍCIOS



3

Questão 3

O RH está avaliando lojas com menos de 15 funcionários para um plano de expansão. Além disso, eles querem listar os nomes das lojas em letras minúsculas para facilitar a integração com um sistema legado.

Crie uma consulta para exibir o ID da loja, o nome da loja em letras minúsculas e o número de funcionários. **(Utilize a tabela lojas)**

4

Questão 4

O setor de CRM quer saber quais clientes fazem aniversário no mês atual para enviar promoções personalizadas.

Utilize a função MONTH na coluna Data_Nascimento para filtrar os clientes que atendem ao critério. **(Utilize a tabela clientes)**

EXERCÍCIOS

5

Questão 5

A equipe de logística está revisando pedidos antigos e precisa identificar aqueles que foram realizados há mais de 30 dias a partir de uma determinada data.

Para esta análise, suponha que a data determinada seja 10/02/2019 e armazene-a em uma variável. Em seguida, filtre todos os pedidos cuja diferença entre a data da venda e a data determinada seja superior a 30 dias. **(Utilize a tabela pedidos)**

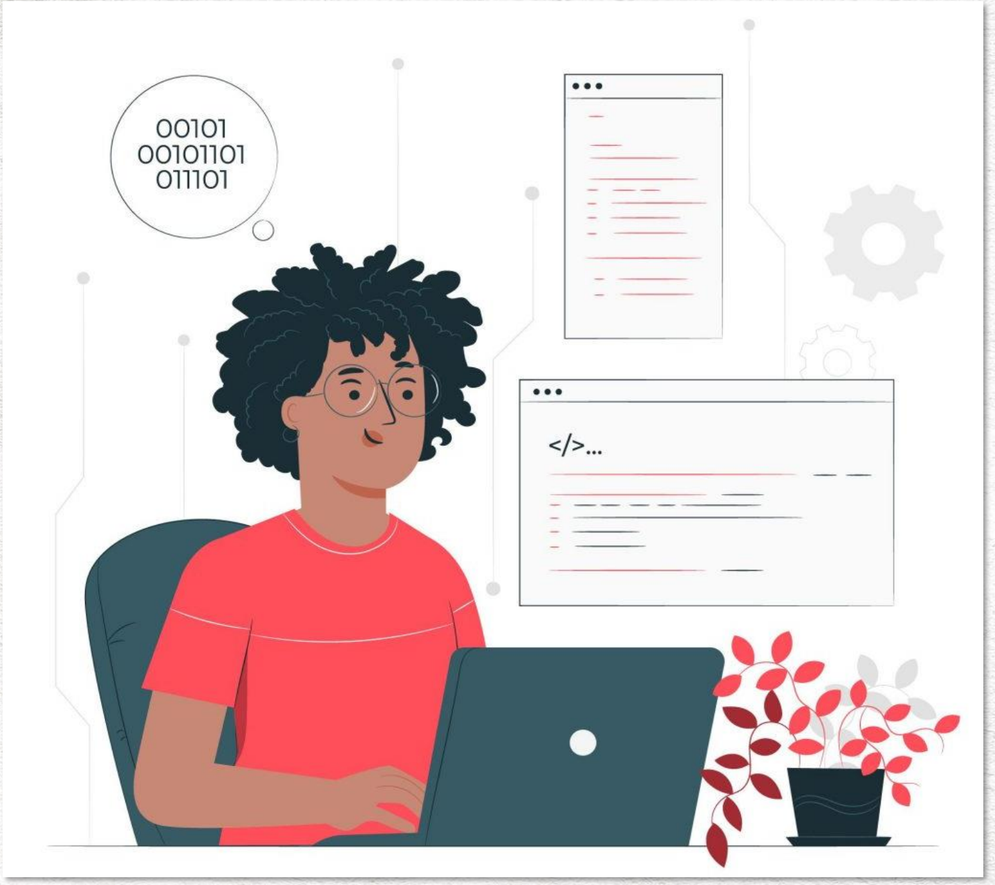
6

Questão 6

A equipe de logística precisa criar identificadores temporários para os produtos em estoque, baseando-se na marca e nos 3 últimos caracteres do número de série.

Utilize as funções CONCAT (ou CONCAT_WS) e RIGHT para construir esses identificadores. **(Utilize a tabela produtos)**

EXERCÍCIOS



7

Questão 7

A equipe de estoque está padronizando o cadastro dos produtos e solicitou que listemos todos os nomes de produtos em letras maiúsculas.

Use a função UCASE() para transformar os nomes. **(Utilize a tabela produtos)**

8

Questão 8

A equipe de marketing deseja identificar clientes nascidos nos dias 1, 15 ou 30 de qualquer mês para uma ação especial.

Use a função DAY() para filtrar esses clientes. **(Utilize a tabela clientes)**

9

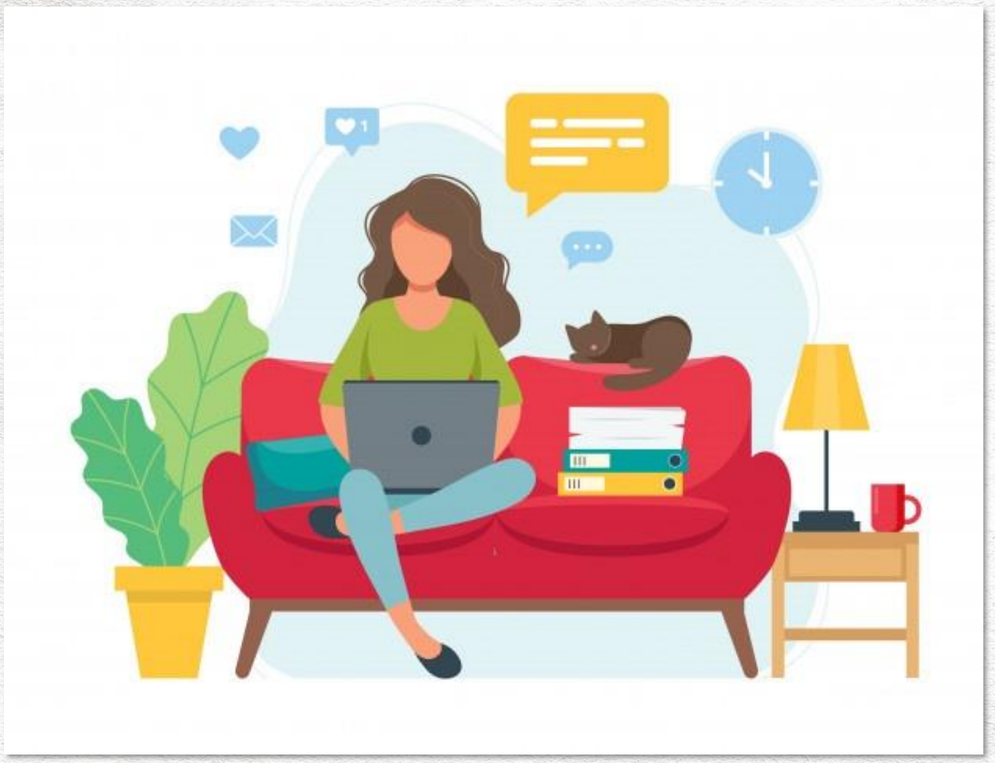
Questão 9

A equipe de marketing está planejando uma campanha direcionada para clientes nascidos na década de 90, visando oferecer produtos personalizados para essa faixa etária. Para isso, precisamos identificar todos os clientes que nasceram entre 1990 e 1999.

Use a função YEAR() para filtrar os clientes por essa faixa de anos e retorne o nome completo e a data de nascimento**. (Utilize a tabela clientes)**

EXERCÍCIOS

10



Questão 10

A equipe de TI está preparando um novo sistema de envio de SMS para as campanhas promocionais e, para isso, precisa dos números de telefone dos clientes no formato simples, sem parênteses, traços ou espaços.

Como a tabela de clientes contém números no formato '(99) 99999-9999', para garantir que o sistema de mensagens funcionará corretamente, é necessário tratar esses telefones para que contenham somente os 11 dígitos.

Sua tarefa é remover os parênteses, o traço e o espaço dos números de telefone e verificar se o comprimento do número após a remoção tem exatamente 11 dígitos. Além disso, você deve retornar apenas os clientes que possuem telefone preenchido, ou seja, excluindo os valores nulos ou em branco. **(Utilize a tabela clientes)**

GABARITO EXPLICATIVO

1

Questão 1

A equipe de estoque precisa identificar produtos cujo número de série inicia com 'MIC' para fazer uma verificação.

Crie uma consulta que utilize a função LEFT para listar os IDs e nomes desses produtos, filtrando os números de série que atendem a essa condição. **(Utilize a tabela produtos)**

Explicação e Resultado

Para retornamos somente **os produtos cujo número de série se inicia com 'MIC'**, podemos consultar a tabela **PRODUTOS**, fazendo um filtro na cláusula **WHERE** com a função **LEFT** utilizando **dois parâmetros**: o nome da coluna (**Num_Serie**) e quantos caracteres queremos retornar, contando da esquerda para a direita (**3 caracteres**). Nesse filtro, ainda informamos o **padrão** que deve ser filtrado: **'MIC'**. Por fim, podemos **selecionar** somente as colunas **ID_Produto**, **Nome_Produto** e **Num_Serie**:

```sql
SELECT
    ID_Produto,
    Nome_Produto,
    Num_Serie
FROM produtos
WHERE LEFT(Num_Serie, 3) = 'MIC';
```

Dessa forma, retornamos o resultado abaixo:

	ID_Produto	Nome_Produto	Num_Serie
▶	10	Microfone Condensador MC1000	MIC-AK-237591
	11	Microfone Condensador com Tripé	MIC-BL-819455
	12	Microfone de mesa com fio condensador	MIC-BL-761411

GABARITO EXPLICATIVO

2

Questão 2

O analista de dados da empresa está verificando a integridade da base de clientes e quer garantir que não existam e-mails duplicados, já que isso pode causar problemas nas comunicações com os clientes.

Crie uma consulta que utilize a função MID para extrair os IDs dos e-mails (a parte antes do caractere '@') e liste aqueles que aparecem mais de uma vez. Caso não haja duplicatas, a consulta não deve retornar resultados. **(Utilize a tabela clientes)**

Explicação e Resultado

Para retornarmos somente **os IDs dos e-mails duplicados, caso existam**, podemos usar a função **MID** para extrair esses IDs, passando para ela **3 parâmetros**: 1) O nome da coluna (**Email**); 2) **A partir** de que posição queremos extrair os caracteres (**posição 1**) e 3) **Até que posição** queremos extrair: aqui, usamos a função **INSTR para descobrir a posição do '@'** nos e-mails. Ao final do parâmetro, acrescentamos **-1** para indicar que os caracteres da coluna Email devem ser extraídos **até o '@', menos o próprio '@'.** Assim descobrimos o ID do Email. Em seguida, fazemos uma contagem (**COUNT(*)**) de ocorrências dos IDs dos Emails, formando um **agrupamento por IDs**. Ao final, **filtramos o agrupamento** com a cláusula **HAVING** para que sejam retornadas apenas as linhas do agrupamento em que **a contagem das ocorrências dos IDs seja maior que 1**. Dessa forma, somente os IDs de Email duplicados são retornados:

```sql
SELECT
    MID(Email, 1, INSTR(Email, '@') - 1) AS 'ID do E-mail',
    COUNT(*) AS 'Ocorrências'
FROM clientes
GROUP BY MID(Email, 1, INSTR(Email, '@') - 1)
HAVING COUNT(*) > 1;
```

	ID do E-mail	Ocorrências

Assim, retornamos o resultado ao lado, que mostra que **NÃO existem e-mails cadastrados em duplicidade na tabela CLIENTES**:

GABARITO EXPLICATIVO

3

Questão 3

O RH está avaliando lojas com menos de 15 funcionários para um plano de expansão. Além disso, eles querem listar os nomes das lojas em letras minúsculas para facilitar a integração com um sistema legado.

Crie uma consulta para exibir o ID da loja, o nome da loja em letras minúsculas e o número de funcionários. **(Utilize a tabela lojas)**

Explicação e Resultado

Nesta consulta, devemos filtrar as colunas **ID_Loja**, **Loja** e **Num_Funcionarios** da tabela **LOJAS**. Como queremos retornar somente **as lojas cuja quantidade de funcionários não exceda a 15**, devemos utilizar a cláusula **WHERE** para fazer o filtro **Num_Funcionarios < 15**. Como precisamos retornar os nomes das lojas em **letras minúsculas**, aplicamos a função **LCASE** na coluna **Loja**:

```sql
SELECT
    ID_Loja,
    LCASE(Loja) AS 'Nome da Loja',
    Num_Funcionarios
FROM lojas
WHERE Num_Funcionarios < 15;
```

Dessa forma, retornamos o resultado abaixo:

	ID_Loja	Nome da Loja	Num_Funcionarios
▶	2	belo horizonte	13
	5	fortaleza	12
	8	goiânia	11

GABARITO EXPLICATIVO

4

Questão 4

O setor de CRM quer saber quais clientes fazem aniversário no mês atual para enviar promoções personalizadas.

Utilize a função MONTH na coluna Data_Nascimento para filtrar os clientes que atendem ao critério. **(Utilize a tabela clientes)**

Explicação e Resultado

Como queremos retornar somente **os clientes que fazem aniversário no mês atual**, devemos retornar as linhas da tabela **CLIENTES**, filtrando na cláusula **WHERE** a coluna **Data_Nascimento** aplicada como parâmetro da função **MONTH**, **comparando-a** com o mês da data atual (**MONTH(CURDATE())**). Se preferirmos, podemos filtrar somente algumas colunas da tabela **CLIENTES**, como, por exemplo, **Nome**, **Sobrenome** e **Data_Nascimento**:

```
SELECT
    Nome,
    Sobrenome,
    Data_Nascimento
FROM clientes
WHERE MONTH(Data_Nascimento) = MONTH(CURDATE());
```

Dessa forma, retornamos o resultado abaixo:

	Nome	Sobrenome	Data_Nascimento
▶	Chloe	Garcia	1997-11-22
	Jeremy	Powell	1970-11-17
	Grace	Butler	1993-11-22
	Felicia	Jimenez	1977-11-11
	Hunter	Davis	1977-11-20

OBS.: Como estamos filtrando pela **data atual** (CURDATE), **o seu resultado pode ser diferente** do apresentado ao lado, **dependendo do mês em que você estiver executando esta consulta.**

GABARITO EXPLICATIVO

5

Questão 5

A equipe de logística está revisando pedidos antigos e precisa identificar aqueles que foram realizados há mais de 30 dias a partir de uma determinada data.

Para esta análise, suponha que a data determinada seja 10/02/2019 e armazene-a em uma variável. Em seguida, filtre todos os pedidos cuja diferença entre a data da venda e a data determinada seja superior a 30 dias. **(Utilize a tabela pedidos)**

Explicação e Resultado

Para verificarmos **todos os pedidos que foram efetuados há mais de 30 dias a partir de 10/02/2019**, primeiramente, podemos armazenar essa data em uma variável (**@dataDeterminada**). Em seguida, consultamos a tabela **PEDIDOS**, filtrando-a com a cláusula **WHERE**. Nesse filtro, usamos a função **DATEDIFF** para calcular a **diferença** entre a **@dataDeterminada** e a **Data_Venda**, retornando apenas as linhas cujo **resultado** da DATEDIFF seja **> 30**. Se preferirmos, podemos retornar apenas as colunas **ID_Pedido** e **Data_Venda**, além de uma coluna que mostra **a quantidade de dias desde a venda**, formada pela função **DATEDIFF** que calcula a diferença entre a **@dataDeterminada** e a **Data_Venda**:

```
SET @dataDeterminada = '2019-02-10';

SELECT
    ID_Pedido,
    Data_Venda,
    DATEDIFF(@dataDeterminada, Data_Venda) AS 'Dias Desde a Venda'
FROM pedidos
WHERE DATEDIFF(@dataDeterminada, Data_Venda) > 30;
```

Result Grid | Filter Rows:

ID_Pedido	Data_Venda	Dias Desde a Venda
1	2019/01/01	40
2	2019/01/01	40
3	2019/01/01	40
4	2019/01/01	40

Result 6

Output

Action Output

#	Time	Action	Message
1	19:34:26	SET @dataDeterminada = '2019-02-10'	0 row(s) affected
2	19:34:26	SELECT ID_Pedido, Data_Venda, DATEDIFF(@dataDeterminada, Data...	602 row(s) returned

Assim, retornamos o resultado ao lado, com **602 pedidos**:

GABARITO EXPLICATIVO

6

Questão 6

A equipe de logística precisa criar identificadores temporários para os produtos em estoque, baseando-se na marca e nos 3 últimos caracteres do número de série.

Utilize as funções CONCAT (ou CONCAT_WS) e RIGHT para construir esses identificadores. **(Utilize a tabela produtos)**

	ID_Produto	Nome_Produto	Identificador Temporário
▶	1	Monitor LED 19,5" Full HD HDMI	DELL-923
	2	Monitor Curvo 24" 144Hz HDMI	SAMSUNG-939
	3	Webcam Full HD 1080p	LOGITECH-GG4
	4	Kit Teclado + Mouse sem fio Wireless	DELL-039
	5	Kit Teclado + Mouse Slim Bluetooth	DELL-924
	6	Cadeira Gamer reclinável Azul/Laranja	ALTURA-14J
	7	Cadeira Gamer PC Racer Vermelha	ALTURA-7FI
	8	Headphone Bluetooth 2000	SONY-821
	9	Fone de Ouvido Tune T5000	JBL-934
	10	Microfone Condensador MC1000	AKG-591
	11	Microfone Condensador com Tripé	BLUE-455
	12	Microfone de mesa com fio condensador	BLUE-411
	13	Notebook LC2100 Intel Core i5 8GB	SAMSUNG-457
	14	Notebook Inspiron 15 5000 4GB	DELL-012
	15	Notebook IdeaPad RF32000	DELL-64I
	16	Notebook Motion Ultra 2	SAMSUNG-39U

Explicação e Resultado

Para criarmos os **identificadores temporários dos produtos** concatenando a marca dos produtos e os 3 últimos caracteres de seus números de série, primeiramente podemos usar a função **RIGHT** para extrair esses **3 caracteres** da coluna **Num_Serie**, contando da direita para a esquerda. Dessa forma, podemos usar a função **CONCAT** para concatenar a coluna **Marca_Produto** com esses **3 caracteres extraídos pela função RIGHT**. Podemos usar **um traço como separador** das partes do identificador temporário, ou qualquer outro caractere de nossa preferência. Se quisermos, podemos retornar somente as colunas **ID_Produto** e **Nome_Produto** da tabela **PRODUTOS**, além da coluna **'Identificador Temporário'** que criamos acima:

```
SELECT
    ID_Produto,
    Nome_Produto,
    CONCAT(Marca_Produto, '-', RIGHT(Num_Serie, 3)) AS 'Identificador Temporário'
FROM produtos;
```

Podemos usar a função CONCAT_WS em vez da CONCAT, se preferirmos:

```
SELECT
    ID_Produto,
    Nome_Produto,
    CONCAT_WS('-', Marca_Produto, RIGHT(Num_Serie, 3)) AS 'Identificador Temporário'
FROM produtos;
```

De qualquer forma, retornamos o resultado ao lado:

Índice Geral

Índice deste Módulo

GABARITO EXPLICATIVO

7

Questão 7

A equipe de estoque está padronizando o cadastro dos produtos e solicitou que listemos todos os nomes de produtos em letras maiúsculas.

Use a função UCASE() para transformar os nomes. **(Utilize a tabela produtos)**

Explicação e Resultado

Para retornarmos **os nomes dos produtos com letras maiúsculas**, podemos fazer um **SELECT** à coluna **Nome_Produto** da tabela **PRODUTOS**, aplicando a função **UCASE** à essa coluna:

```
SELECT
    Nome_Produto,
    UCASE(Nome_Produto) AS 'Nome Maiúsculo'
FROM produtos;
```

Dessa forma, retornamos o resultado abaixo:

Nome_Produto	Nome Maiúsculo
Monitor LED 19,5" Full HD HDMI	MONITOR LED 19,5" FULL HD HDMI
Monitor Curvo 24" 144Hz HDMI	MONITOR CURVO 24" 144HZ HDMI
Webcam Full HD 1080p	WEBCAM FULL HD 1080P
Kit Teclado + Mouse sem fio Wireless	KIT TECLADO + MOUSE SEM FIO WIRELESS
Kit Teclado + Mouse Slim Bluetooth	KIT TECLADO + MOUSE SLIM BLUETOOTH
Cadeira Gamer reclinável Azul/Laranja	CADEIRA GAMER RECLINÁVEL AZUL/LARANJA
Cadeira Gamer PC Racer Vermelha	CADEIRA GAMER PC RACER VERMELHA
Headphone Bluetooth 2000	HEADPHONE BLUETOOTH 2000
Fone de Ouvido Tune T5000	FONE DE OUVIDO TUNE T5000
Microfone Condensador MC1000	MICROFONE CONDENSADOR MC1000
Microfone Condensador com Tripé	MICROFONE CONDENSADOR COM TRIPÉ
Microfone de mesa com fio condensador	MICROFONE DE MESA COM FIO CONDENSADOR
Notebook LC2100 Intel Core i5 8GB	NOTEBOOK LC2100 INTEL CORE I5 8GB
Notebook Inspiron 15 5000 4GB	NOTEBOOK INSPIRON 15 5000 4GB
Notebook IdeaPad RF32000	NOTEBOOK IDEAPAD RF32000
Notebook Motion Ultra 2	NOTEBOOK MOTION ULTRA 2

GABARITO EXPLICATIVO

8

Questão 8

A equipe de marketing deseja identificar clientes nascidos nos dias 1, 15 ou 30 de qualquer mês para uma ação especial.

Use a função DAY() para filtrar esses clientes. **(Utilize a tabela clientes)**

Explicação e Resultado

Para filtramos somente **os clientes nascidos nos dias 1, 15 ou 30 (de qualquer mês)**, podemos utilizar a cláusula **WHERE** para filtrar somente o dia da coluna Data_Nascimento (**DAY(Data_Nascimento)**), usando o operador **IN** para selecionar somente os dias solicitados **(1, 15, 30)**. Se quisermos, podemos retornar somente as colunas **Nome** e **Data_Nascimento** da tabela **CLIENTES**, além de uma coluna com o **'Dia do Nascimento'**:

```
SELECT
    Nome,
    Data_Nascimento,
    DAY(Data_Nascimento) AS 'Dia do Nascimento'
FROM clientes
WHERE DAY(Data_Nascimento) IN (1, 15, 30);
```

Dessa forma, retornamos o resultado ao lado:

	Nome	Data_Nascimento	Dia do Nascimento
▶	Janet	1985-12-01	1
	Jacquelyn	1984-02-01	1
	Curtis	1983-10-30	30
	Ian	1988-08-01	1
	Jordan	1998-09-15	15
	Daniel	1971-07-30	30
	Ana	2000-08-15	15
	Casey	1975-02-01	1
	Angela	1986-07-30	30
	Dalton	1977-03-30	30
	Cheryl	1987-05-01	1

GABARITO EXPLICATIVO

9

Questão 9

A equipe de marketing está planejando uma campanha direcionada para clientes nascidos na década de 90, visando oferecer produtos personalizados para essa faixa etária. Para isso, precisamos identificar todos os clientes que nasceram entre 1990 e 1999.

Use a função YEAR() para filtrar os clientes por essa faixa de anos e retorne o nome completo e a data de nascimento. **(Utilize a tabela clientes)**

Explicação e Resultado

Para filtramos somente **os clientes nascidos na década de 90**, podemos utilizar a cláusula **WHERE** para filtrar apenas o ano da coluna Data_Nascimento (**YEAR(Data_Nascimento)**), usando o operador **BETWEEN** para selecionar **o intervalo entre os anos 1990 e 1999**. Por fim, podemos retornar uma coluna com o **'Nome Completo'** dos clientes, **concatenando** as colunas **Nome** e **Sobrenome** da tabela **CLIENTES**, além da coluna **Data_Nascimento**:

```sql
SELECT
    CONCAT(Nome, ' ', Sobrenome) AS 'Nome Completo',
    Data_Nascimento
FROM clientes
WHERE YEAR(Data_Nascimento) BETWEEN 1990 AND 1999;
```

Dessa forma, retornamos o resultado abaixo:

Nome Completo	Data_Nascimento
Chloe Young	1999-02-22
Wyatt Hill	1999-04-23
Jordan King	1998-09-15
Destiny Wilson	1998-08-29
Ethan Zhang	1998-10-07
Seth Edwards	1998-10-06
Russell Xie	1998-09-12
Jennifer Russell	1998-12-13
Chloe Garcia	1997-11-22
Jesse Murphy	1997-07-27
Amanda Carter	1997-10-11
Megan Sanchez	1997-06-08
Nathan Simmons	1996-02-19
Noah Powell	1995-08-28
Angela Murphy	1995-04-02
Chase Reed	1995-12-02
Jessica Hender...	1993-10-04
Grace Butler	1993-11-22
Caleb Carter	1996-09-20
Gina Martin	1994-01-05

GABARITO EXPLICATIVO

10

Questão 10

A equipe de TI está preparando um novo sistema de envio de SMS para as campanhas promocionais e, para isso, precisa dos números de telefone dos clientes no formato simples, sem parênteses, traços ou espaços.

Como a tabela de clientes contém números no formato '(99) 99999-9999', para garantir que o sistema de mensagens funcionará corretamente, é necessário tratar esses telefones para que contenham somente os 11 dígitos.

Sua tarefa é remover os parênteses, o traço e o espaço dos números de telefone e verificar se o comprimento do número após a remoção tem exatamente 11 dígitos. Além disso, você deve retornar apenas os clientes que possuem telefone preenchido, ou seja, excluindo os valores nulos ou em branco. **(Utilize a tabela clientes)**

Explicação e Resultado

Para removermos **os parênteses, o traço e o espaço dos telefones** da coluna **Telefone** da tabela **CLIENTES**, podemos aninhar vários **REPLACEs**, um para cada caractere a ser **removido** (‘(’, ‘)’, ‘-’ e ‘ ’), **substituindo** tais caracteres por **vazio** (‘’). Para verificar se após a remoção os telefones passaram a ter apenas os **11 dígitos**, podemos aplicar a função **LENGTH** à esse padrão de **REPLACEs** que construímos na coluna acima. Para filtrarmos **somente as linhas cujos clientes informaram seus telefones**, podemos aplicar o filtro **WHERE Telefone IS NOT NULL AND Telefone <> ‘ ’ AND Telefone <> ‘’**. Dessa forma, garantimos que as linhas cujos campos da coluna **Telefone** são **nulos**, **brancos** e **vazios não serão retornadas**:

```
SELECT
    Nome AS 'Nome do Cliente',
    Telefone AS 'Telefone Original',
    REPLACE(REPLACE(REPLACE(REPLACE(Telefone, '(', ''), ')', ''), '-', ''), ' ', '') AS 'Telefone Padronizado',
    LENGTH(REPLACE(REPLACE(REPLACE(REPLACE(Telefone, '(', ''), ')', ''), '-', ''), ' ', '')) AS 'Comprimento do Telefone'
FROM clientes
WHERE Telefone IS NOT NULL AND Telefone <> ' ' AND Telefone <> '';
```

Assim, retornamos o resultado ao lado, com **93 clientes**:

Nome do Cliente	Telefone Original	Telefone Padronizado	Comprimento do Telefone
Ruben	(85) 94132-1176	85941321176	11
Christy	(41) 96270-6086	41962706086	11
Elizabeth	(31) 92039-5832	31920395832	11
Julio	(62) 93391-5891	62933915891	11
Janet	(21) 93379-3743	21933793743	11
Marco	(85) 99019-3803	85990193803	11

Result 14

Output

Action Output

#	Time	Action	Message
1	19:39:36	SELECT Nome AS 'Nome do Cliente', Telefone AS 'Telefone Original', RE...	93 row(s) returned

MÓDULO 9

JOINS

JOINS

JOINS

Índice Geral

Índice deste Módulo

O QUE VEREMOS NESTE MÓDULO

MÓDULO 9 JOINS

01	CHAVE PRIMÁRIA VS. CHAVE ESTRANGEIRA	209
02	TABELA FATO VS. TABELA DIMENSÃO	215
03	JOIN	217
04	INNER JOIN	225
05	INNER JOIN: ESTRUTURA	226
06	INNER JOIN (EXEMPLO 1)	227
07	INNER JOIN (EXEMPLO 2)	229
08	CUIDADOS AO FAZER OS JOINS	231
09	RENOMEANDO AS TABELAS COM O ALIAS	234
10	INNER JOIN + WHERE	235
11	INNER JOIN + GROUP BY	236
12	EXERCÍCIOS	238
13	GABARITO EXPLICATIVO	242

Chave Primária vs. Chave Estrangeira

Uma Chave Primária é uma coluna que identifica as informações distintas em uma tabela. Geralmente é uma coluna de ID. Toda tabela terá uma, e somente uma, chave primária. Essa chave é utilizada como identificador único da tabela, sendo representada por uma coluna que não receberá valores repetidos.

Chave Primária vs. Chave Estrangeira

Uma Chave Primária é uma coluna que identifica as informações distintas em uma tabela. Geralmente é uma coluna de ID. Toda tabela terá uma, e somente uma, chave primária. Essa chave é utilizada como identificador único da tabela, sendo representada por uma coluna que não receberá valores repetidos.

Como pode ser visto abaixo, a tabela **Produtos** possui uma coluna chamada ID_Produto, com valores que não se repetem. Essa será a **Chave Primária**.

Já na tabela de **Pedidos**, temos uma coluna chamada ID_Pedido, e essa também não se repete. Entendemos essa coluna também como uma chave primária.

produtos

ID_Produto	Nome_Produto	ID_Categoria	Marca_Produto	Num_Serie	Preco_Unit	Custo_Unit
1	Monitor LED 19,5" Full HD HDMI	1	DELL	MNT-DL-831923	2300	966
2	Monitor Curvo 24" 144Hz HDMI	1	SAMSUNG	MNT-SS-001939	2800	980
3	Webcam Full HD 1080p	1	LOGITECH	WBC-LT-934GG4	450	90
4	Kit Teclado + Mouse sem fio Wireless	2	DELL	KTM-DL-041039	350	129.5
5	Kit Teclado + Mouse Slim Bluetooth	2	DELL	KTM-DL-111924	280	109.2
6	Cadeira Gamer reclinável Azul/Laranja	3	ALTURA	CGM-AL-9N914J	1800	540
7	Cadeira Gamer PC Racer Vermelha	3	ALTURA	CGM-AL-0147FI	3100	1395
8	Headphone Bluetooth 2000	4	SONY	HDP-SN-194821	600	258
9	Fone de Ouvido Tune T5000	4	JBL	HDP-JB-091934	780	327.6
10	Microfone Condensador MC1000	5	AKG	MIC-AK-237591	1100	275
11	Microfone Condensador com Tripé	5	BLUE	MIC-BL-819455	800	344
12	Microfone de mesa com fio condensa...	5	BLUE	MIC-BL-761411	650	214.5
13	Notebook LC2100 Intel Core i5 8GB	6	SAMSUNG	NOT-SS-918457	3400	850
14	Notebook Inspiron 15 5000 4GB	6	DELL	NOT-DL-000012	3100	1209
15	Notebook IdeaPad RF32000	6	DELL	NOT-DL-77164I	4200	1176
16	Notebook Motion Ultra 2	6	SAMSUNG	NOT-SS-13139U	2900	1160

pedidos

ID_Pedido	Data_Venda	ID_Loja	ID_Produto	ID_Cliente	Qtd_Vendida	Preco_Unit	Custo_Unit	Receita_Venda	Custo_Venda
1	2019/01/01	6	11	45	1	800	344	800	344
2	2019/01/01	2	11	84	1	800	344	800	344
3	2019/01/01	1	14	12	1	3100	1209	3100	1209
4	2019/01/01	8	9	98	1	780	327.6	780	327.6
5	2019/01/01	5	8	21	1	600	258	600	258
6	2019/01/01	6	8	26	1	600	258	600	258
7	2019/01/01	6	13	90	1	3400	850	3400	850
8	2019/01/01	3	3	73	1	450	90	450	90
9	2019/01/01	5	15	17	1	4200	1176	4200	1176
10	2019/01/01	1	4	27	1	350	129.5	350	129.5
11	2019/01/01	5	9	25	1	780	327.6	780	327.6
12	2019/01/01	5	4	75	1	350	129.5	350	129.5
13	2019/01/01	5	11	48	1	800	344	800	344
14	2019/01/01	5	16	8	1	2900	1160	2900	1160
15	2019/01/01	5	9	41	1	780	327.6	780	327.6
16	2019/01/01	8	1	6	1	2300	966	2300	966
17	2019/01/01	6	14	77	1	3100	1209	3100	1209
18	2019/01/01	5	15	55	1	4200	1176	4200	1176
19	2019/01/01	5	3	65	1	450	90	450	90
20	2019/01/01	7	16	16	1	2900	1160	2900	1160
21	2019/01/01	5	11	55	1	800	344	800	344
22	2019/01/01	6	9	36	1	780	327.6	780	327.6
23	2019/01/01	3	4	42	1	350	129.5	350	129.5
24	2019/01/01	5	12	81	1	650	214.5	650	214.5
25	2019/01/01	7	1	77	1	2300	966	2300	966

Chave Primária vs. Chave Estrangeira

Uma Chave Primária é uma coluna que identifica as informações distintas em uma tabela. Geralmente é uma coluna de ID. Toda tabela terá uma, e somente uma, chave primária. Essa chave é utilizada como identificador único da tabela, sendo representada por uma coluna que não receberá valores repetidos.

Como pode ser visto abaixo, a tabela **Produtos** possui uma coluna chamada ID_Produto, com valores que não se repetem. Essa será a **Chave Primária**.

Já na tabela de **Pedidos**, temos uma coluna chamada ID_Pedido, e essa também não se repete. Entendemos essa coluna também como uma chave primária.

Chave Primária

produtos

ID_Produto	Nome_Produto	ID_Categoria	Marca_Produto	Num_Serie	Preco_Unit	Custo_Unit
1	Monitor LED 19,5" Full HD HDMI	1	DELL	MNT-DL-831923	2300	966
2	Monitor Curvo 24" 144Hz HDMI	1	SAMSUNG	MNT-SS-001939	2800	980
3	Webcam Full HD 1080p	1	LOGITECH	WBC-LT-934GG4	450	90
4	Kit Teclado + Mouse sem fio Wireless	2	DELL	KTM-DL-041039	350	129.5
5	Kit Teclado + Mouse Slim Bluetooth	2	DELL	KTM-DL-111924	280	109.2
6	Cadeira Gamer reclinável Azul/Laranja	3	ALTURA	CGM-AL-9N914J	1800	540
7	Cadeira Gamer PC Racer Vermelha	3	ALTURA	CGM-AL-0147FI	3100	1395
8	Headphone Bluetooth 2000	4	SONY	HDP-SN-194821	600	258
9	Fone de Ouvido Tune T5000	4	JBL	HDP-JB-091934	780	327.6
10	Microfone Condensador MC1000	5	AKG	MIC-AK-237591	1100	275
11	Microfone Condensador com Tripé	5	BLUE	MIC-BL-819455	800	344
12	Microfone de mesa com fio condensa...	5	BLUE	MIC-BL-761411	650	214.5
13	Notebook LC2100 Intel Core i5 8GB	6	SAMSUNG	NOT-SS-918457	3400	850
14	Notebook Inspiron 15 5000 4GB	6	DELL	NOT-DL-000012	3100	1209
15	Notebook IdeaPad RF32000	6	DELL	NOT-DL-77164I	4200	1176
16	Notebook Motion Ultra 2	6	SAMSUNG	NOT-SS-13139U	2900	1160

Chave Primária

pedidos

ID_Pedido	Data_Venda	ID_Loja	ID_Produto	ID_Cliente	Qtd_Vendida	Preco_Unit	Custo_Unit	Receita_Venda	Custo_Venda
1	2019/01/01	6	11	45	1	800	344	800	344
2	2019/01/01	2	11	84	1	800	344	800	344
3	2019/01/01	1	14	12	1	3100	1209	3100	1209
4	2019/01/01	8	9	98	1	780	327.6	780	327.6
5	2019/01/01	5	8	21	1	600	258	600	258
6	2019/01/01	6	8	26	1	600	258	600	258
7	2019/01/01	6	13	90	1	3400	850	3400	850
8	2019/01/01	3	3	73	1	450	90	450	90
9	2019/01/01	5	15	17	1	4200	1176	4200	1176
10	2019/01/01	1	4	27	1	350	129.5	350	129.5
11	2019/01/01	5	9	25	1	780	327.6	780	327.6
12	2019/01/01	5	4	75	1	350	129.5	350	129.5
13	2019/01/01	5	11	48	1	800	344	800	344
14	2019/01/01	5	16	8	1	2900	1160	2900	1160
15	2019/01/01	5	9	41	1	780	327.6	780	327.6
16	2019/01/01	8	1	6	1	2300	966	2300	966
17	2019/01/01	6	14	77	1	3100	1209	3100	1209
18	2019/01/01	5	15	55	1	4200	1176	4200	1176
19	2019/01/01	5	3	65	1	450	90	450	90
20	2019/01/01	7	16	16	1	2900	1160	2900	1160
21	2019/01/01	5	11	55	1	800	344	800	344
22	2019/01/01	6	9	36	1	780	327.6	780	327.6
23	2019/01/01	3	4	42	1	350	129.5	350	129.5
24	2019/01/01	5	12	81	1	650	214.5	650	214.5
25	2019/01/01	7	1	77	1	2300	966	2300	966

Chave Primária vs. Chave Estrangeira

Já uma Chave Estrangeira é uma coluna que permite relacionar as linhas de uma segunda tabela com a Chave Primária de uma primeira tabela.

produtos

ID_Produto	Nome_Produto	ID_Categoria	Marca_Produto	Num_Serie	Preco_Unit	Custo_Unit
1	Monitor LED 19,5" Full HD HDMI	1	DELL	MNT-DL-831923	2300	966
2	Monitor Curvo 24" 144Hz HDMI	1	SAMSUNG	MNT-SS-001939	2800	980
3	Webcam Full HD 1080p	1	LOGITECH	WBC-LT-934GG4	450	90
4	Kit Teclado + Mouse sem fio Wireless	2	DELL	KTM-DL-041039	350	129.5
5	Kit Teclado + Mouse Slim Bluetooth	2	DELL	KTM-DL-111924	280	109.2
6	Cadeira Gamer reclinável Azul/Laranja	3	ALTURA	CGM-AL-9N914J	1800	540
7	Cadeira Gamer PC Racer Vermelha	3	ALTURA	CGM-AL-0147FI	3100	1395
8	Headphone Bluetooth 2000	4	SONY	HDP-SN-194821	600	258
9	Fone de Ouvido Tune T5000	4	JBL	HDP-JB-091934	780	327.6
10	Microfone Condensador MC1000	5	AKG	MIC-AK-237591	1100	275
11	Microfone Condensador com Tripé	5	BLUE	MIC-BL-819455	800	344
12	Microfone de mesa com fio condensa...	5	BLUE	MIC-BL-761411	650	214.5
13	Notebook LC2100 Intel Core i5 8GB	6	SAMSUNG	NOT-SS-918457	3400	850
14	Notebook Inspiron 15 5000 4GB	6	DELL	NOT-DL-000012	3100	1209
15	Notebook IdeaPad RF32000	6	DELL	NOT-DL-77164I	4200	1176
16	Notebook Motion Ultra 2	6	SAMSUNG	NOT-SS-13139U	2900	1160

pedidos

ID_Pedido	Data_Venda	ID_Loja	ID_Produto	ID_Cliente	Qtd_Vendida	Preco_Unit	Custo_Unit	Receita_Venda	Custo_Venda
1	2019/01/01	6	11	45	1	800	344	800	344
2	2019/01/01	2	11	84	1	800	344	800	344
3	2019/01/01	1	14	12	1	3100	1209	3100	1209
4	2019/01/01	8	9	98	1	780	327.6	780	327.6
5	2019/01/01	5	8	21	1	600	258	600	258
6	2019/01/01	6	8	26	1	600	258	600	258
7	2019/01/01	6	13	90	1	3400	850	3400	850
8	2019/01/01	3	3	73	1	450	90	450	90
9	2019/01/01	5	15	17	1	4200	1176	4200	1176
10	2019/01/01	1	4	27	1	350	129.5	350	129.5
11	2019/01/01	5	9	25	1	780	327.6	780	327.6
12	2019/01/01	5	4	75	1	350	129.5	350	129.5
13	2019/01/01	5	11	48	1	800	344	800	344
14	2019/01/01	5	16	8	1	2900	1160	2900	1160
15	2019/01/01	5	9	41	1	780	327.6	780	327.6
16	2019/01/01	8	1	6	1	2300	966	2300	966
17	2019/01/01	6	14	77	1	3100	1209	3100	1209
18	2019/01/01	5	15	55	1	4200	1176	4200	1176
19	2019/01/01	5	3	65	1	450	90	450	90
20	2019/01/01	7	16	16	1	2900	1160	2900	1160
21	2019/01/01	5	11	55	1	800	344	800	344
22	2019/01/01	6	9	36	1	780	327.6	780	327.6
23	2019/01/01	3	4	42	1	350	129.5	350	129.5
24	2019/01/01	5	12	81	1	650	214.5	650	214.5
25	2019/01/01	7	1	77	1	2300	966	2300	966

Chave Primária vs. Chave Estrangeira

Já uma Chave Estrangeira é uma coluna que permite relacionar as linhas de uma segunda tabela com a Chave Primária de uma primeira tabela.

Como pode ser visto abaixo, a tabela **Produtos** possui uma coluna chamada ID_Produto, com valores que não se repetem. Essa será a **Chave Primária**. Já na tabela de **Pedidos**, a coluna de ID_Produto também aparece, mas os valores se repetem. Isso porque podemos ter mais de um pedido de um mesmo produto. Na tabela de **Pedidos**, a coluna de ID_Produto vai ser a **Chave Estrangeira** e vai permitir a gente relacionar os valores dessa coluna com a Chave Primária da tabela de **Produtos**.

produtos

ID_Produto	Nome_Produto	ID_Categoria	Marca_Produto	Num_Serie	Preco_Unit	Custo_Unit
1	Monitor LED 19,5" Full HD HDMI	1	DELL	MNT-DL-831923	2300	966
2	Monitor Curvo 24" 144Hz HDMI	1	SAMSUNG	MNT-SS-001939	2800	980
3	Webcam Full HD 1080p	1	LOGITECH	WBC-LT-934GG4	450	90
4	Kit Teclado + Mouse sem fio Wireless	2	DELL	KTM-DL-041039	350	129.5
5	Kit Teclado + Mouse Slim Bluetooth	2	DELL	KTM-DL-111924	280	109.2
6	Cadeira Gamer reclinável Azul/Laranja	3	ALTURA	CGM-AL-9N914J	1800	540
7	Cadeira Gamer PC Racer Vermelha	3	ALTURA	CGM-AL-0147FI	3100	1395
8	Headphone Bluetooth 2000	4	SONY	HDP-SN-194821	600	258
9	Fone de Ouvido Tune T5000	4	JBL	HDP-JB-091934	780	327.6
10	Microfone Condensador MC1000	5	AKG	MIC-AK-237591	1100	275
11	Microfone Condensador com Tripé	5	BLUE	MIC-BL-819455	800	344
12	Microfone de mesa com fio condensa...	5	BLUE	MIC-BL-761411	650	214.5
13	Notebook LC2100 Intel Core i5 8GB	6	SAMSUNG	NOT-SS-918457	3400	850
14	Notebook Inspiron 15 5000 4GB	6	DELL	NOT-DL-000012	3100	1209
15	Notebook IdeaPad RF32000	6	DELL	NOT-DL-77164I	4200	1176
16	Notebook Motion Ultra 2	6	SAMSUNG	NOT-SS-13139U	2900	1160

pedidos

ID_Pedido	Data_Venda	ID_Loja	ID_Produto	ID_Cliente	Qtd_Vendida	Preco_Unit	Custo_Unit	Receita_Venda	Custo_Venda
1	2019/01/01	6	11	45	1	800	344	800	344
2	2019/01/01	2	11	84	1	800	344	800	344
3	2019/01/01	1	14	12	1	3100	1209	3100	1209
4	2019/01/01	8	9	98	1	780	327.6	780	327.6
5	2019/01/01	5	8	21	1	600	258	600	258
6	2019/01/01	6	8	26	1	600	258	600	258
7	2019/01/01	6	13	90	1	3400	850	3400	850
8	2019/01/01	3	3	73	1	450	90	450	90
9	2019/01/01	5	15	17	1	4200	1176	4200	1176
10	2019/01/01	1	4	27	1	350	129.5	350	129.5
11	2019/01/01	5	9	25	1	780	327.6	780	327.6
12	2019/01/01	5	4	75	1	350	129.5	350	129.5
13	2019/01/01	5	11	48	1	800	344	800	344
14	2019/01/01	5	16	8	1	2900	1160	2900	1160
15	2019/01/01	5	9	41	1	780	327.6	780	327.6
16	2019/01/01	8	1	6	1	2300	966	2300	966
17	2019/01/01	6	14	77	1	3100	1209	3100	1209
18	2019/01/01	5	15	55	1	4200	1176	4200	1176
19	2019/01/01	5	3	65	1	450	90	450	90
20	2019/01/01	7	16	16	1	2900	1160	2900	1160
21	2019/01/01	5	11	55	1	800	344	800	344
22	2019/01/01	6	9	36	1	780	327.6	780	327.6
23	2019/01/01	3	4	42	1	350	129.5	350	129.5
24	2019/01/01	5	12	81	1	650	214.5	650	214.5
25	2019/01/01	7	1	77	1	2300	966	2300	966

Chave Primária vs. Chave Estrangeira

Já uma Chave Estrangeira é uma coluna que permite relacionar as linhas de uma segunda tabela com a Chave Primária de uma primeira tabela.

Como pode ser visto abaixo, a tabela **Produtos** possui uma coluna chamada ID_Produto, com valores que não se repetem. Essa será a **Chave Primária**. Já na tabela de **Pedidos**, a coluna de ID_Produto também aparece, mas os valores se repetem. Isso porque podemos ter mais de um pedido de um mesmo produto. Na tabela de **Pedidos**, a coluna de ID_Produto vai ser a **Chave Estrangeira** e vai permitir a gente relacionar os valores dessa coluna com a Chave Primária da tabela de **Produtos**.

Chave Primária

produtos

ID_Produto	Nome_Produto	ID_Categoria	Marca_Produto	Num_Serie	Preco_Unit	Custo_Unit
1	Monitor LED 19,5" Full HD HDMI	1	DELL	MNT-DL-831923	2300	966
2	Monitor Curvo 24" 144Hz HDMI	1	SAMSUNG	MNT-SS-001939	2800	980
3	Webcam Full HD 1080p	1	LOGITECH	WBC-LT-934GG4	450	90
4	Kit Teclado + Mouse sem fio Wireless	2	DELL	KTM-DL-041039	350	129.5
5	Kit Teclado + Mouse Slim Bluetooth	2	DELL	KTM-DL-111924	280	109.2
6	Cadeira Gamer reclinável Azul/Laranja	3	ALTURA	CGM-AL-9N914J	1800	540
7	Cadeira Gamer PC Racer Vermelha	3	ALTURA	CGM-AL-0147FI	3100	1395
8	Headphone Bluetooth 2000	4	SONY	HDP-SN-194821	600	258
9	Fone de Ouvido Tune T5000	4	JBL	HDP-JB-091934	780	327.6
10	Microfone Condensador MC1000	5	AKG	MIC-AK-237591	1100	275
11	Microfone Condensador com Tripé	5	BLUE	MIC-BL-819455	800	344
12	Microfone de mesa com fio condensa...	5	BLUE	MIC-BL-761411	650	214.5
13	Notebook LC2100 Intel Core i5 8GB	6	SAMSUNG	NOT-SS-918457	3400	850
14	Notebook Inspiron 15 5000 4GB	6	DELL	NOT-DL-000012	3100	1209
15	Notebook IdeaPad RF32000	6	DELL	NOT-DL-77164I	4200	1176
16	Notebook Motion Ultra 2	6	SAMSUNG	NOT-SS-13139U	2900	1160

Chave Estrangeira

pedidos

ID_Pedido	Data_Venda	ID_Loja	ID_Produto	ID_Cliente	Qtd_Vendida	Preco_Unit	Custo_Unit	Receita_Venda	Custo_Venda
1	2019/01/01	6	11	45	1	800	344	800	344
2	2019/01/01	2	11	84	1	800	344	800	344
3	2019/01/01	1	14	12	1	3100	1209	3100	1209
4	2019/01/01	8	9	98	1	780	327.6	780	327.6
5	2019/01/01	5	8	21	1	600	258	600	258
6	2019/01/01	6	8	26	1	600	258	600	258
7	2019/01/01	6	13	90	1	3400	850	3400	850
8	2019/01/01	3	3	73	1	450	90	450	90
9	2019/01/01	5	15	17	1	4200	1176	4200	1176
10	2019/01/01	1	4	27	1	350	129.5	350	129.5
11	2019/01/01	5	9	25	1	780	327.6	780	327.6
12	2019/01/01	5	4	75	1	350	129.5	350	129.5
13	2019/01/01	5	11	48	1	800	344	800	344
14	2019/01/01	5	16	8	1	2900	1160	2900	1160
15	2019/01/01	5	9	41	1	780	327.6	780	327.6
16	2019/01/01	8	1	6	1	2300	966	2300	966
17	2019/01/01	6	14	77	1	3100	1209	3100	1209
18	2019/01/01	5	15	55	1	4200	1176	4200	1176
19	2019/01/01	5	3	65	1	450	90	450	90
20	2019/01/01	7	16	16	1	2900	1160	2900	1160
21	2019/01/01	5	11	55	1	800	344	800	344
22	2019/01/01	6	9	36	1	780	327.6	780	327.6
23	2019/01/01	3	4	42	1	350	129.5	350	129.5
24	2019/01/01	5	12	81	1	650	214.5	650	214.5
25	2019/01/01	7	1	77	1	2300	966	2300	966

Tabela Fato vs. Tabela Dimensão

Uma Tabela Dimensão é uma tabela que contém características de um determinado elemento: lojas, produtos, funcionários, clientes, etc.

Nesta tabela, nenhum dos elementos principais irá se repetir. É onde vamos encontrar nossas **chaves primárias**.

Já uma Tabela Fato é uma tabela que vai registrar os fatos ou acontecimentos de uma empresa/negócio em determinados períodos de tempo (vendas, devoluções, aberturas de chamados, receitas, despesas, etc).

Geralmente é uma tabela com milhares de informações e composta essencialmente por colunas de ID usadas para buscar as informações complementares de uma tabela dimensão, conhecidas como **chaves estrangeiras**.

No exemplo ao lado, a FactSales é a nossa tabela Fato e a DimChannel é a nossa tabela Dimensão.

produtos

ID_Produto	Nome_Produto	ID_Categoria	Marca_Produto	Num_Serie	Preco_Unit	Custo_Unit
1	Monitor LED 19,5" Full HD HDMI	1	DELL	MNT-DL-831923	2300	966
2	Monitor Curvo 24" 144Hz HDMI	1	SAMSUNG	MNT-SS-001939	2800	980
3	Webcam Full HD 1080p	1	LOGITECH	WBC-LT-934GG4	450	90
4	Kit Teclado + Mouse sem fio Wireless	2	DELL	KTM-DL-041039	350	129.5
5	Kit Teclado + Mouse Slim Bluetooth	2	DELL	KTM-DL-111924	280	109.2
6	Cadeira Gamer reclinável Azul/Laranja	3	ALTURA	CGM-AL-9N914J	1800	540
7	Cadeira Gamer PC Racer Vermelha	3	ALTURA	CGM-AL-0147FI	3100	1395
8	Headphone Bluetooth 2000	4	SONY	HDP-SN-194821	600	258
9	Fone de Ouvido Tune T5000	4	JBL	HDP-JB-091934	780	327.6
10	Microfone Condensador MC1000	5	AKG	MIC-AK-237591	1100	275
11	Microfone Condensador com Tripé	5	BLUE	MIC-BL-819455	800	344
12	Microfone de mesa com fio condensa...	5	BLUE	MIC-BL-761411	650	214.5
13	Notebook LC2100 Intel Core i5 8GB	6	SAMSUNG	NOT-SS-918457	3400	850
14	Notebook Inspiron 15 5000 4GB	6	DELL	NOT-DL-000012	3100	1209
15	Notebook IdeaPad RF32000	6	DELL	NOT-DL-77164I	4200	1176
16	Notebook Motion Ultra 2	6	SAMSUNG	NOT-SS-13139U	2900	1160

Chave Primária

pedidos

Tabela Fato vs. Tabela Dimensão

Uma Tabela Dimensão é uma tabela que contém características de um determinado elemento: lojas, produtos, funcionários, clientes, etc.

Nesta tabela, nenhum dos elementos principais irá se repetir. É onde vamos encontrar nossas **chaves primárias**.

Já uma Tabela Fato é uma tabela que vai registrar os fatos ou acontecimentos de uma empresa/negócio em determinados períodos de tempo (vendas, devoluções, aberturas de chamados, receitas, despesas, etc).

Geralmente é uma tabela com milhares de informações e composta essencialmente por colunas de ID usadas para buscar as informações complementares de uma tabela dimensão, conhecidas como **chaves estrangeiras**.

No exemplo ao lado, a tabela Pedidos é a nossa tabela Fato e a Produtos é a nossa tabela Dimensão.

produtos

ID_Produto	Nome_Produto	ID_Categoria	Marca_Produto	Num_Serie	Preco_Unit	Custo_Unit
1	Monitor LED 19,5" Full HD HDMI	1	DELL	MNT-DL-831923	2300	966
2	Monitor Curvo 24" 144Hz HDMI	1	SAMSUNG	MNT-SS-001939	2800	980
3	Webcam Full HD 1080p	1	LOGITECH	WBC-LT-834004	450	90
4	Kit Teclado + Mouse sem fio Wireless	2	DELL	KTM-DL-041039	350	129.5
5	Kit Teclado + Mouse Slim Bluetooth	2	DELL	KTM-DL-111924	280	109.2
6	Cadeira Gamer reclinável Azul/Laranja	3	ALTURA	CGM-AL-990142	1800	540
7	Cadeira Gamer PC Racer Vermelha	3	ALTURA	CGM-AL-014PP1	2100	1395
8	Headphone Bluetooth 2000	4	SONY	HDP-SN-194821	600	258
9	Fone de Ouvido Tune T5000	4	JBL	HDP-JB-091934	780	327.6
10	Microfone Condensador MC1000	5	AKG	MIC-AK-237591	1100	275
11	Microfone Condensador com Tripé	5	BLUE	MIC-BL-819455	800	344
12	Microfone de mesa com fio condensa...	5	BLUE	MIC-BL-761411	650	214.5
13	Notebook LC2100 Intel Core i5 8GB	6	SAMSUNG	NOT-SS-918457	3400	850
14	Notebook Inspiron 15 5000 4GB	6	DELL	NOT-DL-000012	3100	1209
15	Notebook IdeaPad RF12000	6	DELL	NOT-DL-771645	4200	1176
16	Notebook Motion Ultra 2	6	SAMSUNG	NOT-SS-131392	2900	1160

Chave Estrangeira

pedidos

ID_Pedido	Data_Venda	ID_Loja	ID_Produto	ID_Cliente	Qtd_Vendida	Preco_Unit	Custo_Unit	Receita_Venda	Custo_Venda
1	2019/01/01	6	11	45	1	800	344	800	344
2	2019/01/01	2	11	84	1	800	344	800	344
3	2019/01/01	1	14	12	1	3100	1209	3100	1209
4	2019/01/01	8	9	98	1	780	327.6	780	327.6
5	2019/01/01	5	8	21	1	600	258	600	258
6	2019/01/01	6	8	26	1	600	258	600	258
7	2019/01/01	6	13	90	1	3400	850	3400	850
8	2019/01/01	3	3	73	1	450	90	450	90
9	2019/01/01	5	15	17	1	4200	1176	4200	1176
10	2019/01/01	1	4	27	1	350	129.5	350	129.5
11	2019/01/01	5	9	25	1	780	327.6	780	327.6
12	2019/01/01	5	4	75	1	350	129.5	350	129.5
13	2019/01/01	5	11	48	1	800	344	800	344
14	2019/01/01	5	16	8	1	2900	1160	2900	1160
15	2019/01/01	5	9	41	1	780	327.6	780	327.6
16	2019/01/01	8	1	6	1	2300	966	2300	966
17	2019/01/01	6	14	77	1	3100	1209	3100	1209
18	2019/01/01	5	15	55	1	4200	1176	4200	1176
19	2019/01/01	5	3	65	1	450	90	450	90
20	2019/01/01	7	16	16	1	2900	1160	2900	1160
21	2019/01/01	5	11	55	1	800	344	800	344
22	2019/01/01	6	9	36	1	780	327.6	780	327.6
23	2019/01/01	3	4	42	1	350	129.5	350	129.5
24	2019/01/01	5	12	81	1	650	214.5	650	214.5
25	2019/01/01	7	1	77	1	2300	966	2300	966

JOIN

Nas aulas anteriores vimos que existem dois tipos de tabelas: Dimensão e Fato. Essas tabelas podem ser relacionadas através de uma coluna: na tabela Dimensão, identificamos a chave Primária, que será relacionada com a chave Estrangeira da tabela Fato.

Nas aulas anteriores vimos que existem dois tipos de tabelas: Dimensão e Fato. Essas tabelas podem ser relacionadas através de uma coluna: na tabela Dimensão, identificamos a chave Primária, que será relacionada com a chave Estrangeira da tabela Fato.

Chave Primária

produtos

ID_Produto	Nome_Produto	ID_Categoria	Marca_Produto	Num_Serie	Preco_Unit	Custo_Unit
1	Monitor LED 19,5" Full HD HDMI	1	DELL	MNT-DL-831923	2300	966
2	Monitor Curvo 24" 144Hz HDMI	1	SAMSUNG	MNT-SS-001939	2800	980
3	Webcam Full HD 1080p	1	LOGITECH	WBC-LT-934GG4	450	90
4	Kit Teclado + Mouse sem fio Wireless	2	DELL	KTM-DL-041039	350	129.5
5	Kit Teclado + Mouse Slim Bluetooth	2	DELL	KTM-DL-111924	280	109.2
6	Cadeira Gamer reclinável Azul/Laranja	3	ALTURA	CGM-AL-9N914J	1800	540
7	Cadeira Gamer PC Racer Vermelha	3	ALTURA	CGM-AL-0147FI	3100	1395
8	Headphone Bluetooth 2000	4	SONY	HDP-SN-194821	600	258
9	Fone de Ouvido Tune T5000	4	JBL	HDP-JB-091934	780	327.6
10	Microfone Condensador MC1000	5	AKG	MIC-AK-237591	1100	275
11	Microfone Condensador com Tripé	5	BLUE	MIC-BL-819455	800	344
12	Microfone de mesa com fio condensa...	5	BLUE	MIC-BL-761411	650	214.5
13	Notebook LC2100 Intel Core i5 8GB	6	SAMSUNG	NOT-SS-918457	3400	850
14	Notebook Inspiron 15 5000 4GB	6	DELL	NOT-DL-000012	3100	1209
15	Notebook IdeaPad RF32000	6	DELL	NOT-DL-77164I	4200	1176
16	Notebook Motion Ultra 2	6	SAMSUNG	NOT-SS-13139U	2900	1160

Tabela Dimensão

Chave Estrangeira

pedidos

ID_Pedido	Data_Venda	ID_Loja	ID_Produto	ID_Cliente	Qtd_Vendida	Preco_Unit	Custo_Unit	Receita_Venda	Custo_Venda
1	2019/01/01	6	11	45	1	800	344	800	344
2	2019/01/01	2	11	84	1	800	344	800	344
3	2019/01/01	1	14	12	1	3100	1209	3100	1209
4	2019/01/01	8	9	98	1	780	327.6	780	327.6
5	2019/01/01	5	8	21	1	600	258	600	258
6	2019/01/01	6	8	26	1	600	258	600	258
7	2019/01/01	6	13	90	1	3400	850	3400	850
8	2019/01/01	3	3	73	1	450	90	450	90
9	2019/01/01	5	15	17	1	4200	1176	4200	1176
10	2019/01/01	1	4	27	1	350	129.5	350	129.5
11	2019/01/01	5	9	25	1	780	327.6	780	327.6
12	2019/01/01	5	4	75	1	350	129.5	350	129.5
13	2019/01/01	5	11	48	1	800	344	800	344
14	2019/01/01	5	16	8	1	2900	1160	2900	1160
15	2019/01/01	5	9	41	1	780	327.6	780	327.6
16	2019/01/01	8	1	6	1	2300	966	2300	966
17	2019/01/01	6	14	77	1	3100	1209	3100	1209
18	2019/01/01	5	15	55	1	4200	1176	4200	1176
19	2019/01/01	5	3	65	1	450	90	450	90
20	2019/01/01	7	16	16	1	2900	1160	2900	1160
21	2019/01/01	5	11	55	1	800	344	800	344
22	2019/01/01	6	9	36	1	780	327.6	780	327.6
23	2019/01/01	3	4	42	1	350	129.5	350	129.5
24	2019/01/01	5	12	81	1	650	214.5	650	214.5
25	2019/01/01	7	1	77	1	2300	966	2300	966

Tabela Fato

JOIN

Nas aulas anteriores vimos que existem dois tipos de tabelas: Dimensão e Fato. Essas tabelas podem ser relacionadas através de uma coluna: na tabela Dimensão, identificamos a chave Primária, que será relacionada com a chave Estrangeira da tabela Fato.

Mas pra que servem essas relações?

Com essas relações, conseguimos utilizar informações de uma tabela em outra tabela. Será muito útil, por exemplo, pra gente descobrir o nome do produto vendido (na tabela de Pedidos) fazendo essa busca lá na tabela de Produtos.

JOIN

Nas aulas anteriores vimos que existem dois tipos de tabelas: Dimensão e Fato. Essas tabelas podem ser relacionadas através de uma coluna: na tabela Dimensão, identificamos a chave Primária, que será relacionada com a chave Estrangeira da tabela Fato.

Mas pra que servem essas relações?

Com essas relações, conseguimos utilizar informações de uma tabela em outra tabela. Será muito útil, por exemplo, pra gente descobrir o nome do produto vendido (na tabela de Pedidos) fazendo essa busca lá na tabela de Produtos.

pedidos

ID_Pedido	Data_Venda	ID_Loja	ID_Produto	ID_Cliente	Qtd_Vendida	Preco_Unit	Custo_Unit	Receita_Venda	Custo_Venda
1	2019/01/01	6	11	45	1	800	344	800	344

JOIN

Nas aulas anteriores vimos que existem dois tipos de tabelas: Dimensão e Fato. Essas tabelas podem ser relacionadas através de uma coluna: na tabela Dimensão, identificamos a chave Primária, que será relacionada com a chave Estrangeira da tabela Fato.

Mas pra que servem essas relações?

Com essas relações, conseguimos utilizar informações de uma tabela em outra tabela. Será muito útil, por exemplo, pra gente descobrir o nome do produto vendido (na tabela de Pedidos) fazendo essa busca lá na tabela de Produtos.

produtos

ID_Produto	Nome_Produto	ID_Categoria	Marca_Produto	Num_Serie	Preco_Unit	Custo_Unit
11	Microfone Condensador com Tripé	5	BLUE	MIC-BL-819455	800	344

pedidos

ID_Pedido	Data_Venda	ID_Loja	ID_Produto	ID_Cliente	Qtd_Vendida	Preco_Unit	Custo_Unit	Receita_Venda	Custo_Venda
1	2019/01/01	6	11	45	1	800	344	800	344

JOIN

Nas aulas anteriores vimos que existem dois tipos de tabelas: Dimensão e Fato. Essas tabelas podem ser relacionadas através de uma coluna: na tabela Dimensão, identificamos a chave Primária, que será relacionada com a chave Estrangeira da tabela Fato.

Mas pra que servem essas relações?

Com essas relações, conseguimos utilizar informações de uma tabela em outra tabela. Será muito útil, por exemplo, pra gente descobrir o nome do produto vendido (na tabela de Pedidos) fazendo essa busca lá na tabela de Produtos.

Essas relações serão criadas por meio do que chamamos de JOIN's. A tradução literal dessa palavra é "juntar", "unir". Os JOINs vão nos permitir fazer exatamente isso: juntar as nossas tabelas Fato e Dimensão, de forma a complementar as informações umas das outras.

JOIN

Nas aulas anteriores vimos que existem dois tipos de tabelas: Dimensão e Fato. Essas tabelas podem ser relacionadas através de uma coluna: na tabela Dimensão, identificamos a chave Primária, que será relacionada com a chave Estrangeira da tabela Fato.

Mas pra que servem essas relações?

Com essas relações, conseguimos utilizar informações de uma tabela em outra tabela. Será muito útil, por exemplo, pra gente descobrir o nome do produto vendido (na tabela de Pedidos) fazendo essa busca lá na tabela de Produtos.

Essas relações serão criadas por meio do que chamamos de JOIN's. A tradução literal dessa palavra é "juntar", "unir". Os JOINs vão nos permitir fazer exatamente isso: juntar as nossas tabelas Fato e Dimensão, de forma a complementar as informações umas das outras.

Existem diversos tipos de Joins no SQL, os quais estão listados abaixo.

- LEFT JOIN
- RIGHT JOIN
- INNER JOIN
- FULL JOIN

JOIN

Nas aulas anteriores vimos que existem dois tipos de tabelas: Dimensão e Fato. Essas tabelas podem ser relacionadas através de uma coluna: na tabela Dimensão, identificamos a chave Primária, que será relacionada com a chave Estrangeira da tabela Fato.

Mas pra que servem essas relações?

Com essas relações, conseguimos utilizar informações de uma tabela em outra tabela. Será muito útil, por exemplo, pra gente descobrir o nome do produto vendido (na tabela de Pedidos) fazendo essa busca lá na tabela de Produtos.

Essas relações serão criadas por meio do que chamamos de JOIN's. A tradução literal dessa palavra é "juntar", "unir". Os JOINs vão nos permitir fazer exatamente isso: juntar as nossas tabelas Fato e Dimensão, de forma a complementar as informações umas das outras.

Existem diversos tipos de Joins no SQL, os quais estão listados abaixo.

LEFT JOIN | RIGHT JOIN | **INNER JOIN** | FULL JOIN

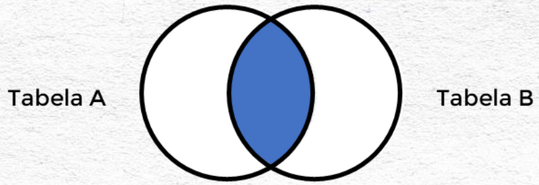
Vamos começar falando sobre o INNER JOIN. Se você desejar se aprofundar nos JOINs, recomendo que assista ao módulo de JOIN do curso de SQL Server.

INNER JOIN

pedidos

ID_Produto	Data_Venda	Qtd_Vendida	Receita_Venda
3	2019/01/01	1	450
4	2019/01/01	1	350
4	2019/01/01	1	350
1	2019/01/01	1	2300
3	2019/01/01	1	450
4	2019/01/01	1	350
1	2019/01/01	1	2300
2	2019/01/01	1	2800
4	2019/01/01	1	350
1	2019/01/01	1	2300

Apenas a interseção entre Tabelas A e B



produtos

ID_Produto	Nome_Produto	Marca_Produto	Preco_Unit
1	Monitor LED 19,5" Full HD HDMI	DELL	2300
2	Monitor Curvo 24" 144Hz HDMI	SAMSUNG	2800
3	Webcam Full HD 1080p	LOGITECH	450
4	Kit Teclado + Mouse sem fio Wireless	DELL	350

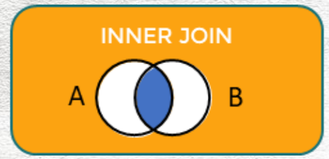
Resultado

ID_Produto	Data_Venda	Qtd_Vendida	Receita_Venda	Nome_Produto	Marca_Produto	Preco_Unit
3	2019/01/01	1	450	Webcam Full HD 1080p	LOGITECH	450
4	2019/01/01	1	350	Kit Teclado + Mouse sem fio Wireless	DELL	350
4	2019/01/01	1	350	Kit Teclado + Mouse sem fio Wireless	DELL	350
1	2019/01/01	1	2300	Monitor LED 19,5" Full HD HDMI	DELL	2300
3	2019/01/01	1	450	Webcam Full HD 1080p	LOGITECH	450
4	2019/01/01	1	350	Kit Teclado + Mouse sem fio Wireless	DELL	350
1	2019/01/01	1	2300	Monitor LED 19,5" Full HD HDMI	DELL	2300
2	2019/01/01	1	2800	Monitor Curvo 24" 144Hz HDMI	SAMSUNG	2800
4	2019/01/01	1	350	Kit Teclado + Mouse sem fio Wireless	DELL	350
1	2019/01/01	1	2300	Monitor LED 19,5" Full HD HDMI	DELL	2300

O INNER JOIN vai nos permitir relacionar duas tabelas (Tabela A e Tabela B) e criar uma nova tabela (Tabela C) que é a junção das duas.

A tabela resultante desse JOIN terá **apenas** as linhas que são a interseção entre a Tabela A e a Tabela B.

INNER JOIN: Estrutura




```
SELECT
    Tabela_A.coluna1,
    Tabela_A.coluna2,
    Tabela_A.coluna3,
    Tabela_B.coluna4
FROM
    Tabela_A
INNER JOIN Tabela_B
    ON Tabela_A.id_chave_estrangeira = Tabela_B.id_chave_primaria
```

A Tabela_A será a tabela que vamos querer complementar com as informações da Tabela_B.

INNER JOIN (Exemplo 1)

Agora vamos ver na prática como se dá a relação entre as tabelas do banco de dados do curso.

Vamos começar relacionando as tabelas produtos e pedidos.

Isso só é possível pois essas duas tabelas possuem uma coluna em comum: a coluna ID_Produto.

```
104     -- Exemplo 1. Relacione as tabelas de Produtos e Pedidos.
105
106 •   SELECT * FROM produtos;
107 •   SELECT * FROM pedidos;
108
```

ID_Produto	Nome_Produto	ID_Categoria	Marca_Produto	Num_Serie	Preco_Unit	Custo_Unit
1	Monitor LED 19,5" Full HD HDMI	1	DELL	MNT-DL-831923	2300	966
2	Monitor Curvo 24" 144Hz HDMI	1	SAMSUNG	MNT-SS-001939	2800	980
3	Webcam Full HD 1080p	1	LOGITECH	WBC-LT-934GG4	450	90
4	Kit Teclado + Mouse sem fio Wireless	2	DELL	KTM-DL-041039	350	129.5
5	Kit Teclado + Mouse Slim Bluetooth	2	DELL	KTM-DL-111924	280	109.2
6						
7						
8						
9						
10						
11						
12						
13						
14						
15						

ID_Pedido	Data_Venda	ID_Loja	ID_Produto	ID_Cliente	Qtd_Vendida	Receita_Venda	Custo_Venda	Custo_Unit	Preco_Unit
1	2019-01-01	2	4	16	1	350	129.5	129.5	350
2	2019-01-01	1	4	17	1	350	129.5	129.5	350
3	2019-01-01	3	4	23	1	350	129.5	129.5	350
4	2019-01-01	8	4	24	1	350	129.5	129.5	350
5	2019-01-01	4	4	25	1	350	129.5	129.5	350
6	2019-01-01	4	4	26	1	350	129.5	129.5	350
7	2019-01-01	2	6	14	1	1800	540	540	1800
8	2019-01-01	8	6	15	1	1800	540	540	1800
9	2019-01-01	7	6	38	1	1800	540	540	1800
10	2019-01-02	3	3	27	1	450	90	90	450
11	2019-01-02	4	3	28	1	450	90	90	450
12	2019-01-02	8	3	29	1	450	90	90	450
13	2019-01-02	2	3	30	1	450	90	90	450
14	2019-01-02	4	3	31	1	450	90	90	450
15	2019-01-02	7	3	32	1	450	90	90	450

INNER JOIN (Exemplo 1)

Seguindo o que foi explicado no início do módulo, podemos criar a relação entre as duas tabelas através da coluna ID_Produto conforme mostrado na imagem ao lado.

Lembrando que a coluna ID_Produto é uma chave primária da tabela Produtos, e uma chave estrangeira da tabela Pedidos.

Além disso, temos como tabela A a nossa tabela fato de Pedidos, enquanto a tabela B, dimensão, será a tabela de Produtos.

Observe agora que conseguimos visualizar na tabela de pedidos as colunas provenientes da tabela de produtos.

```
109 •  SELECT
110        pedidos.ID_Pedido,
111        pedidos.Data_Venda,
112        pedidos.ID_Produto,
113        pedidos.Qtd_Vendida,
114        pedidos.Receita_Venda,
115        produtos.Nome_Produto,
116        produtos.Marca_Produto
117    FROM pedidos
118    INNER JOIN produtos
119        ON pedidos.ID_Produto = produtos.ID_Produto;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ID_Pedido	Data_Venda	ID_Produto	Qtd_Vendida	Receita_Venda	Nome_Produto	Marca_Produto
1	2019-01-01	4	1	350	Kit Teclado + Mouse sem fio Wireless	DELL
2	2019-01-01	4	1	350	Kit Teclado + Mouse sem fio Wireless	DELL
3	2019-01-01	4	1	350	Kit Teclado + Mouse sem fio Wireless	DELL
4	2019-01-01	4	1	350	Kit Teclado + Mouse sem fio Wireless	DELL
5	2019-01-01	4	1	350	Kit Teclado + Mouse sem fio Wireless	DELL
6	2019-01-01	4	1	350	Kit Teclado + Mouse sem fio Wireless	DELL
7	2019-01-01	6	1	1800	Cadeira Gamer reclinável Azul/Laranja	ALTURA
8	2019-01-01	6	1	1800	Cadeira Gamer reclinável Azul/Laranja	ALTURA
9	2019-01-01	6	1	1800	Cadeira Gamer reclinável Azul/Laranja	ALTURA
10	2019-01-02	3	1	450	Webcam Full HD 1080p	LOGITECH
11	2019-01-02	3	1	450	Webcam Full HD 1080p	LOGITECH

INNER JOIN (Exemplo 2)

Agora vamos relacionar as tabelas Clientes e Pedidos. Sabemos que as duas tabelas possuem uma coluna em comum: a coluna ID_Cliente.

Através dela, será possível criar a relação entre as tabelas.

```
104     -- Exemplo 1. Relacione as tabelas de Produtos e Pedidos.
105
106 •   SELECT * FROM produtos;
107 •   SELECT * FROM pedidos;
108
```

ID_Cliente	Nome	Sobrenome	Data_Nascimento	Estado_Civil	Sexo	Email	Telefone	Renda_Anual	Qtd_Filhos	Escolaridade
1	Ruben	[illegible]	1985-08-07	C	M	ruben35@hotmail.com	(85) 94132-1176	60000	3	Pós-graduado
2	Christy	Zhu	1988-02-10	S	F	christy12@hotmail.com	(41) 96270-6086	70000	0	Pós-graduado
3	Elizabeth	Johnson	19[illegible]-08-03	S	F	elizabeth5@gmail.com	(31) 92039-5832	80000	5	Pós-graduado
4	Julio	Ruiz	1985-07-31	S	M	julio1@hotmail.com	(62) 93391-5891	70000	0	Pós-graduado
5	Janet	Alvarez	1985-12-01	S	F	janet9@gmail.com	(21) 93379-3743	70000	0	Pós-graduado
6	Marco	Mehta	1984-05-04	C	M	marco14@yahoo.com.br	(85) 99019-3803	60000	3	Pós-graduado
7	Rob	Verhoff	1984-07-02	S	[illegible]	rob4@hotmail.com	(11) 98162-4760	60000	4	Pós-graduado
8	Shannon	[illegible]								
9	Jacquelyn	[illegible]								
10	Curtis	[illegible]								
11	Lauren	[illegible]								

ID_Pedido	Data_Venda	ID_Loja	ID_Produto	ID_Cliente	Qtd_Vendida	Receita_Venda	Custo_Venda	Custo_Unit	Preco_Unit
1	2019-01-01	2	4	16	1	350	129.5	129.5	350
2	2019-01-01	1	4	17	1	350	129.5	129.5	350
3	2019-01-01	3	4	23	1	350	129.5	129.5	350
4	2019-01-01	8	4	24	1	350	129.5	129.5	350
5	2019-01-01	4	4	25	1	350	129.5	129.5	350
6	2019-01-01	4	4	26	1	350	129.5	129.5	350
7	2019-01-01	2	6	14	1	1800	540	540	1800
8	2019-01-01	8	6	15	1	1800	540	540	1800
9	2019-01-01	7	6	38	1	1800	540	540	1800
10	2019-01-02	3	3	27	1	450	90	90	450
11	2019-01-02	4	3	28	1	450	90	90	450
12	2019-01-02	8	3	29	1	450	90	90	450
13	2019-01-02	2	3	30	1	450	90	90	450
14	2019-01-02	4	3	31	1	450	90	90	450
15	2019-01-02	7	3	32	1	450	90	90	450

INNER JOIN (Exemplo 2)

Seguindo a mesma lógica do Exemplo 1, o código final que permite relacionar as tabelas de Clientes e Pedidos é mostrada ao lado.

```
127 •  SELECT
128        pedidos.ID_Pedido,
129        pedidos.Data_Venda,
130        pedidos.ID_Cliente,
131        pedidos.Qtd_Vendida,
132        clientes.Nome,
133        clientes.Email,
134        clientes.Telefone
135    FROM pedidos
136    INNER JOIN clientes
137        ON pedidos.ID_Cliente = clientes.ID_Cliente;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ID_Pedido	Data_Venda	ID_Cliente	Qtd_Vendida	Nome	Email	Telefone
1	2019-01-01	16	1	Shannon	shannon1@hotmail.com	(71) 94496-1712
2	2019-01-01	17	1	Clarence	clarence32@yahoo.com.br	(21) 98923-7805
3	2019-01-01	23	1	Alejandro	alejandro45@gmail.com	(85) 94492-9582
4	2019-01-01	24	1	Harold	harold3@hotmail.com	(31) 95052-6286
5	2019-01-01	25	1	Jessie	jessie16@hotmail.com	(41) 98751-8234
6	2019-01-01	26	1	Jill	jill13@gmail.com	(41) 97120-5603
7	2019-01-01	14	1	Chloe	chloe23@hotmail.com	(71) 96890-7150
8	2019-01-01	15	1	Wyatt	wyatt32@gmail.com	(71) 98644-2695
9	2019-01-01	38	1	Jesse	jesse15@yahoo.com.br	(62) 99674-9226
10	2019-01-02	27	1	Jimmy	jimmy9@yahoo.com.br	(85) 92931-9277
11	2019-01-02	28	1	Bethany	bethany10@hotmail.com	(62) 92998-6359

Result 35

Cuidados ao fazer os Joins

Será que somos obrigados a incluir o nome das tabelas antes das colunas?

Observe o código da situação 1 ao lado. Selecionamos as colunas:

- ID_Pedido
- Data_Venda
- Qtd_Vendida
- Nome
- Email
- Telefone

Como pode ser visto, não informamos os nomes das tabelas e não tivemos nenhum problema. Mas será que é sempre assim?

1

```
127 •  SELECT
128        ID_Pedido,
129        Data_Venda,
130        Qtd_Vendida,
131        Nome,
132        Email,
133        Telefone
134    FROM pedidos
135    INNER JOIN clientes
136        ON pedidos.ID_Cliente = clientes.ID_Cliente;
137
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ID_Pedido	Data_Venda	Qtd_Vendida	Nome	Email	Telefone
1	2019-01-01	1	Shannon	shannon1@hotmail.com	(71) 94496-1712
2	2019-01-01	1	Clarence	clarence32@yahoo.com.br	(21) 98923-7805
3	2019-01-01	1	Alejandro	alejandro45@gmail.com	(85) 94492-9582
4	2019-01-01	1	Harold	harold3@hotmail.com	(31) 95052-6286
5	2019-01-01	1	Jessie	jessie16@hotmail.com	(41) 98751-8234
6	2019-01-01	1	Jill	jill13@gmail.com	(41) 97120-5603

Result 36

Cuidados ao fazer os Joins

Nessa nova situação, observe que incluímos a coluna ID_Cliente.

Ao executar o código, obtivemos o erro abaixo:

“Column ‘ID_Cliente’ in field list is ambiguous”

Mas o que significa esse erro?

Traduzindo, ele nos diz que o nome ID_Cliente é ambíguo. Isso porque essa coluna existe tanto na tabela pedidos quanto na tabela clientes. Dessa forma, o SQL não sabe de qual dessas duas tabelas selecionar a coluna ID_Cliente.

2

```
127 •  SELECT
128        ID_Pedido,
129        Data_Venda,
130        ID_Cliente,
131        Qtd_Vendida,
132        Nome,
133        Email,
134        Telefone
135    FROM pedidos
136    INNER JOIN clientes
137        ON pedidos.ID_Cliente = clientes.ID_Cliente;
```

```
51  19:11:36  SELECT ID_Pedido,  Data_Venda,  ID_Cliente,  Qtd_Vendida,  Nome,  Email, ...  Error Code: 1052. Column 'ID_Cliente' in field list is ambiguous
```

Cuidados ao fazer os Joins

Para evitar este problema, podemos simplesmente colocar o nome da tabela antes de cada coluna, como pode ser visto ao lado.

Na situação 1, não tivemos problema porque nenhuma das colunas selecionadas existia nas duas tabelas (pedidos e clientes). Por isso não tivemos nenhum erro, pois não havia nenhuma coluna com nome ambíguo.

Já na situação 2, a coluna ID_Cliente existia tanto na tabela pedidos quanto na clientes, por isso o SQL não soube exatamente em qual das duas tabelas buscar, e deu esse erro.

A partir do momento que utilizamos o nome das tabelas, fica mais claro para o próprio SQL.

Portanto, o nome das tabelas não é obrigatório, mas acaba sendo o mais garantido a se fazer.

```
127 •  SELECT
128        pedidos.ID_Pedido,
129        pedidos.Data_Venda,
130        pedidos.Qtd_Vendida,
131        clientes.ID_Cliente,
132        clientes.Nome,
133        clientes.Email,
134        clientes.Telefone
135    FROM pedidos
136    INNER JOIN clientes
137        ON pedidos.ID_Cliente = clientes.ID_Cliente;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ID_Pedido	Data_Venda	Qtd_Vendida	ID_Cliente	Nome	Email	Telefone
1	2019-01-01	1	16	Shannon	shannon1@hotmail.com	(71) 94496-1712
2	2019-01-01	1	17	Clarence	clarence32@yahoo.com.br	(21) 98923-7805
3	2019-01-01	1	23	Alejandro	alejandro45@gmail.com	(85) 94492-9582
4	2019-01-01	1	24	Harold	harold3@hotmail.com	(31) 95052-6286
5	2019-01-01	1	25	Jessie	jessie16@hotmail.com	(41) 98751-8234
6	2019-01-01	1	26	Jill	jill13@gmail.com	(41) 97120-5603

Result 37

Renomeando as tabelas com o ALIAS

Podemos dar um apelido para as tabelas com o objetivo de deixar o nosso código mais enxuto.

Observe no código ao lado. Utilizamos o AS para dar o nome ‘p’ à tabela pedidos e ‘c’ à tabela clientes. Em seguida, usamos esses novos nomes na hora de selecionar as colunas e também na hora de realizar o Join.

Em termos de resultado, temos exatamente a mesma coisa. A única diferença é que com o ALIAS de tabelas, nosso código fica bem mais resumido.

```
127 •  SELECT
128        p.ID_Pedido,
129        p.Data_Venda,
130        p.Qtd_Vendida,
131        c.ID_Cliente,
132        c.Nome,
133        c.Email,
134        c.Telefone
135    FROM pedidos AS p
136    INNER JOIN clientes AS c
137        ON p.ID_Cliente = c.ID_Cliente;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ID_Pedido	Data_Venda	Qtd_Vendida	ID_Cliente	Nome	Email	Telefone
1	2019-01-01	1	16	Shannon	shannon1@hotmail.com	(71) 94496-1712
2	2019-01-01	1	17	Clarence	clarence32@yahoo.com.br	(21) 98923-7805
3	2019-01-01	1	23	Alejandro	alejandro45@gmail.com	(85) 94492-9582
4	2019-01-01	1	24	Harold	harold3@hotmail.com	(31) 95052-6286
5	2019-01-01	1	25	Jessie	jessie16@hotmail.com	(41) 98751-8234
6	2019-01-01	1	26	Jill	jill13@gmail.com	(41) 97120-5603

Result 39

INNER JOIN + WHERE

Os Joins podem ser combinados com o WHERE para filtrar a tabela.

No exemplo ao lado, incluímos no SELECT a coluna de sexo e fizemos um filtro WHERE para mostrar apenas os clientes do sexo masculino (WHERE c.Sexo = 'M').

Lembrando que a letra 'c' indica o nome da tabela clientes, tabela onde temos a coluna Sexo.

O WHERE é aplicado logo depois do Join pois é apenas depois do Join que temos a visibilidade da coluna Sexo em nossa consulta.

```
127    -- Selecione todas os pedidos feitos pelos clientes do sexo masculino.
128
129 •  SELECT
130        p.ID_Pedido,
131        p.Data_Venda,
132        p.Qtd_Vendida,
133        c.ID_Cliente,
134        c.Nome,
135        c.Email,
136        c.Telefone,
137        c.Sexo
138    FROM pedidos AS p
139    INNER JOIN clientes AS c
140        ON p.ID_Cliente = c.ID_Cliente
141    WHERE c.Sexo = 'M';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ID_Pedido	Data_Venda	Qtd_Vendida	ID_Cliente	Nome	Email	Telefone	Sexo
2	2019-01-01	1	17	Clarence	clarence32@yahoo.com.br	(21) 98923-7805	M
3	2019-01-01	1	23	Alejandro	alejandro45@gmail.com	(85) 94492-9582	M
4	2019-01-01	1	24	Harold	harold3@hotmail.com	(31) 95052-6286	M
5	2019-01-01	1	25	Jessie	jessie16@hotmail.com	(41) 98751-8234	M
8	2019-01-01	1	15	Wyatt	wyatt32@gmail.com	(71) 98644-2695	M
9	2019-01-01	1	38	Jesse	jesse15@yahoo.com.br	(62) 99674-9226	M
10	2019-01-02	1	27	Jimmy	jimmy9@yahoo.com.br	(85) 92931-9277	M
14	2019-01-02	1	31	Jaime	jaime41@yahoo.com.br	(21) 96759-5499	M
18	2019-01-03	1	37	Marc	marc3@yahoo.com.br	(11) 92792-7732	M

Result 40

INNER JOIN + GROUP BY

Agora vamos ver como utilizar Joins com o Group By.

No exemplo ao lado, fizemos um agrupamento de ID_Loja na tabela pedidos, para mostrar a Receita Total e o Custo Total para cada loja.

Poderíamos melhorar essa consulta e mostrar nesse agrupamento, ao invés do ID da Loja, o seu respectivo nome.

```
143   -- Faça um agrupamento para mostrar o total de receita e custo por ID da loja.
144
145 • SELECT
146       ID_Loja,
147       SUM(Receita_Venda) AS 'Receita Total',
148       SUM(Custo_Venda) AS 'Custo Total'
149   FROM pedidos
150   GROUP BY ID_Loja;
151
```

Result Grid | Filter Rows:

ID_Loja	Receita Total	Custo Total
2	34400	14199.5
1	30100	12901
3	27800	11595.5
8	27650	11468
4	28600	11815
7	17250	7080
6	34300	14707
5	28800	12150

INNER JOIN + GROUP BY

Para isso, vamos trocar dentro do SELECT a coluna ID_Loja por Loja.

Em seguida, é necessário fazer o INNER JOIN para que o código saiba onde buscar a coluna Loja.

Por fim, no GROUP BY, especificamos o nome da coluna a ser agrupado, que no caso, será Loja.

O resultado final é mostrado ao lado.

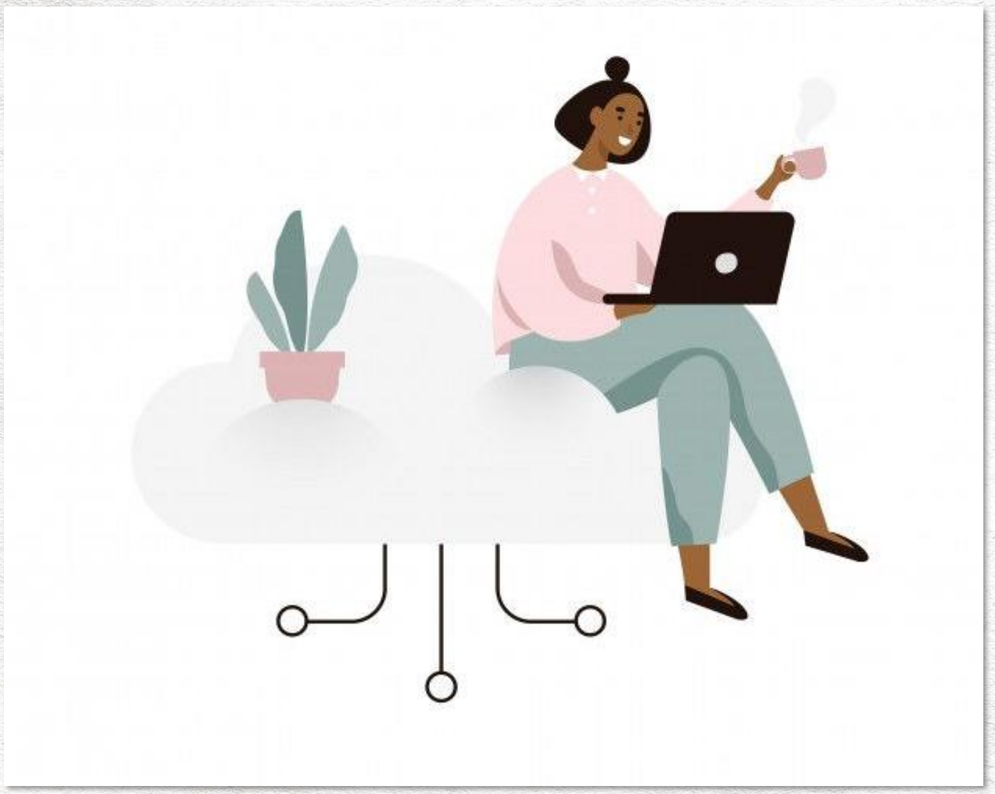
```
143    -- Faça um agrupamento para mostrar o total de receita e custo por Nome da loja.
144
145 •  SELECT
146        Loja,
147        SUM(Receita_Venda) AS 'Receita Total',
148        SUM(Custo_Venda) AS 'Custo Total'
149    FROM pedidos
150    INNER JOIN lojas
151        ON pedidos.ID_Loja = lojas.ID_Loja
152    GROUP BY Loja;
```

Loja	Receita Total	Custo Total
Belo Horizonte	34400	14199.5
Rio de Janeiro	30100	12901
Salvador	27800	11595.5
Goiânia	27650	11468
Curitiba	28600	11815
Niterói	17250	7080
São Paulo	34300	14707
Fortaleza	28800	12150

Result 42

Wrap Cell Content:

EXERCÍCIOS



1

Questão 1

A equipe de produtos está revisando o catálogo e deseja listar o nome do produto, sua marca e a categoria à qual pertence.

Crie uma consulta para exibir essas informações. **(Utilize as tabelas produtos e categorias)**

2

Questão 2

O setor de marketing está planejando campanhas específicas para clientes com alta escolaridade e deseja analisar o perfil de vendas desse público.

Crie uma consulta que liste o nome completo dos clientes com escolaridade "Pós-graduado", a data da venda realizada por cada cliente e o valor da receita gerada por pedido. **(Utilize as tabelas pedidos e clientes)**

EXERCÍCIOS

3

Questão 3

O setor de análise de vendas deseja um relatório detalhado do desempenho dos produtos da marca Samsung. O objetivo é identificar a quantidade total vendida e a receita gerada por cada produto dessa marca.

Crie uma consulta SQL que liste o nome de cada produto da Samsung, a quantidade total vendida e a receita total. **(Utilize as tabelas pedidos e produtos)**

4a

Questão 4-a

A equipe de planejamento quer identificar os produtos mais vendidos.

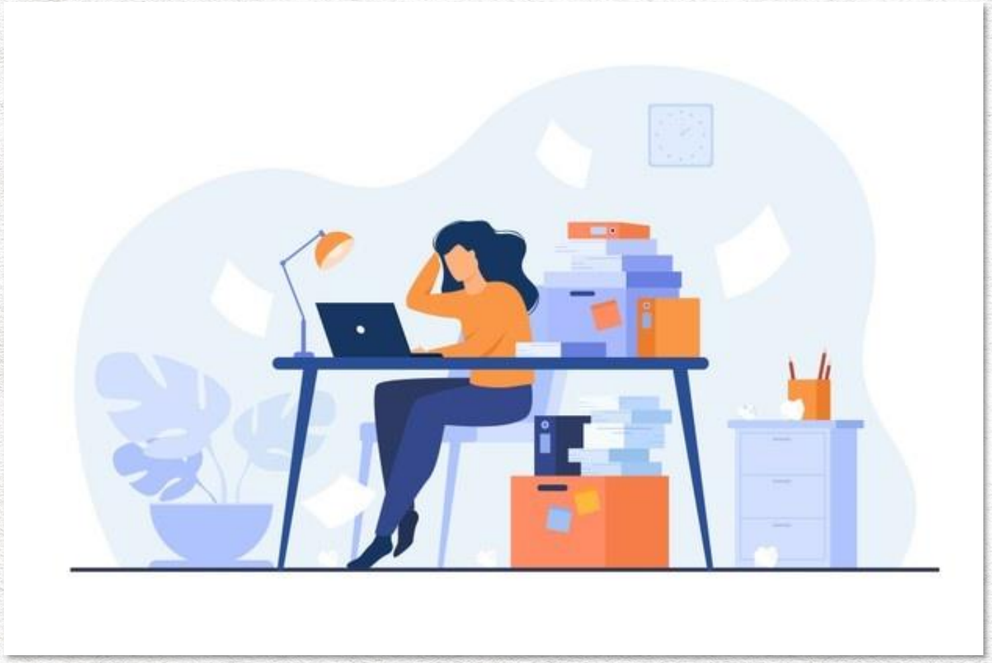
Liste o nome do produto e a quantidade total vendida, ordenando em ordem decrescente de quantidade. **(Utilize as tabelas pedidos e produtos)**

4b

Questão 4-b

Adapte o código do item A para acrescentar as categorias dos produtos. **(Utilize as tabelas pedidos, produtos e categorias)**

EXERCÍCIOS



5

Questão 5

A diretoria quer avaliar o desempenho de cada gerente.

Crie uma consulta SQL que retorne o nome do gerente, o nome da loja e a receita total, ordenando os resultados pela maior receita. **(Utilize as tabelas pedidos e lojas)**

6

Questão 6

O setor estratégico quer entender o desempenho por região.

Crie uma consulta que exiba a região e o total de vendas por região. **(Utilize as tabelas pedidos, lojas e locais)**

7

Questão 7

O setor de produtos quer avaliar as vendas por categoria.

Crie uma consulta que exiba os nomes das categorias e o total de vendas para cada uma. **(Utilize as tabelas pedidos, produtos e categorias)**

EXERCÍCIOS

8

Questão 8

O setor de inteligência de mercado quer identificar quais clientes geraram maior receita para cada produto adquirido. Esse relatório será utilizado para entender o comportamento de compra e valorizar os clientes mais lucrativos da empresa.

Crie uma consulta SQL que retorne o nome completo dos clientes, os produtos adquiridos e a receita total gerada por cliente e produto. Organize os resultados em ordem decrescente de receita gerada. **(Utilize as tabelas clientes, pedidos e produtos)**

9

Questão 9

DESAFIO: O time de planejamento estratégico está desenvolvendo uma análise detalhada para entender a performance de vendas considerando múltiplos fatores.

Eles precisam de um relatório que exiba o nome completo dos clientes, a região onde realizaram suas compras, o produto adquirido e a receita total gerada por cliente, produto e região.

Além disso, o relatório deve listar apenas os agrupamentos (combinação de cliente, produto e região) que geraram uma receita total acima de R$ 25.000,00.

Ordene os resultados em ordem decrescente de receita total. **(Utilize as tabelas clientes, pedidos, produtos, lojas e locais)**

GABARITO EXPLICATIVO

1

Questão 1

A equipe de produtos está revisando o catálogo e deseja listar o nome do produto, sua marca e a categoria à qual pertence.

Crie uma consulta para exibir essas informações. **(Utilize as tabelas produtos e categorias)**

Produto	Marca	Categoria
Monitor LED 19,5" Full HD HDMI	DELL	Monitor
Monitor Curvo 24" 144Hz HDMI	SAMSUNG	Monitor
Webcam Full HD 1080p	LOGITECH	Monitor
Kit Teclado + Mouse sem fio Wireless	DELL	Kit Teclado e Mouse
Kit Teclado + Mouse Slim Bluetooth	DELL	Kit Teclado e Mouse
Cadeira Gamer reclinável Azul/Laranja	ALTURA	Cadeira Gamer
Cadeira Gamer PC Racer Vermelha	ALTURA	Cadeira Gamer
Headphone Bluetooth 2000	SONY	Headphone
Fone de Ouvido Tune T5000	JBL	Headphone
Microfone Condensador MC1000	AKG	Microfone
Microfone Condensador com Tripé	BLUE	Microfone
Microfone de mesa com fio condensa...	BLUE	Microfone
Notebook LC2100 Intel Core i5 8GB	SAMSUNG	Notebook
Notebook Inspiron 15 5000 4GB	DELL	Notebook
Notebook IdeaPad RF32000	DELL	Notebook
Notebook Motion Ultra 2	SAMSUNG	Notebook

Explicação e Resultado

Para **listar o nome do produto, sua marca e a categoria à qual pertence**, devemos retornar as colunas **Nome_Produto** e **Marca_Produto** da tabela **PRODUTOS**, assim como a coluna **Categoria** da tabela **CATEGORIAS**.

Para fazer isso, podemos fazer um **INNER JOIN** para conectar as duas tabelas, relacionando na cláusula **ON** a coluna **ID_Categoria** da tabela **PRODUTOS** com a coluna **ID_Categoria** da tabela **CATEGORIAS**.

Além disso, opcionalmente, atribuímos *aliases* às tabelas (**p** para **PRODUTOS** e **c** para **CATEGORIAS**) para facilitar a leitura do código:

```
SELECT
    p.Nome_Produto AS 'Produto',
    p.Marca_Produto AS 'Marca',
    c.Categoria AS 'Categoria'
FROM produtos AS p
INNER JOIN categorias AS c
    ON p.ID_Categoria = c.ID_Categoria;
```

Dessa forma, retornamos o resultado ao lado:

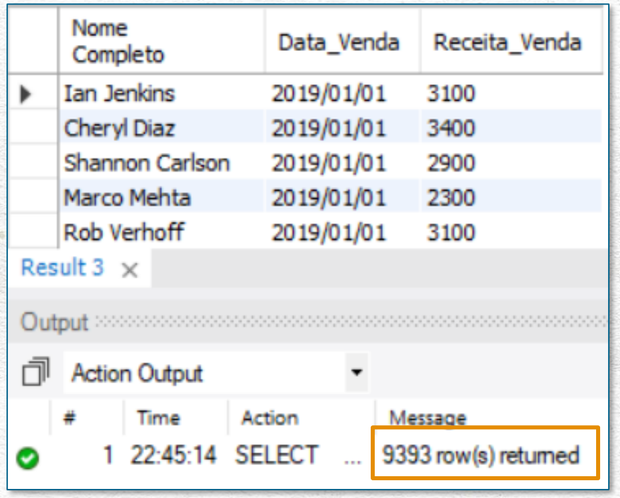
GABARITO EXPLICATIVO

2

Questão 2

O setor de marketing está planejando campanhas específicas para clientes com alta escolaridade e deseja analisar o perfil de vendas desse público.

Crie uma consulta que liste o nome completo dos clientes com escolaridade "Pós-graduado", a data da venda realizada por cada cliente e o valor da receita gerada por pedido. **(Utilize as tabelas pedidos e clientes)**



	Nome Completo	Data_Venda	Receita_Venda
▶	Ian Jenkins	2019/01/01	3100
	Cheryl Diaz	2019/01/01	3400
	Shannon Carlson	2019/01/01	2900
	Marco Mehta	2019/01/01	2300
	Rob Verhoff	2019/01/01	3100

Result 3 ×

Output

Action Output

#	Time	Action	Message
1	22:45:14	SELECT ...	9393 row(s) returned

Explicação e Resultado

Para **listar o nome completo dos clientes, a data da venda e o valor da receita gerada por clientes com escolaridade "Pós-graduado"**, precisamos recuperar dados de duas tabelas: **PEDIDOS** e **CLIENTES**.

Na cláusula **SELECT**, usamos a função **CONCAT** para combinar as colunas **Nome** e **Sobrenome** da tabela **CLIENTES** em uma única coluna chamada **"Nome Completo"**. Também retornamos as colunas **Data_Venda** e **Receita_Venda** da tabela **PEDIDOS**.

Para conectar as tabelas **PEDIDOS** e **CLIENTES**, utilizamos um **INNER JOIN**, relacionando a coluna **ID_Cliente** da tabela **PEDIDOS** com a coluna **ID_Cliente** da tabela **CLIENTES** na cláusula **ON**.

Para filtrar os clientes com escolaridade "Pós-graduado", adicionamos a condição **WHERE c.Escolaridade = 'Pós-graduado'**, considerando que a tabela **CLIENTES** contém a informação de escolaridade.

Os *aliases* **p** e **c** foram atribuídos às tabelas **PEDIDOS** e **CLIENTES**, respectivamente, para simplificar a escrita do código:

```sql
SELECT
    CONCAT(c.Nome, ' ', c.Sobrenome) AS 'Nome Completo',
    p.Data_Venda,
    p.Receita_Venda
FROM pedidos AS p
INNER JOIN clientes AS c
    ON p.ID_Cliente = c.ID_Cliente
WHERE c.Escolaridade = 'Pós-graduado';
```

Dessa forma, retornamos o resultado ao lado, com 9393 pedidos efetuados somente pelos clientes pós-graduados:

Índice Geral

Índice deste Módulo

GABARITO EXPLICATIVO

3

Questão 3

O setor de análise de vendas deseja um relatório detalhado do desempenho dos produtos da marca Samsung. O objetivo é identificar a quantidade total vendida e a receita gerada por cada produto dessa marca.

Crie uma consulta SQL que liste o nome de cada produto da Samsung, a quantidade total vendida e a receita total. **(Utilize as tabelas pedidos e produtos)**

	Nome_Produto	Qtd. Total	Receita Total
▶	Notebook LC2100 Intel Core i5 8GB	2827	9611800
	Notebook Motion Ultra 2	2808	8143200
	Monitor Curvo 24" 144Hz HDMI	2806	7856800

Explicação e Resultado

Para **criar o relatório de desempenho dos produtos da marca Samsung**, utilizamos as tabelas **PEDIDOS** e **PRODUTOS** e seguimos os passos abaixo:

No **SELECT**, retornamos o nome do produto (**prod.Nome_Produto**) e aplicamos as funções **SUM()** para calcular a quantidade total vendida (**SUM(p.Qtd_Vendida)**) e a receita total gerada (**SUM(p.Receita_Venda)**). Renomeamos essas colunas como **'Qtd. Total'** e **'Receita Total'** para facilitar a leitura.

Fizemos um **INNER JOIN** entre as tabelas **PEDIDOS** e **PRODUTOS**, relacionando a coluna **ID_Produto** de ambas as tabelas na cláusula **ON**.

Utilizamos a cláusula **WHERE** para filtrar apenas os produtos da marca "Samsung".

Para agrupar os resultados por produto e calcular os totais, utilizamos a cláusula **GROUP BY prod.Nome_Produto**.

Além disso, atribuímos os *aliases* **ped** e **prod** às tabelas para tornar o código mais legível:

```
SELECT
    prod.Nome_Produto,
    SUM(ped.Qtd_Vendida) AS 'Qtd. Total',
    SUM(ped.Receita_Venda) AS 'Receita Total'
FROM pedidos AS ped
INNER JOIN produtos AS prod
    ON ped.ID_Produto = prod.ID_Produto
WHERE prod.Marca_Produto = 'Samsung'
GROUP BY prod.Nome_Produto;
```

Dessa forma, retornamos o resultado ao lado:

GABARITO EXPLICATIVO

4a

Questão 4-a

A equipe de planejamento quer identificar os produtos mais vendidos.

Liste o nome do produto e a quantidade total vendida, ordenando em ordem decrescente de quantidade. **(Utilize as tabelas pedidos e produtos)**

	Produto	Quantidade Total
▶	Kit Teclado + Mouse sem fio Wireless	2906
	Cadeira Gamer PC Racer Vermelha	2898
	Headphone Bluetooth 2000	2888
	Monitor LED 19,5" Full HD HDMI	2875
	Notebook IdeaPad RF32000	2855
	Notebook LC2100 Intel Core i5 8GB	2827
	Webcam Full HD 1080p	2816
	Microfone de mesa com fio condensador	2816
	Notebook Motion Ultra 2	2808
	Monitor Curvo 24" 144Hz HDMI	2806
	Microfone Condensador com Tripé	2804
	Notebook Inspiron 15 5000 4GB	2799
	Kit Teclado + Mouse Slim Bluetooth	2787
	Fone de Ouvido Tune T5000	2776
	Cadeira Gamer reclinável Azul/Laranja	2768
	Microfone Condensador MC1000	2767

Explicação e Resultado

Para **identificar os produtos mais vendidos**, utilizamos as tabelas **PEDIDOS** e **PRODUTOS** com os seguintes passos:

No **SELECT**, retornamos o nome do produto (**prod.Nome_Produto**) e utilizamos a função **SUM()** para somar a quantidade vendida de cada produto (**SUM(ped.Qtd_Vendida)**). Renomeamos as colunas como **"Produto"** e **"Quantidade Total"** para facilitar a leitura.

Estabelecemos um **INNER JOIN** entre as tabelas **PEDIDOS** e **PRODUTOS**, conectando a coluna **ID_Produto** de ambas na cláusula **ON.**

Utilizamos a cláusula **GROUP BY** para agrupar os dados pelo nome do produto (**prod.Nome_Produto**) e garantir que os totais sejam calculados corretamente.

Ordenamos os resultados em ordem decrescente com a cláusula **ORDER BY SUM(ped.Qtd_Vendida) DESC**, para que os produtos com maior quantidade total apareçam primeiro.

Os *aliases* **ped** e **prod** foram atribuídos às tabelas para tornar o código mais claro e compacto:

Assim, retornamos o resultado ao lado:

```
SELECT
    prod.Nome_Produto AS 'Produto',
    SUM(ped.Qtd_Vendida) AS 'Quantidade Total'
FROM pedidos AS ped
INNER JOIN produtos AS prod
    ON ped.ID_Produto = prod.ID_Produto
GROUP BY prod.Nome_Produto
ORDER BY SUM(ped.Qtd_Vendida) DESC;
```

GABARITO EXPLICATIVO

4b

Questão 4-b

Adapte o código do item A para acrescentar as categorias dos produtos. **(Utilize as tabelas pedidos, produtos e categorias)**

Produto	Categoria	Quantidade Total
Kit Teclado + Mouse sem fio Wireless	Kit Teclado e Mouse	2906
Cadeira Gamer PC Racer Vermelha	Cadeira Gamer	2898
Headphone Bluetooth 2000	Headphone	2888
Monitor LED 19,5" Full HD HDMI	Monitor	2875
Notebook IdeaPad RF32000	Notebook	2855
Notebook LC2100 Intel Core i5 8GB	Notebook	2827
Webcam Full HD 1080p	Monitor	2816
Microfone de mesa com fio condensador	Microfone	2816
Notebook Motion Ultra 2	Notebook	2808
Monitor Curvo 24" 144Hz HDMI	Monitor	2806
Microfone Condensador com Tripé	Microfone	2804
Notebook Inspiron 15 5000 4GB	Notebook	2799
Kit Teclado + Mouse Slim Bluetooth	Kit Teclado e Mouse	2787
Fone de Ouvido Tune T5000	Headphone	2776
Cadeira Gamer reclinável Azul/Laranja	Cadeira Gamer	2768
Microfone Condensador MC1000	Microfone	2767

Explicação e Resultado

No item B, adaptamos o código do item A para **incluir a categoria de cada produto**. A estrutura base permanece a mesma, mas adicionamos alguns ajustes:

No **SELECT**, além de retornar o nome do produto e a quantidade total vendida, incluímos também a coluna **cat.Categoria** da tabela **CATEGORIAS**, que corresponde à categoria de cada produto.

Foi necessário adicionar mais um **INNER JOIN** com a tabela **CATEGORIAS**, conectando a coluna **ID_Categoria** da tabela **PRODUTOS** com a coluna **ID_Categoria** da tabela **CATEGORIAS** na cláusula **ON**. Esse relacionamento permite obter as categorias associadas a cada produto.

Ajustamos a cláusula **GROUP BY** para incluir **prod.Nome_Produto** e **cat.Categoria**, garantindo que os totais sejam agrupados corretamente por produto e categoria.

A cláusula **ORDER BY** permanece inalterada, ordenando os resultados pela quantidade total em ordem decrescente (**SUM(ped.Qtd_Vendida) DESC**).

Atribuímos os *aliases* **ped**, **prod** e **cat** às tabelas para tornar o código mais legível:

Assim, retornamos o resultado ao lado:

```
SELECT
    prod.Nome_Produto AS 'Produto',
    cat.Categoria AS 'Categoria',
    SUM(ped.Qtd_Vendida) AS 'Quantidade Total'
FROM pedidos AS ped
INNER JOIN produtos AS prod
    ON ped.ID_Produto = prod.ID_Produto
INNER JOIN categorias AS cat
    ON prod.ID_Categoria = cat.ID_Categoria
GROUP BY prod.Nome_Produto, cat.Categoria
ORDER BY SUM(ped.Qtd_Vendida) DESC;
```

GABARITO EXPLICATIVO

5

Questão 5

A diretoria quer avaliar o desempenho de cada gerente.

Crie uma consulta SQL que retorne o nome do gerente, o nome da loja e a receita total, ordenando os resultados pela maior receita. **(Utilize as tabelas pedidos e lojas)**

Gerente	Loja	Receita Total
Rogério Lopes	São Paulo	10443680
Letícia Gomes	Fortaleza	10217870
Amanda Lima	Curitiba	10139870
Marcelo Castro	Salvador	10107860
Julia Freitas	Belo Horizonte	10061430
Pedro Martins	Rio de Janeiro	10037160
Matheus Leal	Niterói	9986950
Carlos Henrique	Goiânia	9943620

Explicação e Resultado

Para **criar o relatório de desempenho dos gerentes**, utilizamos as tabelas **PEDIDOS** e **LOJAS** e seguimos os passos abaixo:

No **SELECT**, retornamos o nome do gerente (**l.Gerente**), o nome da loja (**l.Loja**) e aplicamos a função **SUM()** para calcular a receita total gerada (**SUM(p.Receita_Venda)**). Renomeamos a coluna da receita como **'Receita Total'** para facilitar a leitura.

Fizemos um **INNER JOIN** entre as tabelas **PEDIDOS** e **LOJAS**, relacionando a coluna **ID_Loja** de ambas as tabelas na cláusula **ON**.

Utilizamos a cláusula **GROUP BY** para agrupar os resultados pelo nome do gerente (**l.Gerente**) e pela loja (**l.Loja**) e calcular a receita total para cada grupo.

Por fim, aplicamos o comando **ORDER BY SUM(p.Receita_Venda) DESC** para ordenar os resultados pela maior receita, da maior para a menor.

Atribuímos os *aliases* **p** para a tabela **PEDIDOS** e **l** para a tabela **LOJAS** para tornar o código mais legível:

Dessa forma, retornamos o resultado ao lado:

```sql
SELECT
    l.Gerente AS 'Gerente',
    l.Loja AS 'Loja',
    SUM(p.Receita_Venda) AS 'Receita Total'
FROM pedidos AS p
INNER JOIN lojas AS l
    ON p.ID_Loja = l.ID_Loja
GROUP BY l.Gerente, l.Loja
ORDER BY SUM(p.Receita_Venda) DESC;
```

GABARITO EXPLICATIVO

6

Questão 6

O setor estratégico quer entender o desempenho por região.

Crie uma consulta que exiba a região e o total de vendas por região. **(Utilize as tabelas pedidos, lojas e locais)**

	Região	Receita Total
▶	Sudeste	40529220
	Centro-Oeste	9943620
	Nordeste	20325730
	Sul	10139870

Explicação e Resultado

Para **criar o relatório de desempenho por região**, utilizamos as tabelas **PEDIDOS**, **LOJAS** e **LOCAIS** e seguimos os passos abaixo:

No **SELECT**, retornamos a região (**loc.Região**) e aplicamos a função **SUM()** para calcular a receita total por região (**SUM(p.Receita_Venda)**). Renomeamos a coluna da receita como **'Receita Total'** para facilitar a leitura.

Fizemos dois **INNER JOINs**: o primeiro entre as tabelas **PEDIDOS** e **LOJAS**, relacionando a coluna **ID_Loja** de ambas as tabelas na cláusula **ON**. O segundo **INNER JOIN** é entre as tabelas **LOJAS** e **LOCAIS**, relacionando a coluna **Loja** da tabela **LOJAS** com a coluna **Cidade** da tabela **LOCAIS**.

Utilizamos a cláusula **GROUP BY** para agrupar os resultados pela região (loc.Região) e calcular a receita total para cada região.

Atribuímos os *aliases* **ped**, **loj** e **loc** às tabelas **PEDIDOS**, **LOJAS** e **LOCAIS**, respectivamente, para tornar o código mais enxuto:

Assim, retornamos o resultado ao lado:

```
SELECT
    loc.Região,
    SUM(ped.Receita_Venda) AS 'Receita Total'
FROM pedidos AS ped
INNER JOIN lojas AS loj
    ON ped.ID_Loja = loj.ID_Loja
INNER JOIN locais AS loc
    ON loj.Loja = loc.Cidade
GROUP BY loc.Região;
```

GABARITO EXPLICATIVO

7

Questão 7

O setor de produtos quer avaliar as vendas por categoria.

Crie uma consulta que exiba os nomes das categorias e o total de vendas para cada uma. **(Utilize as tabelas pedidos, produtos e categorias)**

Categoria	Receita Total
Microfone	7117300
Notebook	38422900
Headphone	3898080
Monitor	15736500
Kit Teclado e Mouse	1797460
Cadeira Gamer	13966200

Explicação e Resultado

Para **criar o relatório de vendas por categoria**, utilizamos as tabelas **PEDIDOS**, **PRODUTOS** e **CATEGORIAS** e seguimos os passos abaixo:

No **SELECT**, retornamos o nome da categoria (**cat.Categoria**) e aplicamos a função **SUM()** para calcular a receita total de vendas por categoria (**SUM(ped.Receita_Venda)**). Renomeamos a coluna da receita como **'Receita Total'** para facilitar a leitura.

Fizemos dois **INNER JOINs**: o primeiro entre as tabelas **PEDIDOS** e **PRODUTOS**, relacionando a coluna **ID_Produto** de ambas as tabelas na cláusula **ON**. O segundo **INNER JOIN** é entre as tabelas **PRODUTOS** e **CATEGORIAS**, relacionando a coluna **ID_Categoria** de ambas as tabelas na cláusula **ON**.

Utilizamos a cláusula **GROUP BY** para agrupar os resultados pela categoria (**cat.Categoria**) e calcular a receita total de vendas para cada categoria.

Atribuímos os *aliases* **ped**, **prod** e **cat** às tabelas **PEDIDOS**, **PRODUTOS** e **CATEGORIAS**, respectivamente, para tornar o código mais compacto:

Dessa forma, retornamos o resultado ao lado:

```
SELECT
    cat.Categoria,
    SUM(ped.Receita_Venda) AS 'Receita Total'
FROM pedidos AS ped
INNER JOIN produtos AS prod
    ON ped.ID_Produto = prod.ID_Produto
INNER JOIN categorias AS cat
    ON prod.ID_Categoria = cat.ID_Categoria
GROUP BY cat.Categoria;
```

GABARITO EXPLICATIVO

8

Questão 8

O setor de inteligência de mercado quer identificar quais clientes geraram maior receita para cada produto adquirido. Esse relatório será utilizado para entender o comportamento de compra e valorizar os clientes mais lucrativos da empresa.

Crie uma consulta SQL que retorne o nome completo dos clientes, os produtos adquiridos e a receita total gerada por cliente e produto. Organize os resultados em ordem decrescente de receita gerada. **(Utilize as tabelas clientes, pedidos e produtos)**

Explicação e Resultado

Para **criar o relatório de clientes que geraram maior receita por produto adquirido**, utilizamos as tabelas **CLIENTES**, **PEDIDOS** e **PRODUTOS** e seguimos os passos abaixo:

No **SELECT**, retornamos o nome completo do cliente (**CONCAT(cli.Nome, ' ', cli.Sobrenome)**), o nome do produto adquirido (**prod.Nome_Produto**) e aplicamos a função **SUM()** para calcular a receita total gerada por cliente e produto (**SUM(ped.Receita_Venda)**).

Fizemos dois **INNER JOINs**: o primeiro entre as tabelas **CLIENTES** e **PEDIDOS**, relacionando a coluna **ID_Cliente** de ambas as tabelas na cláusula **ON**. O segundo **INNER JOIN** é entre as tabelas **PEDIDOS** e **PRODUTOS**, relacionando a coluna **ID_Produto** de ambas as tabelas na cláusula **ON**.

Utilizamos a cláusula **GROUP BY** para agrupar os resultados pelo nome completo do cliente (**CONCAT(cli.Nome, ' ', cli.Sobrenome)**) e pelo nome do produto adquirido (**prod.Nome_Produto**), a fim de calcular a receita gerada por cada cliente e produto.

Por fim, aplicamos o comando **ORDER BY Receita_Gerada DESC** para ordenar os resultados pela receita gerada, da maior para a menor.

Atribuímos os *aliases* **cli**, **ped** e **prod** às tabelas **CLIENTES**, **PEDIDOS** e **PRODUTOS**, respectivamente, para tornar o código mais enxuto:

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

8

Questão 8

O setor de inteligência de mercado quer identificar quais clientes geraram maior receita para cada produto adquirido. Esse relatório será utilizado para entender o comportamento de compra e valorizar os clientes mais lucrativos da empresa.

Crie uma consulta SQL que retorne o nome completo dos clientes, os produtos adquiridos e a receita total gerada por cliente e produto. Organize os resultados em ordem decrescente de receita gerada. **(Utilize as tabelas clientes, pedidos e produtos)**

Explicação e Resultado

```sql
SELECT
    CONCAT(cli.Nome, ' ', cli.Sobrenome) AS 'Cliente',
    prod.Nome_Produto AS 'Produto',
    SUM(ped.Receita_Venda) AS 'Receita_Gerada'
FROM clientes AS cli
INNER JOIN pedidos AS ped
    ON cli.ID_Cliente = ped.ID_Cliente
INNER JOIN produtos AS prod
    ON ped.ID_Produto = prod.ID_Produto
GROUP BY CONCAT(cli.Nome, ' ', cli.Sobrenome), prod.Nome_Produto
ORDER BY Receita_Gerada DESC;
```

Dessa forma, retornamos o resultado ao lado, com 1600 linhas que mostram a receita gerada por cada combinação de clientes e produtos, ordenadas da maior para a menor receita:

Cliente	Produto	Receita_Gerada
Sydney Bennett	Notebook IdeaPad RF32000	172200
Alyssa Cox	Notebook IdeaPad RF32000	172200
Harold Sai	Notebook IdeaPad RF32000	168000
Wendy Dominguez	Notebook IdeaPad RF32000	168000
Michele Nath	Notebook IdeaPad RF32000	163800
Alejandro Beck	Notebook IdeaPad RF32000	159600

Result 1

Output

Action Output

#	Time	Action		Message
1	00:03:16	SELECT	CONCAT(cli.N...	1600 row(s) returned

GABARITO EXPLICATIVO

9

Questão 9

DESAFIO: O time de planejamento estratégico está desenvolvendo uma análise detalhada para entender a performance de vendas considerando múltiplos fatores.

Eles precisam de um relatório que exiba o nome completo dos clientes, a região onde realizaram suas compras, o produto adquirido e a receita total gerada por cliente, produto e região.

Além disso, o relatório deve listar apenas os agrupamentos (combinação de cliente, produto e região) que geraram uma receita total acima de R$ 25.000,00.

Ordene os resultados em ordem decrescente de receita total. **(Utilize as tabelas clientes, pedidos, produtos, lojas e locais)**

Explicação e Resultado

Para resolver este **DESAFIO**, precisamos utilizar as tabelas **CLIENTES**, **PEDIDOS**, **PRODUTOS**, **LOJAS** e **LOCAIS** e seguir os passos abaixo:

No **SELECT**, retornamos o nome completo do cliente (**CONCAT(cli.Nome, ' ', cli.Sobrenome)**), a região onde a compra foi realizada (**loc.Região**), o nome do produto adquirido (**prod.Nome_Produto**) e aplicamos a função **SUM()** para calcular a receita total gerada pela combinação de cliente, produto e região (**SUM(ped.Receita_Venda)**).

Fizemos quatro **INNER JOINs**: o primeiro entre as tabelas **PEDIDOS e CLIENTES**, relacionando a coluna **ID_Cliente** de ambas as tabelas na cláusula **ON**. O segundo **INNER JOIN** é entre as tabelas **PEDIDOS** e **PRODUTOS**, relacionando a coluna **ID_Produto** de ambas as tabelas na cláusula **ON**. O terceiro **INNER JOIN** é entre as tabelas **PEDIDOS** e **LOJAS**, relacionando a coluna **ID_Loja** de ambas as tabelas na cláusula **ON**. O quarto **INNER JOIN** é entre as tabelas **LOJAS** e **LOCAIS**, relacionando a coluna **Loja** da tabela **LOJAS** com a coluna **Cidade** da tabela **LOCAIS**.

Utilizamos a cláusula **GROUP BY** para **agrupar os resultados pelo nome completo do cliente, pela região e pelo nome do produto**, a fim de calcular a receita total gerada por cada combinação de cliente, produto e região.

Utilizamos a cláusula **HAVING** para filtrar apenas os agrupamentos que geraram uma receita total superior a R$ 25.000,00 (**SUM(ped.Receita_Venda) > 25000**).

Por fim, aplicamos o comando **ORDER BY SUM(ped.Receita_Venda) DESC** para ordenar os resultados pela receita gerada, da maior para a menor.

Atribuímos os *aliases* **cli**, **ped**, **prod**, **loj** e **loc** às tabelas **CLIENTES**, **PEDIDOS**, **PRODUTOS**, **LOJAS** e **LOCAIS**, respectivamente, para tornar o código mais enxuto e legível:

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

9

Questão 9

DESAFIO: O time de planejamento estratégico está desenvolvendo uma análise detalhada para entender a performance de vendas considerando múltiplos fatores.

Eles precisam de um relatório que exiba o nome completo dos clientes, a região onde realizaram suas compras, o produto adquirido e a receita total gerada por cliente, produto e região.

Além disso, o relatório deve listar apenas os agrupamentos (combinação de cliente, produto e região) que geraram uma receita total acima de R$ 25.000,00.

Ordene os resultados em ordem decrescente de receita total. **(Utilize as tabelas clientes, pedidos, produtos, lojas e locais)**

Explicação e Resultado

```sql
SELECT
    CONCAT(cli.Nome, ' ', cli.Sobrenome) AS 'Cliente',
    loc.Região AS 'Região',
    prod.Nome_Produto AS 'Produto',
    SUM(ped.Receita_Venda) AS 'Receita_Total'
FROM pedidos AS ped
INNER JOIN clientes AS cli
    ON ped.ID_Cliente = cli.ID_Cliente
INNER JOIN produtos AS prod
    ON ped.ID_Produto = prod.ID_Produto
INNER JOIN lojas AS loj
    ON ped.ID_Loja = loj.ID_Loja
INNER JOIN locais AS loc
    ON loj.Loja = loc.Cidade
GROUP BY CONCAT(cli.Nome, ' ', cli.Sobrenome), loc.Região, prod.Nome_Produto
HAVING SUM(ped.Receita_Venda) > 25000
ORDER BY SUM(ped.Receita_Venda) DESC;
```

Dessa forma, retornamos o resultado ao lado, com 999 linhas que mostram a receita total gerada por cada combinação de cliente, região e produto, ordenadas da maior para a menor receita, considerando-se somente as receitas maiores que R$ 25.000,00:

Cliente	Região	Produto	Receita_Total
Alyssa Cox	Sudeste	Notebook IdeaPad RF32000	96600
Alejandro Beck	Sudeste	Notebook IdeaPad RF32000	96600
Julia Nelson	Sudeste	Notebook IdeaPad RF32000	92400
Noah Powell	Sudeste	Notebook IdeaPad RF32000	92400
Levi Arun	Sudeste	Notebook IdeaPad RF32000	88200

Result 12

Output

Action Output

#	Time	Action		Message
1	00:11:00	SELECT	CONCAT(cli....	999 row(s) returned

Índice Geral

Índice deste Módulo

MÓDULO 10

FUNÇÕES CONDICIONAIS

FUNÇÕES CONDICIONAIS

FUNÇÕES CONDICIONAIS

Índice Geral

Índice deste Módulo

O QUE VEREMOS NESTE MÓDULO

MÓDULO 10 FUNÇÕES CONDICIONAIS

01	FUNÇÕES CONDICIONAIS	256
02	IF	257
03	IF (EXEMPLO 1)	258
04	IF (EXEMPLO 2)	259
05	IF (EXEMPLO 3)	260
06	IFNULL	261
07	ISNULL	262
08	NULLIF	263
09	CASE WHEN	264
10	CASE WHEN (EXEMPLO 1)	265
11	CASE WHEN (EXEMPLO 2)	266
12	CASE + AND E OR	267
13	CASE + AND (EXEMPLO)	268
14	CASE + OR (EXEMPLO)	270
15	EXERCÍCIOS	272
16	GABARITO EXPLICATIVO	276

Funções Condicionais

As funções condicionais nos permitem tratar condições dentro do MySQL. As funções condicionais que temos são:

- IF
- IFNULL
- ISNULL
- NULLIF
- CASE

A seguir, vamos ver na prática como que cada uma delas vai funcionar.

IF

A função IF permite tratar condições no MySQL. Para usá-la, precisamos informar 3 argumentos:

IF(teste_logico, valor_se_verdadeiro, valor_se_falso)

1 2 3

O teste_logico nada mais é do que a comparação entre dois valores. Por exemplo, 10 > 20 (ou seja, o valor 10 é maior que o valor 20?).

Se a comparação do teste_logico for verdadeira, o resultado será o especificado no segundo argumento, valor_se_verdadeiro.

Se a comparação do teste_logico for falsa, o resultado será o especificado no terceiro argumento, valor_se_falso.

IF (Exemplo 1)

Observe o exemplo ao lado.

Temos uma comparação simples, onde queremos verificar se o valor 10 é maior que o valor 20.

Como isso não é verdade, então o resultado retornado é o do terceiro argumento.

```
155    -- Exemplo 1: Faça um IF simples para verificar o seu funcionamento.
156
157 •  SELECT
158        IF(10 > 20, '10 é maior que 20', '10 não é maior que 20') AS Resposta;
159
160
161
162
```

Resposta
10 não é maior que 20

IF (Exemplo 2)

No próximo exemplo, vamos verificar se um determinado funcionário vai receber ou não um bônus pelo seu desempenho.

A regra é simples: se a nota for maior ou igual a 7, o funcionário recebe um bônus de 10% (0.1), caso contrário, não recebe bônus (0).

O resultado final está mostrado ao lado.

```
161    -- Exemplo 2: Uma empresa oferece um bônus de 10% para todos os funcionários que tiveram
       uma avaliação do RH de acordo com a seguinte regra.
162    -- NotaDesempenho >= 7 ----> Recebe bônus de 10%
163    -- NotaDesempenho < 7  ----> Não recebe bônus
164
165 •  SET @varNotaDesempenho = 8.5;
166
167 •  SELECT
168        IF(@varNotaDesempenho >= 7, 0.1, 0) AS Resultado;
169
170
```

Wrap Cell Content:

Resultado
0.1

IF (Exemplo 3)

No próximo exemplo, vamos complementar o exemplo anterior e criar uma nova categoria de bônus.

Quem não tiver uma nota acima de 7, ainda poderá receber um bônus se tiver a nota acima de 5.

Perceba que agora, se um funcionário ficar com 6.5, por exemplo, ele não ganha o bônus de 10%, mas ganha o de 5%.

```
161    -- Exemplo 2: Uma empresa oferece um bônus de 10% para todos os funcionários que tiveram
       uma avaliação do RH de acordo com a seguinte regra.
162    -- NotaDesempenho >= 7 ----> Recebe bônus de 10%
163    -- NotaDesempenho >= 5 ----> Recebe bônus de 5%
164    -- NotaDesempenho < 7  ----> Não recebe bônus
165
166 •  SET @varNotaDesempenho = 6.5;
167
168 •  SELECT
169        IF(@varNotaDesempenho >= 7, 0.1, IF(@varNotaDesempenho >= 5, 0.05, 0)) AS Resultado;
170
```

Resultado
0.05

IFNULL

A função IFNULL tem um funcionamento bem simples.

Ela pede dois argumentos. Se o primeiro argumento informado for NULL, então a função vai retornar o valor no segundo argumento, caso contrário vai retornar o próprio primeiro argumento.

Na imagem ao lado temos um exemplo de aplicação desta função.

Observe que para todos os Telefones que eram diferentes de NULL, a função retornou o próprio telefone da Loja.

Agora para a Loja em que o Telefone era NULL, ela retornou o telefone alternativo especificado no segundo argumento da função

```
172    -- IFNULL
173    -- Exemplo: A tabela de lojas pode ter lojas que não possuem telefone de contato (NULL). Nestes casos, todas
       as lojas que tiverem NULL na coluna Telefone devem ter como ponto de contato o telefone padrão da central de
       atendimento: 0800-999-9999. Utilize a função IFNULL para fazer este ajuste.
174
175 •  SELECT
176        *,
177        IFNULL(Telefone, '0800-999-9999') AS 'Telefone Corrigido'
178    FROM lojas;
179
```

ID_Loja	Loja	Gerente	Endereco	Num_Funcionarios	Telefone	Telefone Corrigido
1	Rio de Janeiro	Pedro Martins	R. Visc. de Pirajá, 136 - Ipanema, Rio de Janeir...	23	(21) 3198-1239	(21) 3198-1239
2	Belo Horizonte	Julia Freitas	Av. Barão Homem de Melo, 1389 - Nova Granad...	13	(31) 2891-2394	(31) 2891-2394
3	Salvador	Marcelo Castro	Shopping Barra - Av. Centenário, 2992 - Barra, ...	19	(71) 3465-8748	(71) 3465-8748
4	Curitiba	Amanda Lima	R. Brasílio Itiberê, 3279 - Água Verde, Curitiba -...	31	(41) 2923-2137	(41) 2923-2137
5	Fortaleza	Letícia Gomes	Av. Dom Luís, 500 - Aldeota, Fortaleza - CE, 60...	12	(85) 3294-1481	(85) 3294-1481
6	São Paulo	Rogério Lopes	R. Teodoro Sampaio, 954 - Pinheiros, São Paulo...	33	(11) 3295-3945	(11) 3295-3945
7	Niterói	Matheus Leal	Rua Quinze de Novembro, 8 - Loja 301a Loja 30...	18	(21) 3134-7263	(21) 3134-7263
8	Goiânia	Carlos Henrique	Av. Goiás Norte, 3.592 - quadra 2.1 - St. Mal. ...	11	NULL	0800-999-9999

ISNULL

A função ISNULL verifica se um determinado valor é NULL. Se for, ela retorna 1 (verdadeiro), caso contrário, retorna 0 (falso).

No exemplo ao lado, utilizamos a função ISNULL para conseguir identificar os clientes que têm o telefone NULL, pois estes vamos precisar verificar o que pode ter acontecido.

```
181   -- ISNULL
182   -- Exemplo: Alguns clientes não cadastraram  telefone de contato. Faça uma consulta com uma coluna extra que
      identifique esses clientes de alguma forma.
183
184 • SELECT
185       *,
186       IF(ISNULL(Telefone), 'Verificar', 'Ok') AS Status
187   FROM clientes;
188
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ID_Cliente	Nome	Sobrenome	Data_Nascimento	Estado_Civil	Sexo	Email	Telefone	Renda_Anual	Qtd_Filhos	Escolaridade	Status
58	Jon	Zhou	1974-03-12	C	M	jon28@hotmail.com	(85) 96675-4865	80000	2	Parcial	Ok
59	Todd	Gao	1974-02-22	C	M	todd14@gmail.com	(11) 96585-5605	80000	2	Parcial	Ok
60	Noah	Powell	1995-08-28	C	M	noah5@yahoo.com.br	(85) 98162-6253	40000	0	Ensino médio	Ok
61	Angela	Murphy	1995-04-02	S	F	angela41@yahoo.com.br	NULL	40000	0	Ensino médio	Verificar
62	Chase	Reed	1995-12-02	C	M	chase21@hotmail.com		40000	0	Ensino médio	Ok
63	Jessica	Henderson	1993-10-04	C	F	jessica29@gmail.com	(21) 93673-6903	60000	0	Parcial	Ok
64	Grace	Butler	1993-11-22	C	F	grace62@gmail.com	NULL	70000	0	Parcial	Verificar
65	Caleb	Carter	1996-09-20	S	M	caleb40@gmail.com	(31) 97809-1800	60000	0	Parcial	Ok
66	Tiffany	Liang	1975-09-18	S	F	tiffany17@gmail.com	(21) 97231-3503	80000	2	Ensino médio	Ok
67	Carolyn	Navarro	1975-09-16	S	F	carolyn30@yahoo.com.br	(21) 94139-6777	80000	2	Ensino médio	Ok
68	Willie	Raii	1975-03-31	C	M	willie40@yahoo.com.br	(71) 97614-1995	80000	2	Ensino médio	Ok

Result 50

NULLIF

Uma outra função condicional é a NULLIF. Essa função compara duas expressões e retorna NULL se as duas forem iguais. Se forem, retorna NULL; caso contrário, retorna a primeira expressão. Observe na situação 1 que os dois valores de @var1 e @var2 são iguais. Portanto, ao usar a função NULLIF, o resultado é NULL. Já na segunda situação, o valor de @var1 é diferente do valor de @var2. Portanto, nesse caso, o resultado será o valor do primeiro argumento; ou seja, o valor de @var1.

1

```
191    -- NULLIF
192
193 •  SET @var1 = 100;
194 •  SET @var2 = 100;
195
196 •  SELECT
197        NULLIF(@var1, @var2);
198
199
```

NULLIF(@var1, @var2)
NULL

2

```
191    -- NULLIF
192
193 •  SET @var1 = 100;
194 •  SET @var2 = 50;
195
196 •  SELECT
197        NULLIF(@var1, @var2);
198
199
```

NULLIF(@var1, @var2)
100

CASE WHEN

O CASE é uma outra forma de tratar condições dentro do MySQL. Abaixo, temos a sua sintaxe:

```
CASE
        WHEN condicao1 THEN resultado1
        WHEN condicao2 THEN resultado2
        WHEN condicao3 THEN resultado3
        ELSE resultado4
END;
```

Podemos fazer uma “tradução” da estrutura acima da seguinte forma:

CASO a condição 1 seja verdadeira, ENTÃO retorna o resultado1. CASO a condição 2 seja verdadeira, ENTÃO retorna o resultado2. CASO a condição 3 seja verdadeira, ENTÃO retorna o resultado3. CASO CONTRÁRIO, retorna o resultado4.

CASE WHEN (Exemplo 1)

No primeiro exemplo ao lado, utilizamos a estrutura CASE para substituir, na tabela clientes, as letras M e F pelos textos Masculino e Feminino.

Chamamos de 'Sexo 2' a coluna resultante do CASE.

```
212  -- Exemplo 1: Utilize a estrutura CASE para fazer uma consulta à tabela clientes, substituindo as letras M e F
     por Masculino e Feminino.
213
214  SELECT
215      *,
216      CASE
217          WHEN Sexo = 'M' THEN 'Masculino'
218          ELSE 'Feminino'
219      END AS 'Sexo 2'
220  FROM clientes;
221
222
223
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ID_Cliente	Nome	Sobrenome	Data_Nascimento	Estado_Civil	Sexo	Email	Telefone	Renda_Anual	Qtd_Filhos	Escolaridade	Sexo 2
1	Ruben	Torres	1985-08-07	C	M	ruben35@hotmail.com	(85) 94132-1176	60000	3	Pós-graduado	Masculino
2	Christy	Zhu	1988-02-10	S	F	christy12@hotmail.com	(41) 96270-6086	70000	0	Pós-graduado	Feminino
3	Elizabeth	Johnson	1988-08-03	S	F	elizabeth5@gmail.com	(31) 92039-5832	80000	5	Pós-graduado	Feminino
4	Julio	Ruiz	1985-07-31	S	M	julio1@hotmail.com	(62) 93391-5891	70000	0	Pós-graduado	Masculino
5	Janet	Alvarez	1985-12-01	S	F	janet9@gmail.com	(21) 93379-3743	70000	0	Pós-graduado	Feminino
6	Marco	Mehta	1984-05-04	C	M	marco14@yahoo.com.br	(85) 99019-3803	60000	3	Pós-graduado	Masculino
7	Rob	Verhoff	1984-07-02	S	F	rob4@hotmail.com	(11) 98162-4760	60000	4	Pós-graduado	Feminino
8	Shannon	Carlson	1984-03-27	S	M	shannon38@gmail.com	(85) 96795-9950	70000	0	Pós-graduado	Masculino

Result 60

CASE WHEN (Exemplo 2)

No segundo exemplo, queremos categorizar os pedidos de acordo com a sua receita de venda.

Ao lado, temos toda a lógica do exercício e como resolvê-lo.

```sql
-- Exemplo 2: Na tabela pedidos, crie uma estrutura CASE para avaliar as seguintes situações de Receita_Venda:
-- Caso Receita_Venda >= 3000 ---> 'Faturamento Alto'
-- Caso Receita_Venda >= 1000 ---> 'Faturamento Médio'
-- Caso Receita_Venda < 1000  ---> 'Faturamento Baixo'

SELECT
    *,
    CASE
        WHEN Receita_Venda >= 3000 THEN 'Faturamento Alto'
        WHEN Receita_Venda >= 1000 THEN 'Faturamento Médio'
        ELSE 'Faturamento Baixo'
    END AS Resultado
FROM pedidos;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ID_Pedido	Data_Venda	ID_Loja	ID_Produto	ID_Cliente	Qtd_Vendida	Receita_Venda	Custo_Venda	Custo_Unit	Preco_Unit	Resultado
1	2019-01-01	2	4	16	1	350	129.5	129.5	350	Faturamento Baixo
2	2019-01-01	1	4	17	1	350	129.5	129.5	350	Faturamento Baixo
3	2019-01-01	3	4	23	1	350	129.5	129.5	350	Faturamento Baixo
4	2019-01-01	8	4	24	1	350	129.5	129.5	350	Faturamento Baixo
5	2019-01-01	4	4	25	1	350	129.5	129.5	350	Faturamento Baixo
6	2019-01-01	4	4	26	1	350	129.5	129.5	350	Faturamento Baixo
7	2019-01-01	2	6	14	1	1800	540	540	1800	Faturamento Médio
8	2019-01-01	8	6	15	1	1800	540	540	1800	Faturamento Médio
9	2019-01-01	7	6	38	1	1800	540	540	1800	Faturamento Médio

Result 62 | Read

CASE + AND e OR

Podemos combinar a estrutura CASE com o AND ou o OR para criar condições mais complexas.

A sintaxe dessas combinações é mostrada abaixo.

AND

```
CASE
        WHEN condicao1 AND condicao2 THEN resultado1
        ELSE resultado2
END;
```

Podemos fazer uma “tradução” da estrutura acima da seguinte forma:

CASO a condição 1 seja verdadeira E TAMBÉM a condição 2 seja verdadeira, ENTÃO retorna o resultado1. CASO CONTRÁRIO, retorna o resultado2.

OR

```
CASE
        WHEN condicao1 OR condicao2 THEN resultado1
        ELSE resultado2
END;
```

Podemos fazer uma “tradução” da estrutura acima da seguinte forma:

CASO a condição 1 seja verdadeira OU a condição 2 seja verdadeira, ENTÃO retorna o resultado1. CASO CONTRÁRIO, retorna o resultado2.

CASE + AND (Exemplo)

O enunciado do primeiro exemplo é mostrado abaixo. Precisaremos utilizar a combinação CASE + AND para resolvê-lo.

```
-- APLICAÇÃO AND
-- Exemplo 1. A empresa está com uma ação de dia das mães/dia dos pais. Como vai funcionar?

-- Caso o cliente seja do sexo Feminino E tenha filhos, receberá ofertas de dia das mães
-- Caso o cliente seja do sexo Masculino E tenha filhos, receberá ofertas de dia dos pais
-- Caso contrário, não receberá oferta

-- As ofertas serão enviadas por e-mail, por isso o setor responsável precisa de uma tabela identificando
facilmente quem receberá qual oferta:

-- 'Oferta dia das mães' ---> Mulher com filhos
-- 'Oferta dia dos pais' ---> Homem com filhos
-- 'Sem oferta'          ---> Clientes sem filhos

-- Você deve criar uma coluna extra identificando cada status.
```

CASE + AND (Exemplo)

A estrutura final da solução é mostrada ao lado.

A lógica é basicamente o seguinte:

Quando o sexo do cliente for Feminino E (AND) a quantidade de filhos maior que zero, então a promoção é de dia das mães.

Agora se o sexo do cliente for Masculino E (AND) a quantidade de filhos maior que zero, então a promoção é de dia dos pais.

Caso contrário, não há uma oferta disponível.

```
35 • SELECT
36       *,
37       CASE
38           WHEN Sexo = 'F' AND Qtd_Filhos > 0 THEN 'Oferta dia das mães'
39           WHEN Sexo = 'M' AND Qtd_Filhos > 0 THEN 'Oferta dia dos pais'
40           ELSE 'Sem oferta'
41       END AS 'Situação'
42   FROM clientes;
43
44
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ID_Cliente	Nome	Sobrenome	Data_Nascimento	Estado_Civil	Sexo	Email	Telefone	Renda_Anual	Qtd_Filhos	Escolaridade	Situação
62	Chase	Reed	1995-12-02	C	M	jessica29@gmail.com	(21) 93673-6903	60000	0	Parcial	Sem oferta
63	Jessica	Henderson	1993-10-04	C	F	jessica29@gmail.com	(21) 93673-6903	60000	0	Parcial	Sem oferta
64	Grace	Butler	1993-11-22	C	F	grace62@gmail.com	[NULL]	70000	0	Parcial	Sem oferta
65	Caleb	Carter	1996-09-20	S	M	caleb40@gmail.com	(31) 97809-1800	60000	0	Parcial	Sem oferta
66	Tiffany	Liang	1975-09-18	S	F	tiffany17@gmail.com	(21) 97231-3503	80000	2	Ensino médio	Oferta dia das mães
67	Carolyn	Navarro	1975-09-16	S	F	carolyn30@yahoo.com.br	(21) 94139-6777	80000	2	Ensino médio	Oferta dia das mães
68	Willie	Raji	1975-03-31	C	M	willie40@yahoo.com.br	(71) 97614-1995	80000	2	Ensino médio	Oferta dia dos pais
69	Linda	Serrano	1975-06-21	S	F	linda31@hotmail.com	(21) 92428-2938	80000	2	Ensino médio	Oferta dia das mães
70	Casey	Luo	1975-02-01	S	F	casey6@yahoo.com.br	(31) 96609-9916	80000	2	Ensino médio	Oferta dia das mães
71	Amy	Ye	1976-08-09	S	F	amy16@hotmal.com	(41) 92958-3807	70000	2	Ensino médio	Oferta dia das mães
72	Levi	Arun	1976-08-23	S	M	levi6@gmail.com	(71) 92754-9983	70000	2	Ensino médio	Oferta dia dos pais
73	Felicia	Jimenez	1977-11-11	S	F	felicia4@gmail.com	(85) 95647-1806	80000	2	Ensino médio	Oferta dia das mães
74	Blake	Anderson	1977-07-08	S	M	blake9@gmail.com	(11) 98232-2736	80000	2	Ensino médio	Oferta dia dos pais
75	Leah	Ye	1977-09-14	S	F	leah7@hotmal.com	(31) 95663-3082	80000	2	Ensino médio	Oferta dia das mães

Result 48

CASE + OR (Exemplo)

O enunciado do segundo exemplo é mostrado abaixo. Precisaremos utilizar a combinação CASE + OR para resolvê-lo.

```
-- APLICAÇÃO OR
-- Exemplo 2. A empresa está com uma parceria com as empresas das marcas 'DELL' e 'SAMSUNG'. Isso significa que os produtos dessas marcas receberão um desconto de 15% em seu custo de aquisição. Faça uma consulta que retorne uma coluna extra de desconto no custo de aquisição de cada produto.
```

CASE + OR (Exemplo)

Neste novo exemplo, queremos oferecer um desconto de 15% para os produtos de qualquer uma das marcas: DELL ou SAMSUNG.

Para isso, usamos a combinação do CASE + OR.

O resultado final é mostrado ao lado.

```
64 • SELECT
65       *,
66       CASE
67           WHEN Marca_Produto = 'DELL' OR Marca_Produto = 'SAMSUNG' THEN (1 - 0.15) * Custo_Unit
68           ELSE Custo_Unit
69       END AS 'Desconto?'
70   FROM produtos;
71
72
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ID_Produto	Nome_Produto	ID_Categoria	Marca_Produto	Num_Serie	Preco_Unit	Custo_Unit	Desconto?
1	Monitor LED 19,5" Full HD HDMI	1	DELL	MNT-DL-831923	2300	966	821.1
2	Monitor Curvo 24" 144Hz HDMI	1	SAMSUNG	MNT-SS-001939	2800	980	833
3	Webcam Full HD 1080p	1	LOGITECH	WBC-LT-934GG4	450	90	90
4	Kit Teclado + Mouse sem fio Wireless	2	DELL	KTM-DL-041039	350	129.5	110.075
5	Kit Teclado + Mouse Slim Bluetooth	2	DELL	KTM-DL-111924	280	109.2	92.82
6	Cadeira Gamer reclinável Azul/Laranja	3	ALTURA	CGM-AL-9N9143	1800	540	540
7	Cadeira Gamer PC Racer Vermelha	3	ALTURA	CGM-AL-0147FI	3100	1395	1395
8	Headphone Bluetooth 2000	4	SONY	HDP-SN-194821	600	258	258
9	Fone de Ouvido Tune T5000	4	JBL	HDP-JB-091934	780	327.6	327.6
10	Microfone Condensador MC1000	5	AKG	MIC-AK-237591	1100	275	275
11	Microfone Condensador com Tripé	5	BLUE	MIC-BL-819455	800	344	344
12	Microfone de mesa com fio condensa...	5	BLUE	MIC-BL-761411	650	214.5	214.5
13	Notebook LC2100 Intel Core i5 8GB	6	SAMSUNG	NOT-SS-918457	3400	850	722.5
14	Notebook Inspiron 15 5000 4GB	6	DELL	NOT-DL-000012	3100	1209	1027.649...

Result 51

EXERCÍCIOS

1

Questão 1

O setor de compras quer classificar os produtos de acordo com o custo unitário.

Crie uma consulta que categorize os produtos como "Alto Custo" (Custo_Unit > 1000), "Médio Custo" (Custo_Unit entre 500 e 1000) e "Baixo Custo" (Custo_Unit < 500).

Liste o nome do produto, o custo unitário e a classificação. **(Utilize a tabela produtos)**

2

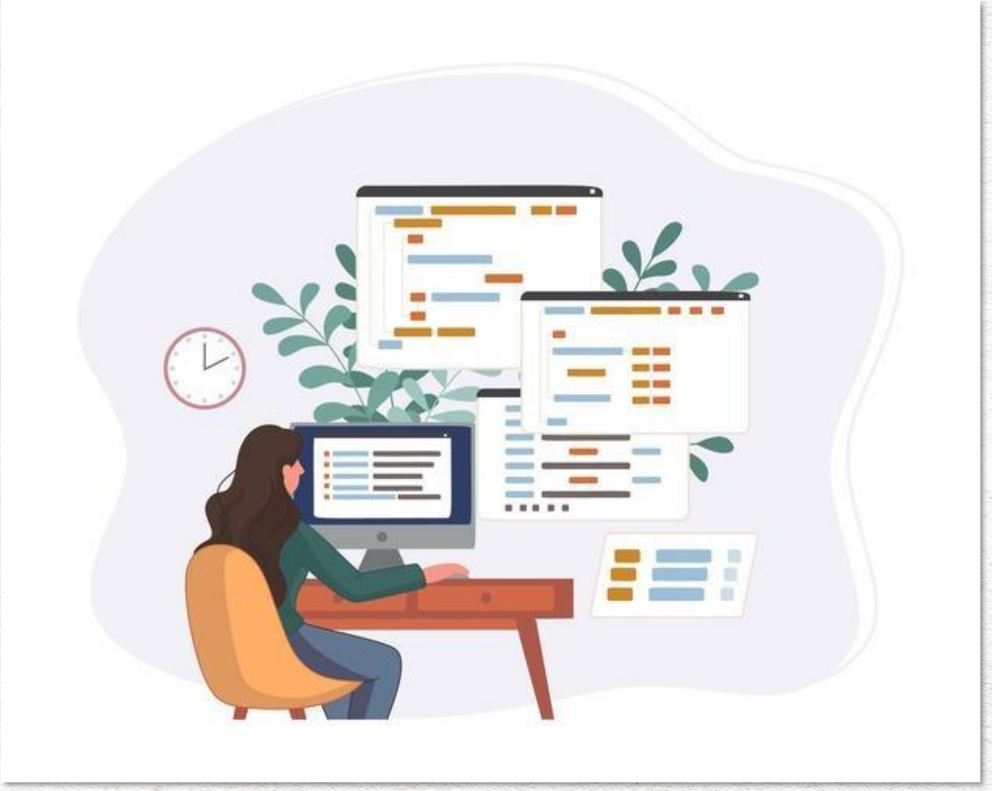
Questão 2

O setor de atendimento identificou que alguns clientes não cadastraram um telefone.

Crie uma consulta que liste o ID do cliente, o nome completo, o telefone e uma coluna de status que indique se o telefone foi cadastrado ou não ("Não informado" para campos sem telefone e "OK" para os demais).

Lembre-se que existem tanto campos nulos quanto com *strings* vazias na coluna Telefone. **(Utilize a tabela clientes)**

EXERCÍCIOS



3

Questão 3

A empresa decidiu aplicar um reajuste de 10% no preço dos produtos que não pertencem às marcas "DELL" e "SAMSUNG". Para isso, é necessário criar uma consulta que exiba o nome do produto, a marca, o preço atual e o novo preço com o reajuste aplicado.

Caso o produto pertença às marcas "DELL" ou "SAMSUNG", o preço permanece inalterado.

Elabore uma consulta que atenda a essa necessidade. **(Utilize a tabela produtos)**

4

Questão 4

A diretoria quer identificar os gerentes de lojas que administram equipes com menos de 15 funcionários e mais de 20 funcionários.

Crie uma consulta que exiba o nome do gerente, a loja, a quantidade de funcionários e o status:

- ✓ "Equipe pequena" para menos de 15 funcionários;
- ✓ "Equipe grande" para mais de 20;
- ✓ "Equipe média" nos demais casos.

Se possível, tente resolver utilizando IFs. **(Utilize a tabela lojas)**

EXERCÍCIOS

5

Questão 5

A equipe financeira quer calcular a margem de lucro unitária para cada produto e categorizá-la como:

- ✓ "Alta margem" para produtos com margem acima de R$1000;
- ✓ "Média margem" entre R$300 e R$1000;
- ✓ "Baixa margem" para margens abaixo de R$300.

(Utilize a tabela produtos)

6

Questão 6

A empresa quer identificar clientes prioritários, definidos como aqueles com uma renda anual acima de R$80.000 ou que já realizaram no mínimo 450 pedidos.

Crie uma consulta que exiba o nome completo do cliente, a renda anual, a quantidade de pedidos, além de uma coluna que indique se ele é 'prioritário' ou 'comum'. **(Utilize as tabelas clientes e pedidos)**

EXERCÍCIOS

7

Questão 7

A diretoria deseja identificar a performance de cada loja, levando em consideração o número de funcionários e a receita gerada por cada um deles.

Crie uma consulta que exiba o nome da loja, a quantidade de funcionários, a receita total da loja, a receita gerada por funcionário e um indicador de desempenho:

- ✓ "Alta Performance" para lojas com mais de R$700.000 em vendas por funcionário;
- ✓ "Média Performance" para vendas por funcionário entre R$400.000 e R$700.000;
- ✓ "Baixa Performance" para valores abaixo de R$400.000.

(Utilize as tabelas lojas e pedidos)

GABARITO EXPLICATIVO

1

Questão 1

O setor de compras quer classificar os produtos de acordo com o custo unitário.

Crie uma consulta que categorize os produtos como "Alto Custo" (Custo_Unit > 1000), "Médio Custo" (Custo_Unit entre 500 e 1000) e "Baixo Custo" (Custo_Unit < 500). Liste o nome do produto, o custo unitário e a classificação. **(Utilize a tabela produtos)**

Nome_Produto	Custo_Unit	Classificação Custo
Monitor LED 19,5" Full HD HDMI	966	Médio Custo
Monitor Curvo 24" 144Hz HDMI	980	Médio Custo
Webcam Full HD 1080p	90	Baixo Custo
Kit Teclado + Mouse sem fio Wireless	129.5	Baixo Custo
Kit Teclado + Mouse Slim Bluetooth	109.2	Baixo Custo
Cadeira Gamer reclinável Azul/Laranja	540	Médio Custo
Cadeira Gamer PC Racer Vermelha	1395	Alto Custo
Headphone Bluetooth 2000	258	Baixo Custo
Fone de Ouvido Tune T5000	327.6	Baixo Custo
Microfone Condensador MC1000	275	Baixo Custo
Microfone Condensador com Tripé	344	Baixo Custo
Microfone de mesa com fio condensador	214.5	Baixo Custo
Notebook LC2100 Intel Core i5 8GB	850	Médio Custo
Notebook Inspiron 15 5000 4GB	1209	Alto Custo
Notebook IdeaPad RF32000	1176	Alto Custo
Notebook Motion Ultra 2	1160	Alto Custo

Explicação e Resultado

Para **classificar os produtos de acordo com seu custo unitário**, podemos aplicar a função condicional **CASE**, categorizando os produtos em três faixas de custo que serão retornadas na coluna 'Classificação Custo' (formada pelo CASE):

Quando o custo unitário for superior a 1000 (**Custo_Unit > 1000**), retornamos a informação **'Alto Custo'**; quando o custo unitário estiver entre 500 e 1000 (**Custo_Unit BETWEEN 500 AND 1000**), retornamos **'Médio Custo'**; e para todos os outros casos (**ELSE**), ou seja, quando o custo unitário for inferior a 500, retornamos **'Baixo Custo'**.

Além disso, atendendo ao solicitado no enunciado, também **retornamos** as colunas **Nome_Produto** e **Custo_Unit** da tabela **PRODUTOS**:

```sql
SELECT
    Nome_Produto,
    Custo_Unit,
    CASE
        WHEN Custo_Unit > 1000 THEN 'Alto Custo'
        WHEN Custo_Unit BETWEEN 500 AND 1000 THEN 'Médio Custo'
        ELSE 'Baixo Custo'
    END AS 'Classificação Custo'
FROM produtos;
```

Dessa forma, retornamos o resultado ao lado:

Índice Geral
Índice deste Módulo

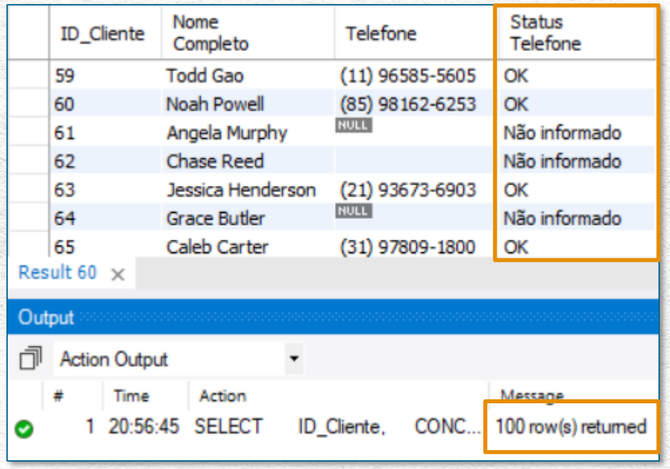
GABARITO EXPLICATIVO

2

Questão 2

O setor de atendimento identificou que alguns clientes não cadastraram um telefone.

Crie uma consulta que liste o ID do cliente, o nome completo, o telefone e uma coluna de status que indique se o telefone foi cadastrado ou não ("Não informado" para campos sem telefone e "OK" para os demais). Lembre-se que existem tanto campos nulos quanto com *strings* vazias na coluna Telefone. **(Utilize a tabela clientes)**



ID_Cliente	Nome Completo	Telefone	Status Telefone
59	Todd Gao	(11) 96585-5605	OK
60	Noah Powell	(85) 98162-6253	OK
61	Angela Murphy	NULL	Não informado
62	Chase Reed		Não informado
63	Jessica Henderson	(21) 93673-6903	OK
64	Grace Butler	NULL	Não informado
65	Caleb Carter	(31) 97809-1800	OK

Explicação e Resultado

Para **verificar se os telefones dos clientes foram cadastrados na tabela CLIENTES**, seguimos os seguintes passos:

No **SELECT**, escolhemos o **ID_Cliente**, o nome completo concatenado a partir do nome e sobrenome (**CONCAT(Nome, ' ', Sobrenome) AS 'Nome Completo'**) e **Telefone** para exibir as informações solicitadas.

Utilizamos a função condicional **IF()** para verificar se o telefone foi informado: usamos **ISNULL(Telefone)** para identificar valores **nulos** e **Telefone = ''** para identificar ***strings* vazias**. Se o telefone for nulo ou vazio, a coluna **'Status Telefone'** recebe a informação **"Não informado"**. Caso contrário, se o telefone estiver preenchido, a coluna **'Status Telefone'** recebe **"OK"**:

```
SELECT
    ID_Cliente,
    CONCAT(Nome, ' ', Sobrenome) AS 'Nome Completo',
    Telefone,
    IF(ISNULL(Telefone) OR Telefone = '', 'Não informado', 'OK') AS 'Status Telefone'
FROM clientes;
```

Dessa forma, retornamos o resultado ao lado:

Índice Geral
Índice deste Módulo

GABARITO EXPLICATIVO

3

Questão 3

A empresa decidiu aplicar um reajuste de 10% no preço dos produtos que não pertencem às marcas "DELL" e "SAMSUNG". Para isso, é necessário criar uma consulta que exiba o nome do produto, a marca, o preço atual e o novo preço com o reajuste aplicado.

Caso o produto pertença às marcas "DELL" ou "SAMSUNG", o preço permanece inalterado. Elabore uma consulta que atenda a essa necessidade. **(Utilize a tabela produtos)**

Nome_Produto	Marca_Produto	Preco_Unit	Preço com reajuste
Monitor LED 19,5" Full HD HDMI	DELL	2300	2300
Monitor Curvo 24" 144Hz HDMI	SAMSUNG	2800	2800
Webcam Full HD 1080p	LOGITECH	450	495.0
Kit Teclado + Mouse sem fio Wireless	DELL	350	350
Kit Teclado + Mouse Slim Bluetooth	DELL	280	280
Cadeira Gamer reclinável Azul/Laranja	ALTURA	1800	1980.0
Cadeira Gamer PC Racer Vermelha	ALTURA	3100	3410.0
Headphone Bluetooth 2000	SONY	600	660.0
Fone de Ouvido Tune T5000	JBL	780	858.0
Microfone Condensador MC1000	AKG	1100	1210.0
Microfone Condensador com Tripé	BLUE	800	880.0
Microfone de mesa com fio condensador	BLUE	650	715.0
Notebook LC2100 Intel Core i5 8GB	SAMSUNG	3400	3400
Notebook Inspiron 15 5000 4GB	DELL	3100	3100
Notebook IdeaPad RF32000	DELL	4200	4200
Notebook Motion Ultra 2	SAMSUNG	2900	2900

Explicação e Resultado

Para **aplicar o reajuste de 10% nos produtos que não pertencem às marcas "DELL" ou "SAMSUNG"**, seguimos os seguintes passos:

No **SELECT**, selecionamos as colunas **Nome_Produto**, **Marca_Produto** e **Preco_Unit** da tabela **PRODUTOS** para exibir as informações solicitadas.

Utilizamos a função CASE para calcular o novo preço: a condição **WHEN Marca_Produto NOT IN ('DELL', 'SAMSUNG')** verifica se o produto **não pertence** às marcas **"DELL"** ou **"SAMSUNG"**. Para esses produtos, **o preço é multiplicado por 1.1**, aplicando o **reajuste de 10%**. Caso o produto **pertença** a essas marcas, o **preço permanece inalterado**, usando **ELSE Preco_Unit**. A coluna resultante, chamada **"Preço com reajuste"**, exibe o preço ajustado ou o preço original, dependendo da marca do produto:

```
SELECT
    Nome_Produto,
    Marca_Produto,
    Preco_Unit,
    CASE
        WHEN Marca_Produto NOT IN ('DELL', 'SAMSUNG') THEN Preco_Unit * 1.1
        ELSE Preco_Unit
    END AS 'Preço com reajuste'
FROM produtos;
```

Dessa forma, retornamos o resultado ao lado:

Índice Geral

Índice deste Módulo

GABARITO EXPLICATIVO

4

Questão 4

A diretoria quer identificar os gerentes de lojas que administram equipes com menos de 15 funcionários e mais de 20 funcionários.

Crie uma consulta que exiba o nome do gerente, a loja, a quantidade de funcionários e o status:

- ✓ "Equipe pequena" para menos de 15 funcionários;
- ✓ "Equipe grande" para mais de 20;
- ✓ "Equipe média" nos demais casos.

Se possível, tente resolver utilizando IFs. **(Utilize a tabela lojas)**

	Gerente	Loja	Num_Funcionarios	Status
▶	Pedro Martins	Rio de Janeiro	23	Equipe grande
	Julia Freitas	Belo Horizonte	13	Equipe pequena
	Marcelo Castro	Salvador	19	Equipe média
	Amanda Lima	Curitiba	31	Equipe grande
	Letícia Gomes	Fortaleza	12	Equipe pequena
	Rogério Lopes	São Paulo	33	Equipe grande
	Matheus Leal	Niterói	18	Equipe média
	Carlos Henrique	Goiânia	11	Equipe pequena

Explicação e Resultado

Para **identificar os gerentes de lojas com equipes pequenas, médias ou grandes**, seguimos os seguintes passos:

No **SELECT**, selecionamos as colunas **Gerente**, **Loja** e **Num_Funcionarios** da tabela **LOJAS** para exibir as informações solicitadas.

Utilizamos a função **IF()** para atribuir um **status** à equipe com base na quantidade de funcionários: se a quantidade de funcionários for menor que 15 (**Num_Funcionarios < 15**), a coluna **"Status"** recebe o valor **'Equipe pequena'**; se a quantidade de funcionários for maior que 20 (**Num_Funcionarios > 20**), a coluna **"Status"** recebe o valor **'Equipe grande'**; nos **demais casos**, a coluna **"Status"** recebe o valor **'Equipe média'**:

```sql
SELECT
    Gerente,
    Loja,
    Num_Funcionarios,
    IF(Num_Funcionarios < 15, 'Equipe pequena',
        IF(Num_Funcionarios > 20, 'Equipe grande', 'Equipe média')
    ) AS 'Status'
FROM lojas;
```

Dessa forma, retornamos o resultado ao lado:

Índice Geral

Índice deste Módulo

GABARITO EXPLICATIVO

5

Questão 5

A equipe financeira quer calcular a margem de lucro unitária para cada produto e categorizá-la como:

- ✓ "Alta margem" para produtos com margem acima de R$1000;
- ✓ "Média margem" entre R$300 e R$1000;
- ✓ "Baixa margem" para margens abaixo de R$300.

(Utilize a tabela produtos)

	Nome_Produto	Preco_Unit	Custo_Unit	Margem Unitária	Classificação Margem
▶	Monitor LED 19,5" Full HD HDMI	2300	966	1334	Alta margem
	Monitor Curvo 24" 144Hz HDMI	2800	980	1820	Alta margem
	Webcam Full HD 1080p	450	90	360	Média margem
	Kit Teclado + Mouse sem fio Wireless	350	129.5	220.5	Baixa margem
	Kit Teclado + Mouse Slim Bluetooth	280	109.2	170.8	Baixa margem
	Cadeira Gamer reclinável Azul/Laranja	1800	540	1260	Alta margem
	Cadeira Gamer PC Racer Vermelha	3100	1395	1705	Alta margem
	Headphone Bluetooth 2000	600	258	342	Média margem
	Fone de Ouvido Tune T5000	780	327.6	452.4	Média margem
	Microfone Condensador MC1000	1100	275	825	Média margem
	Microfone Condensador com Tripé	800	344	456	Média margem
	Microfone de mesa com fio condensador	650	214.5	435.5	Média margem
	Notebook LC2100 Intel Core i5 8GB	3400	850	2550	Alta margem
	Notebook Inspiron 15 5000 4GB	3100	1209	1891	Alta margem
	Notebook IdeaPad RF32000	4200	1176	3024	Alta margem
	Notebook Motion Ultra 2	2900	1160	1740	Alta margem

Explicação e Resultado

Para **calcular a margem de lucro unitária de cada produto e categorizá-la**, seguimos os seguintes passos:

No **SELECT**, selecionamos as colunas **Nome_Produto**, **Preco_Unit** e **Custo_Unit** da tabela **PRODUTOS**.

Calculamos a **margem de lucro unitária** subtraindo o custo unitário do preço unitário (**Preco_Unit - Custo_Unit**), e a coluna resultante é renomeada como **'Margem Unitária'**.

Utilizamos a função **CASE** para classificar a margem de lucro: quando a **margem unitária for superior a R$1000**, a coluna **"Classificação Margem"** recebe **'Alta margem'**; quando a **margem unitária estiver entre R$300 e R$1000**, a coluna **"Classificação Margem"** recebe **'Média margem'**; para **margens abaixo de R$300**, a coluna **"Classificação Margem"** recebe **'Baixa margem'**:

```sql
SELECT
    Nome_Produto,
    Preco_Unit,
    Custo_Unit,
    Preco_Unit - Custo_Unit AS 'Margem Unitária',
    CASE
        WHEN Preco_Unit - Custo_Unit > 1000 THEN 'Alta margem'
        WHEN Preco_Unit - Custo_Unit BETWEEN 300 AND 1000 THEN 'Média margem'
        ELSE 'Baixa margem'
    END AS 'Classificação Margem'
FROM produtos;
```

Assim, retornamos o resultado ao lado:

Índice Geral

Índice deste Módulo

GABARITO EXPLICATIVO

6

Questão 6

A empresa quer identificar clientes prioritários, definidos como aqueles com uma renda anual acima de R$80.000 ou que já realizaram no mínimo 450 pedidos.

Crie uma consulta que exiba o nome completo do cliente, a renda anual, a quantidade de pedidos, além de uma coluna que indique se ele é 'prioritário' ou 'comum'. **(Utilize as tabelas clientes e pedidos)**

Explicação e Resultado

Para **identificar os clientes prioritários com base na renda anual ou na quantidade de pedidos**, seguimos os seguintes passos:

No **SELECT**, selecionamos o nome completo do cliente (concatenando as colunas **Nome** e **Sobrenome** da tabela **CLIENTES**, utilizando a função **CONCAT()**), a coluna **Renda_Anual** e a quantidade de pedidos realizados por cada cliente (função **COUNT(p.ID_Pedido)**).

Utilizamos a função **CASE** para definir se o cliente é **prioritário** ou **comum**: a condição **WHEN c.Renda_Anual > 80000 OR COUNT(p.ID_Pedido) > 450** verifica se o cliente tem uma renda anual superior a R$80.000 ou se ele realizou mais de 450 pedidos. **Se qualquer uma das condições for verdadeira**, o cliente é classificado como **'Prioritário'**; caso contrário (**ELSE**), o cliente é classificado como **'Comum'**.

Fazemos um **INNER JOIN** entre as tabelas **CLIENTES** e **PEDIDOS**, estabelecendo a relação pela coluna **ID_Cliente**, que é comum nas duas tabelas.

A consulta é **agrupada por c.ID_Cliente** para garantir que os cálculos sejam feitos por cliente, exibindo uma linha por cliente.

A consulta exibe o nome completo do cliente, a renda anual, a quantidade de pedidos realizados por cada cliente e o status, classificando o cliente como 'Prioritário' ou 'Comum' conforme os critérios estabelecidos:

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

6

Questão 6

A empresa quer identificar clientes prioritários, definidos como aqueles com uma renda anual acima de R$80.000 ou que já realizaram no mínimo 450 pedidos.

Crie uma consulta que exiba o nome completo do cliente, a renda anual, a quantidade de pedidos, além de uma coluna que indique se ele é 'prioritário' ou 'comum'. **(Utilize as tabelas clientes e pedidos)**

Explicação e Resultado

```sql
SELECT
    CONCAT(c.Nome, ' ', c.Sobrenome) AS 'Cliente',
    Renda_Anual,
    COUNT(p.ID_Pedido) AS 'Qtd Pedidos',
    CASE
        WHEN c.Renda_Anual > 80000 OR COUNT(p.ID_Pedido) > 450 THEN 'Prioritário'
        ELSE 'Comum'
    END AS 'Status'
FROM clientes AS c
INNER JOIN pedidos AS p
    ON c.ID_Cliente = p.ID_Cliente
GROUP BY c.ID_Cliente;
```

Dessa forma, retornamos o resultado ao lado:

Cliente	Renda_Anual	Qtd Pedidos	Status
Seth Edwards	40000	434	Comum
Jill Jimenez	30000	452	Prioritário
Cheryl Diaz	90000	400	Prioritário
Felicia Jimenez	80000	430	Comum
Clarence Rai	30000	458	Prioritário
Jimmy Moreno	30000	425	Comum

Result 72

Output

Action Output

#	Time	Action		Message
1	21:54:39	SELECT	CONCAT(...	100 row(s) returned

GABARITO EXPLICATIVO

7

Questão 7

A diretoria deseja identificar a performance de cada loja, levando em consideração o número de funcionários e a receita gerada por cada um deles.

Crie uma consulta que exiba o nome da loja, a quantidade de funcionários, a receita total da loja, a receita gerada por funcionário e um indicador de desempenho:

- ✓ "Alta Performance" para lojas com mais de R$700.000 em vendas por funcionário;
- ✓ "Média Performance" para vendas por funcionário entre R$400.000 e R$700.000;
- ✓ "Baixa Performance" para valores abaixo de R$400.000.

(Utilize as tabelas lojas e pedidos)

Explicação e Resultado

Para **analisar a performance das lojas com base na receita gerada por funcionário**, seguimos os seguintes passos:

No **SELECT**, selecionamos as colunas **Loja** e **Num_Funcionarios** da tabela **LOJAS**, além de calcularmos a receita total da loja aplicando a função **SUM()** a coluna **Receita_Venda** da tabela **PEDIDOS** (**SUM(Receita_Venda)**), renomeando a coluna como **'Receita Total'**.

Calculamos a **receita gerada por funcionário** dividindo a receita total da loja pela quantidade de funcionários (**SUM(Receita_Venda) / Num_Funcionarios**). A coluna resultante é chamada de **'Receita por Funcionário'**.

Utilizamos a função **CASE** para classificar a performance das lojas: quando a **receita por funcionário for superior a R$700.000**, a loja é classificada como **'Alta Performance'**; quando a **receita por funcionário estiver entre R$400.000 e R$700.000**, a loja é classificada como **'Média Performance'**; para **valores abaixo de R$400.000**, a loja é classificada como **'Baixa Performance'**.

Realizamos um **INNER JOIN** entre as tabelas **LOJAS** e **PEDIDOS**, relacionando as colunas **ID_Loja** de ambas as tabelas para associar as vendas a cada loja.

A consulta é **agrupada** pelas colunas **Loja** e **Num_Funcionarios** para garantir que os cálculos de receita e a classificação de desempenho sejam feitos por loja.

A consulta exibe o nome da loja, o número de funcionários, a receita total da loja, a receita gerada por funcionário e a classificação de desempenho, conforme os critérios definidos:

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

7

Questão 7

A diretoria deseja identificar a performance de cada loja, levando em consideração o número de funcionários e a receita gerada por cada um deles.

Crie uma consulta que exiba o nome da loja, a quantidade de funcionários, a receita total da loja, a receita gerada por funcionário e um indicador de desempenho:

- ✓ "Alta Performance" para lojas com mais de R$700.000 em vendas por funcionário;
- ✓ "Média Performance" para vendas por funcionário entre R$400.000 e R$700.000;
- ✓ "Baixa Performance" para valores abaixo de R$400.000.

(Utilize as tabelas lojas e pedidos)

Explicação e Resultado

```sql
SELECT
    Loja,
    Num_Funcionarios,
    SUM(Receita_Venda) AS 'Receita Total',
    SUM(Receita_Venda) / Num_Funcionarios AS 'Receita por Funcionário',
    CASE
        WHEN SUM(Receita_Venda) / Num_Funcionarios > 700000 THEN 'Alta Performance'
        WHEN SUM(Receita_Venda) / Num_Funcionarios BETWEEN 400000 AND 700000 THEN 'Média Performance'
        ELSE 'Baixa Performance'
    END AS 'Desempenho'
FROM lojas AS l
INNER JOIN pedidos AS p
    ON l.ID_Loja = p.ID_Loja
GROUP BY Loja, Num_Funcionarios;
```

Dessa forma, retornamos o resultado abaixo:

Loja	Num_Funcionarios	Receita Total	Receita por Funcionário	Desempenho
São Paulo	33	10443680	316475.1515	Baixa Performance
Belo Horizonte	13	10061430	773956.1538	Alta Performance
Rio de Janeiro	23	10037160	436398.2609	Média Performance
Goiânia	11	9943620	903965.4545	Alta Performance
Fortaleza	12	10217870	851489.1667	Alta Performance
Salvador	19	10107860	531992.6316	Média Performance
Niterói	18	9986950	554830.5556	Média Performance
Curitiba	31	10139870	327092.5806	Baixa Performance

MÓDULO 11

VIEWS

VIEWS

VIEWS

Índice Geral

Índice deste Módulo

O QUE VEREMOS NESTE MÓDULO

MÓDULO 11 VIEWS

01	VIEWS: INTRODUÇÃO	287
02	O QUE É UMA VIEW?	288
03	POR QUE CRIAR UMA VIEW?	289
04	CREATE VIEW: CRIANDO A PRIMEIRA VIEW	290
05	ALTER VIEW: ALTERANDO A VIEW CRIADA	291
06	DROP VIEW: EXCLUINDO UMA VIEW	293
07	CRIANDO UMA VIEW COM WHERE (EXEMPLO 1)	294
08	CRIANDO UMA VIEW COM WHERE (EXEMPLO 2)	296
09	CRIANDO UMA VIEW COM O COMANDO GROUP BY	298
10	ALTERANDO A VIEW CRIADA COM O WHERE	300
11	ALTERANDO A VIEW CRIADA COM O HAVING	301
12	CRIANDO UMA VIEW COM JOIN	302
13	CRIANDO UMA VIEW COM JOIN + GROUP BY	304
14	EXERCÍCIOS	305
15	GABARITO EXPLICATIVO	310

VIEWS: Introdução

Até aqui vimos como criar diferentes consultas aos bancos de dados. Utilizamos comandos como o SELECT, GROUP BY, JOINs, etc para criar tabelas como a mostrada ao lado.

Mas pra onde foram todas essas “tabelas” que a gente criou? Elas estão em algum lugar?

A resposta é: elas não estão em nenhum lugar!

Tudo o que fizemos até agora foi apenas visualizar alguns dados das nossas tabelas do Banco de Dados, nada além disso. Quando executamos um SELECT e logo em seguida executamos outro SELECT, o resultado do primeiro é perdido.

Nenhuma das consultas que fizemos ficou salvo em algum lugar. Inclusive, diversas vezes precisamos criar as mesmas consultas, pois elas se perdem a cada novo SELECT, ou quando fechamos uma consulta e abrimos uma nova.

Existe uma solução pra gente conseguir salvar essas tabelas em algum lugar, e essa solução é a **View**.

```
1 • SELECT
2       Nome AS 'Nome do Cliente',
3       Data_Nascimento AS 'Data de Nascimento',
4       Email AS 'E-mail'
5   FROM clientes;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Nome do Cliente	Data de Nascimento	E-mail
Ruben	1985-08-07	ruben35@hotmail.com
Christy	1988-02-10	christy12@hotmail.com
Elizabeth	1988-08-03	elizabeth5@gmail.com
Julio	1985-07-31	julio1@hotmail.com
Janet	1985-12-01	janet9@gmail.com
Marco	1984-05-04	marco14@yahoo.com.br
Rob	1984-07-02	rob4@hotmail.com
Shannon	1984-03-27	shannon38@gmail.com
Jacquelyn	1984-02-01	jacquelyn20@hotmail.com
Curtis	1983-10-30	curtis9@yahoo.com.br
Lauren	1988-01-13	lauren41@hotmail.com
Ian	1988-08-01	ian47@gmail.com
Sydney	1988-05-04	sydney23@gmail.com
Chloe	1999-02-22	chloe23@hotmail.com
Wyatt	1999-04-23	wyatt32@gmail.com
Shannon	1964-06-21	shannon1@hotmail.com
Clarence	1964-10-04	clarence32@yahoo.com.br
Jordan	1998-09-15	jordan73@gmail.com

O que é uma View?

- Uma View (ou traduzindo, uma exibição), é uma tabela virtual criada a partir de uma consulta a uma ou mais tabelas (ou até mesmo de outras views) no banco de dados.

- Ela contém linhas e colunas assim como uma tabela real. Nela podemos utilizar comandos como o JOIN, WHERE e outras funções.

- Sempre mostra resultados atualizados dos dados, ou seja, uma vez criada uma View, caso haja alterações no Banco de Dados, as Views são atualizadas automaticamente.

- Caso o servidor seja desligado (ou o SSMS fechado), a view continua armazenada no sistema.

Através de uma View, conseguimos armazenar o resultado de um SELECT (como por exemplo, a tabela à direita) e acessar sempre que precisar, como se fosse uma tabela, com a vantagem de não precisar criar esse SELECT do zero.

```sql
SELECT
    Nome AS 'Nome do Cliente',
    Data_Nascimento AS 'Data de Nascimento',
    Email AS 'E-mail'
FROM clientes;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Nome do Cliente	Data de Nascimento	E-mail
Ruben	1985-08-07	ruben35@hotmail.com
Christy	1988-02-10	christy12@hotmail.com
Elizabeth	1988-08-03	elizabeth5@gmail.com
Julio	1985-07-31	julio1@hotmail.com
Janet	1985-12-01	janet9@gmail.com
Marco	1984-05-04	marco14@yahoo.com.br
Rob	1984-07-02	rob4@hotmail.com
Shannon	1984-03-27	shannon38@gmail.com
Jacquelyn	1984-02-01	jacquelyn20@hotmail.com
Curtis	1983-10-30	curtis9@yahoo.com.br
Lauren	1988-01-13	lauren41@hotmail.com
Ian	1988-08-01	ian47@gmail.com
Sydney	1988-05-04	sydney23@gmail.com
Chloe	1999-02-22	chloe23@hotmail.com
Wyatt	1999-04-23	wyatt32@gmail.com
Shannon	1964-06-21	shannon1@hotmail.com
Clarence	1964-10-04	clarence32@yahoo.com.br
Jordan	1998-09-15	jordan73@gmail.com

Por que criar uma View?

São muitas as vantagens de uma View. Abaixo temos algumas das principais:

Reutilização

Sempre que necessário, podemos consultar aquela View, pois ela fica armazenada no sistema.

Segurança

Ao criar uma View, estamos ocultando linhas ou colunas da tabela original do banco de dados. Desta forma, apenas algumas informações relevantes serão visualizadas na View.

Ganho de tempo

Quando criamos Views, estamos poupando o tempo de recriar vários SELECTs, o que aumenta a produtividade.

CREATE VIEW: Criando a primeira view

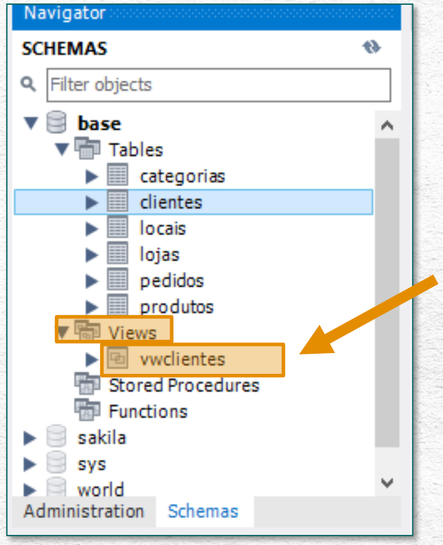
Para criar uma View, utilizamos o comando **CREATE VIEW**. Na imagem abaixo temos a estrutura padrão:

```
CREATE VIEW nome_da_view AS
SELECT
        Coluna1,
        Coluna2,
        Coluna3
FROM
        Tabela
```

```
1 • CREATE VIEW vwClientes AS
2   SELECT
3       Nome AS 'Nome do Cliente',
4       Data_Nascimento AS 'Data de Nascimento',
5       Email AS 'E-mail'
6   FROM clientes;
```

Uma vez criada, a View ficará armazenada no banco de dados, na pasta de **Views**.

E para selecionar essa View, utilizamos o comando SELECT.




```
1 • SELECT * FROM vwClientes;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Nome do Cliente	Data de Nascimento	E-mail
Ruben	1985-08-07	ruben35@hotmail.com
Christy	1988-02-10	christy12@hotmail.com
Elizabeth	1988-08-03	elizabeth5@gmail.com
Julio	1985-07-31	julio1@hotmail.com
Janet	1985-12-01	janet9@gmail.com
Marco	1984-05-04	marco14@yahoo.com.br
Rob	1984-07-02	rob4@hotmail.com
Shannon	1984-03-27	shannon38@gmail.com
Jacquelyn	1984-02-01	jacquelyn20@hotmail.com
Curtis	1983-10-30	curtis9@yahoo.com.br
Lauren	1988-01-13	lauren41@hotmail.com

ALTER VIEW: Alterando a view criada

Para alterar uma View criada, usamos o comando **ALTER VIEW**.

Na imagem ao lado, temos um exemplo. A vwClientes, criada anteriormente, foi alterada para considerar apenas os clientes do sexo feminino.

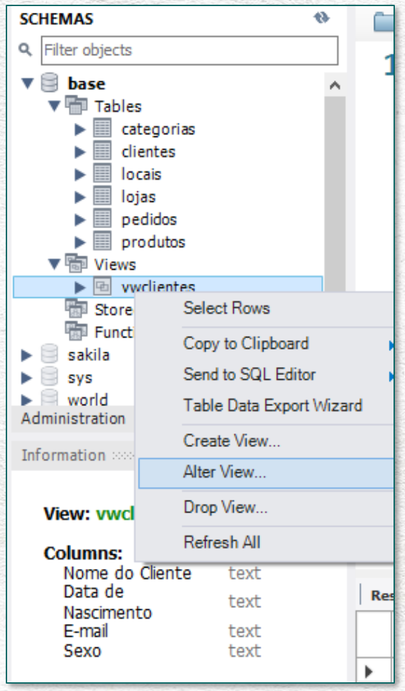
```
1  ALTER VIEW vwClientes AS
2  SELECT
3       Nome AS 'Nome do Cliente',
4       Data_Nascimento AS 'Data de Nascimento',
5       Email AS 'E-mail',
6       Sexo AS 'Sexo'
7  FROM clientes
8  WHERE Sexo = 'F';
9
10 SELECT * FROM vwclientes;
```

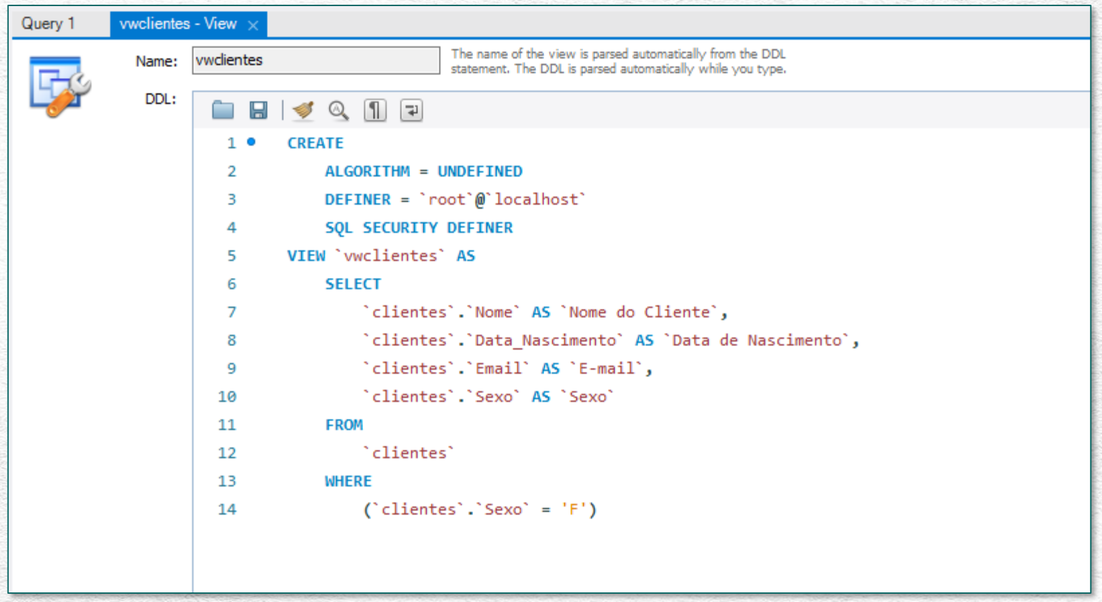
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Nome do Cliente	Data de Nascimento	E-mail	Sexo
Christy	1988-02-10	christy12@hotmail.com	F
Elizabeth	1988-08-03	elizabeth5@gmail.com	F
Janet	1985-12-01	janet9@gmail.com	F
Rob	1984-07-02	rob4@hotmail.com	F
Jacquelyn	1984-02-01	jacquelyn20@hotmail.com	F
Lauren	1988-01-13	lauren41@hotmail.com	F
Sydney	1988-05-04	sydney23@gmail.com	F
Chloe	1999-02-22	chloe23@hotmail.com	F
Shannon	1964-06-21	shannon1@hotmail.com	F
Destiny	1998-08-29	destiny7@hotmail.com	F
Jill	1966-04-06	jill13@gmail.com	F
Bethany	1967-02-17	bethany10@hotmail.com	F
Theresa	1967-08-17	theresa13@hotmail.com	F

ALTER VIEW: Alterando a view criada

Para alterar a view também podemos clicar nela com o botão direito e ir na opção Alter View. A janela que se abre mostra inclusive o código que deu origem à view criada.





DROP VIEW: Excluindo uma view

Para excluir uma View criada, usamos o comando **DROP VIEW**.

O comando é bem simples e é ilustrado na imagem ao lado.

```
1   DROP VIEW vwclientes;
```

Criando uma View com o comando WHERE (Exemplo 1)

No exemplo a seguir vamos criar uma View que combina o comando SELECT com o WHERE.

```
1   # Exemplos Views
2
3   # 1. VIEWS + WHERE: Criando Views com consultas filtradas.
4
5   -- Exemplo 1. Crie uma View chamada vwReceitaAcima4000 que armazene todas as colunas da tabela
    Pedidos. A sua View deverá conter apenas as vendas com receita acima de R$4.000.
```

Criando uma View com o comando WHERE (Exemplo 1)

Aplicando o que aprendemos até então, o resultado final do código é mostrado abaixo.

```
5   -- Exemplo 1. Crie uma View chamada vwReceitaAcima4000 que armazene todas as colunas da tabela
    Pedidos. A sua View deverá conter apenas as vendas com receita acima de R$4.000.
6
7•  SELECT * FROM pedidos;
8
9•  CREATE VIEW vwReceitaAcima4000 AS
0   SELECT
1       *
2   FROM pedidos
3   WHERE Receita_Venda >= 4000;
```

Criando uma View com o comando WHERE (Exemplo 2)

No exemplo 2 vamos mais uma vez criar uma View que utilize o comando WHERE. O enunciado do exemplo é mostrado abaixo.

```
-- Exemplo 2. Crie uma View chamada vwProdutosAtualizada que armazene todas as colunas da tabela
Produtos. A sua view deverá conter apenas os produtos das marcas DELL, SAMSUNG e SONY.
```

Criando uma View com o comando WHERE (Exemplo 2)

Aplicando o que aprendemos até então, o resultado final do código é mostrado abaixo.

```
-- Exemplo 2. Crie uma View chamada vwProdutosAtualizada que armazene todas as colunas da tabela
Produtos. A sua view deverá conter apenas os produtos das marcas DELL, SAMSUNG e SONY.

CREATE VIEW vwProdutosAtualizada AS
SELECT
    *
FROM produtos
WHERE Marca_Produto IN ('DELL', 'SAMSUNG', 'SONY');
```

Criando uma View com o comando GROUP BY

Agora vamos ver um exemplo de como criar uma View utilizando o GROUP BY.

```
# 1. VIEWS + GROUP BY: Criando Views com consultas filtradas.

-- Exemplo 1. Crie uma view que será o resultado de um agrupamento da tabela de pedidos. A ideia
é que você tenha nessa view o total de Receita e Custo agrupado por ID_Produto.
```

Criando uma View com o comando GROUP BY

Aplicando o que já sabemos, o gabarito é mostrado abaixo. Observe que não há nenhuma grande diferença na criação das views com comandos como o WHERE, GROUP BY, etc. Sempre começaremos com o CREATE VIEW, e em seguida o código da consulta que queremos realizar.

```
# 1. VIEWS + GROUP BY: Criando Views com consultas filtradas.

-- Exemplo 1. Crie uma view que será o resultado de um agrupamento da tabela de pedidos. A ideia
é que você tenha nessa view o total de Receita e Custo agrupado por ID_Produto.
```

```
CREATE VIEW vwReceitaECustoTotal AS
SELECT
    ID_Produto,
    SUM(Receita_Venda) AS 'Total Receita',
    SUM(Custo_Venda) AS 'Total Custo'
FROM pedidos
GROUP BY ID_Produto;
```

Alterando a View criada com o WHERE

Criada uma View, podemos alterá-la usando o comando ALTER VIEW. Abaixo, temos um exemplo de como incluir o comando WHERE para trabalharmos em conjunto com o GROUP BY.

```
-- Exemplo 2. Altere a view anterior para mostrar o agrupamento apenas para os produtos da loja 2.

ALTER VIEW vwReceitaECustoTotal AS
SELECT
    ID_Produto,
    SUM(Receita_Venda) AS 'Total Receita',
    SUM(Custo_Venda) AS 'Total Custo'
FROM pedidos
WHERE ID_Loja = 2
GROUP BY ID_Produto;
```

Alterando a View criada com o HAVING

Agora, podemos alterar nossa view usando o comando ALTER VIEW para realizar um filtro com o HAVING, conforme mostrado no exemplo abaixo.

```
-- Exemplo 3. Altere a view anterior para mostrar o agrupamento apenas para os produtos que tiveram uma
receita total maior que 1 milhão, na loja 2.

ALTER VIEW vwReceitaECustoTotal AS
SELECT
    ID_Produto,
    SUM(Receita_Venda) AS 'Total Receita',
    SUM(Custo_Venda) AS 'Total Custo'
FROM pedidos
WHERE ID_Loja = 2
GROUP BY ID_Produto
HAVING SUM(Receita_Venda) >= 1000000;
```

Criando uma View com o comando JOIN

Vejamos agora um exemplo de View onde usamos o comando Join.

```
# 1. VIEWS + JOIN e GROUP BY: Aplicando Join nas views criadas.

-- Exemplo 1. Crie uma view que seja a junção entre as tabelas de pedidos e de produtos. Ou seja, essa view
deve conter todas as colunas da tabela pedidos e as colunas Nome_Produto, Marca_Produto e Num_Serie da tabela
de produtos.
```

Criando uma View com o comando JOIN

Mais uma vez, nenhuma grande novidade na criação da View. Podemos aplicar exatamente o que já sabemos sobre Joins e chegar no resultado abaixo.

```
# 1. VIEWS + JOIN e GROUP BY: Aplicando Join nas views criadas.

-- Exemplo 1. Crie uma view que seja a junção entre as tabelas de pedidos e de produtos. Ou seja, essa view
deve conter todas as colunas da tabela pedidos e as colunas Nome_Produto, Marca_Produto e Num_Serie da tabela
de produtos.
```

```
CREATE VIEW vwPedidosCompleta AS
SELECT
    pe.*,
    pr.Nome_Produto,
    pr.Marca_Produto,
    pr.Num_Serie
FROM pedidos AS pe
INNER JOIN produtos AS pr
    ON pe.ID_Produto = pr.ID_Produto;
```

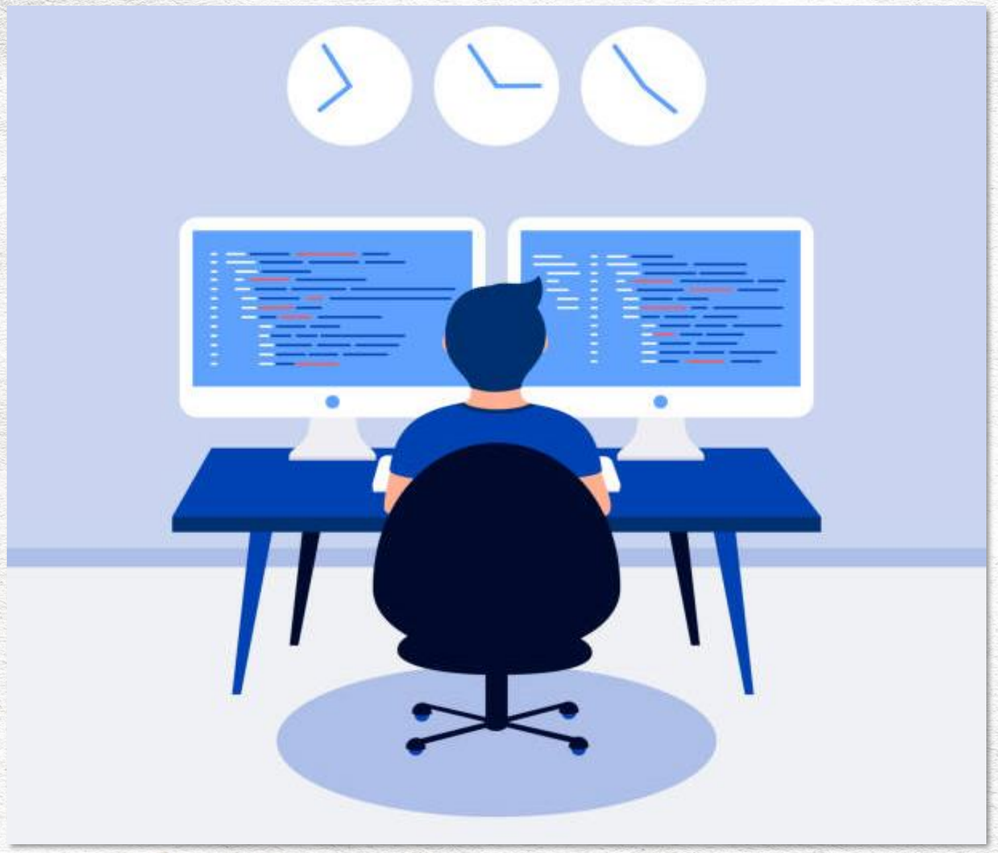
Criando uma View com o comando JOIN + GROUP BY

Por fim, temos um exemplo de criação de View utilizando o comando Join combinado com o Group By.

```
-- Exemplo 2. Crie uma view que será o resultado de um agrupamento da tabela de pedidos. A ideia é que você
tenha nessa view o total de Receita e Custo agrupados por Nome_Produto.

CREATE VIEW vwResultadoFinal AS
SELECT
    pr.Nome_Produto,
    SUM(Receita_Venda) AS 'Receita_Total',
    SUM(Custo_Venda) AS 'Custo_Total'
FROM pedidos AS pe
INNER JOIN produtos AS pr
    ON pe.ID_Produto = pr.ID_Produto
GROUP BY Nome_Produto;
```

EXERCÍCIOS



1

Questão 1

A equipe de marketing deseja uma visão centralizada dos nomes e e-mails de todos os clientes cadastrados.

Crie uma View chamada vwClientesContato que exiba apenas os campos Nome, Sobrenome e Email dos clientes. **(Utilize a tabela clientes)**

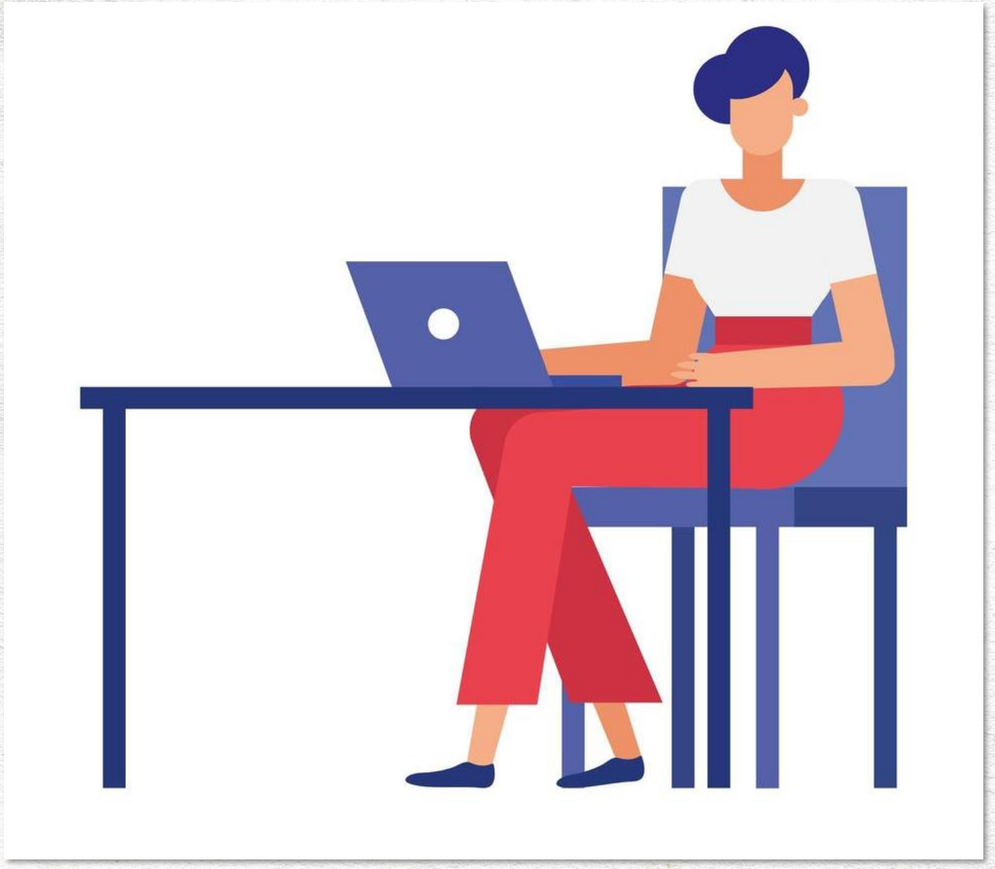
2

Questão 2

O RH deseja identificar as lojas com o menor número de funcionários para possíveis otimizações.

Crie uma View chamada vwLojasFuncionarios que exiba o nome da loja, o gerente e o número de funcionários, ordenados de forma crescente pelo número de funcionários. **(Utilize a tabela lojas)**

EXERCÍCIOS



3

Questão 3

O setor financeiro quer uma amostra dos cinco produtos mais caros da tabela produtos.

Crie uma View chamada vwProdutosAltoValor que exiba o nome do produto, a marca e o preço unitário desses produtos. **(Utilize a tabela produtos)**

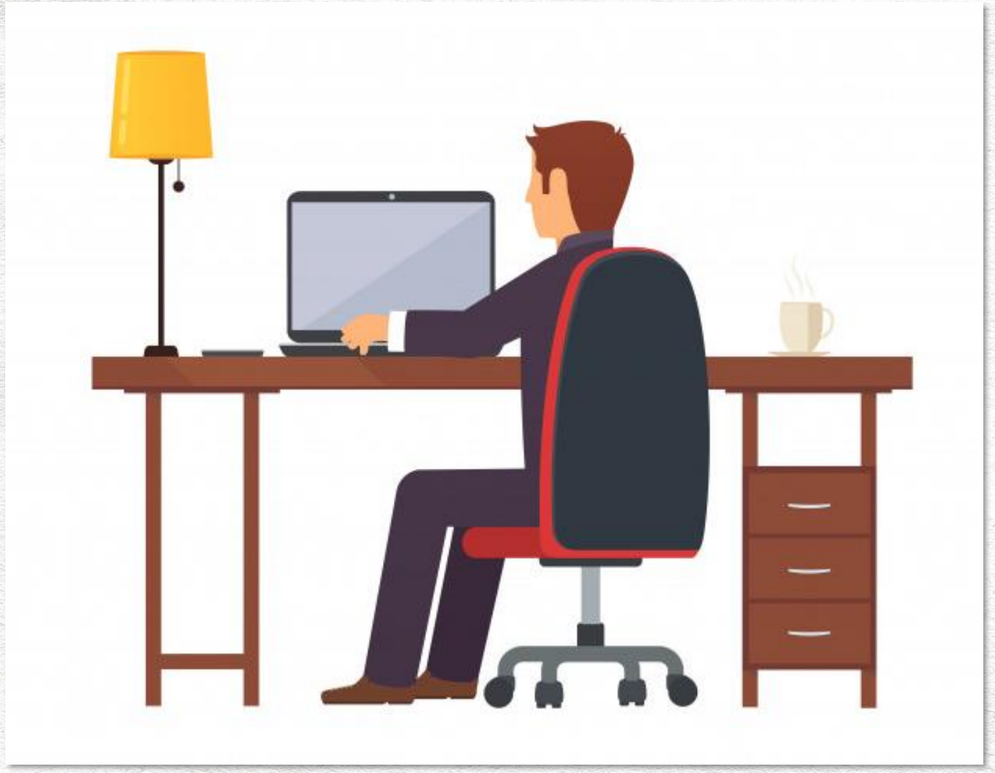
4

Questão 4

O setor financeiro deseja calcular a margem de lucro por produto.

Crie uma View chamada vwMargemLucro que exiba o nome do produto, o preço unitário, o custo unitário e a margem de lucro calculada. **(Utilize a tabela produtos)**

EXERCÍCIOS



5a

Questão 5-a

O setor de vendas precisa de uma visão consolidada das vendas por loja.

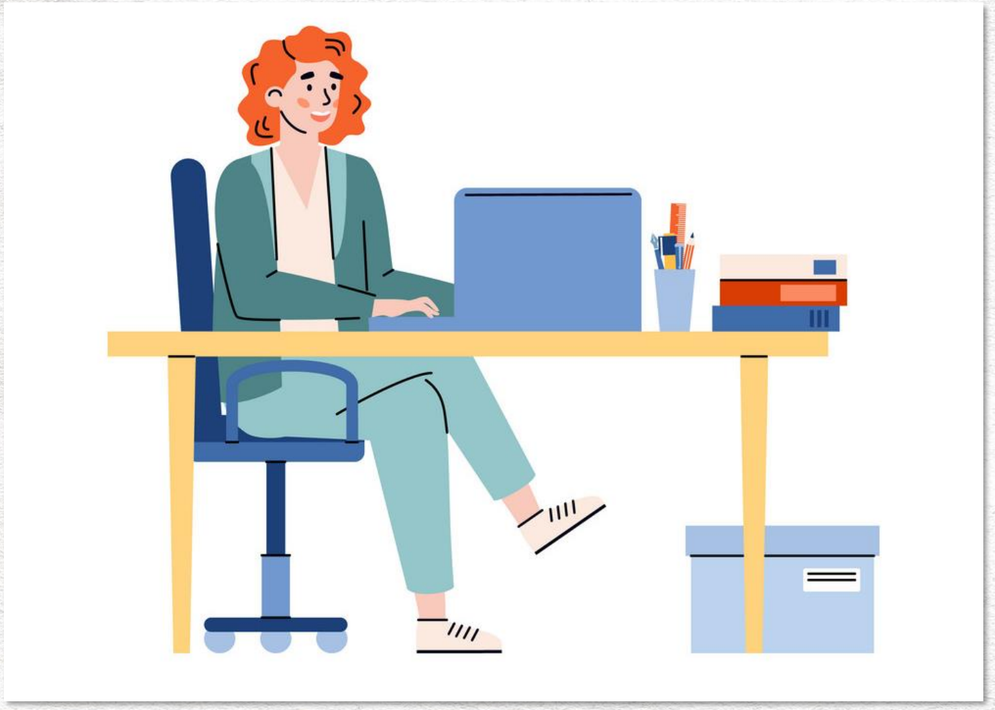
Crie uma View chamada vwVendasPorLoja que exiba o ID_Loja, a soma das receitas e a quantidade de pedidos. **(Utilize a tabela pedidos)**

5b

Questão 5-b

Altere a View criada no item A para incluir o Nome da Loja. **(Utilize as tabelas pedidos e lojas)**

EXERCÍCIOS



6

Questão 6

A equipe de compras precisa classificar os produtos como "Custo Alto" ou "Custo Baixo" com base em seu custo unitário.

Crie uma View chamada vwClassificacaoProdutos que exiba o nome do produto, a marca, o custo unitário e a classificação baseada no custo unitário. Considere "Custo Alto" para custos maiores que 500. **(Utilize a tabela produtos)**

7

Questão 7

A empresa deseja monitorar o volume de vendas mensais de cada loja.

Crie uma View chamada vwVolumeVendasLojas que contenha as colunas Mes_Ano, Loja, e o total de itens vendidos agrupados por mês/ano e loja. **(Utilize as tabelas pedidos e lojas)**

EXERCÍCIOS

8

Questão 8

A equipe de marketing precisa analisar os clientes que possuem uma renda anual maior ou igual a R$90.000 e têm ao menos um filho.

Crie uma View chamada vwClientesAltaRendaComFilhos que inclua o nome completo, e-mail, renda anual e quantidade de filhos desses clientes. **(Utilize a tabela clientes)**

9

Questão 9

Exclua todas as Views criadas nesta lista de exercícios:

vwClientesContato

vwLojasFuncionarios

vwProdutosAltoValor

vwMargemLucro

vwVendasPorLoja

vwClassificacaoProdutos

vwVolumeVendasLojas

vwClientesAltaRendaComFilhos

(Não é necessário utilizar tabelas)

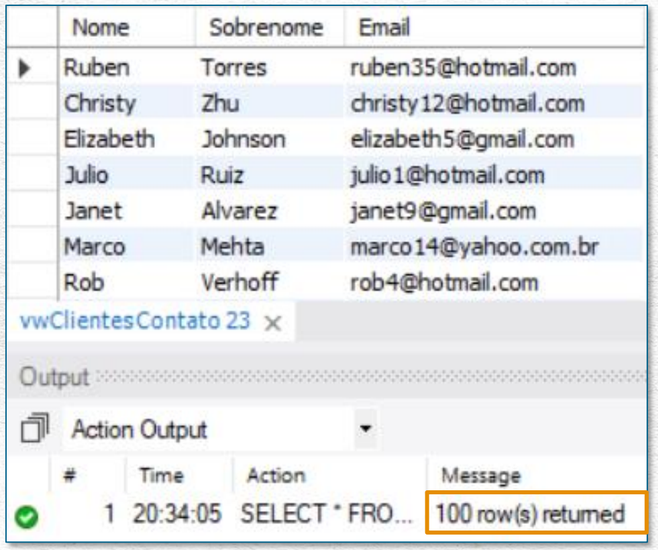
GABARITO EXPLICATIVO

1

Questão 1

A equipe de marketing deseja uma visão centralizada dos nomes e e-mails de todos os clientes cadastrados.

Crie uma View chamada vwClientesContato que exiba apenas os campos Nome, Sobrenome e Email dos clientes. **(Utilize a tabela clientes)**



Nome	Sobrenome	Email
Ruben	Torres	ruben35@hotmail.com
Christy	Zhu	christy12@hotmail.com
Elizabeth	Johnson	elizabeth5@gmail.com
Julio	Ruiz	julio1@hotmail.com
Janet	Alvarez	janet9@gmail.com
Marco	Mehta	marco14@yahoo.com.br
Rob	Verhoff	rob4@hotmail.com

vwClientesContato 23

Output

Action Output

#	Time	Action	Message
1	20:34:05	SELECT * FRO...	100 row(s) returned

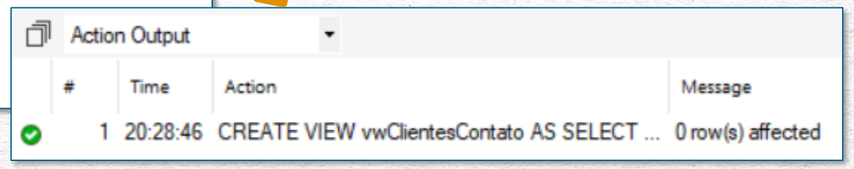
Explicação e Resultado

Para **criar uma View com os nomes e e-mails dos clientes**, seguimos os passos abaixo:

Utilizamos o comando **CREATE VIEW** para criar uma nova View chamada **vwClientesContato**. No comando **SELECT** dentro da definição da View, selecionamos as colunas **Nome**, **Sobrenome** e **Email** da tabela **CLIENTES**.

Executamos o código para criar a View no banco de dados:

```
CREATE VIEW vwClientesContato AS
SELECT
    Nome,
    Sobrenome,
    Email
FROM clientes;
```



Action Output

#	Time	Action	Message
1	20:28:46	CREATE VIEW vwClientesContato AS SELECT ...	0 row(s) affected

Após criar a View, utilizamos o comando **SELECT * FROM vwClientesContato** para verificar o conteúdo da View:

```
SELECT * FROM vwClientesContato;
```

Dessa forma, retornamos o resultado ao lado:

Índice Geral

Índice deste Módulo

GABARITO EXPLICATIVO

2

Questão 2

O RH deseja identificar as lojas com o menor número de funcionários para possíveis otimizações.

Crie uma View chamada vwLojasFuncionarios que exiba o nome da loja, o gerente e o número de funcionários, ordenados de forma crescente pelo número de funcionários. **(Utilize a tabela lojas)**

Loja	Gerente	Num_Funcionarios
Goiânia	Carlos Henrique	11
Fortaleza	Letícia Gomes	12
Belo Horizonte	Julia Freitas	13
Niterói	Matheus Leal	18
Salvador	Marcelo Castro	19
Rio de Janeiro	Pedro Martins	23
Curitiba	Amanda Lima	31
São Paulo	Rogério Lopes	33

Explicação e Resultado

Para **criar uma View que identifique as lojas de acordo com seu número de funcionários**, seguimos os passos abaixo:

Utilizamos o comando **CREATE VIEW** para criar uma nova View chamada **vwLojasFuncionarios**. No comando **SELECT** dentro da definição da View, selecionamos as colunas **Loja**, **Gerente** e **Num_Funcionarios** da tabela **LOJAS**. Incluímos a cláusula **ORDER BY Num_Funcionarios ASC** para ordenar as lojas de forma crescente com base no número de funcionários.

Executamos o código para criar a View no banco de dados:

```
CREATE VIEW vwLojasFuncionarios AS
SELECT
    Loja,
    Gerente,
    Num_Funcionarios
FROM lojas
ORDER BY Num_Funcionarios ASC;
```

Action Output

#	Time	Action	Message
1	20:41:42	CREATE VIEW vwLojasFuncionarios AS SEL...	0 row(s) affected

Após criar a View, utilizamos o comando **SELECT * FROM vwLojasFuncionarios** para verificar o conteúdo da View:

```
SELECT * FROM vwLojasFuncionarios;
```

Dessa forma, retornamos o resultado ao lado:

GABARITO EXPLICATIVO

3

Questão 3

O setor financeiro quer uma amostra dos cinco produtos mais caros da tabela produtos.

Crie uma View chamada vwProdutosAltoValor que exiba o nome do produto, a marca e o preço unitário desses produtos. **(Utilize a tabela produtos)**

Explicação e Resultado

Para **criar uma View que exiba os cinco produtos mais caros**, seguimos os passos abaixo:

Utilizamos o comando **CREATE VIEW** para criar uma nova View chamada **vwProdutosAltoValor**. No comando **SELECT** dentro da definição da View, selecionamos as colunas **Nome_Produto**, **Marca_Produto** e **Preco_Unit** da tabela **PRODUTOS**. Incluímos a cláusula **ORDER BY Preco_Unit DESC** para ordenar os produtos de forma decrescente com base no preço unitário. Utilizamos a cláusula **LIMIT 5** para restringir o resultado aos cinco produtos mais caros.

Executamos o código para criar a View no banco de dados:

```
CREATE VIEW vwProdutosAltoValor AS
SELECT
    Nome_Produto,
    Marca_Produto,
    Preco_Unit
FROM produtos
ORDER BY Preco_Unit DESC
LIMIT 5;
```

Action Output

#	Time	Action	Message
1	20:49:35	CREATE VIEW vwProdutosAltoValor AS SEL...	0 row(s) affected

Após criar a View, utilizamos o comando **SELECT * FROM vwProdutosAltoValor** para verificar o conteúdo da View:

```
SELECT * FROM vwProdutosAltoValor;
```

Dessa forma, retornamos o resultado ao lado:

Nome_Produto	Marca_Produto	Preco_Unit
Notebook IdeaPad RF32000	DELL	4200
Notebook LC2100 Intel Core i5 8GB	SAMSUNG	3400
Cadeira Gamer PC Racer Vermelha	ALTURA	3100
Notebook Inspiron 15 5000 4GB	DELL	3100
Notebook Motion Ultra 2	SAMSUNG	2900

Índice Geral
Índice deste Módulo

GABARITO EXPLICATIVO

4

Questão 4

O setor financeiro deseja calcular a margem de lucro por produto.

Crie uma View chamada vwMargemLucro que exiba o nome do produto, o preço unitário, o custo unitário e a margem de lucro calculada. **(Utilize a tabela produtos)**

	Nome_Produto	Preco_Unit	Custo_Unit	Margem_Lucro
▶	Monitor LED 19,5" Full HD HDMI	2300	966	1334
	Monitor Curvo 24" 144Hz HDMI	2800	980	1820
	Webcam Full HD 1080p	450	90	360
	Kit Teclado + Mouse sem fio Wireless	350	129.5	220.5
	Kit Teclado + Mouse Slim Bluetooth	280	109.2	170.8
	Cadeira Gamer reclinável Azul/Laranja	1800	540	1260
	Cadeira Gamer PC Racer Vermelha	3100	1395	1705
	Headphone Bluetooth 2000	600	258	342
	Fone de Ouvido Tune T5000	780	327.6	452.4
	Microfone Condensador MC1000	1100	275	825
	Microfone Condensador com Tripé	800	344	456
	Microfone de mesa com fio condensa...	650	214.5	435.5
	Notebook LC2100 Intel Core i5 8GB	3400	850	2550
	Notebook Inspiron 15 5000 4GB	3100	1209	1891
	Notebook IdeaPad RF32000	4200	1176	3024
	Notebook Motion Ultra 2	2900	1160	1740

Explicação e Resultado

Para **criar uma View que calcula a margem de lucro por produto**, seguimos os passos abaixo:

Utilizamos o comando **CREATE VIEW** para criar uma nova View chamada **vwMargemLucro**. No comando **SELECT** dentro da definição da View, selecionamos as colunas **Nome_Produto**, **Preco_Unit** e **Custo_Unit** da tabela **PRODUTOS**. Adicionamos uma coluna calculada chamada **Margem_Lucro**, obtida pela **subtração entre o Preco_Unit e o Custo_Unit**, para determinar a margem de lucro de cada produto.

Executamos o código para criar a View no banco de dados:

```
CREATE VIEW vwMargemLucro AS
SELECT
    Nome_Produto,
    Preco_Unit,
    Custo_Unit,
    (Preco_Unit - Custo_Unit) AS Margem_Lucro
FROM produtos;
```

Action Output

#	Time	Action	Message
1	20:55:01	CREATE VIEW vwMargemLucro AS SELECT...	0 row(s) affected

Após criar a View, utilizamos o comando **SELECT * FROM vwMargemLucro** para verificar o conteúdo da View:

```
SELECT * FROM vwMargemLucro;
```

Dessa forma, retornamos o resultado ao lado:

Índice Geral

Índice deste Módulo

GABARITO EXPLICATIVO

5a

Questão 5-a

O setor de vendas precisa de uma visão consolidada das vendas por loja.

Crie uma View chamada vwVendasPorLoja que exiba o ID_Loja, a soma das receitas e a quantidade de pedidos. **(Utilize a tabela pedidos)**

	ID_Loja	Total_Receita	Numero_Pedidos
▶	6	10443680	5854
	2	10061430	5604
	1	10037160	5645
	8	9943620	5609
	5	10217870	5679
	3	10107860	5606
	7	9986950	5577
	4	10139870	5622

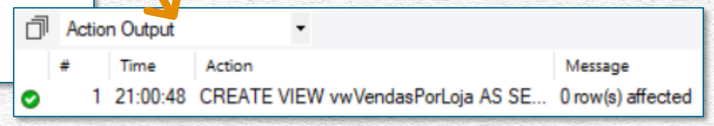
Explicação e Resultado

Para **criar uma View que consolida as vendas por loja**, seguimos os passos abaixo:

Utilizamos o comando **CREATE VIEW** para criar uma nova View chamada **vwVendasPorLoja**. No comando **SELECT** dentro da definição da View, selecionamos a coluna **ID_Loja** da tabela **PEDIDOS**. Adicionamos duas colunas agregadas: **SUM(Receita_Venda)** para calcular a soma total das receitas por loja (nomeada como **Total_Receita**) e **COUNT(ID_Pedido)** para contar o número de pedidos por loja (nomeada como **Numero_Pedidos**). Utilizamos o comando **GROUP BY ID_Loja** para agrupar os dados por loja.

Executamos o código para criar a View no banco de dados:

```
CREATE VIEW vwVendasPorLoja AS
SELECT
    ID_Loja,
    SUM(Receita_Venda) AS Total_Receita,
    COUNT(ID_Pedido) AS Numero_Pedidos
FROM pedidos
GROUP BY ID_Loja;
```



Após criar a View, utilizamos o comando **SELECT * FROM vwVendasPorLoja** para verificar o conteúdo da View:

```
SELECT * FROM vwVendasPorLoja;
```

Dessa forma, retornamos o resultado ao lado:

Índice Geral
Índice deste Módulo

GABARITO EXPLICATIVO

5b

Questão 5-b

Altere a View criada no item A para incluir o Nome da Loja. **(Utilize as tabelas pedidos e lojas)**

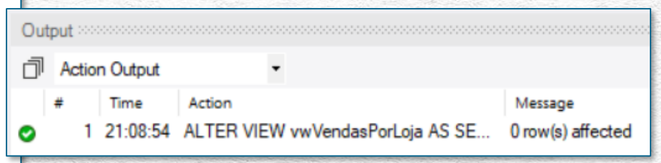
Explicação e Resultado

Utilizamos o comando **ALTER VIEW** para **modificar a View vwVendasPorLoja já existente**.

No comando **SELECT** dentro da definição da View, continuamos selecionando a coluna **ID_Loja** da tabela **PEDIDOS** e, agora, também incluímos a coluna **Loja** da tabela **LOJAS** (para exibir o nome da loja). Mantemos as colunas agregadas **Total_Receita** e **Numero_Pedidos** que criamos no **item A**. Realizamos um **INNER JOIN** entre as tabelas **PEDIDOS** e **LOJAS**, utilizando a coluna **ID_Loja** para relacioná-las. Mantemos o **GROUP BY** para agrupar os dados por loja, agora incluindo tanto **pe.ID_Loja** quanto **lo.Loja**.

Executamos o código para alterar a View no banco de dados:

```
ALTER VIEW vwVendasPorLoja AS
SELECT
    pe.ID_Loja,
    lo.Loja,
    SUM(Receita_Venda) AS Total_Receita,
    COUNT(ID_Pedido) AS Numero_Pedidos
FROM pedidos pe
INNER JOIN lojas lo
    ON pe.ID_Loja = lo.ID_Loja
GROUP BY pe.ID_Loja, lo.Loja;
```



Após alterar a View, utilizamos o comando **SELECT * FROM vwVendasPorLoja** para verificar o conteúdo da View com a inclusão do nome da loja:

Dessa forma, retornamos o resultado ao lado:

```
SELECT * FROM vwVendasPorLoja;
```

ID_Loja	Loja	Total_Receita	Numero_Pedidos
6	São Paulo	10443680	5854
2	Belo Horizonte	10061430	5604
1	Rio de Janeiro	10037160	5645
8	Goiânia	9943620	5609
5	Fortaleza	10217870	5679
3	Salvador	10107860	5606
7	Niterói	9986950	5577
4	Curitiba	10139870	5622

Índice Geral
Índice deste Módulo

GABARITO EXPLICATIVO

6

Questão 6

A equipe de compras precisa classificar os produtos como "Custo Alto" ou "Custo Baixo" com base em seu custo unitário.

Crie uma View chamada vwClassificacaoProdutos que exiba o nome do produto, a marca, o custo unitário e a classificação baseada no custo unitário. Considere "Custo Alto" para custos maiores que 500. **(Utilize a tabela produtos)**

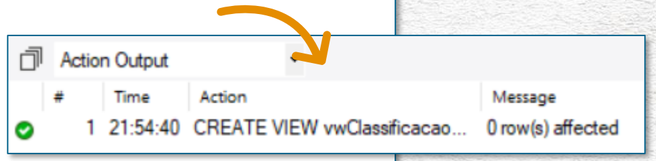
Nome_Produto	Marca_Produto	Custo_Unit	Classificacao
Monitor LED 19,5" Full HD HDMI	DELL	966	Custo Alto
Monitor Curvo 24" 144Hz HDMI	SAMSUNG	980	Custo Alto
Webcam Full HD 1080p	LOGITECH	90	Custo Baixo
Kit Teclado + Mouse sem fio Wireless	DELL	129.5	Custo Baixo
Kit Teclado + Mouse Slim Bluetooth	DELL	109.2	Custo Baixo
Cadeira Gamer reclinável Azul/Laranja	ALTURA	540	Custo Alto
Cadeira Gamer PC Racer Vermelha	ALTURA	1395	Custo Alto
Headphone Bluetooth 2000	SONY	258	Custo Baixo
Fone de Ouvido Tune T5000	JBL	327.6	Custo Baixo
Microfone Condensador MC1000	AKG	275	Custo Baixo
Microfone Condensador com Tripé	BLUE	344	Custo Baixo
Microfone de mesa com fio condensa...	BLUE	214.5	Custo Baixo
Notebook LC2100 Intel Core i5 8GB	SAMSUNG	850	Custo Alto
Notebook Inspiron 15 5000 4GB	DELL	1209	Custo Alto
Notebook IdeaPad RF32000	DELL	1176	Custo Alto
Notebook Motion Ultra 2	SAMSUNG	1160	Custo Alto

Explicação e Resultado

Para **criar a View que classifica os produtos com base no custo unitário**, seguimos os seguintes passos: utilizamos o comando **CREATE VIEW** para criar a nova View chamada **vwClassificacaoProdutos**. No comando **SELECT** dentro da definição da View, selecionamos as colunas **Nome_Produto**, **Marca_Produto** e **Custo_Unit** da tabela **PRODUTOS**. Aplicamos a função condicional **IF** para classificar os produtos: se o **Custo_Unit** for **maior que 500**, a classificação será **"Custo Alto"**; **caso contrário**, a classificação será **"Custo Baixo"** (chamamos a coluna resultante de **Classificacao**).

Executamos o código para criar a View no banco de dados:

```
CREATE VIEW vwClassificacaoProdutos AS
SELECT
    Nome_Produto,
    Marca_Produto,
    Custo_Unit,
    IF(Custo_Unit > 500, 'Custo Alto', 'Custo Baixo') AS Classificacao
FROM produtos;
```



Após criar a View, utilizamos o comando **SELECT * FROM vwClassificacaoProdutos** para verificar o conteúdo da View:

```
SELECT * FROM vwClassificacaoProdutos;
```

Dessa forma, retornamos o resultado ao lado:

Índice Geral
Índice deste Módulo

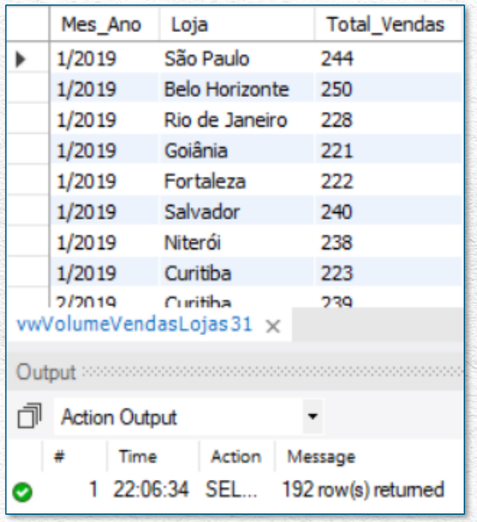
GABARITO EXPLICATIVO

7

Questão 7

A empresa deseja monitorar o volume de vendas mensais de cada loja.

Crie uma View chamada vwVolumeVendasLojas que contenha as colunas Mes_Ano, Loja, e o total de itens vendidos agrupados por mês/ano e loja. **(Utilize as tabelas pedidos e lojas)**



Explicação e Resultado

Para **criar a View que monitora o volume de vendas mensais de cada loja**, seguimos os seguintes passos: utilizamos o comando **CREATE VIEW** para criar a View chamada **vwVolumeVendasLojas**. No **SELECT** dentro da definição da View, usamos a função **CONCAT(MONTH(pe.Data_Venda), '/', YEAR(pe.Data_Venda))** para combinar o mês e o ano da data de venda, criando a coluna **Mes_Ano**. Selecionamos a coluna **Loja** da tabela **LOJAS**. Calculamos o total de itens vendidos por loja e mês/ano utilizando a função **SUM(pe.Qtd_Vendida)**. Fazemos um **INNER JOIN** entre as tabelas **PEDIDOS** e **LOJAS** usando a coluna **ID_Loja**, comum entre as tabelas. Agrupamos os resultados por mês/ano e loja com o **GROUP BY**, utilizando as funções **MONTH(pe.Data_Venda)**, **YEAR(pe.Data_Venda)** e a coluna **lo.loja**.

```
CREATE VIEW vwVolumeVendasLojas AS
SELECT
    CONCAT(MONTH(pe.Data_Venda), '/', YEAR(pe.Data_Venda)) AS Mes_Ano,
    lo.Loja,
    SUM(pe.Qtd_Vendida) AS Total_Vendas
FROM pedidos AS pe
INNER JOIN lojas AS lo
    ON pe.ID_Loja = lo.ID_Loja
GROUP BY MONTH(pe.Data_Venda), YEAR(pe.Data_Venda), lo.Loja;
```

```
Action Output
# Time Action Message
1 22:05:11 CREATE VIEW vwVolumeVend... 0 row(s) affected
```

Após criar a View, utilizamos o comando **SELECT * FROM vwVolumeVendasLojas** para verificar o conteúdo da View:

```
SELECT * FROM vwVolumeVendasLojas;
```

Dessa forma, retornamos o resultado ao lado:

Índice Geral
Índice deste Módulo

GABARITO EXPLICATIVO

8

Questão 8

A equipe de marketing precisa analisar os clientes que possuem uma renda anual maior ou igual a R$90.000 e têm ao menos um filho.

Crie uma View chamada vwClientesAltaRendaComFilhos que inclua o nome completo, e-mail, renda anual e quantidade de filhos desses clientes. **(Utilize a tabela clientes)**

Explicação e Resultado

Para **criar a View que analisa os clientes com renda anual maior ou igual a R$90.000 e ao menos um filho**, aplicamos os passos: utilizamos o comando **CREATE VIEW** para criar a View chamada **vwClientesAltaRendaComFilhos**. No comando **SELECT** dentro da definição da View, usamos a função **CONCAT(Nome, ' ', Sobrenome)** para criar a coluna **Nome_Completo** e selecionamos as colunas **Email**, **Renda_Anual** e **Qtd_Filhos** da tabela **CLIENTES**. Aplicamos a condição **WHERE Renda_Anual >= 90000 AND Qtd_Filhos > 0** para filtrar apenas os clientes que atendem aos critérios de renda anual e filhos.

Executamos o código para criar a View no banco de dados:

```sql
CREATE VIEW vwClientesAltaRendaComFilhos AS
SELECT
    CONCAT(Nome, ' ', Sobrenome) AS Nome_Completo,
    Email,
    Renda_Anual,
    Qtd_Filhos
FROM clientes
WHERE Renda_Anual >= 90000 AND Qtd_Filhos > 0;
```

Action Output

#	Time	Action	Message
1	22:14:33	CREATE VIEW vwClient...	0 row(s) affected

Após criar a View, utilizamos o comando **SELECT * FROM vwClientesAltaRendaComFilhos** para verificar o conteúdo da View:

```sql
SELECT * FROM vwClientesAltaRendaComFilhos;
```

Dessa forma, retornamos o resultado ao lado:

Nome_Completo	Email	Renda_Anual	Qtd_Filhos
Lauren Walker	lauren41@hotmail.com	100000	2
Ian Jenkins	ian47@gmail.com	100000	2
Sydney Bennett	sydney23@gmail.com	100000	3
Savannah Baker	savannah39@gmail.com	120000	2
Trevor Bryant	trevor18@gmail.com	90000	2
Dalton Perez	dalton37@yahoo.com.br	90000	2
Cheryl Diaz	cheryl4@gmail.com	90000	2

Índice Geral
Índice deste Módulo

GABARITO EXPLICATIVO

9

Questão 9

Exclua todas as Views criadas nesta lista de exercícios:

vwClientesContato

vwLojasFuncionarios

vwProdutosAltoValor

vwMargemLucro

vwVendasPorLoja

vwClassificacaoProdutos

vwVolumeVendasLojas

vwClientesAltaRendaComFilhos

(Não é necessário utilizar tabelas)

Explicação e Resultado

Para **excluir todas as Views criadas**, utilizamos o comando **DROP VIEW** para cada uma delas:

```
DROP VIEW vwClientesContato;
```

Action Output

#	Time	Action	Message
1	22:21:05	DROP VIEW vwClientesContato	0 row(s) affected

```
DROP VIEW vwLojasFuncionarios;
```

Action Output

#	Time	Action	Message
1	22:24:58	DROP VIEW vwLojasFuncionarios	0 row(s) affected

```
DROP VIEW vwProdutosAltoValor;
```

Action Output

#	Time	Action	Message
1	22:25:40	DROP VIEW vwProdutosAltoValor	0 row(s) affected

```
DROP VIEW vwMargemLucro;
```

Action Output

#	Time	Action	Message
1	22:26:06	DROP VIEW vwMargemLucro	0 row(s) affected

```
DROP VIEW vwVendasPorLoja;
```

Action Output

#	Time	Action	Message
1	22:26:27	DROP VIEW vwVendasPorLoja	0 row(s) affected

```
DROP VIEW vwClassificacaoProdutos;
```

Action Output

#	Time	Action	Message
1	22:26:55	DROP VIEW vwClassificacaoPro...	0 row(s) affected

```
DROP VIEW vwVolumeVendasLojas;
```

Action Output

#	Time	Action	Message
1	22:27:14	DROP VIEW vwVolumeVendasL...	0 row(s) affected

```
DROP VIEW vwClientesAltaRendaComFilhos;
```

Action Output

#	Time	Action	Message
1	22:27:35	DROP VIEW vwClientesAltaRen...	0 row(s) affected

MÓDULO 12

CRUD

CRUD

CRUD

Índice Geral

Índice deste Módulo

O QUE VEREMOS NESTE MÓDULO

MÓDULO 12 CRUD

01	O QUE SIGNIFICA CRUD?	322
02	CRUD EM BANCOS DE DADOS	323
03	CREATE DATABASE	324
04	SHOW DATABASES	325
05	USE	326
06	DROP DATABASE	327
07	CRUD EM TABELAS	328
08	CREATE DATABASE	329
09	TIPOS DE DADOS	330
10	CREATE TABLE	331
11	SHOW TABLES	333
12	CONSTRAINTS	334
13	PREPARAÇÃO PARA CRIAR AS CONSTRAINTS	341
14	RECRIANDO A TABELA DALUNOS	342
15	RECRIANDO A TABELA DCURSOS	343
16	RECRIANDO A TABELA FMATRICULAS	344
16	VISUALIZANDO RELACIONAMENTOS (TABELAS)	345
17	INSERT INTO	352
18	UPDATE	353
19	DELETE	354
20	DROP TABLE x TRUNCATE TABLE	355
21	EXERCÍCIOS	356
22	GABARITO EXPLICATIVO	362

O que significa CRUD?

Operações CRUD são operações que conseguimos fazer em um Banco de Dados. Essa sigla significa o seguinte:

C

CREATE

Permite criar Bancos de Dados, Tabelas ou Exibições (Views)

R

READ

Permite ler os dados do banco de dados, tabelas e views.

U

UPDATE

Permite atualizar os dados do banco de dados ou tabelas.

D

DELETE

Permite excluir dados de um banco de dados ou tabelas.

No módulo anterior, vimos como criar Views (exibições) com os operadores CRUD. Essas exibições não são entendidas exatamente como tabelas do banco de dados. Agora o que vamos fazer é aprender como criar tabelas propriamente ditas.

CRUD em bancos de dados

Podemos utilizar comandos SQL para criar, mostrar, selecionar e excluir bancos de dados.

CREATE DATABASE	SHOW DATABASES	USE	SELECT DATABASE	DROP DATABASE
Permite criar bancos de dados	Mostra os bancos de dados	Define um banco de dados como o padrão	Seleciona o banco de dados padrão	Exclui um banco de dados

A seguir, vejamos como utilizar esses comandos na prática.

CREATE DATABASE

Para criar um banco de dados, usamos o comando CREATE DATABASE, como mostrado em 1.

```
CREATE DATABASE db_Exemplo;
```

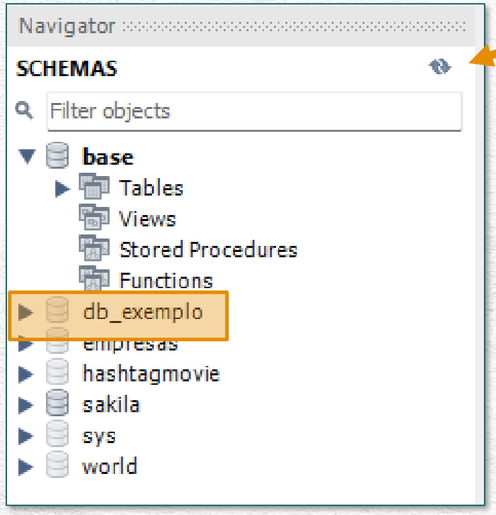
1

Podemos complementar esse comando usando o IF NOT EXISTS para que o banco de dados seja criado apenas se ele já não existir, como mostrado em 2. A vantagem é que, se o banco de dados já existir, não será retornado um erro.

```
CREATE DATABASE IF NOT EXISTS db_Exemplo;
```

2

Por fim, em 3, clique no botão de atualizar indicado pela seta para visualizar o banco de dados criado.



3

SHOW DATABASES

Com o comando SHOW DATABASES podemos listar todos os bancos de dados existentes.

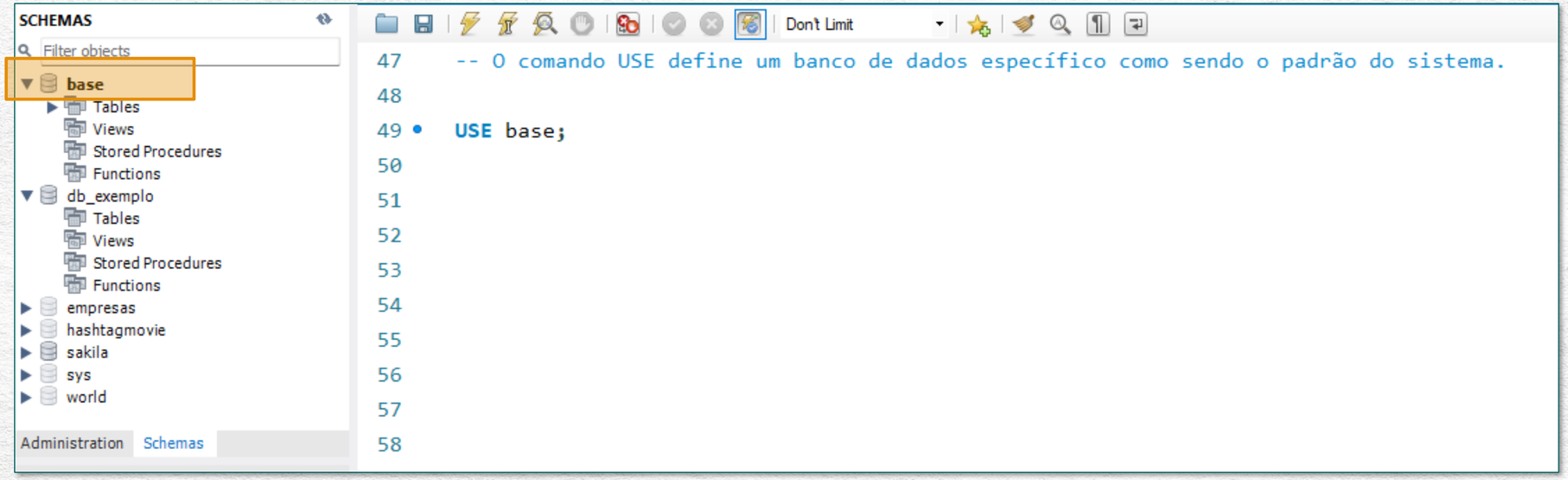
```
43 •   SHOW DATABASES;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Database
base
db_exemplo
empresas
hashtagmovie
information_schema
mysql
performance_schema
sakila
sys
world

USE

Utilizamos o comando USE para definir como padrão um determinado banco de dados. Basta usar o comando USE seguido do nome do banco de dados, conforme exemplo abaixo.



DROP DATABASE

Para excluir um banco de dados, utilizamos o comando DROP DATABASE, seguido do nome do banco de dados que queremos excluir, conforme exemplo abaixo.

```
74    -- Para excluir um bando de dados, usamos o seguinte comando:
75
76 •  DROP DATABASE IF EXISTS db_Exemplo;
77
```

CRUD em tabelas

Podemos utilizar comandos SQL para criar, mostrar, selecionar, atualizar, incluir dados e excluir tabelas.

CREATE TABLE

Permite criar uma tabela

SHOW TABLES

Mostra as tabelas do banco de dados

SELECT

Permite selecionar tabelas

INSERT INTO

Insere dados dentro de tabelas

UPDATE

Atualiza dados em uma tabela

TRUNCATE

Exclui todas as linhas de uma tabela

DROP

Exclui uma tabela

A seguir, vejamos como utilizar esses comandos na prática.

CREATE DATABASE

Primeiramente, vamos recriar o banco de dados db_Exemplo para que possamos adicionar dentro dele as novas tabelas. Utilize o código abaixo.

Lembre-se do comando USE para tornar esse banco de dados o padrão do sistema.

```
-- Crie o banco de dados db_Exemplo e defina-o como padrão:
CREATE DATABASE IF NOT EXISTS db_Exemplo;
USE db_Exemplo;
--
```

Tipos de Dados

Quando criamos uma nova tabela, precisamos especificar quais são as colunas que essa tabela deve conter. Cada uma dessas colunas vai armazenar um tipo de dados específico. Os principais tipos de dados são os listados abaixo:

TIPO	DESCRIÇÃO
INT	Inteiros entre -2.147.483.648 e 2.147.483.647
DECIMAL(M, D)	Número decimal com M dígitos no total e D casas decimais. O padrão é (10, 2). Exemplo: o valor 100,34 poderia ser declarado como DECIMAL(5, 2).
FLOAT(M, D)	Número decimal com M dígitos no total e D casas decimais. O padrão é (10, 2). Exemplo: o valor 12345,67 poderia ser declarado como FLOAT(7, 2).
VARCHAR(N)	String de tamanho variável, até 65535 caracteres.
DATE	Uma data de 01/01/1000 a 31/12/9999, no formato YYYY-MM-DD
DATETIME	Uma combinação de data e hora de 01/01/1000 00:00:00 a 31/12/9999 23:59:59, no formato YYYY-MM-DD HH:MM:SS

CREATE TABLE

Agora vamos criar as tabelas do nosso banco de dados.

O banco será composto por 3 tabelas, que vão armazenar informações de uma empresa que oferece treinamentos online. As tabelas criadas armazenarão os seguintes dados:

1) Alunos
2) Cursos
3) Matrículas

Ao lado, temos a estrutura para criação de cada uma das tabelas.

```
CREATE TABLE IF NOT EXISTS dAlunos(
    ID_Aluno INT,
    Nome_Aluno VARCHAR(100),
    Email VARCHAR(100)
);

CREATE TABLE IF NOT EXISTS  dCursos(
    ID_Curso INT,
    Nome_Curso VARCHAR(100),
    Preco_Curso DECIMAL(10, 2)
);

CREATE TABLE IF NOT EXISTS fMatriculas(
    ID_Matricula INT,
    ID_Aluno INT,
    ID_Curso INT,
    Data_Cadastro DATE
);
```

CREATE TABLE

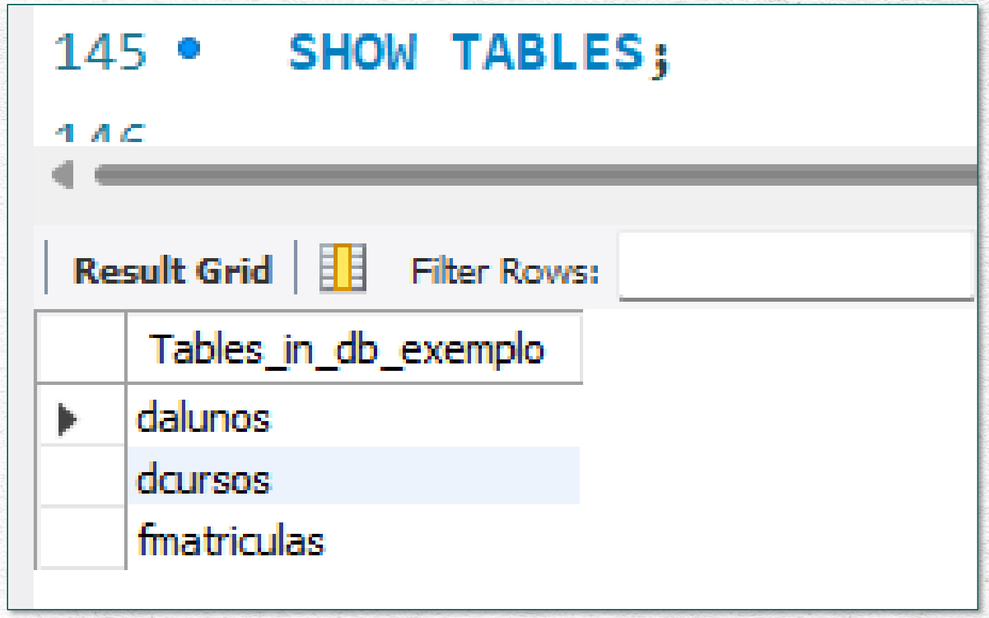
A estrutura será sempre:

1) O comando CREATE TABLE
2) Comando IF NOT EXISTS (opcional)
3) Nome da tabela
4) Nome da coluna seguido do tipo daquela coluna

```
CREATE TABLE IF NOT EXISTS dAlunos(
    ID_Aluno INT,
    Nome_Aluno VARCHAR(100),
    Email VARCHAR(100)
);
```

SHOW TABLES

Para mostrar as tabelas de um banco de dados, usamos o comando SHOW TABLES, conforme mostrado ao lado.



CONSTRAINTS

Quando criamos tabelas, podemos definir regras para os valores adicionados naquelas tabelas.

É aqui que entram as Restrições (Constraints). Elas se tratam de regras aplicadas nas colunas de uma tabela, proporcionando um melhor controle sobre os dados adicionados naquelas tabelas.

Exemplo 1: Podemos especificar que uma coluna não pode ter valores NULL.

Exemplo 2: Podemos especificar que uma coluna deverá ser uma chave primária ou chave estrangeira.

Basicamente, as constraints são usadas para limitar os tipos de dados que são inseridos e temos alguns tipos de constraints que veremos a seguir.

CONSTRAINTS

Abaixo, temos listadas as principais constraints usadas na criação de tabelas.

- NOT NULL
- UNIQUE
- DEFAULT
- PRIMARY KEY
- FOREIGN KEY

CONSTRAINTS

Abaixo, temos listadas as principais constraints usadas na criação de tabelas.

NOT NULL | UNIQUE | DEFAULT | PRIMARY KEY | FOREIGN KEY

A constraint NOT NULL faz com que uma coluna não aceite valores NULL. Um valor NULL é diferente de zero ou vazio, ele identifica que nenhum valor foi definido.

A constraint NOT NULL obriga um campo a sempre possuir um valor. Dessa forma, não é possível inserir um registro ou atualizar sem entrar com um valor neste campo.

```
Nome_Cliente VARCHAR(100) NOT NULL
```

CONSTRAINTS

Abaixo, temos listadas as principais constraints usadas na criação de tabelas.

NOT NULL | **UNIQUE** | DEFAULT | PRIMARY KEY | FOREIGN KEY

A restrição UNIQUE impede que em uma mesma coluna sejam adicionados valores repetidos.

Podemos usar esse tipo de constraint em uma coluna de CPF, por exemplo. Dois clientes não podem ter um mesmo CPF, por isso podemos usar a constraint UNIQUE na coluna CPF para impedir que valores duplicados sejam cadastrados.

```
CPF VARCHAR(100) UNIQUE
```

CONSTRAINTS

Abaixo, temos listadas as principais constraints usadas na criação de tabelas.

NOT NULL | UNIQUE | **DEFAULT** | PRIMARY KEY | FOREIGN KEY

Essa restrição permite definir um valor padrão para uma determinada coluna. Dessa forma, caso um valor não seja adicionado na coluna, automaticamente ela receberá o valor padrão informado.

```
Cidade VARCHAR(100) DEFAULT 'São Paulo'
```

CONSTRAINTS

Abaixo, temos listadas as principais constraints usadas na criação de tabelas.

NOT NULL | UNIQUE | DEFAULT | **PRIMARY KEY** | FOREIGN KEY

A constraint PRIMARY KEY identifica de forma única cada registro em uma tabela do banco de dados.

Chaves primárias devem conter valores únicos. Uma coluna de chave primária não pode conter valores NULL. Cada tabela deve conter 1 e apenas 1 chave primária.

```
PRIMARY KEY(ID_Curso)
```

CONSTRAINTS

Abaixo, temos listadas as principais constraints usadas na criação de tabelas.

NOT NULL | UNIQUE | DEFAULT | PRIMARY KEY | **FOREIGN KEY**

Uma FOREIGN KEY em uma tabela é um campo que aponta para uma chave primária em outra tabela.

```
FOREIGN KEY(ID_Curso) REFERENCES dCursos(ID_Curso)
```

Preparação para criar as constraints

Agora que entendemos o que são as constraints, a ideia é recriar as tabelas dAlunos, dCursos e fMatriculas incluindo constraints.

Portanto, o primeiro passo é excluir as tabelas para poder recriá-las. Vamos utilizar o comando DROP TABLE, como mostrado ao lado, para excluir cada uma dessas tabelas.

```
DROP TABLE IF EXISTS dAlunos;
DROP TABLE IF EXISTS dCursos;
DROP TABLE IF EXISTS fMatriculas;
```

Recriando a tabela dAlunos

Vamos começar recriando a tabela dAlunos.

Essa tabela terá duas restrições principais:

1) A coluna ID_Aluno será uma chave primária (Primary Key).
2) As colunas Nome_Aluno e Email não aceitarão valores nulos.

O código final é mostrado ao lado.

```
CREATE TABLE dAlunos(
    ID_Aluno INT,
    Nome_Aluno VARCHAR(100) NOT NULL,
    Email VARCHAR(100) NOT NULL,
    PRIMARY KEY(ID_Aluno)
);
```

Recriando a tabela dCursos

Vamos agora recriar a tabela dCursos.

Essa tabela terá duas restrições principais:

1) A coluna ID_Curso será uma chave primária (Primary Key).
2) As colunas Nome_Curso e Preco_Curso não aceitarão valores nulos.

O código final é mostrado ao lado.

```
CREATE TABLE IF NOT EXISTS dCursos(
    ID_Curso INT,
    Nome_Curso VARCHAR(100) NOT NULL,
    Preco_Curso DECIMAL(10, 2) NOT NULL,
    PRIMARY KEY(ID_Curso)
);
```

Recriando a tabela fMatriculas

Vamos agora recriar a tabela fMatriculas.

Essa tabela terá três restrições principais:

1) A coluna ID_Matricula será uma chave primária (Primary Key).
2) As colunas ID_Aluno, ID_Curso e Data_Cadastro não aceitarão valores nulos.
3) As colunas ID_Aluno e ID_Curso serão chaves estrangeiras que farão referência às colunas de mesmos nomes das tabelas dAlunos e dCursos, respectivamente.

O código final é mostrado ao lado.

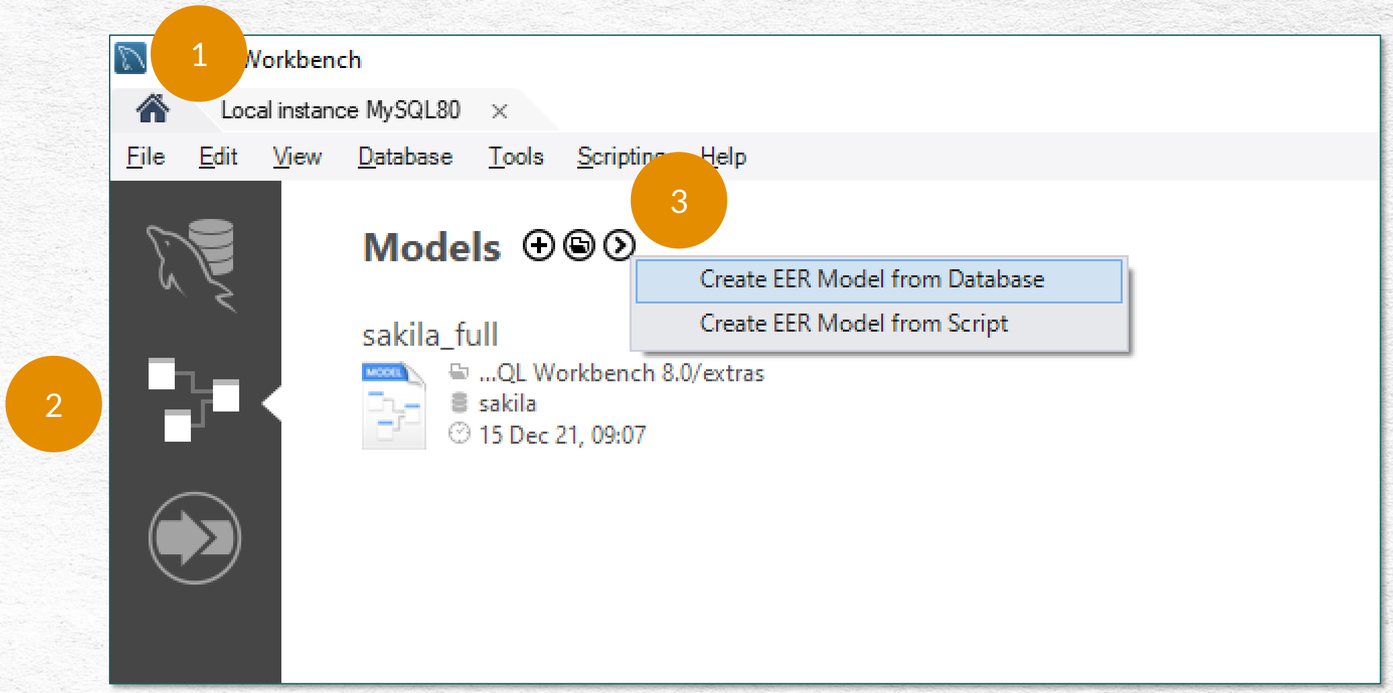
```
CREATE TABLE IF NOT EXISTS fMatriculas(
    ID_Matricula INT,
    ID_Aluno INT NOT NULL,
    ID_Curso INT NOT NULL,
    Data_Cadastro DATE NOT NULL,
    PRIMARY KEY(ID_Matricula),
    FOREIGN KEY(ID_Aluno) REFERENCES dAlunos(ID_Aluno),
    FOREIGN KEY(ID_Curso) REFERENCES dCursos(ID_Curso)
);
```

Visualizando os relacionamentos entre tabelas

Agora vamos ver a diferença prática que a criação de constraints proporciona.

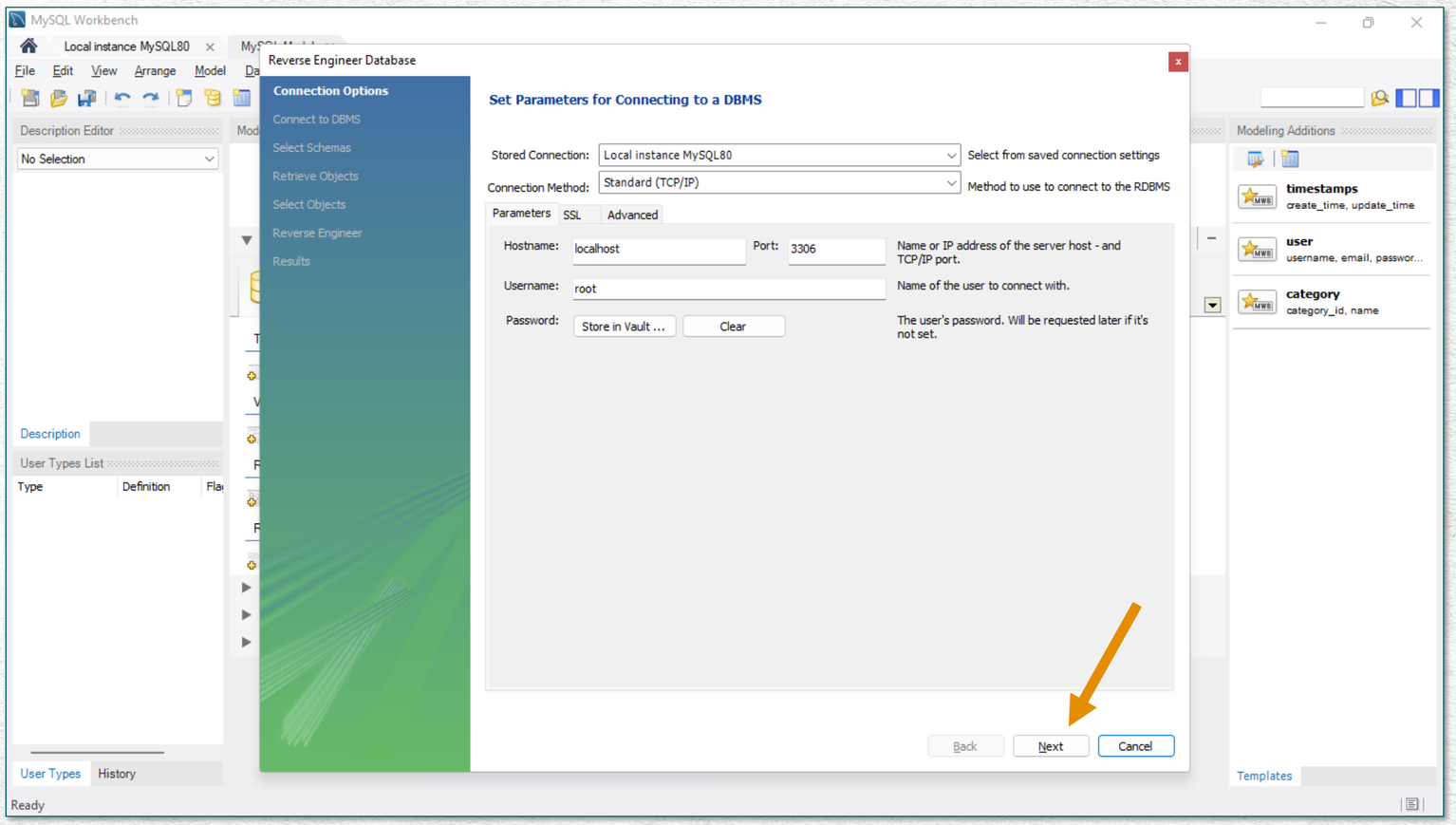
As constraints de primary key e foreign key criadas vão gerar relacionamentos entre essas tabelas, que podemos visualizar através do caminho ao lado.

Clique na casinha (1), depois clique em (2) e por fim, na setinha em (3) clique em 'Create EER Model from Database'.



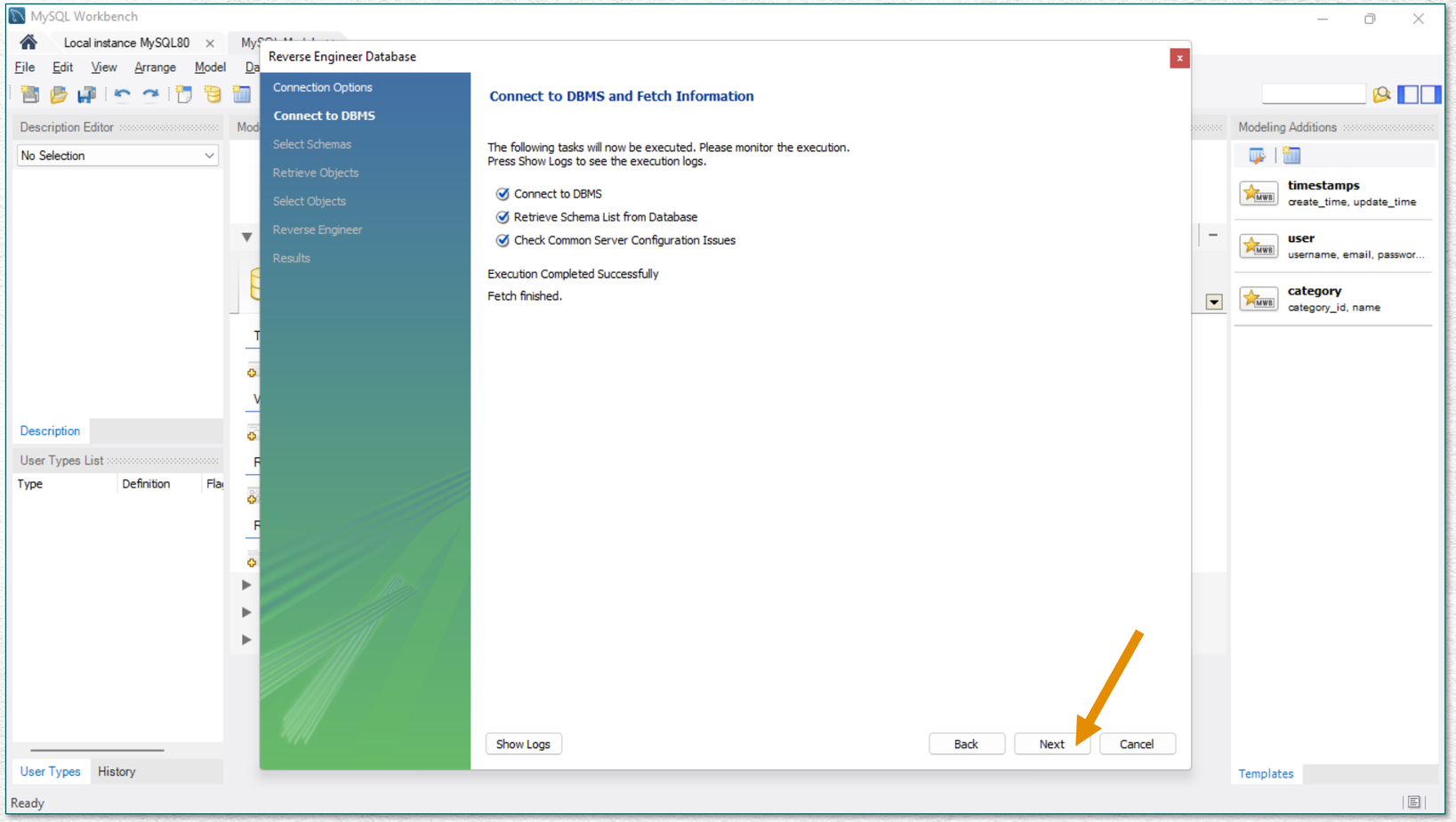
Visualizando os relacionamentos entre tabelas

Em seguida, na janela que abrir, clique em Next.



Visualizando os relacionamentos entre tabelas

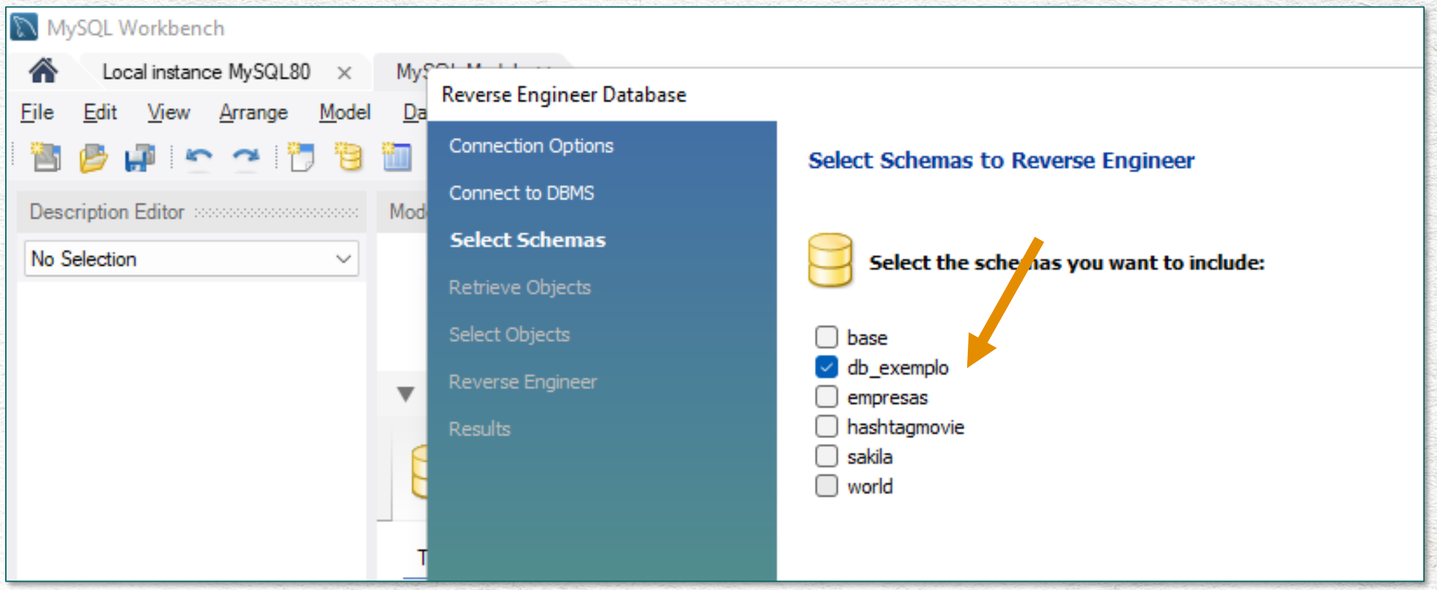
Em seguida, na janela que abrir, clique em Next novamente.



Visualizando os relacionamentos entre tabelas

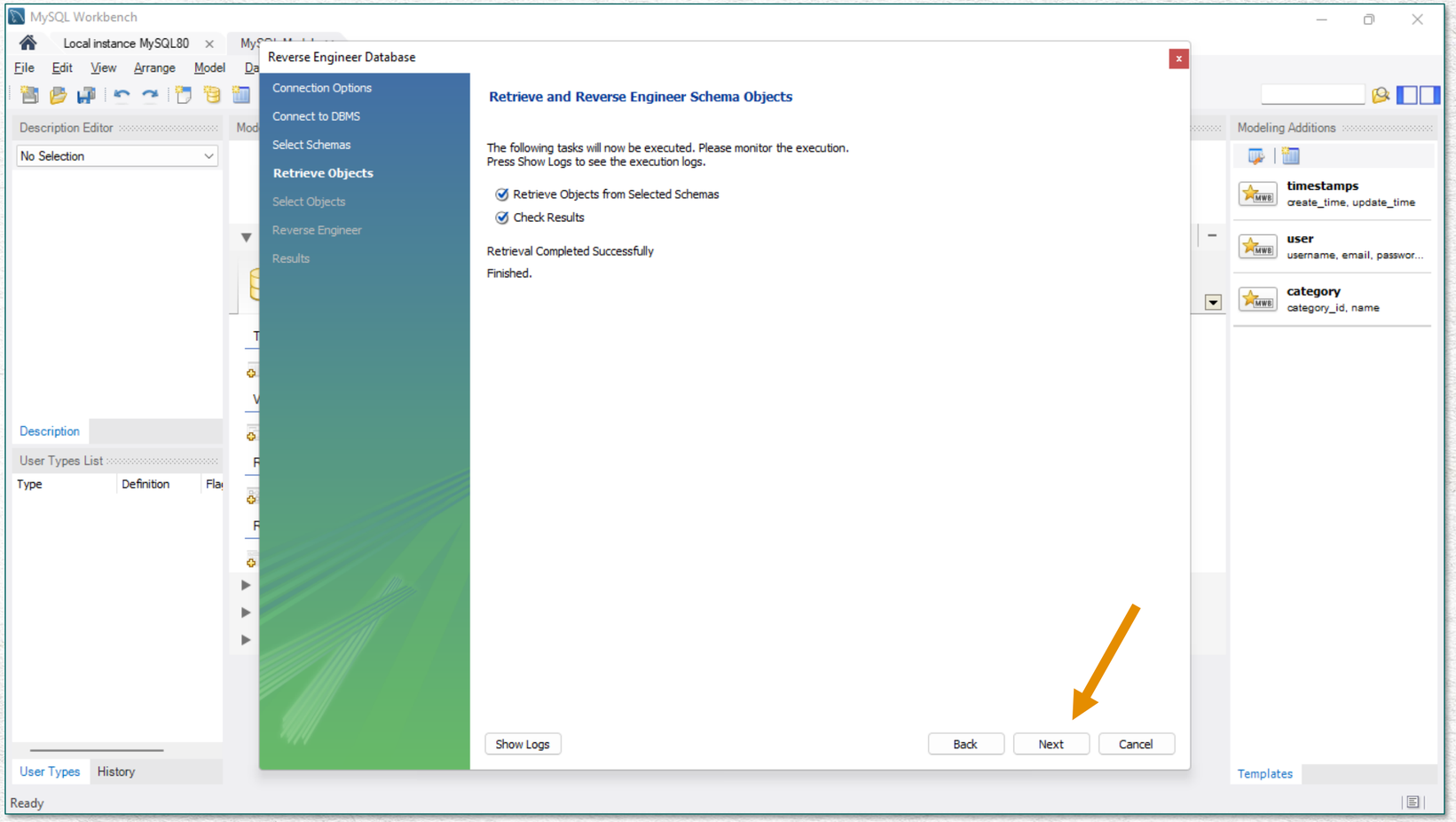
Na lista de bancos de dados, escolha aquele que você deseja visualizar os relacionamentos. No caso, o db_Exemplo.

Depois é só clicar em Next.



Visualizando os relacionamentos entre tabelas

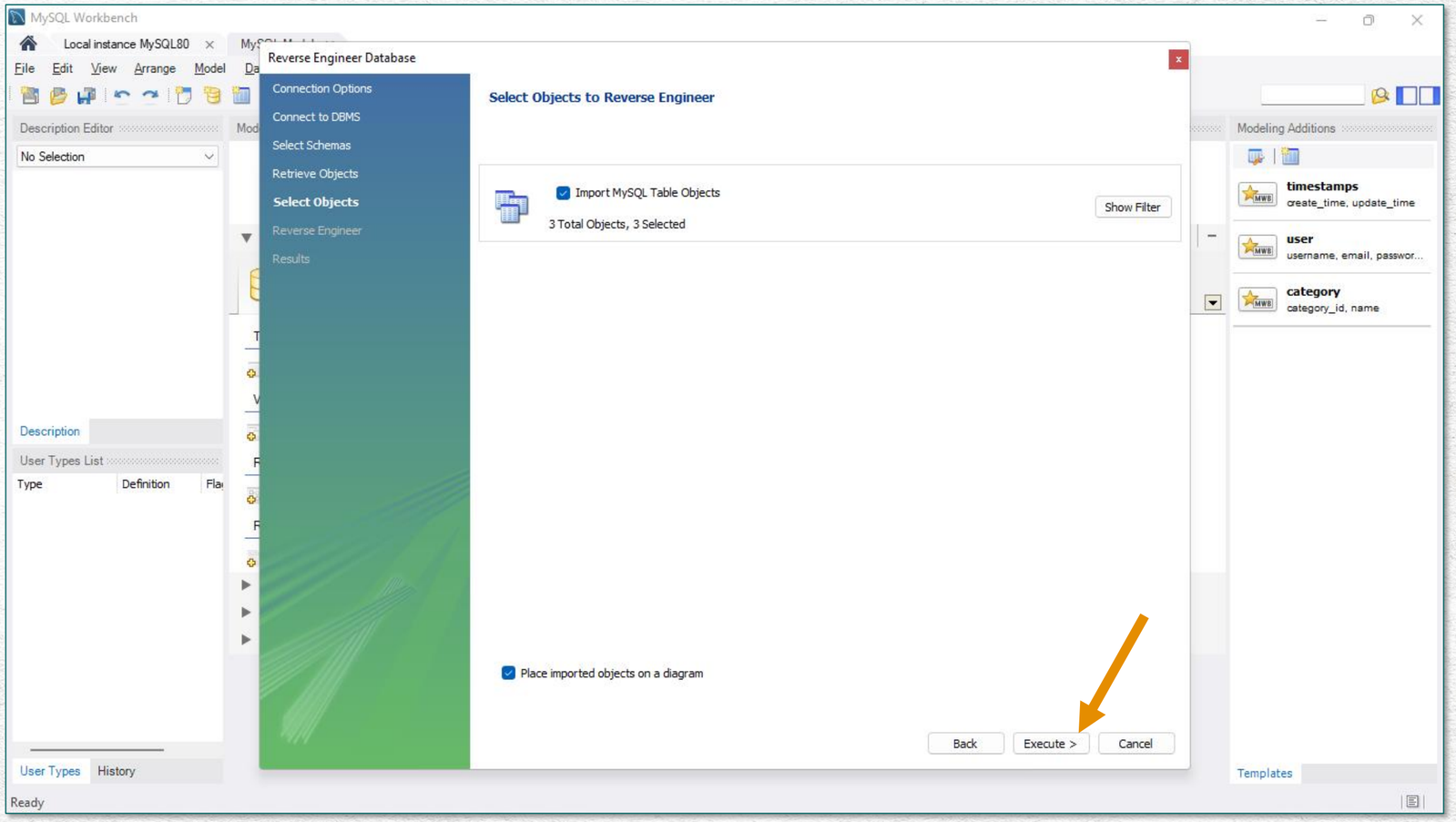
Na janela seguinte, clique em Next novamente.



Visualizando os relacionamentos entre tabelas

Por fim, em Execute.

Depois, é só seguir com o Next até que finalize as etapas.

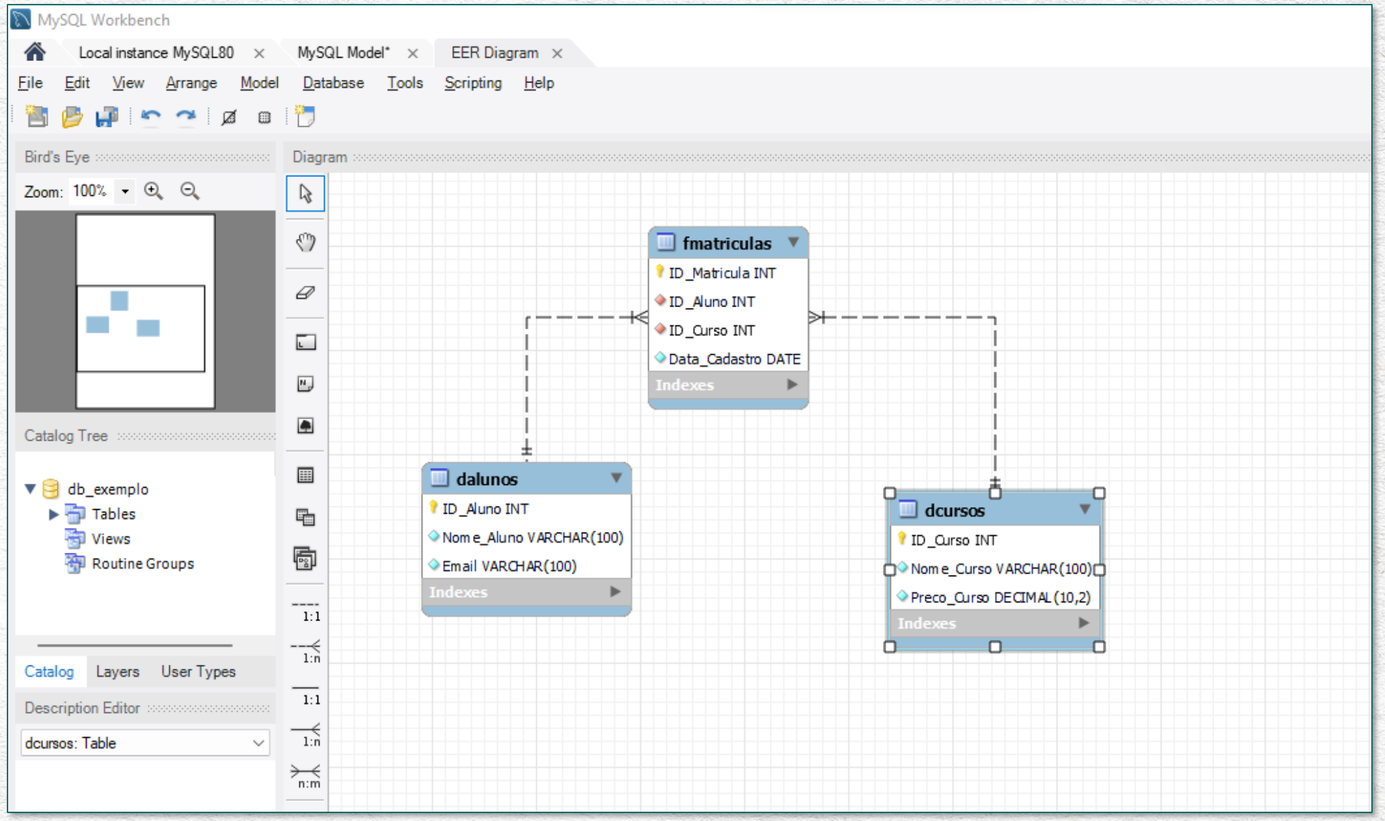


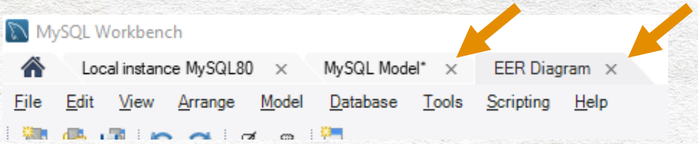
Visualizando os relacionamentos entre tabelas

Ao terminar as etapas, será aberta uma janela com o diagrama onde você poderá visualizar as tabelas do banco, relacionadas entre si.

Essa relação surgiu por conta das restrições de chave primária e chave estrangeira criadas.

Por fim, você pode simplesmente fechar a janela para voltar para o MySQL, onde estamos criando nossos códigos.





Índice Geral
Índice deste Módulo

INSERT INTO

Agora que criamos as tabelas, vamos adicionar valores nelas usando o comando INSERT INTO.

Para cada uma das tabelas, o código é mostrado ao lado.

```
247     # 5. Inserindo dados nas tabelas.
248
249 •   INSERT INTO dAlunos(ID_Aluno, Nome_Aluno, Email)
250     VALUES
251         (1, 'Ana'    ,   'ana123@gmail.com'          ),
252         (2, 'Bruno' ,   'bruno_vargas@outlook.com'  ),
253         (3, 'Carla' ,   'carlinha1999@gmail.com'    ),
254         (4, 'Diego' ,   'diicastroneves@gmail.com'  );
255
256 •   INSERT INTO dCursos(ID_Curso, Nome_Curso, Preco_Curso)
257     VALUES
258         (1, 'Excel'     ,   100),
259         (2, 'VBA'       ,   200),
260         (3, 'Power BI'  ,   150);
261
262 •   INSERT INTO fMatriculas(ID_Matricula, ID_Aluno, ID_Curso, Data_Cadastro)
263     VALUES
264         (1, 1, 1, '2021/03/11'  ),
265         (2, 1, 2, '201/06/21'   ),
266         (3, 2, 3, '2021/01/08'  ),
267         (4, 3, 1, '2021/04/03'  ),
268         (5, 4, 1, '2021/05/10'  ),
269         (6, 4, 3, '2021/05/10'  );
270
```

UPDATE

Podemos atualizar dados de uma tabela usando o comando UPDATE, conforme a estrutura mostrada ao lado.

Um detalhe importante é que você não pode esquecer de incluir o comando WHERE para garantir que você vai atualizar apenas a linha desejada.

Caso você esqueça do WHERE, todas as linhas da tabela serão atualizadas.

```
# 6. Atualizando dados de uma tabela com o UPDATE

UPDATE dCursos
SET Preco_Curso = 300
WHERE ID_Curso = 1;
```

DELETE

Para excluir linhas da tabela, usamos o DELETE conforme a estrutura ao lado.

Assim como no UPDATE, é muito importante que você especifique a linha que deseja excluir através do WHERE para não correr o risco de excluir todas as linhas da tabela.

```
# DELETE. Deletando registros de uma tabela

DELETE FROM fMatriculas
WHERE ID_Matricula = 6;
```

DROP TABLE x TRUNCATE TABLE

Por fim, temos outros dois comandos que nos permitem excluir dados.

O DROP TABLE é um comando que já vimos e que permite excluir definitivamente uma tabela.

Já o TRUNCATE TABLE é um comando que exclui de uma vez todas as linhas de uma tabela, mas mantém essa tabela no banco de dados.

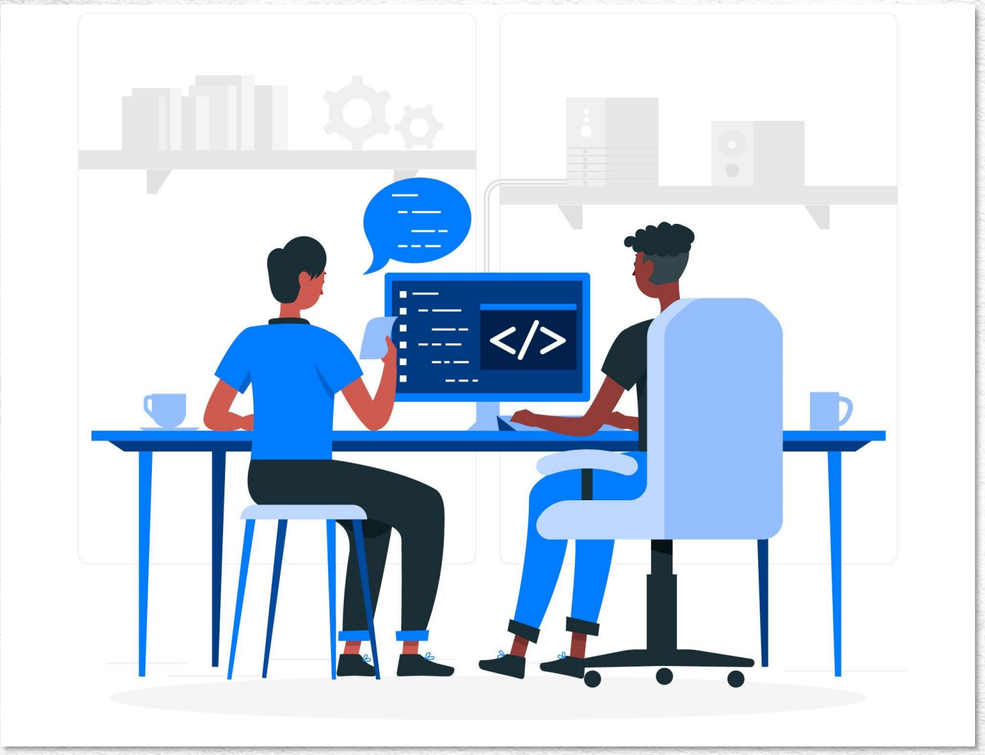
```
# TRUNCATE TABLE x DROP TABLE

-- TRUNCATE TABLE: Deleta todos os registros da tabela de uma vez, mas a tabela
continua existindo.

-- DROP TABLE: Deleta todos os registros da tabela, inclusive a própria tabela.

TRUNCATE TABLE fMatriculas;
DROP TABLE fMatriculas;
```

EXERCÍCIOS



1

Questão 1

Crie um banco de dados chamado **db_Projetos** e o coloque em uso.

2a

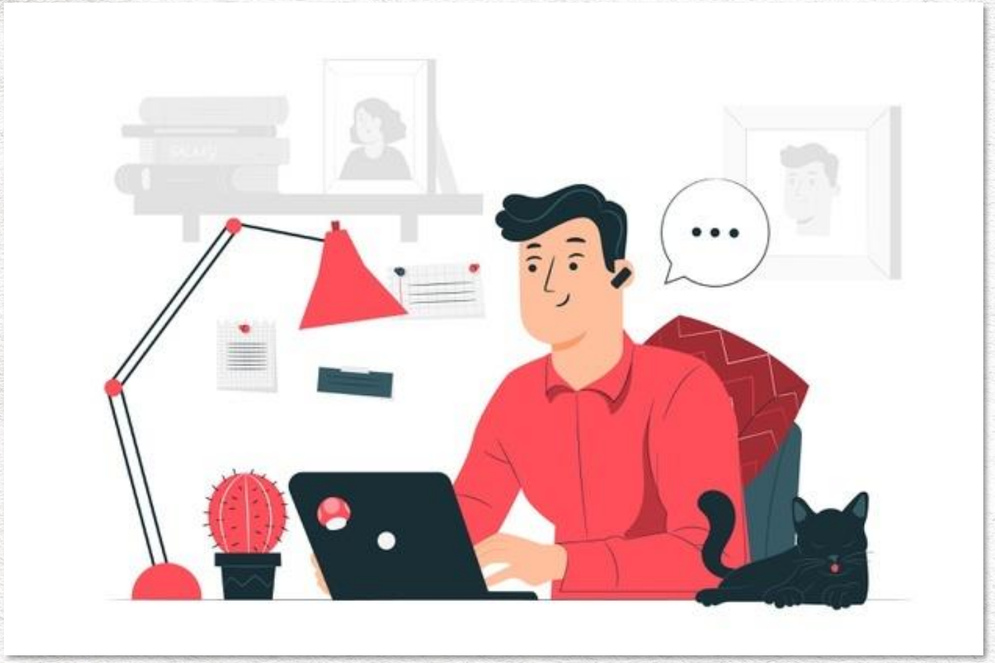
Questão 2-a

Crie as seguintes tabelas no banco db_Projetos:

a) Tabela Funcionarios

NOME DA COLUNA	ESPECIFICAÇÕES
ID_Funcionario	INT, será a chave primária da tabela.
Nome	VARCHAR(100), não pode ser nulo.
Sobrenome	VARCHAR(100), não pode ser nulo.
Email	VARCHAR(50), não pode ser nulo e deve ser único.
Data_Nascimento	DATE, não pode ser nula.
Data_Admissao	DATE, não pode ser nula.
Salario	DECIMAL(10, 2), não pode ser nulo.

EXERCÍCIOS



2b

Questão 2-b

Crie as seguintes tabelas no banco db_Projetos:

b) Tabela Projetos

NOME DA COLUNA	ESPECIFICAÇÕES
ID_Projeto	INT, será a chave primária da tabela.
Nome_Projeto	VARCHAR(100), não pode ser nulo e deve ser único.
Descricao	VARCHAR(250), pode ser nula.
Data_Inicio	DATE, não pode ser nula.
Data_Fim	DATE, pode ser nula.

2c

Questão 2-c

Crie as seguintes tabelas no banco db_Projetos:

c) Tabela Alocacoes

NOME DA COLUNA	ESPECIFICAÇÕES
ID_Alocacao	INT, será a chave primária da tabela.
ID_Funcionario	INT, será a chave estrangeira que fará referência à coluna ID_Funcionario da tabela Funcionarios, não pode ser nula.
ID_Projeto	INT, será a chave estrangeira que fará referência à coluna ID_Projeto da tabela Projetos, não pode ser nula.
Horas_Trabalhadas	INT, não pode ser nula.

EXERCÍCIOS

3a

Questão 3-a

Agora que as tabelas foram criadas, insira os dados dos funcionários, projetos e alocações:

a) Insira **10 funcionários** na **tabela Funcionarios** e faça uma consulta para verificar se os dados foram inseridos corretamente:

ID_Funcionario	Nome	Sobrenome	Email	Data_Nascimento	Data_Admissao	Salario
1	Ana	Silva	ana.silva@email.com	1985-02-20	2010-04-10	3500.50
2	Carlos	Oliveira	carlos.oliveira@email.com	1990-05-15	2015-08-01	4000.00
3	Lucas	Souza	lucas.souza@email.com	1988-07-22	2012-09-10	4500.75
4	Maria	Pereira	maria.pereira@email.com	1995-11-30	2017-02-14	3800.00
5	João	Costa	joao.costa@email.com	1980-01-10	2011-03-18	5000.00
6	Fernanda	Gomes	fernanda.gomes@email.com	1992-04-05	2016-06-30	4200.00
7	Rodrigo	Martins	rodrigo.martins@email.com	1987-09-25	2013-07-20	4600.00
8	Juliana	Alves	juliana.alves@email.com	1994-10-12	2018-01-15	3700.00
9	Ricardo	Santos	ricardo.santos@email.com	1983-12-05	2010-11-20	4800.50
10	Patrícia	Lima	patricia.lima@email.com	1991-03-14	2014-04-25	4100.25

3b

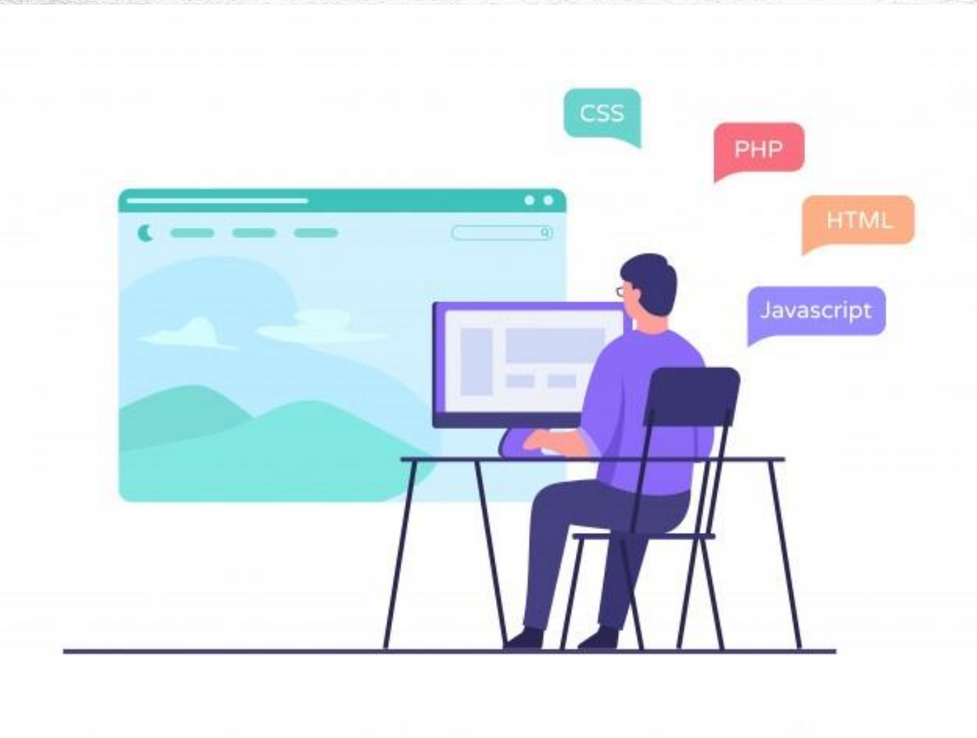
Questão 3-b

b) Insira **8 projetos** na **tabela Projetos** e faça uma consulta para verificar se os dados foram inseridos corretamente:

ID_Projeto	Nome_Projeto	Descricao	Data_Inicio	Data_Fim
1	Sistema ERP	Desenvolvimento de um ERP para a empresa	2022-01-01	2023-01-01
2	Reestruturação Financeira	Projeto de reestruturação do setor financeiro	2021-06-01	2022-12-31
3	Desenvolvimento Web	Criação de um site institucional	2022-03-15	NULL
4	Automação de Processos	Automação dos processos internos da empresa	2022-07-01	2023-05-01
5	Plataforma E-Commerce	Desenvolvimento de plataforma para vendas online	2022-05-10	NULL
6	Integração de Sistemas	Integração de sistemas de gestão com fornecedores	2022-08-01	2023-06-30
7	Expansão de Mercado	Projeto para expansão da empresa para novas regiões	2023-01-01	NULL
8	Segurança de TI	Melhoria da infraestrutura de segurança de TI	2022-11-01	2023-11-01

EXERCÍCIOS

3c

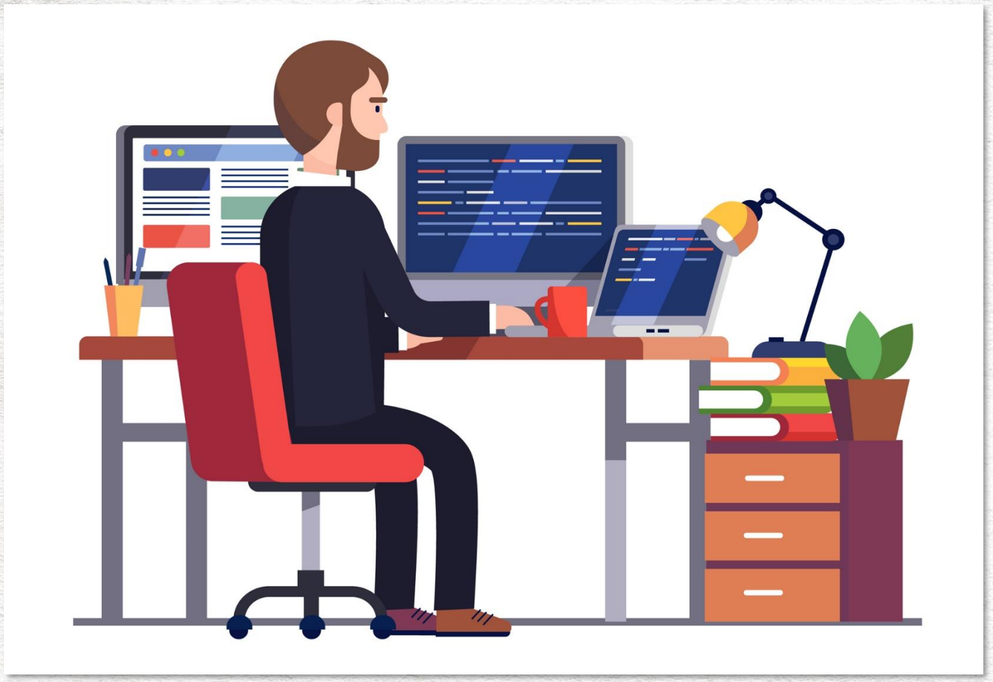


Questão 3-c

c) Insira **20 alocações** na **tabela Alocacoes** e faça uma consulta para verificar se os dados foram inseridos corretamente:

ID_Alocacao	ID_Funcionario	ID_Projeto	Horas_Trabalhadas
1	1	1	40
2	2	2	35
3	3	3	45
4	4	4	40
5	5	5	38
6	6	6	40
7	7	7	42
8	8	8	36
9	9	1	40
10	10	2	35
11	1	3	40
12	2	4	38
13	3	5	40
14	4	6	35
15	5	7	38
16	6	8	40
17	7	3	36
18	8	4	45
19	9	5	40
20	10	6	42

EXERCÍCIOS



4

Questão 4

Você recebeu uma solicitação do departamento de recursos humanos para ajustar o salário de um dos funcionários. O funcionário com o ID_Funcionario igual a 3 (Lucas Souza) deve ter o seu salário aumentado de R$ 4500,75 para R$ 5000,00.
Realize a atualização da tabela Funcionarios para refletir esse aumento salarial.

5

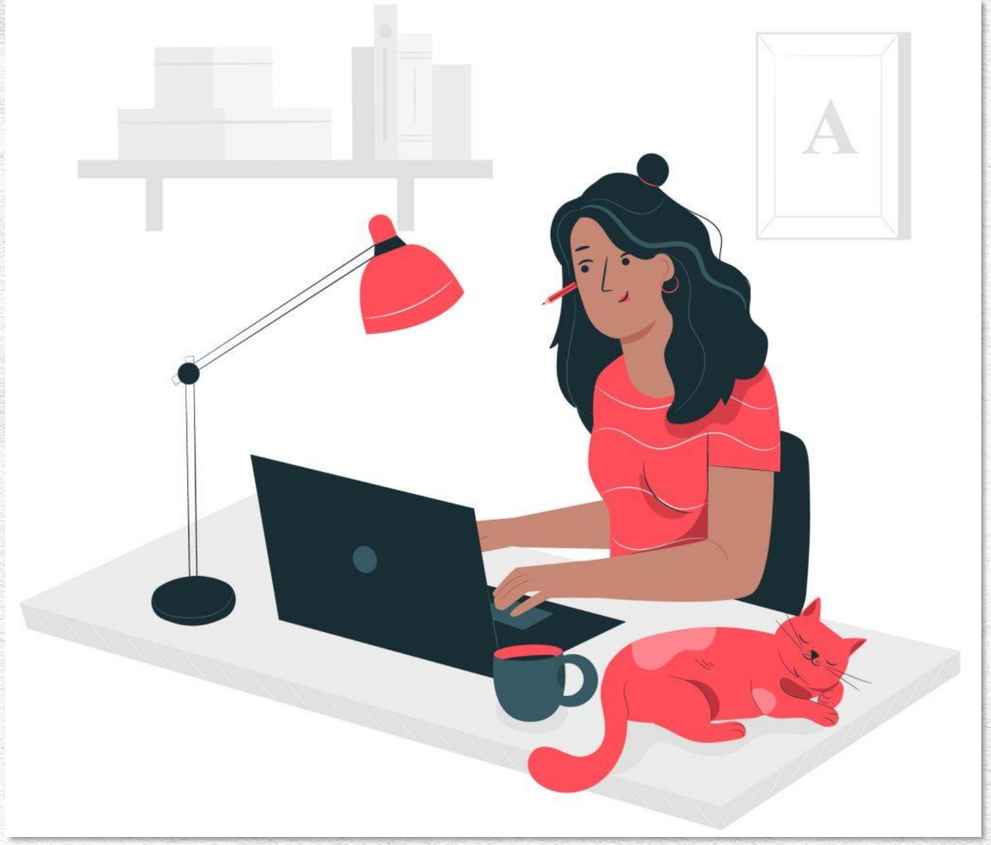
Questão 5

O projeto "Expansão de Mercado" (ID_Projeto = 7) foi cancelado e, como resultado, precisa ser removido do banco de dados.
Como este projeto está referenciado na tabela Alocacoes, é necessário seguir os seguintes passos para realizar sua exclusão:

a) Excluir todas as alocações que fazem referência a esse projeto na tabela Alocacoes.

b) Após a exclusão das alocações, excluir o projeto da tabela Projetos.

EXERCÍCIOS



6

Questão 6

A empresa realizou uma reestruturação em sua gestão de alocações de recursos. O processo agora é gerenciado por um sistema externo integrado, e, por isso, os dados armazenados na tabela Alocacoes já foram migrados para essa nova plataforma.

Para liberar espaço de armazenamento e simplificar o banco de dados, a tabela Alocacoes deve ser excluída. Efetue a exclusão dessa tabela.

7

Questão 7

Após uma revisão do planejamento de TI, a empresa decidiu descontinuar o banco de dados db_Projetos. Isso ocorreu porque a gestão de projetos foi reorganizada e um novo sistema de gestão foi implementado, que utiliza uma plataforma diferente e mais integrada.

Como o banco de dados db_Projetos não será mais utilizado, é necessário excluí-lo completamente do sistema, removendo todas as tabelas e os dados nele contidos. Proceda à essa exclusão.

GABARITO EXPLICATIVO

1

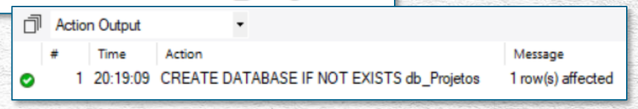
Questão 1

Crie um banco de dados chamado **db_Projetos** e o coloque em uso.

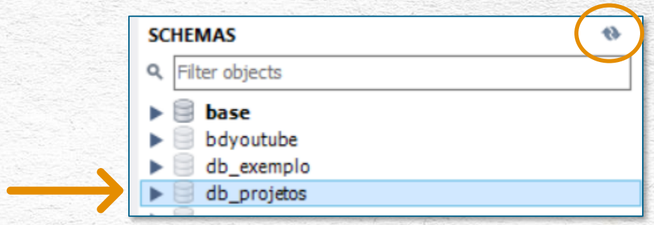
Explicação e Resultado

Para criar o banco de dados **db_Projetos**, podemos executar o seguinte comando (sendo o IF NOT EXISTS opcional):

```
CREATE DATABASE IF NOT EXISTS db_Projetos;
```

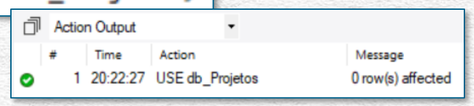


Clicando no ícone de **Atualizar** no menu lateral esquerdo do MySQL Workbench, veremos que o banco de dados **db_projetos foi criado**, porém, ele não está em uso (pois não está aparecendo em negrito):

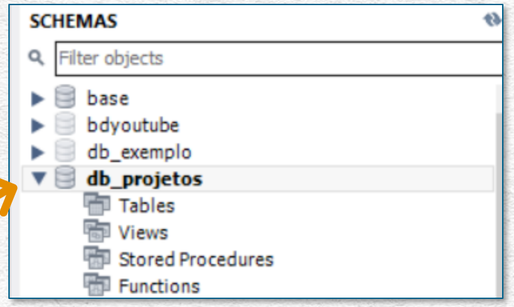


Para colocar o banco de dados **db_projetos** em uso, executamos o seguinte comando:

```
USE db_Projetos;
```



Perceba que **agora ele aparece em uso** (negrito):



GABARITO EXPLICATIVO

2a

Questão 2-a

Crie as seguintes tabelas no banco db_Projetos:

a) Tabela Funcionarios

NOME DA COLUNA	ESPECIFICAÇÕES
ID_Funcionario	INT, será a chave primária da tabela.
Nome	VARCHAR(100), não pode ser nulo.
Sobrenome	VARCHAR(100), não pode ser nulo.
Email	VARCHAR(50), não pode ser nulo e deve ser único.
Data_Nascimento	DATE, não pode ser nula.
Data_Admissao	DATE, não pode ser nula.
Salario	DECIMAL(10, 2), não pode ser nulo.

Explicação e Resultado

Para criar a tabela **Funcionarios** com as colunas solicitadas e com as especificações apresentadas ao lado, executamos um **CREATE TABLE IF NOT EXISTS** (sendo o IF NOT EXISTS opcional), passando para o comando os **nomes das colunas** e as **especificações** de cada uma, ou seja, **seus tipos de dados e constraints**:

```
-- a) Criando a tabela Funcionarios
CREATE TABLE IF NOT EXISTS Funcionarios (
    ID_Funcionario INT,
    Nome VARCHAR(100) NOT NULL,
    Sobrenome VARCHAR(100) NOT NULL,
    Email VARCHAR(50) NOT NULL UNIQUE,
    Data_Nascimento DATE NOT NULL,
    Data_Admissao DATE NOT NULL,
    Salario DECIMAL(10, 2) NOT NULL,
    PRIMARY KEY(ID_Funcionario)
);
```

Action Output

#	Time	Action	Message
1	20:31:43	CREATE TABLE IF ...	0 row(s) affected

GABARITO EXPLICATIVO

2b

Questão 2-b

Crie as seguintes tabelas no banco db_Projetos:

b) Tabela Projetos

NOME DA COLUNA	ESPECIFICAÇÕES
ID_Projeto	INT, será a chave primária da tabela.
Nome_Projeto	VARCHAR(100), não pode ser nulo e deve ser único.
Descricao	VARCHAR(250), pode ser nula.
Data_Inicio	DATE, não pode ser nula.
Data_Fim	DATE, pode ser nula.

Explicação e Resultado

Para criar a tabela **Projetos** com as colunas solicitadas e com as especificações apresentadas ao lado, executamos um **CREATE TABLE IF NOT EXISTS** (sendo o IF NOT EXISTS opcional), passando para o comando os **nomes das colunas** e as **especificações** de cada uma, ou seja, **seus tipos de dados e constraints**:

```
-- b) Criando a tabela Projetos
CREATE TABLE IF NOT EXISTS Projetos (
    ID_Projeto INT,
    Nome_Projeto VARCHAR(100) NOT NULL UNIQUE,
    Descricao VARCHAR(250),
    Data_Inicio DATE NOT NULL,
    Data_Fim DATE,
    PRIMARY KEY(ID_Projeto)
);
```

Action Output

#	Time	Action	Message
1	20:34:13	CREATE TABLE IF ...	0 row(s) affected

GABARITO EXPLICATIVO

2c

Questão 2-c

Crie as seguintes tabelas no banco db_Projetos:

c) Tabela Alocacoes

NOME DA COLUNA	ESPECIFICAÇÕES
ID_Alocacao	INT, será a chave primária da tabela.
ID_Funcionario	INT, será a chave estrangeira que fará referência à coluna ID_Funcionario da tabela Funcionarios, não pode ser nula.
ID_Projeto	INT, será a chave estrangeira que fará referência à coluna ID_Projeto da tabela Projetos, não pode ser nula.
Horas_Trabalhadas	INT, não pode ser nula.

Explicação e Resultado

Para criar a tabela **Alocacoes** com as colunas solicitadas e com as especificações apresentadas ao lado, executamos um **CREATE TABLE IF NOT EXISTS** (sendo o IF NOT EXISTS opcional), passando para o comando os **nomes das colunas** e as **especificações** de cada uma, ou seja, **seus tipos de dados e constraints**:

```
-- c) Criando a tabela Alocacoes
CREATE TABLE IF NOT EXISTS Alocacoes (
    ID_Alocacao INT,
    ID_Funcionario INT NOT NULL,
    ID_Projeto INT NOT NULL,
    Horas_Trabalhadas INT NOT NULL,
    PRIMARY KEY(ID_Alocacao),
    FOREIGN KEY(ID_Funcionario) REFERENCES Funcionarios(ID_Funcionario),
    FOREIGN KEY(ID_Projeto) REFERENCES Projetos(ID_Projeto)
);
```

Action Output

#	Time	Action	Message
1	20:35:26	CREATE TABLE IF ...	0 row(s) affected

GABARITO EXPLICATIVO

3a

Questão 3-a

Agora que as tabelas foram criadas, insira os dados dos funcionários, projetos e alocações:

a) Insira **10 funcionários** na **tabela Funcionarios** e faça uma consulta para verificar se os dados foram inseridos corretamente:

ID_Funcionario	Nome	Sobrenome	Email	Data_Nascimento	Data_Admissao	Salario
1	Ana	Silva	ana.silva@email.com	1985-02-20	2010-04-10	3500.50
2	Carlos	Oliveira	carlos.oliveira@email.com	1990-05-15	2015-08-01	4000.00
3	Lucas	Souza	lucas.souza@email.com	1988-07-22	2012-09-10	4500.75
4	Maria	Pereira	maria.pereira@email.com	1995-11-30	2017-02-14	3800.00
5	João	Costa	joao.costa@email.com	1980-01-10	2011-03-18	5000.00
6	Fernanda	Gomes	fernanda.gomes@email.com	1992-04-05	2016-06-30	4200.00
7	Rodrigo	Martins	rodrigo.martins@email.com	1987-09-25	2013-07-20	4600.00
8	Juliana	Alves	juliana.alves@email.com	1994-10-12	2018-01-15	3700.00
9	Ricardo	Santos	ricardo.santos@email.com	1983-12-05	2010-11-20	4800.50
10	Patrícia	Lima	patricia.lima@email.com	1991-03-14	2014-04-25	4100.25

Explicação e Resultado

Para inserir as **10 linhas** mostradas ao lado na **tabela Funcionarios**, executamos o comando **INSERT INTO** conforme mostrado abaixo:

```
INSERT INTO Funcionarios (ID_Funcionario, Nome, Sobrenome, Email, Data_Nascimento, Data_Admissao, Salario)
VALUES
(1, 'Ana', 'Silva', 'ana.silva@email.com', '1985-02-20', '2010-04-10', 3500.50),
(2, 'Carlos', 'Oliveira', 'carlos.oliveira@email.com', '1990-05-15', '2015-08-01', 4000.00),
(3, 'Lucas', 'Souza', 'lucas.souza@email.com', '1988-07-22', '2012-09-10', 4500.75),
(4, 'Maria', 'Pereira', 'maria.pereira@email.com', '1995-11-30', '2017-02-14', 3800.00),
(5, 'João', 'Costa', 'joao.costa@email.com', '1980-01-10', '2011-03-18', 5000.00),
(6, 'Fernanda', 'Gomes', 'fernanda.gomes@email.com', '1992-04-05', '2016-06-30', 4200.00),
(7, 'Rodrigo', 'Martins', 'rodrigo.martins@email.com', '1987-09-25', '2013-07-20', 4600.00),
(8, 'Juliana', 'Alves', 'juliana.alves@email.com', '1994-10-12', '2018-01-15', 3700.00),
(9, 'Ricardo', 'Santos', 'ricardo.santos@email.com', '1983-12-05', '2010-11-20', 4800.50),
(10, 'Patrícia', 'Lima', 'patricia.lima@email.com', '1991-03-14', '2014-04-25', 4100.25);
```

Action Output

#	Time	Action	Message
1	20:42:26	INSERT INTO Fun...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

3a

Questão 3-a

Agora que as tabelas foram criadas, insira os dados dos funcionários, projetos e alocações:

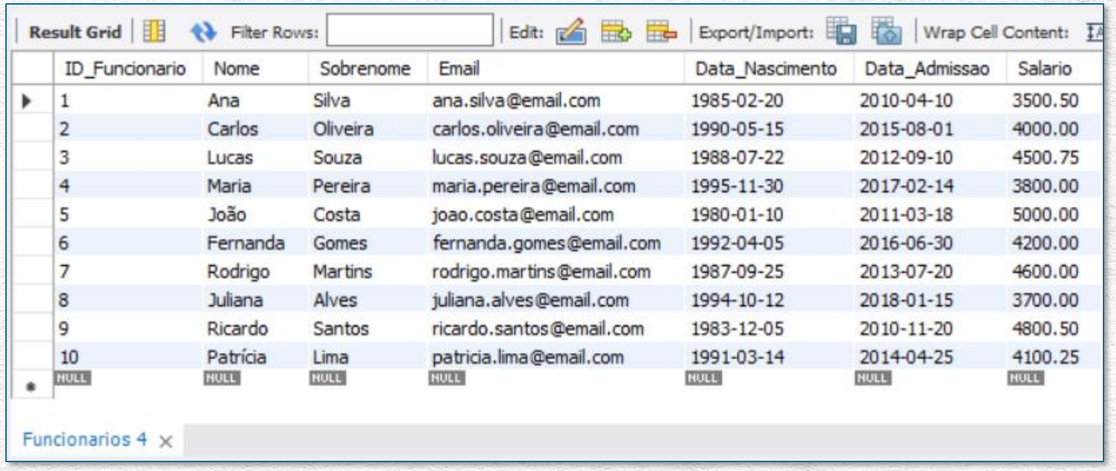
a) Insira **10 funcionários** na **tabela Funcionarios** e faça uma consulta para verificar se os dados foram inseridos corretamente:

ID_Funcionario	Nome	Sobrenome	Email	Data_Nascimento	Data_Admissao	Salario
1	Ana	Silva	ana.silva@email.com	1985-02-20	2010-04-10	3500.50
2	Carlos	Oliveira	carlos.oliveira@email.com	1990-05-15	2015-08-01	4000.00
3	Lucas	Souza	lucas.souza@email.com	1988-07-22	2012-09-10	4500.75
4	Maria	Pereira	maria.pereira@email.com	1995-11-30	2017-02-14	3800.00
5	João	Costa	joao.costa@email.com	1980-01-10	2011-03-18	5000.00
6	Fernanda	Gomes	fernanda.gomes@email.com	1992-04-05	2016-06-30	4200.00
7	Rodrigo	Martins	rodrigo.martins@email.com	1987-09-25	2013-07-20	4600.00
8	Juliana	Alves	juliana.alves@email.com	1994-10-12	2018-01-15	3700.00
9	Ricardo	Santos	ricardo.santos@email.com	1983-12-05	2010-11-20	4800.50
10	Patrícia	Lima	patricia.lima@email.com	1991-03-14	2014-04-25	4100.25

Explicação e Resultado

Assim, ao consultarmos a **tabela Funcionarios**, vemos que **as 10 linhas foram inseridas:**

```
SELECT * FROM Funcionarios;
```



ID_Funcionario	Nome	Sobrenome	Email	Data_Nascimento	Data_Admissao	Salario
1	Ana	Silva	ana.silva@email.com	1985-02-20	2010-04-10	3500.50
2	Carlos	Oliveira	carlos.oliveira@email.com	1990-05-15	2015-08-01	4000.00
3	Lucas	Souza	lucas.souza@email.com	1988-07-22	2012-09-10	4500.75
4	Maria	Pereira	maria.pereira@email.com	1995-11-30	2017-02-14	3800.00
5	João	Costa	joao.costa@email.com	1980-01-10	2011-03-18	5000.00
6	Fernanda	Gomes	fernanda.gomes@email.com	1992-04-05	2016-06-30	4200.00
7	Rodrigo	Martins	rodrigo.martins@email.com	1987-09-25	2013-07-20	4600.00
8	Juliana	Alves	juliana.alves@email.com	1994-10-12	2018-01-15	3700.00
9	Ricardo	Santos	ricardo.santos@email.com	1983-12-05	2010-11-20	4800.50
10	Patrícia	Lima	patricia.lima@email.com	1991-03-14	2014-04-25	4100.25
NULL	NULL	NULL	NULL	NULL	NULL	NULL

GABARITO EXPLICATIVO

3b

Questão 3-b

b) Insira **8 projetos** na **tabela Projetos** e faça uma consulta para verificar se os dados foram inseridos corretamente:

ID_Projeto	Nome_Projeto	Descricao	Data_Inicio	Data_Fim
1	Sistema ERP	Desenvolvimento de um ERP para a empresa	2022-01-01	2023-01-01
2	Reestruturação Financeira	Projeto de reestruturação do setor financeiro	2021-06-01	2022-12-31
3	Desenvolvimento Web	Criação de um site institucional	2022-03-15	NULL
4	Automação de Processos	Automação dos processos internos da empresa	2022-07-01	2023-05-01
5	Plataforma E-Commerce	Desenvolvimento de plataforma para vendas online	2022-05-10	NULL
6	Integração de Sistemas	Integração de sistemas de gestão com fornecedores	2022-08-01	2023-06-30
7	Expansão de Mercado	Projeto para expansão da empresa para novas regiões	2023-01-01	NULL
8	Segurança de TI	Melhoria da infraestrutura de segurança de TI	2022-11-01	2023-11-01

Explicação e Resultado

Para inserir as **8 linhas** mostradas ao lado na **tabela Projetos**, executamos o comando **INSERT INTO** conforme mostrado abaixo:

```
INSERT INTO Projetos (ID_Projeto, Nome_Projeto, Descricao, Data_Inicio, Data_Fim)
VALUES
(1, 'Sistema ERP', 'Desenvolvimento de um ERP para a empresa', '2022-01-01', '2023-01-01'),
(2, 'Reestruturação Financeira', 'Projeto de reestruturação do setor financeiro', '2021-06-01', '2022-12-31'),
(3, 'Desenvolvimento Web', 'Criação de um site institucional', '2022-03-15', NULL),
(4, 'Automação de Processos', 'Automação dos processos internos da empresa', '2022-07-01', '2023-05-01'),
(5, 'Plataforma E-Commerce', 'Desenvolvimento de plataforma para vendas online', '2022-05-10', NULL),
(6, 'Integração de Sistemas', 'Integração de sistemas de gestão com fornecedores', '2022-08-01', '2023-06-30'),
(7, 'Expansão de Mercado', 'Projeto para expansão da empresa para novas regiões', '2023-01-01', NULL),
(8, 'Segurança de TI', 'Melhoria da infraestrutura de segurança de TI', '2022-11-01', '2023-11-01');
```

Action Output

#	Time	Action	Message
1	20:46:46	INSERT INTO Proj...	8 row(s) affected Records: 8 Duplicates: 0 Warnings: 0

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

3b

Questão 3-b

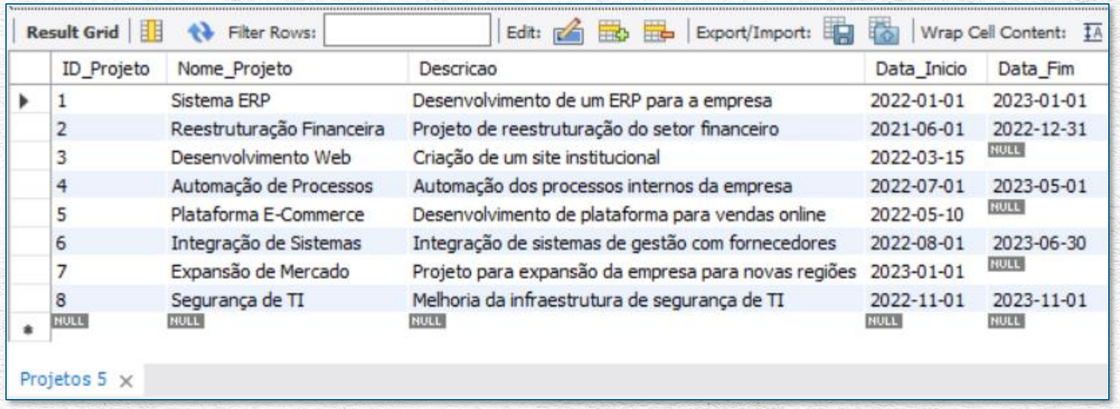
b) Insira **8 projetos** na **tabela Projetos** e faça uma consulta para verificar se os dados foram inseridos corretamente:

ID_Projeto	Nome_Projeto	Descricao	Data_Inicio	Data_Fim
1	Sistema ERP	Desenvolvimento de um ERP para a empresa	2022-01-01	2023-01-01
2	Reestruturação Financeira	Projeto de reestruturação do setor financeiro	2021-06-01	2022-12-31
3	Desenvolvimento Web	Criação de um site institucional	2022-03-15	NULL
4	Automação de Processos	Automação dos processos internos da empresa	2022-07-01	2023-05-01
5	Plataforma E-Commerce	Desenvolvimento de plataforma para vendas online	2022-05-10	NULL
6	Integração de Sistemas	Integração de sistemas de gestão com fornecedores	2022-08-01	2023-06-30
7	Expansão de Mercado	Projeto para expansão da empresa para novas regiões	2023-01-01	NULL
8	Segurança de TI	Melhoria da infraestrutura de segurança de TI	2022-11-01	2023-11-01

Explicação e Resultado

Assim, ao consultarmos a **tabela Projetos**, vemos que **as 8 linhas foram inseridas**:

```
SELECT * FROM Projetos;
```



ID_Projeto	Nome_Projeto	Descricao	Data_Inicio	Data_Fim
1	Sistema ERP	Desenvolvimento de um ERP para a empresa	2022-01-01	2023-01-01
2	Reestruturação Financeira	Projeto de reestruturação do setor financeiro	2021-06-01	2022-12-31
3	Desenvolvimento Web	Criação de um site institucional	2022-03-15	NULL
4	Automação de Processos	Automação dos processos internos da empresa	2022-07-01	2023-05-01
5	Plataforma E-Commerce	Desenvolvimento de plataforma para vendas online	2022-05-10	NULL
6	Integração de Sistemas	Integração de sistemas de gestão com fornecedores	2022-08-01	2023-06-30
7	Expansão de Mercado	Projeto para expansão da empresa para novas regiões	2023-01-01	NULL
8	Segurança de TI	Melhoria da infraestrutura de segurança de TI	2022-11-01	2023-11-01
NULL	NULL	NULL	NULL	NULL

GABARITO EXPLICATIVO

3c

Questão 3-c

c) Insira **20 alocações** na **tabela Alocacoes** e faça uma consulta para verificar se os dados foram inseridos corretamente:

ID_Alocacao	ID_Funcionario	ID_Projeto	Horas_Trabalhadas
1	1	1	40
2	2	2	35
3	3	3	45
4	4	4	40
5	5	5	38
6	6	6	40
7	7	7	42
8	8	8	36
9	9	1	40
10	10	2	35
11	1	3	40
12	2	4	38
13	3	5	40
14	4	6	35
15	5	7	38
16	6	8	40
17	7	3	36
18	8	4	45
19	9	5	40
20	10	6	42

Explicação e Resultado

Para inserir as **20 linhas** mostradas ao lado na **tabela Alocacoes**, executamos o comando **INSERT INTO** conforme mostrado abaixo:

```
INSERT INTO Alocacoes (ID_Alocacao, ID_Funcionario, ID_Projeto, Horas_Trabalhadas)
VALUES
(1, 1, 1, 40),
(2, 2, 2, 35),
(3, 3, 3, 45),
(4, 4, 4, 40),
(5, 5, 5, 38),
(6, 6, 6, 40),
(7, 7, 7, 42),
(8, 8, 8, 36),
(9, 9, 1, 40),
(10, 10, 2, 35),
(11, 1, 3, 40),
(12, 2, 4, 38),
(13, 3, 5, 40),
(14, 4, 6, 35),
(15, 5, 7, 38),
(16, 6, 8, 40),
(17, 7, 3, 36),
(18, 8, 4, 45),
(19, 9, 5, 40),
(20, 10, 6, 42);
```

Action Output

#	Time	Action	Message
1	20:50:32	INSERT INTO Aloc...	20 row(s) affected Records: 20 Duplicates: 0 Warnings: 0

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

3c

Questão 3-c

c) Insira **20 alocações** na **tabela Alocacoes** e faça uma consulta para verificar se os dados foram inseridos corretamente:

ID_Alocacao	ID_Funcionario	ID_Projeto	Horas_Trabalhadas
1	1	1	40
2	2	2	35
3	3	3	45
4	4	4	40
5	5	5	38
6	6	6	40
7	7	7	42
8	8	8	36
9	9	1	40
10	10	2	35
11	1	3	40
12	2	4	38
13	3	5	40
14	4	6	35
15	5	7	38
16	6	8	40
17	7	3	36
18	8	4	45
19	9	5	40
20	10	6	42

Explicação e Resultado

Assim, ao consultarmos a **tabela Alocacoes**, vemos que **as 20 linhas foram inseridas**:

```
SELECT * FROM Alocacoes;
```

Result Grid | Filter Rows: | Edit:

ID_Alocacao	ID_Funcionario	ID_Projeto	Horas_Trabalhadas
1	1	1	40
2	2	2	35
3	3	3	45
4	4	4	40
5	5	5	38
6	6	6	40
7	7	7	42
8	8	8	36
9	9	1	40
10	10	2	35
11	1	3	40
12	2	4	38
13	3	5	40
14	4	6	35
15	5	7	38
16	6	8	40
17	7	3	36
18	8	4	45
19	9	5	40
20	10	6	42
NULL	NULL	NULL	NULL

Alocacoes 6

GABARITO EXPLICATIVO

4

Questão 4

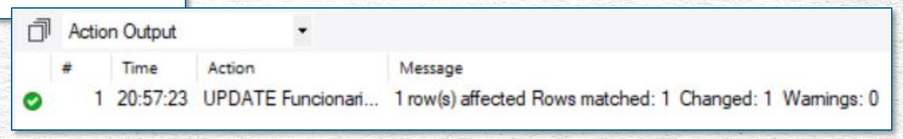
Você recebeu uma solicitação do departamento de recursos humanos para ajustar o salário de um dos funcionários. O funcionário com o ID_Funcionario igual a 3 (Lucas Souza) deve ter o seu salário aumentado de R$ 4500,75 para R$ 5000,00.

Realize a atualização da tabela Funcionarios para refletir esse aumento salarial.

Explicação e Resultado

Para **atualizarmos o salário do funcionário de ID_Funcionario = 3 (Lucas Souza) para R$ 5000,00**, executamos o comando **UPDATE** abaixo, passando para ele o nome da tabela a ser atualizada (**Funcionarios**), seguido do comando **SET** com a alteração a ser efetuada (**SET Salario = 5000.00**) e a condição que deve ser respeitada para que somente a linha referente ao funcionário Lucas Souza receba a atualização (**WHERE ID_Funcionario = 3**):

```
UPDATE Funcionarios
SET Salario = 5000.00
WHERE ID_Funcionario = 3;
```



CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

4

Questão 4

Você recebeu uma solicitação do departamento de recursos humanos para ajustar o salário de um dos funcionários. O funcionário com o ID_Funcionario igual a 3 (Lucas Souza) deve ter o seu salário aumentado de R$ 4500,75 para R$ 5000,00.

Realize a atualização da tabela Funcionarios para refletir esse aumento salarial.

Explicação e Resultado

Dessa forma, **ao consultarmos a tabela Funcionarios** novamente, vemos que a **alteração foi de fato efetuada**:

```
SELECT * FROM Funcionarios;
```

ID_Funcionario	Nome	Sobrenome	Email	Data_Nascimento	Data_Admissao	Salario
1	Ana	Silva	ana.silva@email.com	1985-02-20	2010-04-10	3500.50
2	Carlos	Oliveira	carlos.oliveira@email.com	1990-05-15	2015-08-01	4000.00
3	Lucas	Souza	lucas.souza@email.com	1988-07-22	2012-09-10	5000.00
4	Maria	Pereira	maria.pereira@email.com	1995-11-30	2017-02-14	3800.00
5	João	Costa	joao.costa@email.com	1980-01-10	2011-03-18	5000.00
6	Fernanda	Gomes	fernanda.gomes@email.com	1992-04-05	2016-06-30	4200.00
7	Rodrigo	Martins	rodrigo.martins@email.com	1987-09-25	2013-07-20	4600.00
8	Juliana	Alves	juliana.alves@email.com	1994-10-12	2018-01-15	3700.00
9	Ricardo	Santos	ricardo.santos@email.com	1983-12-05	2010-11-20	4800.50
10	Patrícia	Lima	patricia.lima@email.com	1991-03-14	2014-04-25	4100.25
NULL	NULL	NULL	NULL	NULL	NULL	NULL

GABARITO EXPLICATIVO

5

Questão 5

O projeto "Expansão de Mercado" (ID_Projeto = 7) foi cancelado e, como resultado, precisa ser removido do banco de dados.

Como este projeto está referenciado na tabela Alocacoes, é necessário seguir os seguintes passos para realizar sua exclusão:

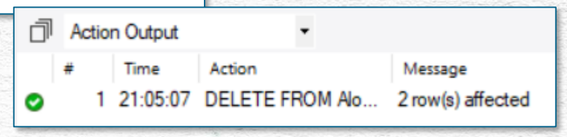
a) Excluir todas as alocações que fazem referência a esse projeto na tabela Alocacoes.

b) Após a exclusão das alocações, excluir o projeto da tabela Projetos.

Explicação e Resultado

a) Para **excluirmos todas as alocações** que fazem referência ao projeto que queremos deletar (**"Expansão de Mercado"**), primeiro executamos o comando **DELETE** na **tabela Alocacoes**, passando como **condição** o ID referente à esse projeto (**ID_Projeto = 7**):

```
DELETE FROM Alocacoes
WHERE ID_Projeto = 7;
```



CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

5

Questão 5

O projeto "Expansão de Mercado" (ID_Projeto = 7) foi cancelado e, como resultado, precisa ser removido do banco de dados.

Como este projeto está referenciado na tabela Alocacoes, é necessário seguir os seguintes passos para realizar sua exclusão:

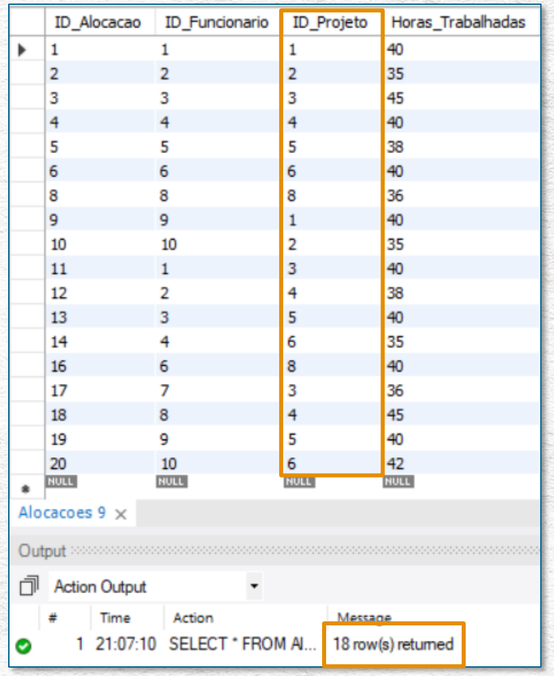
a) Excluir todas as alocações que fazem referência a esse projeto na tabela Alocacoes.

b) Após a exclusão das alocações, excluir o projeto da tabela Projetos.

Explicação e Resultado

Consultando a **tabela Alocacoes** novamente, vemos que as **duas linhas** que se referiam a esse projeto foram **excluídas**, restando somente **18 linhas** na tabela (nenhuma delas se referindo ao ID_Projeto = 7):

```
SELECT * FROM Alocacoes;
```



ID_Alocacao	ID_Funcionario	ID_Projeto	Horas_Trabalhadas
1	1	1	40
2	2	2	35
3	3	3	45
4	4	4	40
5	5	5	38
6	6	6	40
8	8	8	36
9	9	1	40
10	10	2	35
11	1	3	40
12	2	4	38
13	3	5	40
14	4	6	35
16	6	8	40
17	7	3	36
18	8	4	45
19	9	5	40
20	10	6	42
NULL	NULL	NULL	NULL

Alocacoes 9

Output

Action Output

#	Time	Action	Message
1	21:07:10	SELECT * FROM Al...	18 row(s) returned

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

5

Questão 5

O projeto "Expansão de Mercado" (ID_Projeto = 7) foi cancelado e, como resultado, precisa ser removido do banco de dados.

Como este projeto está referenciado na tabela Alocacoes, é necessário seguir os seguintes passos para realizar sua exclusão:

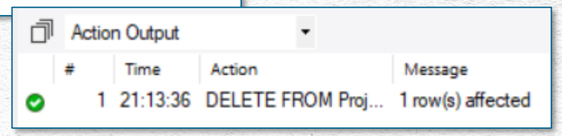
a) Excluir todas as alocações que fazem referência a esse projeto na tabela Alocacoes.

b) Após a exclusão das alocações, excluir o projeto da tabela Projetos.

Explicação e Resultado

b) Agora, para **excluirmos** o projeto **"Expansão de Mercado"** da tabela Projetos, executamos o comando **DELETE** na **tabela Projetos**, passando como **condição** o ID referente à esse projeto (**ID_Projeto = 7**):

```
DELETE FROM Projetos
WHERE ID_Projeto = 7;
```



CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

5

Questão 5

O projeto "Expansão de Mercado" (ID_Projeto = 7) foi cancelado e, como resultado, precisa ser removido do banco de dados.

Como este projeto está referenciado na tabela Alocacoes, é necessário seguir os seguintes passos para realizar sua exclusão:

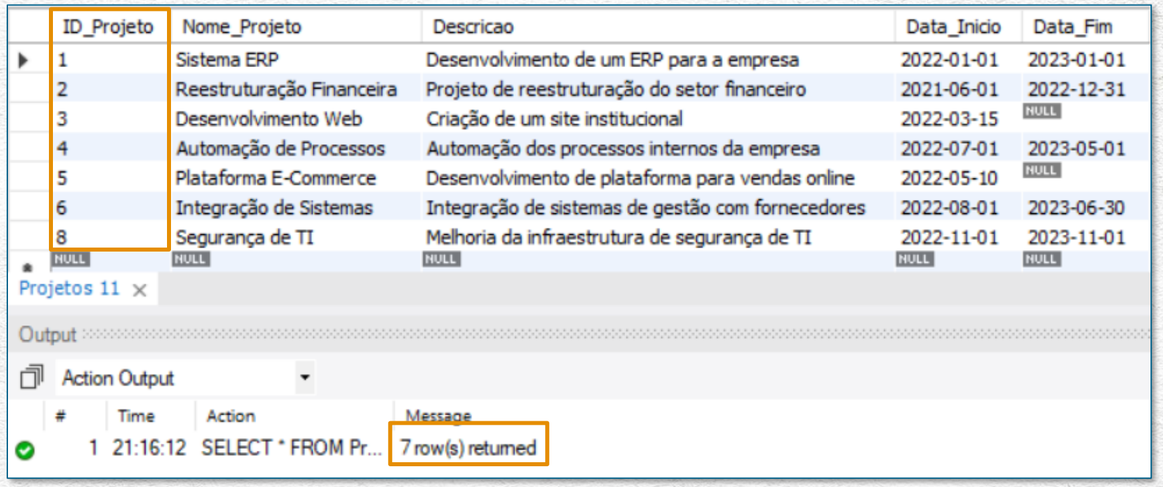
a) Excluir todas as alocações que fazem referência a esse projeto na tabela Alocacoes.

b) Após a exclusão das alocações, excluir o projeto da tabela Projetos.

Explicação e Resultado

Consultando a **tabela Projetos** novamente, vemos que **a linha** que se referia a esse projeto foi **excluída**, restando somente **7 linhas** na tabela (nenhuma delas se referindo ao ID_Projeto = 7):

```
SELECT * FROM Projetos;
```



ID_Projeto	Nome_Projeto	Descricao	Data_Inicio	Data_Fim
1	Sistema ERP	Desenvolvimento de um ERP para a empresa	2022-01-01	2023-01-01
2	Reestruturação Financeira	Projeto de reestruturação do setor financeiro	2021-06-01	2022-12-31
3	Desenvolvimento Web	Criação de um site institucional	2022-03-15	NULL
4	Automação de Processos	Automação dos processos internos da empresa	2022-07-01	2023-05-01
5	Plataforma E-Commerce	Desenvolvimento de plataforma para vendas online	2022-05-10	NULL
6	Integração de Sistemas	Integração de sistemas de gestão com fornecedores	2022-08-01	2023-06-30
8	Segurança de TI	Melhoria da infraestrutura de segurança de TI	2022-11-01	2023-11-01
NULL	NULL	NULL	NULL	NULL

Projetos 11

Output

Action Output

#	Time	Action	Message
1	21:16:12	SELECT * FROM Pr...	7 row(s) returned

GABARITO EXPLICATIVO

6

Questão 6

A empresa realizou uma reestruturação em sua gestão de alocações de recursos. O processo agora é gerenciado por um sistema externo integrado, e, por isso, os dados armazenados na tabela Alocacoes já foram migrados para essa nova plataforma.

Para liberar espaço de armazenamento e simplificar o banco de dados, a tabela Alocacoes deve ser excluída. Efetue a exclusão dessa tabela.

Explicação e Resultado

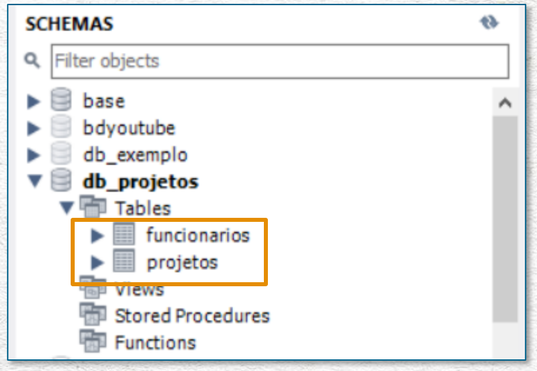
Para **excluir a tabela Alocacoes** do banco de dados db_projetos, executamos o comando **DROP TABLE IF EXISTS** (sendo o IF EXISTS opcional) **e o nome da tabela**, conforme mostrado abaixo:

```
DROP TABLE IF EXISTS Alocacoes;
```

Action Output

#	Time	Action	Message
1	21:19:56	DROP TABLE IF EXISTS Alocacoes	0 row(s) affected

No menu lateral esquerdo do MySQL Workbench, vemos que **a tabela Alocacoes foi excluída**, restando somente as tabelas Funcionarios e Projetos:



GABARITO EXPLICATIVO

7

Questão 7

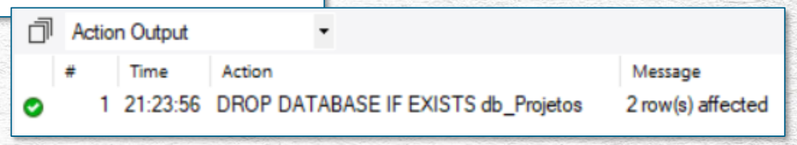
Após uma revisão do planejamento de TI, a empresa decidiu descontinuar o banco de dados db_Projetos. Isso ocorreu porque a gestão de projetos foi reorganizada e um novo sistema de gestão foi implementado, que utiliza uma plataforma diferente e mais integrada.

Como o banco de dados db_Projetos não será mais utilizado, é necessário excluí-lo completamente do sistema, removendo todas as tabelas e os dados nele contidos. Proceda à essa exclusão.

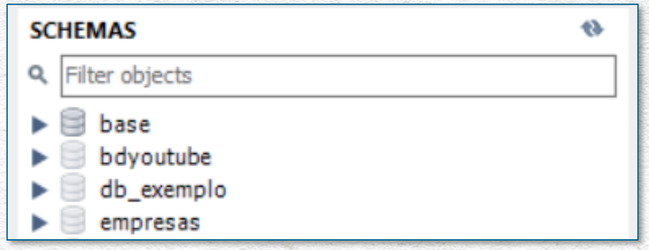
Explicação e Resultado

Para **excluir o banco de dados db_projetos**, executamos o comado **DROP DATABASE IF EXISTS** (sendo o IF EXISTS opcional) **e o nome do banco de dados**, conforme mostrado abaixo:

```
DROP DATABASE IF EXISTS db_Projetos;
```



No menu lateral esquerdo do MySQL Workbench, vemos que **o banco de dados db_projetos foi excluído** do servidor:



MÓDULO 13

FUNCTIONS E PROCEDURES

FUNCTIONS E PROCEDURES

FUNCTIONS E PROCEDURES

Índice Geral

Índice deste Módulo

O QUE VEREMOS NESTE MÓDULO

MÓDULO 13 FUNCTIONS E PROCEDURES

01	FUNCTIONS E PROCEDURES	382
02	BEGIN... END;	383
03	BEGIN... END: FUNCTION	384
04	BEGIN... END: STORED PROCEDURE	385
05	ESCOPO DAS VARIÁVEIS: REVISÃO	386
06	DECLARAÇÃO DE VARIÁVEIS LOCAIS	388
07	FUNCTIONS	389
08	FUNCTIONS: EXEMPLO 1	390
09	FUNCTIONS: EXEMPLO 2	391
10	FUNCTIONS: EXEMPLO 3	392
11	ALTERANDO UMA FUNCTION	393
12	PARA QUE SERVE O DETERMINISTIC	395
13	STORED PROCEDURES	396
14	STORED PROCEDURES: EXEMPLO 1	398
15	STORED PROCEDURES: EXEMPLO 2	399
16	ALTERANDO UMA PROCEDURE	400
17	EXCLUINDO UMA PROCEDURE	402
18	FUNCTIONS VS STORED PROCEDURES	403
19	EXERCÍCIOS	404
20	GABARITO EXPLICATIVO	409

Functions e Procedures

Se tratam de dois tipos de rotinas. Uma rotina é um conjunto de instruções que você pode salvar no seu banco de dados e executar quando quiser, sem a necessidade de criar o código do zero toda vez que você precisar dele.

Funções (UDFs) e Procedimentos (Stored Procedures) são um pouco similares, mas com algumas diferenças em termos de aplicações.

BEGIN... END;

Os comandos BEGIN e END têm como objetivo delimitar um bloco de comandos a serem executados por uma Function ou Stored Procedure.

Cada bloco BEGIN / END possui um delimitador (;).

Porém, o delimitador ; pode ser problemático pois, ao ser encontrado em um procedimento ou função, ele a finaliza imediatamente. Para não termos problemas, devemos então mudar esse delimitador e para isso usamos o comando DELIMITER para criar rotinas com declarações compostas.

BEGIN... END: Function

Ao lado, temos um exemplo de Function criada, contendo o bloco BEGIN..END.

```
1   -- Criar função para cálculo de receita de produto
2
3   DELIMITER //
4 • CREATE FUNCTION fn_CalculaReceita(Quantidade INT, Preco DECIMAL(10, 2))
5   RETURNS DECIMAL(10, 2) DETERMINISTIC
6   BEGIN
7       RETURN quantidade * preco;
8   END //
9   DELIMITER ;
10
11 • SELECT fn_CalculaReceita(10, 99.90);
12
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

fn_CalculaReceita(10, 99.90)
999.00

BEGIN... END: Stored Procedure

Ao lado, temos um exemplo de Procedure criada, contendo o bloco BEGIN..END.

```
14    -- Criar procedimento para busca de preço do produto
15    DELIMITER $$
16 •  CREATE PROCEDURE sp_BuscaPreco(ID INT)
17    BEGIN
18        SELECT CONCAT('O preço do produto é: ', Preco_Unit) AS 'Resultado'
19        FROM produtos
20        WHERE ID_Produto = ID;
21        SELECT 'Procedimento finalizado com sucesso!!' AS 'Mensagem';
22    END $$
23    DELIMITER ;
24
25 •  CALL sp_BuscaPreco(4);
26
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Resultado
O preço do produto é: 350

Escopo das variáveis: Revisão

O escopo de uma variável se refere ao local onde essa variável existe, ou seja, o local onde ela pode ser acessada.

Temos os seguintes níveis de escopo:

1. USER-DEFINED VARIABLES
2. LOCAL VARIABLES

Escopo das variáveis: Revisão

O escopo de uma variável se refere ao local onde essa variável existe, ou seja, o local onde ela pode ser acessada.

Temos os seguintes níveis de escopo:

1. USER-DEFINED VARIABLES
2. LOCAL VARIABLES → **USADA DENTRO DE FUNCTIONS E STORED PROCEDURES**

Declaração de variáveis locais

Podemos criar variáveis locais em um procedimento ou função usando uma declaração DECLARE dentro de um bloco BEGIN.

A variável pode ser criada e inicializada com um valor se desejado.

Ficam disponíveis apenas dentro do bloco onde foram criadas, e em blocos que existam dentro do bloco onde a variável foi criada.

Após o bloco ter sido executado e encerrado, a variável é desalocada da memória.

FUNCTION

```
DELIMITER $$
CREATE FUNCTION fn_Exemplo(valor INT)
RETURNS INT
BEGIN
    DECLARE var1 INT;
    DECLARE var2 DECIMAL(10, 2);
    DECLARE var3 VARCHAR(100);
    DECLARE var4 DATE;

    SET var1 = 10;
    SET var2 = 9.98;
    SET var3 = 'SQL';
    SET var4 = '2999-12-31';
END $$
```

PROCEDURE

```
DELIMITER $$
CREATE PROCEDURE sp_Exemplo()
BEGIN
    DECLARE var1 INT;
    DECLARE var2 DECIMAL(10, 2);
    DECLARE var3 VARCHAR(100);
    DECLARE var4 DATE;

    SET var1 = 10;
    SET var2 = 9.98;
    SET var3 = 'SQL';
    SET var4 = '2999-12-31';
END $$
```

Functions

Uma função é uma rotina, um conjunto de instruções que você pode salvar no seu banco de dados e executar quando quiser, sem a necessidade de criar o código do zero toda vez que você precisa dele.

A estrutura para criação de uma Function é mostrada abaixo:

```
DELIMITER $$

CREATE FUNCTION nome_function(parametro1 TIPO1, parametro2 TIPO2)
RETURNS TIPO DETERMINISTIC
BEGIN

        .... Bloco de códigos

END $$

DELIMITER ;
```

Functions: Exemplo 1

Vamos começar criando a primeira Function, usando o código ao lado.

Essa function será bem simples e retornará uma mensagem de boas vindas.

Podemos chamar uma function diretamente dentro de um SELECT, como mostrado ao lado. Basta informar o parâmetro de entrada para que ela possa retornar o resultado.

```
-- Exemplo 1. Crie uma função que retorna o seguinte texto: "Olá ____, tudo bem?".

DELIMITER $$

CREATE FUNCTION fn_BoasVindas(nome VARCHAR(100))
RETURNS VARCHAR(100) DETERMINISTIC
BEGIN
    RETURN CONCAT('Olá ', nome, ', tudo bem?');
END $$

DELIMITER ;
```

```
1   SELECT fn_BoasVindas('Marcus');
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

fn_BoasVindas('Marcus')
Olá Marcus, tudo bem?

Functions: Exemplo 2

Uma função é usada para gerar um valor que pode ser usado em uma expressão.

O valor (resultado) é gerado baseado em um ou mais parâmetros de entrada fornecidos à função.

```
1 •  DROP FUNCTION IF EXISTS fn_Faturamento;
2
3 •  CREATE FUNCTION fn_Faturamento(preco DECIMAL(10, 2), quantidade INT)
4    RETURNS DECIMAL(10, 2) DETERMINISTIC
5    RETURN preco * quantidade;
6
7
8 •  SELECT fn_Faturamento(2.5, 10) AS 'Faturamento';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Faturamento
25.00

Functions: Exemplo 3

Uma função é usada para gerar um valor que pode ser usado em uma expressão.

O valor (resultado) é gerado baseado em um ou mais parâmetros de entrada fornecidos à função.

```
1 • CREATE FUNCTION fn_CalculaDesconto(preco DECIMAL(10, 2), desconto DECIMAL(10, 2))
2   RETURNS DECIMAL(10, 2) DETERMINISTIC
3   RETURN preco * (1 - desconto);
4
5 • SELECT Nome_Produto, Preco_Unit, fn_CalculaDesconto(Preco_Unit, 0.5) FROM produtos;
```

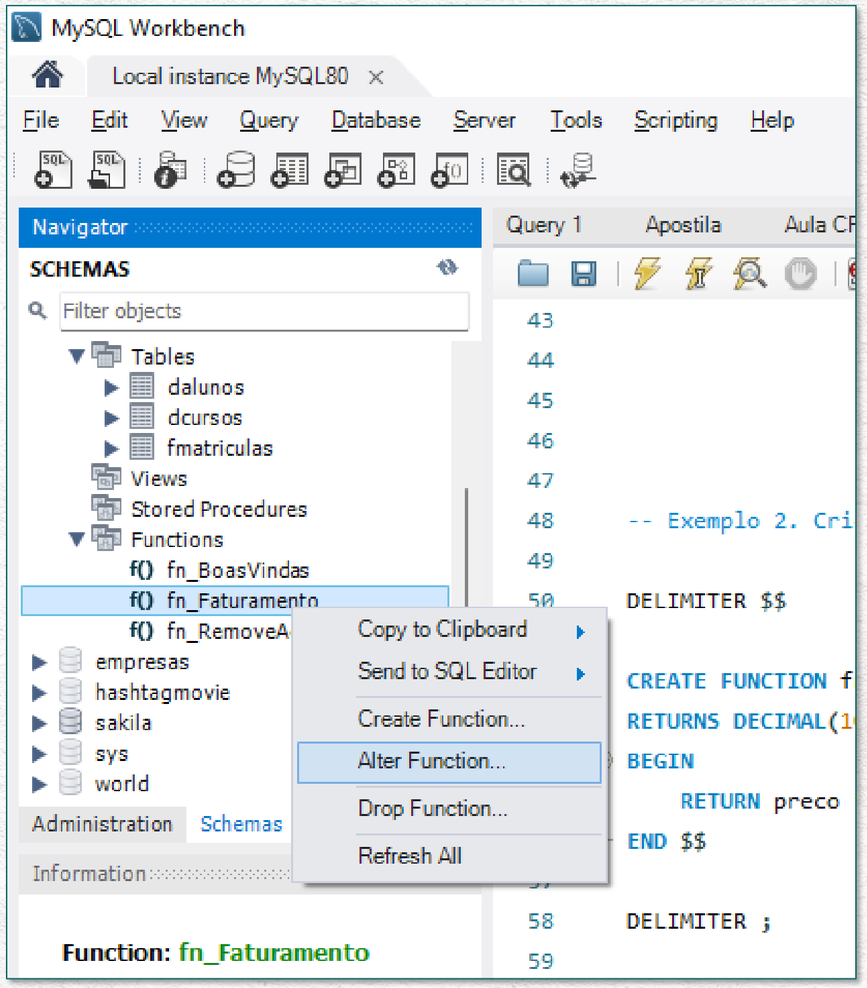
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Nome_Produto	Preco_Unit	fn_CalculaDesconto(Preco_Unit, 0.5)
Monitor LED 19,5" Full HD HDMI	2300	1150.00
Monitor Curvo 24" 144Hz HDMI	2800	1400.00
Webcam Full HD 1080p	450	225.00
Kit Teclado + Mouse sem fio Wireless	350	175.00
Kit Teclado + Mouse Slim Bluetooth	280	140.00
Cadeira Gamer reclinável Azul/Laranja	1800	900.00

Result 1

Alterando uma function

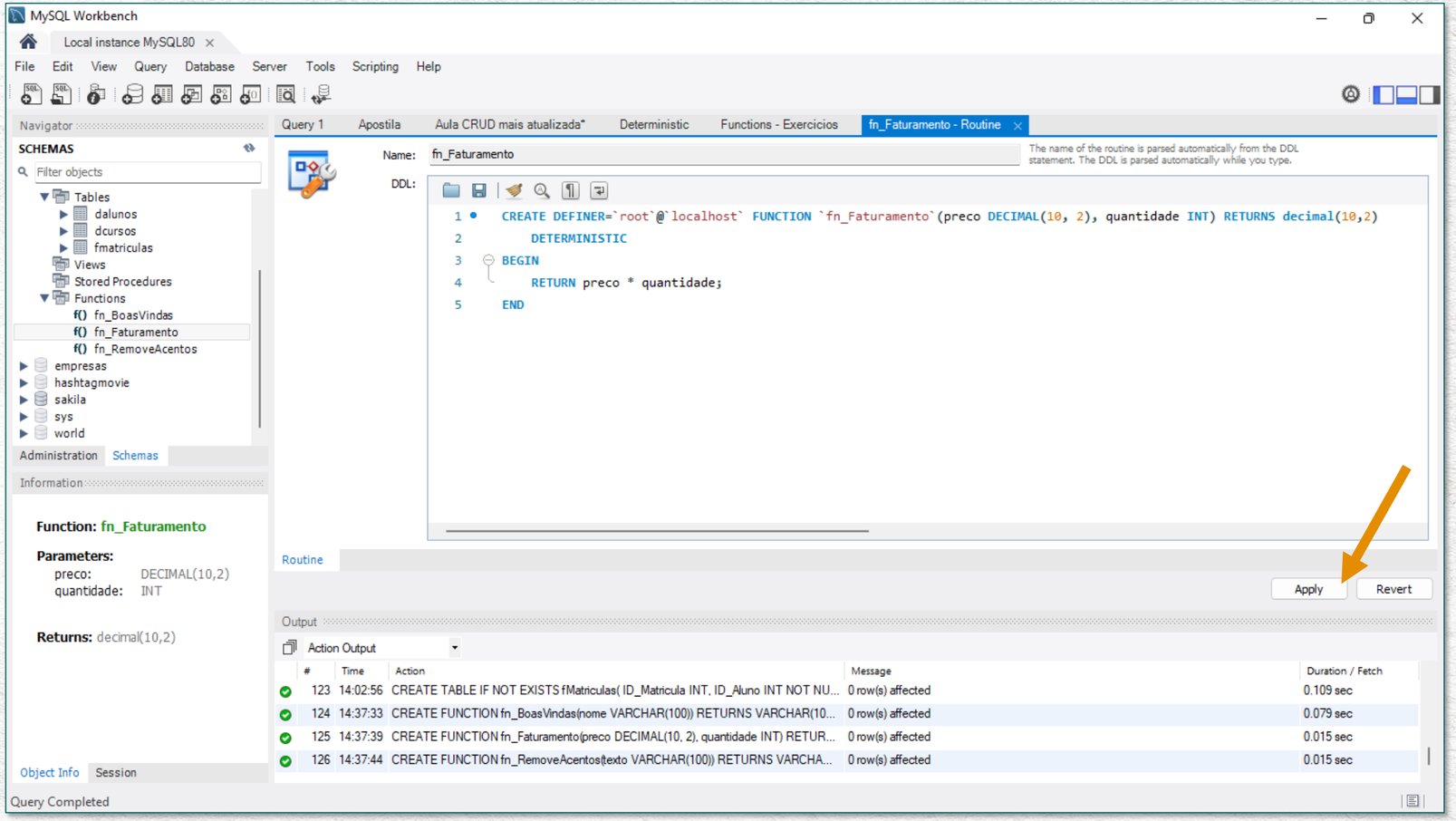
Para alterar uma Function, do lado esquerdo, na lista de Functions criadas, clique com o botão direito na que você deseja alterar e no menu que aparecer, utilize a opção Alter Function.



Alterando uma function

Feito isso, uma janela se abrirá onde você pode visualizar a Function criada, e consequentemente alterar essa function.

Após as alterações, basta clicar em Apply (conforme indicado pela seta ao lado).



Para que serve o DETERMINISTIC

Ao criar uma função armazenada, você deve declarar que ela é determinística ou que não modifica os dados do banco de dados. Caso contrário, pode ser inseguro para recuperação ou replicação de dados.

Caso a gente não declare a function como DETERMINISTIC, será retornado o erro 1418.

Para corrigir este problema, temos dois caminhos:

1) Seguir utilizando o DETERMINISTC como fizemos até agora.
2) Utilizar o código abaixo para dizer para o MySQL que as funções criadas são confiáveis.

```
SET GLOBAL log_bin_trust_function_creators = 1;
```

Stored Procedures: (Procedimentos Armazenados)

As Procedures podem ser vistas como programas/scripts. Ela permite alterar de forma global o banco de dados.

Com elas, conseguimos utilizar instruções INSERT, UPDATE, DELETE.

Procedures são utilizadas normalmente para juntar várias queries em um único bloco de código e executar alguma tarefa repetitiva ou complexa dentro do banco de dados.

Stored Procedures: (Procedimentos Armazenados)

A sintaxe de uma Procedure é mostrada abaixo:

```
DELIMITER $$

CREATE PROCEDURE nome_storedprocedure(param1 tipo1, param2 tipo2)
BEGIN
        DECLARE var1 tipo1;
        DECLARE var2 tipo2;

        instruções1;
        instruções2;
        instruções3;
END $$

DELIMITER ;
```

Stored Procedures: Exemplo 1

No primeiro exemplo, vamos ver como criar uma Procedure que seja capaz de receber dois parâmetros de entrada (Preço e ID do produto) e atualizar no banco de dados o Preço para o ID do produto informado.

A solução é mostrada ao lado.

Para chamar uma Procedure, usamos a instrução CALL.

```
1   DELIMITER $$
2   CREATE PROCEDURE sp_AtualizaPreco(NovoPreco DECIMAL(10, 2), ID INT)
3   BEGIN
4       UPDATE dCursos
5       SET Preco_Curso = NovoPreco
6       WHERE ID_Curso = ID;
7       SELECT 'Preço atualizado com sucesso!';
8   END $$
9   DELIMITER ;
10
11  CALL sp_AtualizaPreco(600, 1);
12
13  SELECT * FROM dCursos;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

ID_Curso	Nome_Curso	Preco_Curso
1	Excel	600.00
2	VBA	200.00
3	Power BI	150.00
NULL	NULL	NULL

Stored Procedures: Exemplo 2

No próximo exemplo, criamos uma Procedure que aplica um desconto a um determinado curso, de acordo com o seu ID.

A Procedure sp_AplicaDesconto recebe dois parâmetros (ID e Desconto) e através de uma sequência de códigos aplica o novo preço do curso dentro da tabela dCursos.

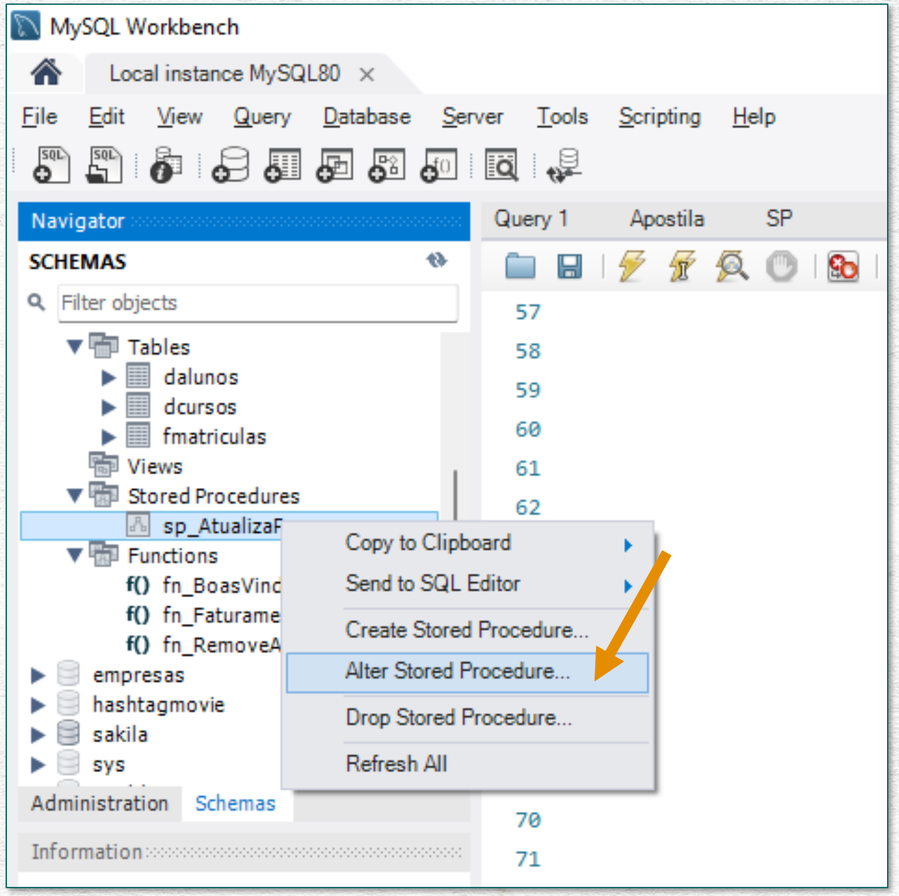
```
16  ####################
17  DELIMITER $$
18  CREATE PROCEDURE sp_AplicaDesconto(ID INT, Desconto DECIMAL(10, 2))
19  BEGIN
20      DECLARE varPrecoComDesconto DECIMAL(10, 2);
21      DECLARE varNomeCurso VARCHAR(100);
22
23      SET varPrecoComDesconto = (SELECT Preco_Curso FROM dCursos WHERE ID_Curso = ID) * (1 - Desconto);
24      SET varNomeCurso = (SELECT Nome_Curso FROM dCursos WHERE ID_Curso = ID);
25
26      UPDATE dCursos
27      SET Preco_Curso = varPrecoComDesconto
28      WHERE ID_Curso = ID;
29      SELECT 'Desconto aplicado com sucesso!';
30  END $$
31  DELIMITER ;
32
33  SELECT * FROM dCursos;
34
35  CALL sp_AplicaDesconto(1, 0.5);
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

ID_Curso	Nome_Curso	Preco_Curso
1	Excel	300.00
2	VBA	200.00
3	Power BI	150.00
NULL	NULL	NULL

Alterando uma Procedure

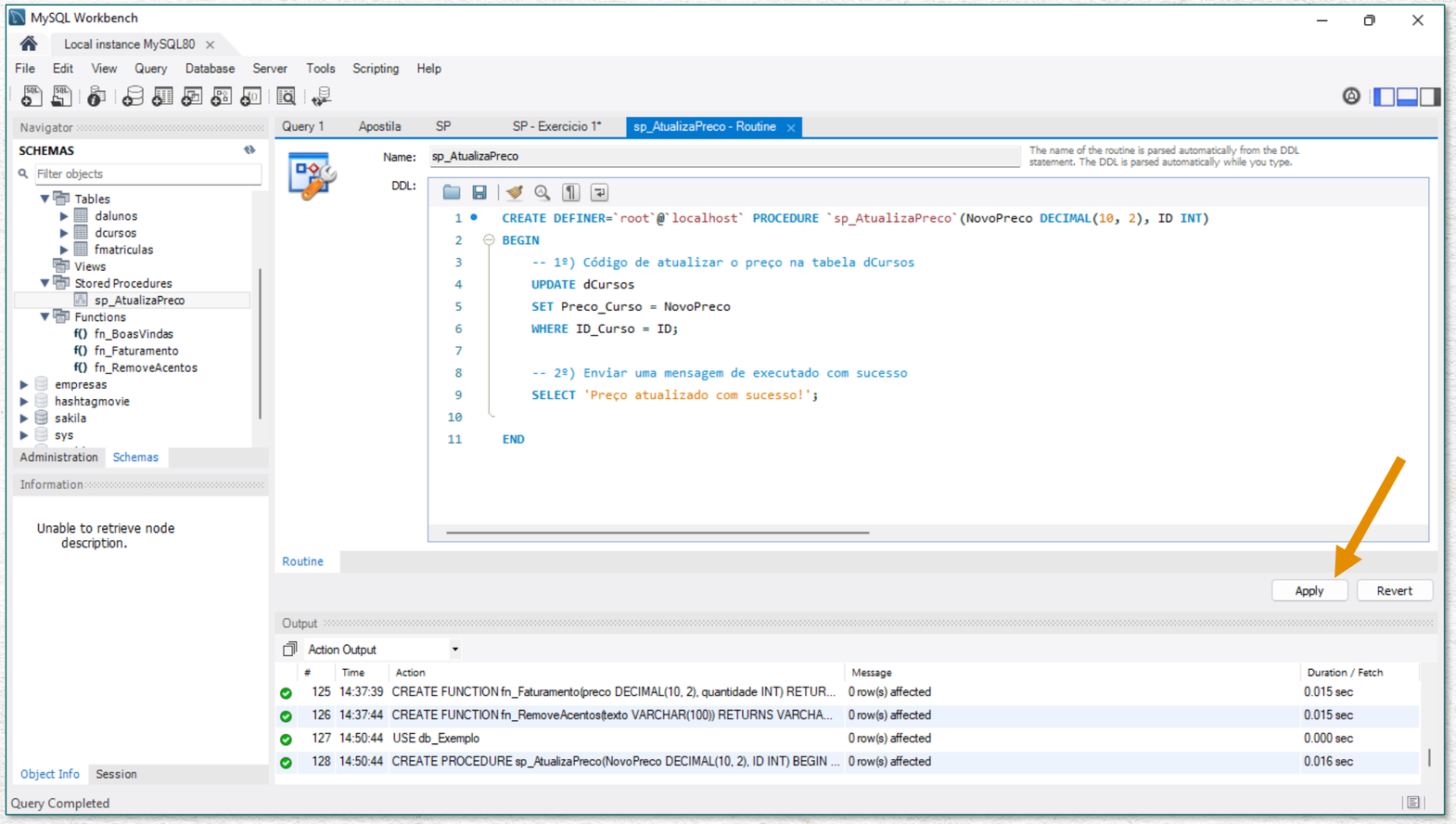
Para alterar uma Procedure, do lado esquerdo, na lista de Procedures criadas, clique com o botão direito na que você deseja alterar e no menu que aparecer, utilize a opção Alter Procedure.



Alterando uma Procedure

Feito isso, uma janela se abrirá onde você pode visualizar a Procedure criada, e consequentemente alterar essa procedure.

Após as alterações, basta clicar em Apply (conforme indicado pela seta ao lado).



Excluindo uma Procedure

Para excluir uma Procedure, basta usar o comando DROP PROCEDURE, conforme exemplo ao lado.

```
DROP PROCEDURE ConsultaID;
```

Functions vs Stored Procedures

Functions

- Uma function é usada para calcular um valor baseado em alguns inputs.
- Functions não permitem a utilização de instruções INSERT, UPDATE e DELETE para alteração de um banco de dados.
- Usamos functions em conjunto com as instruções SELECT, WHERE, HAVING mas não é possível fazer o mesmo com procedures.
- Uma function pode ser executada dentro de uma procedure, mas o contrário não é válido.

Procedures

- Uma procedure pode retornar um ou mais valores, ou não retornar nenhum valor.
- Procedures podem ser vistas como programas/scripts. Uma procedure permite alterar o estado global da base de dados (por exemplo, a utilização das instruções INSERT, UPDATE, DELETE.
- Procedures são utilizadas normalmente para juntar várias queries em um único bloco de comando.

EXERCÍCIOS

Para resolver os exercícios a seguir, utilize as tabelas do DB_Exemplo que criamos no módulo de CRUD.

1

Questão 1

O gestor deseja saber o preço de um curso pelo nome para facilitar consultas rápidas na interface do sistema.

Crie uma **function** que receba o nome do curso a ser consultado e retorne seu preço.

(Utilize a tabela dCursos no código da function)

2

Questão 2

Um relatório precisa exibir os nomes dos alunos em letras maiúsculas para padronização.

Crie uma **function** que receba o nome do aluno como parâmetro e faça essa padronização.

(Utilize a tabela dAlunos para testar se a function está funcionando)

EXERCÍCIOS

3

Questão 3

Antes de inserir matrículas, o sistema precisa validar se o curso existe no banco de dados.

Crie uma **function** que receba o nome do curso e verifique sua existência, retornando 1 para quando o curso existir na tabela e 0 para quando não existir na tabela.

(Utilize a tabela dCursos no código da function)

4

Questão 4

O gestor precisa calcular o valor do curso com um desconto aplicado para campanhas promocionais.

Crie uma **function** que receba o ID do Curso e o percentual do desconto, calcule e retorne o preço com desconto.

(Utilize a tabela dCursos no código da function)

EXERCÍCIOS

5

Questão 5

O coordenador quer saber quantos alunos estão matriculados em cada curso para ajustar a lotação das turmas.

Crie uma **function** que receba o ID do curso e retorne a contagem da quantidade de matrículas.

(Utilize a tabela fMatriculas no código da function e a tabela dCursos para testar se a function está funcionando)

6

Questão 6

A administração decidiu aumentar o preço de todos os cursos por um fator percentual.

Crie uma **procedure** que receba o percentual do aumento e faça a atualização dos preços na tabela.

(Utilize a tabela dCursos no código da procedure)

EXERCÍCIOS

7

Questão 7

A secretaria precisa corrigir e-mails de alunos registrados de forma incorreta no sistema.

Crie uma **procedure** que receba os parâmetros necessários (ID do aluno e e-mail correto) e proceda à alteração na tabela.

(Utilize a tabela dAlunos no código da procedure)

8

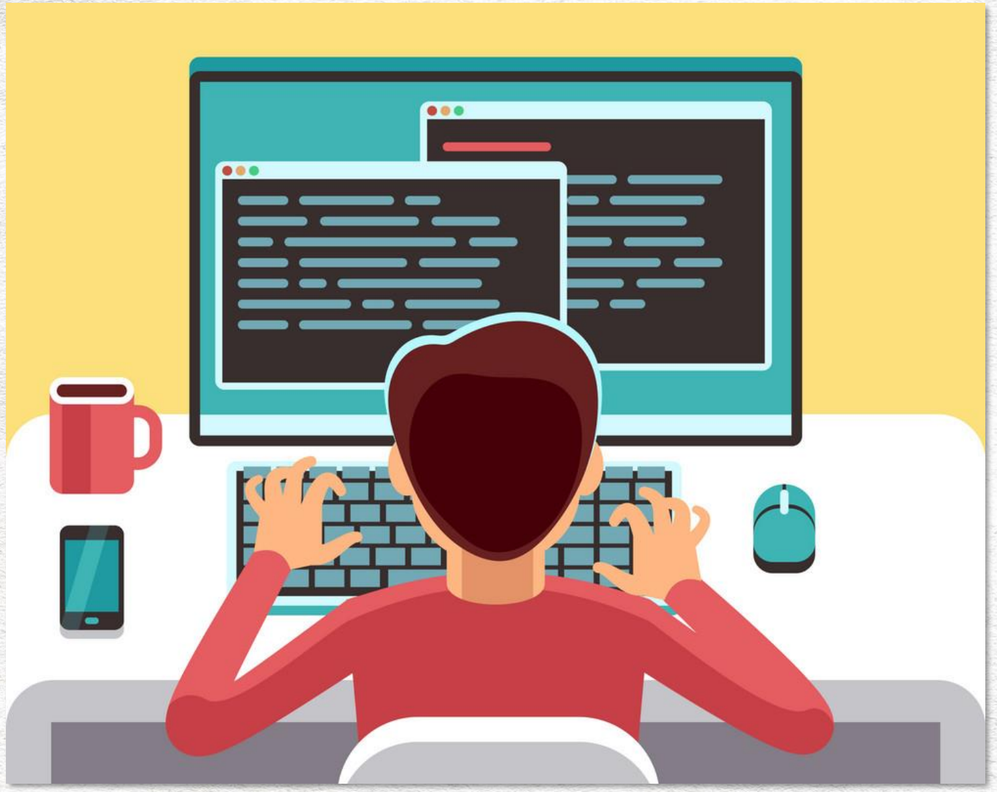
Questão 8

Quando um aluno solicitar a exclusão completa de seus dados, todos os seus dados pessoais, incluindo as matrículas associadas a esse aluno, devem ser excluídos do banco de dados.

Crie uma **procedure** que receba o ID do aluno e exclua todos os seus dados pessoais e suas matrículas.

(Utilize as tabelas fMatriculas e dAlunos no código da procedure)

EXERCÍCIOS



9

Questão 9

A secretaria precisa cadastrar novos cursos no sistema.

Crie uma **procedure** que receba os dados do novo curso (ID, nome e preço) e os insira na tabela.

(Utilize a tabela dCursos no código da procedure)

10

Questão 10

Um aluno solicitou sua transferência de curso e a secretaria precisa atualizar as informações.

Crie uma **procedure** que receba os parâmetros necessários (ID do aluno, ID do curso atual, ID do novo curso para o qual o aluno será transferido e data do novo cadastro) e efetue a alteração na tabela.

(Utilize a tabela fMatriculas no código da procedure)

GABARITO EXPLICATIVO

1

Questão 1

O gestor deseja saber o preço de um curso pelo nome para facilitar consultas rápidas na interface do sistema.

Crie uma **function** que receba o nome do curso a ser consultado e retorne seu preço.

(Utilize a tabela dCursos no código da function)

Explicação e Resultado

```
DELIMITER $$

CREATE FUNCTION fn_CursoPreco(nomeCurso VARCHAR(100))
RETURNS DECIMAL(10, 2) DETERMINISTIC
BEGIN
    DECLARE preco DECIMAL(10, 2);
    SET preco = (SELECT preco_curso FROM dCursos WHERE nome_curso = nomeCurso);
    RETURN preco;
END $$

DELIMITER ;
```

Para **criar uma function que receba o nome do curso e retorne seu preço**, seguimos os passos a seguir:

Alteração do delimitador padrão:

Utilizamos o comando **DELIMITER** para alterar o delimitador padrão do MySQL (que é o ponto e vírgula) para **$$**, permitindo a criação de uma função com múltiplas linhas.

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

1

Questão 1

O gestor deseja saber o preço de um curso pelo nome para facilitar consultas rápidas na interface do sistema.

Crie uma **function** que receba o nome do curso a ser consultado e retorne seu preço.

(Utilize a tabela dCursos no código da function)

Explicação e Resultado

Definição da função:

Iniciamos o código com **CREATE FUNCTION** seguido pelo nome da função, neste caso, **fn_CursoPreco**, e o parâmetro de entrada (**nomeCurso VARCHAR(100)**).

Especificamos o tipo de dado que a função retornará com o comando **RETURNS DECIMAL(10, 2)**, pois o preço é um valor numérico com duas casas decimais.

Adicionamos o modificador **DETERMINISTIC**, obrigatório para funções que sempre retornam o mesmo resultado para a mesma entrada.

Instruções no bloco:

Utilizamos o comando **BEGIN** para iniciar o bloco de instruções da função.

Declaramos uma **variável** chamada **preco** com o tipo **DECIMAL(10, 2)** para armazenar o resultado da consulta.

Atribuímos à variável **preco** o preço do curso retornado pela consulta **SELECT preco_curso FROM dCursos WHERE nome_curso = nomeCurso** utilizando o comando **SET**.

Retornamos o valor da variável **preco** com o comando **RETURN**.

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

1

Questão 1

O gestor deseja saber o preço de um curso pelo nome para facilitar consultas rápidas na interface do sistema.

Crie uma **function** que receba o nome do curso a ser consultado e retorne seu preço.

(Utilize a tabela dCursos no código da function)

Explicação e Resultado

Encerramento do bloco e restauração do delimitador:

Finalizamos o bloco de instruções com o comando **END $$**.

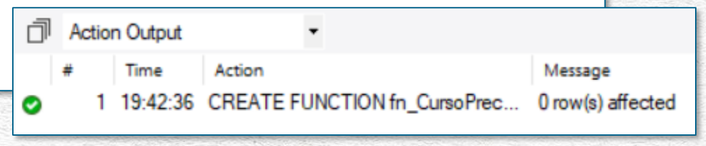
Alteramos o delimitador do MySQL de volta para o padrão (;) utilizando **DELIMITER ;**.

Executamos o código para **criar** a função no banco de dados:

```
DELIMITER $$

CREATE FUNCTION fn_CursoPreco(nomeCurso VARCHAR(100))
RETURNS DECIMAL(10, 2) DETERMINISTIC
BEGIN
    DECLARE preco DECIMAL(10, 2);
    SET preco = (SELECT preco_curso FROM dCursos WHERE nome_curso = nomeCurso);
    RETURN preco;
END $$

DELIMITER ;
```



CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

1

Questão 1

O gestor deseja saber o preço de um curso pelo nome para facilitar consultas rápidas na interface do sistema.

Crie uma **function** que receba o nome do curso a ser consultado e retorne seu preço.

(Utilize a tabela dCursos no código da function)

Explicação e Resultado

Para **testar** a função, podemos executar uma consulta chamando a função **fn_CursoPreco** e passando ao parâmetro de entrada o nome de um curso cadastrado na tabela dCursos (no exemplo abaixo, 'VBA'):

```
SELECT fn_CursoPreco('VBA');
```

O **resultado** da consulta será o preço do curso correspondente:

	fn_CursoPreco('VBA')
▶	200.00

GABARITO EXPLICATIVO

2

Questão 2

Um relatório precisa exibir os nomes dos alunos em letras maiúsculas para padronização.

Crie uma **function** que receba o nome do aluno como parâmetro e faça essa padronização.

(Utilize a tabela dAlunos para testar se a function está funcionando)

Explicação e Resultado

```
DELIMITER $$

CREATE FUNCTION fn_AlunoMaiusculo(nomeAluno VARCHAR(100))
RETURNS VARCHAR(100) DETERMINISTIC
BEGIN
    RETURN UCASE(nomeAluno);
END $$

DELIMITER ;
```

Para **criar uma function que receba o nome do aluno e retorne esse nome em letras maiúsculas**, seguimos os passos a seguir:

Alteração do delimitador padrão:

Utilizamos o comando **DELIMITER** para mudar o delimitador padrão do MySQL (ponto e vírgula) para **$$,** permitindo a criação de uma function com múltiplas linhas.

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

2

Questão 2

Um relatório precisa exibir os nomes dos alunos em letras maiúsculas para padronização.

Crie uma **function** que receba o nome do aluno como parâmetro e faça essa padronização.

(Utilize a tabela dAlunos para testar se a function está funcionando)

Explicação e Resultado

Definição da função:

Iniciamos o código com **CREATE FUNCTION** seguido pelo nome da função, neste caso, **fn_AlunoMaiusculo**, e o parâmetro de entrada (**nomeAluno VARCHAR(100)**).

Especificamos o tipo de dado que a função retornará com o comando **RETURNS VARCHAR(100)**.

Adicionamos o modificador **DETERMINISTIC**, obrigatório para funções que sempre retornam o mesmo resultado para a mesma entrada.

Instruções no bloco:

Utilizamos o comando **BEGIN** para iniciar o bloco de instruções da função.

Retornamos o nome do aluno em letras maiúsculas usando a função **UCASE(nomeAluno)** com o comando **RETURN**.

Encerramento do bloco e restauração do delimitador:

Finalizamos o bloco de instruções com o comando **END $$**.

Alteramos o delimitador do MySQL de volta para o padrão (;) utilizando **DELIMITER ;**.

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

2

Questão 2

Um relatório precisa exibir os nomes dos alunos em letras maiúsculas para padronização.

Crie uma **function** que receba o nome do aluno como parâmetro e faça essa padronização.

(Utilize a tabela dAlunos para testar se a function está funcionando)

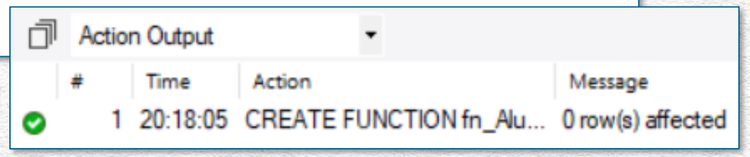
Explicação e Resultado

Executamos o código para **criar** a função no banco de dados:

```
DELIMITER $$

CREATE FUNCTION fn_AlunoMaiusculo(nomeAluno VARCHAR(100))
RETURNS VARCHAR(100) DETERMINISTIC
BEGIN
    RETURN UCASE(nomeAluno);
END $$

DELIMITER ;
```



CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

2

Questão 2

Um relatório precisa exibir os nomes dos alunos em letras maiúsculas para padronização.

Crie uma **function** que receba o nome do aluno como parâmetro e faça essa padronização.

(Utilize a tabela dAlunos para testar se a function está funcionando)

Explicação e Resultado

Para **testar** a function, podemos executar uma consulta na tabela **dAlunos** para verificar como os nomes são convertidos para letras maiúsculas:

```
SELECT
    Nome_Aluno,
    fn_AlunoMaiusculo(Nome_Aluno) AS 'Nome_Aluno_Maiúsculo'
FROM dAlunos;
```

No exemplo acima, **consultamos** os **nomes** dos alunos na tabela **dAlunos** e **aplicamos** a função **fn_AlunoMaiusculo** em cada um deles. O **resultado** da consulta será uma **tabela** com o **nome original** e o **nome convertido em letras maiúsculas**:

Nome_Aluno	Nome_Aluno_Maiúsculo
Ana	ANA
Bruno	BRUNO
carla	CARLA
Diego	DIEGO

GABARITO EXPLICATIVO

3

Questão 3

Antes de inserir matrículas, o sistema precisa validar se o curso existe no banco de dados.

Crie uma **function** que receba o nome do curso e verifique sua existência, retornando 1 para quando o curso existir na tabela e 0 para quando não existir na tabela.

(Utilize a tabela dCursos no código da function)

Explicação e Resultado

```
DELIMITER $$

CREATE FUNCTION fn_CursoExistente(nomeCurso VARCHAR(100))
RETURNS INT DETERMINISTIC
BEGIN
    DECLARE existe INT;
    SET existe = (SELECT COUNT(*) FROM dCursos WHERE Nome_Curso = nomeCurso);
    RETURN existe;
END $$

DELIMITER ;
```

Para **criar uma function que verifica se um curso existe no banco de dados**, seguimos os passos a seguir:

Alteração do delimitador padrão:

Utilizamos o comando **DELIMITER** para alterar o delimitador do MySQL para **$$**, permitindo que a função seja escrita com múltiplas linhas sem conflito com os delimitadores padrão.

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

3

Questão 3

Antes de inserir matrículas, o sistema precisa validar se o curso existe no banco de dados.

Crie uma **function** que receba o nome do curso e verifique sua existência, retornando 1 para quando o curso existir na tabela e 0 para quando não existir na tabela.

(Utilize a tabela dCursos no código da function)

Explicação e Resultado

Definição da função:

Iniciamos o código com **CREATE FUNCTION**, seguido pelo nome da função, **fn_CursoExistente**, e o parâmetro de entrada (**nomeCurso VARCHAR(100)**).

Como **o objetivo é retornar 1 ou 0** (indicando a existência ou não do curso), definimos o tipo de retorno como INT com o comando **RETURNS INT**.

Incluímos o modificador **DETERMINISTIC**, pois a função sempre retornará o mesmo valor para uma mesma entrada.

Instruções no bloco:

Iniciamos o bloco de instruções com o comando **BEGIN**.

Declaramos uma variável chamada **existe** do tipo **INT**, que armazenará o resultado da validação.

Utilizamos o comando **SET** para atribuir à variável **existe** o valor retornado por uma consulta que verifica a existência do curso na tabela **dCursos** através do comando **SELECT COUNT(*) FROM dCursos WHERE Nome_Curso = nomeCurso**. O COUNT(*) contará a existência de uma linha e retornará 1 se o curso existir, ou não encontrará nenhuma linha para contar, retornando 0, se o curso não existir.

Retornamos o valor da variável **existe** com o comando **RETURN**.

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

3

Questão 3

Antes de inserir matrículas, o sistema precisa validar se o curso existe no banco de dados.

Crie uma **function** que receba o nome do curso e verifique sua existência, retornando 1 para quando o curso existir na tabela e 0 para quando não existir na tabela.

(Utilize a tabela dCursos no código da function)

Explicação e Resultado

Encerramento do bloco e restauração do delimitador:

Finalizamos o bloco de instruções com **END $$.**

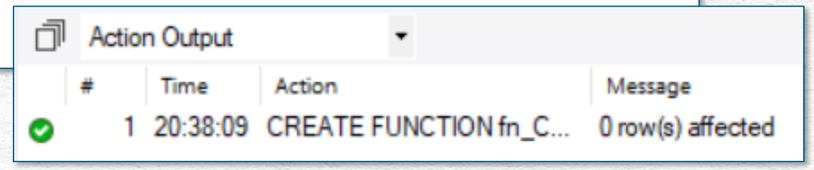
Alteramos o delimitador do MySQL de volta para o padrão (;) com o comando **DELIMITER ;.**

Executamos o código para **criar** a função no banco de dados:

```
DELIMITER $$

CREATE FUNCTION fn_CursoExistente(nomeCurso VARCHAR(100))
RETURNS INT DETERMINISTIC
BEGIN
    DECLARE existe INT;
    SET existe = (SELECT COUNT(*) FROM dCursos WHERE Nome_Curso = nomeCurso);
    RETURN existe;
END $$

DELIMITER ;
```



CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

3

Questão 3

Antes de inserir matrículas, o sistema precisa validar se o curso existe no banco de dados.

Crie uma **function** que receba o nome do curso e verifique sua existência, retornando 1 para quando o curso existir na tabela e 0 para quando não existir na tabela.

(Utilize a tabela dCursos no código da function)

Explicação e Resultado

Para **testar** a função, podemos executar consultas para verificar se cursos específicos existem na tabela dCursos. Por exemplo:

Podemos **procurar** pelo curso de 'VBA'...

```
SELECT fn_CursoExistente('VBA');
```

... E **retornaremos o valor 1**, pois o curso de **VBA** está cadastrado (**existe**) na tabela dCursos:

	fn_CursoExistente('VBA')
▶	1

Mas se procurarmos pelo curso 'SQL'...

```
SELECT fn_CursoExistente('SQL');
```

... **Retornaremos o valor 0**, pois o curso de **SQL** não está cadastrado (**não existe**) na tabela dCursos:

	fn_CursoExistente('SQL')
▶	0

GABARITO EXPLICATIVO

4

Questão 4

O gestor precisa calcular o valor do curso com um desconto aplicado para campanhas promocionais.

Crie uma **function** que receba o ID do Curso e o percentual do desconto, calcule e retorne o preço com desconto.

(Utilize a tabela dCursos no código da function)

Explicação e Resultado

```
DELIMITER $$

CREATE FUNCTION fn_PrecoComDesconto(idCurso INT, desconto DECIMAL(5, 2))
RETURNS DECIMAL(10, 2) DETERMINISTIC
BEGIN
    DECLARE preco DECIMAL(10, 2);
    SET preco = (SELECT preco_curso FROM dCursos WHERE id_curso = idCurso);
    RETURN preco - (preco * desconto);
END $$

DELIMITER ;
```

Para **criar uma function que calcula o valor de um curso com desconto**, seguimos os passos a seguir:

Alteração do delimitador padrão:

Utilizamos o comando **DELIMITER** para alterar o delimitador do MySQL para **$$**, permitindo a escrita da função em múltiplas linhas sem conflitos com o delimitador padrão (;).

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

4

Questão 4

O gestor precisa calcular o valor do curso com um desconto aplicado para campanhas promocionais.

Crie uma **function** que receba o ID do Curso e o percentual do desconto, calcule e retorne o preço com desconto.

(Utilize a tabela dCursos no código da function)

Explicação e Resultado

Definição da função:

Iniciamos o código com **CREATE FUNCTION**, seguido pelo nome da função, **fn_PrecoComDesconto**, e parâmetros de entrada (**idCurso INT, desconto DECIMAL(5, 2)**).

O parâmetro **idCurso** é usado para identificar o curso, enquanto **desconto** representa o percentual do desconto (por exemplo, 0.1 para 10%).

No **RETURNS**, definimos o tipo de retorno como **DECIMAL(10, 2)** para retornar o preço com duas casas decimais.

Incluímos o modificador **DETERMINISTIC**, pois a função retornará o mesmo resultado para os mesmos valores de entrada.

Instruções no bloco:

Iniciamos o bloco de instruções com o comando **BEGIN**.

Declaramos uma variável chamada **preco** do tipo **DECIMAL(10, 2)**, que armazenará o valor do curso.

Utilizamos o comando **SET** para atribuir à variável **preco** o valor da coluna **preco_curso** da tabela **dCursos**, onde o valor da coluna **id_curso** corresponde ao parâmetro **idCurso**.

Calculamos o valor com desconto como **preco - (preco * desconto)** e o retornamos com o comando **RETURN**.

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

4

Questão 4

O gestor precisa calcular o valor do curso com um desconto aplicado para campanhas promocionais.

Crie uma **function** que receba o ID do Curso e o percentual do desconto, calcule e retorne o preço com desconto.

(Utilize a tabela dCursos no código da function)

Explicação e Resultado

Encerramento do bloco e restauração do delimitador:

Finalizamos o bloco de instruções com **END $$**.

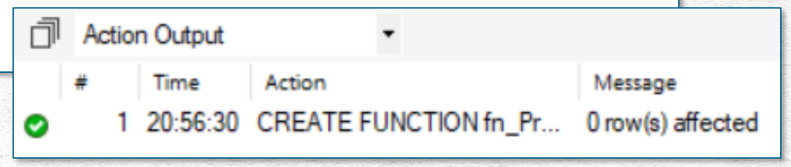
Alteramos o delimitador do MySQL de volta para o padrão (;) com o comando **DELIMITER ;**.

Executamos o código para **criar** a função no banco de dados:

```
DELIMITER $$

CREATE FUNCTION fn_PrecoComDesconto(idCurso INT, desconto DECIMAL(5, 2))
RETURNS DECIMAL(10, 2) DETERMINISTIC
BEGIN
    DECLARE preco DECIMAL(10, 2);
    SET preco = (SELECT preco_curso FROM dCursos WHERE id_curso = idCurso);
    RETURN preco - (preco * desconto);
END $$

DELIMITER ;
```



CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

4

Questão 4

O gestor precisa calcular o valor do curso com um desconto aplicado para campanhas promocionais.

Crie uma **function** que receba o ID do Curso e o percentual do desconto, calcule e retorne o preço com desconto.

(Utilize a tabela dCursos no código da function)

Explicação e Resultado

Para **testar** a função, podemos executar uma consulta, passando como parâmetros o ID de um curso e o percentual de desconto. Por exemplo:

```
SELECT fn_PrecoComDesconto(3, 0.1);
```

O resultado retornará o preço com desconto aplicado:

	fn_PrecoComDesconto(3, 0.1)
▶	135.00

Obs.: Se consultarmos a tabela dCursos, veremos que o preço do curso de id_curso = 3 (Power BI) é de R$ 150,00.

Aplicados os 10% de desconto (0.1), o preço com desconto retornado foi de R$ 135,00.

Portanto, nossa function está retornando resultados corretamente.

GABARITO EXPLICATIVO

5

Questão 5

O coordenador quer saber quantos alunos estão matriculados em cada curso para ajustar a lotação das turmas.

Crie uma **function** que receba o ID do curso e retorne a contagem da quantidade de matrículas.

(Utilize a tabela fMatriculas no código da function e a tabela dCursos para testar se a function está funcionando)

Explicação e Resultado

```
DELIMITER $$

CREATE FUNCTION fn_QuantidadeMatriculasCurso(idCurso INT)
RETURNS INT DETERMINISTIC
BEGIN
    DECLARE total INT;
    SET total = (SELECT COUNT(*) FROM fMatriculas WHERE id_curso = idCurso);
    RETURN total;
END $$

DELIMITER ;
```

Para **criar uma função que calcula a quantidade de alunos matriculados em cada curso**, realizamos as etapas a seguir:

Alteração do delimitador padrão:

Utilizamos o comando **DELIMITER** para alterar o delimitador padrão do MySQL para **$$**, permitindo a escrita de blocos de instrução sem conflitos com o ponto e vírgula (;).

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

5

Questão 5

O coordenador quer saber quantos alunos estão matriculados em cada curso para ajustar a lotação das turmas.

Crie uma **function** que receba o ID do curso e retorne a contagem da quantidade de matrículas.

(Utilize a tabela fMatriculas no código da function e a tabela dCursos para testar se a function está funcionando)

Explicação e Resultado

Definição da função:

Iniciamos com o comando **CREATE FUNCTION**, seguido pelo nome da função **fn_QuantidadeMatriculasCurso** e pelo parâmetro de entrada (**idCurso INT**). O parâmetro **idCurso** é usado para especificar o curso cuja quantidade de matrículas será verificada.

No **RETURNS**, definimos o tipo de retorno como **INT**, pois o resultado será um número inteiro representando a contagem de matrículas.

Incluímos o modificador **DETERMINISTIC**, que indica que a função sempre retorna o mesmo valor para os mesmos parâmetros de entrada.

Instruções no bloco:

Iniciamos o bloco de instruções com o comando **BEGIN**.

Declaramos uma variável chamada **total** do tipo **INT**, que armazenará a contagem de matrículas.

Atribuímos à variável **total** o resultado da consulta SQL que utiliza a função de agregação **COUNT(*)** na tabela **fMatriculas**, filtrando os registros com base no **id_curso** informado no parâmetro de entrada **idCurso**.

Utilizamos o comando **RETURN** para retornar o valor armazenado na variável **total**.

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

5

Questão 5

O coordenador quer saber quantos alunos estão matriculados em cada curso para ajustar a lotação das turmas.

Crie uma **function** que receba o ID do curso e retorne a contagem da quantidade de matrículas.

(Utilize a tabela fMatriculas no código da function e a tabela dCursos para testar se a function está funcionando)

Explicação e Resultado

Encerramento do bloco e restauração do delimitador:

Finalizamos o bloco de instruções com o comando **END $$**.

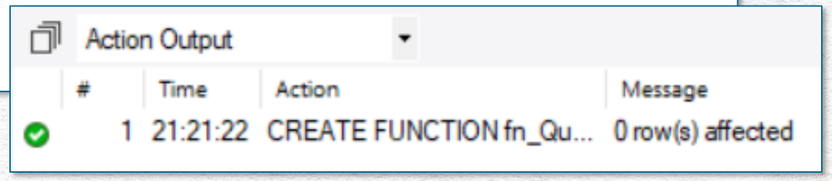
Alteramos o delimitador do MySQL de volta para o padrão (;) com o comando **DELIMITER ;**.

Executamos o código para **criar** a função no banco de dados:

```
DELIMITER $$

CREATE FUNCTION fn_QuantidadeMatriculasCurso(idCurso INT)
RETURNS INT DETERMINISTIC
BEGIN
    DECLARE total INT;
    SET total = (SELECT COUNT(*) FROM fMatriculas WHERE id_curso = idCurso);
    RETURN total;
END $$

DELIMITER ;
```



CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

5

Questão 5

O coordenador quer saber quantos alunos estão matriculados em cada curso para ajustar a lotação das turmas.

Crie uma **function** que receba o ID do curso e retorne a contagem da quantidade de matrículas.

(Utilize a tabela fMatriculas no código da function e a tabela dCursos para testar se a function está funcionando)

Explicação e Resultado

Para **testar** a função, podemos executar uma consulta que utiliza a tabela **dCursos** e a função **fn_QuantidadeMatriculasCurso**, passando o **ID** do curso para verificar a quantidade de alunos matriculados:

```
SELECT
    *,
    fn_QuantidadeMatriculasCurso(id_Curso) AS 'Qtd_Alunos_Matriculados'
FROM dCursos;
```

O **resultado** exibirá a lista de cursos, junto com a quantidade de alunos matriculados em cada um:

	ID_Curso	Nome_Curso	Preco_Curso	Qtd_Alunos_Matriculados
▶	1	Excel	100.00	3
	2	VBA	200.00	1
	3	Power BI	150.00	2

GABARITO EXPLICATIVO

6

Questão 6

A administração decidiu aumentar o preço de todos os cursos por um fator percentual.

Crie uma **procedure** que receba o percentual do aumento e faça a atualização dos preços na tabela.

(Utilize a tabela dCursos no código da procedure)

Explicação e Resultado

```
DELIMITER $$

CREATE PROCEDURE sp_IncrementaPreco(percentual DECIMAL(5, 2))
BEGIN
    UPDATE dCursos
    SET preco_curso = preco_curso + (preco_curso * percentual);

    SELECT 'Preços atualizados com sucesso!';
END $$

DELIMITER ;
```

Para **criar uma procedure que atualiza o preço de todos os cursos aplicando um percentual de aumento**, seguimos os passos descritos a seguir:

Alteração do delimitador padrão:

Utilizamos o comando **DELIMITER** para alterar o delimitador padrão do MySQL para **$$**. Isso permite a inclusão de blocos de instrução que contenham múltiplos comandos separados por ponto e vírgula (;).

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

6

Questão 6

A administração decidiu aumentar o preço de todos os cursos por um fator percentual.

Crie uma **procedure** que receba o percentual do aumento e faça a atualização dos preços na tabela.

(Utilize a tabela dCursos no código da procedure)

Explicação e Resultado

Definição da procedure:

Iniciamos com o comando **CREATE PROCEDURE**, seguido pelo nome da procedure **sp_IncrementaPreco** e pelo parâmetro de entrada (**percentual DECIMAL(5, 2)**). O parâmetro **percentual** é usado para definir o percentual de aumento a ser aplicado nos preços dos cursos.

Instruções no bloco:

Iniciamos o bloco de instruções com o comando **BEGIN**.

Incluímos o comando **UPDATE** para **atualizar os preços** dos cursos na tabela **dCursos**. A lógica de atualização consiste em **somar** o valor atual (**preco_curso**) à **porcentagem** correspondente, calculada como **preco_curso * percentual**.

Adicionamos um comando **SELECT** que **retorna uma mensagem** confirmando que os preços foram atualizados com sucesso. Isso é útil para o usuário verificar se a operação foi concluída corretamente.

Encerramento do bloco e restauração do delimitador:

Finalizamos o bloco de instruções com o comando **END $$**.

Alteramos o delimitador do MySQL de volta para o padrão (;) com o comando **DELIMITER ;**.

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

6

Questão 6

A administração decidiu aumentar o preço de todos os cursos por um fator percentual.

Crie uma **procedure** que receba o percentual do aumento e faça a atualização dos preços na tabela.

(Utilize a tabela dCursos no código da procedure)

Explicação e Resultado

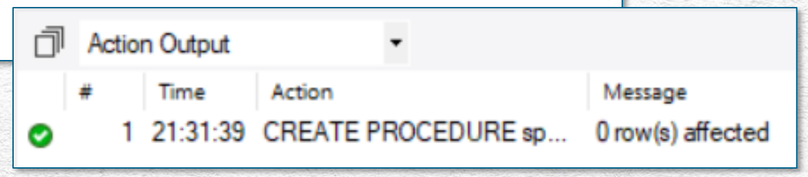
Executamos o código para **criar** a procedure no banco de dados:

```
DELIMITER $$

CREATE PROCEDURE sp_IncrementaPreco(percentual DECIMAL(5, 2))
BEGIN
    UPDATE dCursos
    SET preco_curso = preco_curso + (preco_curso * percentual);

    SELECT 'Preços atualizados com sucesso!';
END $$

DELIMITER ;
```



CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

6

Questão 6

A administração decidiu aumentar o preço de todos os cursos por um fator percentual.

Crie uma **procedure** que receba o percentual do aumento e faça a atualização dos preços na tabela.

(Utilize a tabela dCursos no código da procedure)

Explicação e Resultado

Para **testar** a procedure, podemos fazer o seguinte:

Realizamos uma consulta à tabela dCursos para **verificar os preços dos cursos antes de aplicar a alteração**:

```
SELECT * FROM dCursos;
```

ID_Curso	Nome_Curso	Preco_Curso
1	Excel	100.00
2	VBA	200.00
3	Power BI	150.00
NULL	NULL	NULL

Chamamos a procedure sp_IncrementaPreco, passando o percentual de aumento como argumento (por exemplo, **0.1,** que corresponde a 10%):

```
CALL sp_IncrementaPreco(0.1);
```

Preços atualizados com sucesso!
Preços atualizados com sucesso!

Realizamos novamente uma consulta à tabela dCursos para **verificar os preços atualizados**, e o resultado mostrará que **os preços foram ajustados** de acordo com o percentual informado:

```
SELECT * FROM dCursos;
```

ID_Curso	Nome_Curso	Preco_Curso
1	Excel	110.00
2	VBA	220.00
3	Power BI	165.00
NULL	NULL	NULL

GABARITO EXPLICATIVO

7

Questão 7

A secretaria precisa corrigir e-mails de alunos registrados de forma incorreta no sistema.

Crie uma **procedure** que receba os parâmetros necessários (ID do aluno e e-mail correto) e proceda à alteração na tabela.

(Utilize a tabela dAlunos no código da procedure)

Explicação e Resultado

```
DELIMITER $$

CREATE PROCEDURE sp_AtualizaEmail(alunoID INT, novoEmail VARCHAR(100))
BEGIN
    UPDATE dAlunos
    SET email = novoEmail
    WHERE id_aluno = alunoID;

    SELECT 'E-mail alterado com sucesso!';
END $$

DELIMITER ;
```

Para **criar uma procedure que permite a atualização do e-mail de um aluno registrado de forma incorreta no sistema**, seguimos os passos a seguir:

Alteração do delimitador padrão:

Alteramos o delimitador padrão do MySQL para **$$** com o comando **DELIMITER**. Isso possibilita que a procedure contenha múltiplos comandos separados por ponto e vírgula (;).

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

7

Questão 7

A secretaria precisa corrigir e-mails de alunos registrados de forma incorreta no sistema.

Crie uma **procedure** que receba os parâmetros necessários (ID do aluno e e-mail correto) e proceda à alteração na tabela.

(Utilize a tabela dAlunos no código da procedure)

Explicação e Resultado

Definição da procedure:

Utilizamos o comando **CREATE PROCEDURE**, seguido do nome da procedure **sp_AtualizaEmail** e de dois parâmetros de entrada:

alunoID INT: identifica o aluno cujo e-mail será corrigido.

novoEmail VARCHAR(100): contém o e-mail atualizado e correto que será registrado.

Instruções no bloco:

Abrimos o bloco de instruções com o comando **BEGIN**.

Incluímos um comando **UPDATE** para realizar a alteração do **e-mail** na tabela **dAlunos**. A alteração ocorre na linha correspondente ao **id_aluno** informado no parâmetro **alunoID**.

Adicionamos um comando **SELECT** que retorna uma **mensagem** confirmando que o e-mail foi atualizado com sucesso, permitindo uma validação visual por parte do usuário.

Encerramento do bloco e restauração do delimitador:

Encerramos o bloco de instruções com **END $$**.

Restauramos o delimitador padrão para o ponto e vírgula (;) com o comando **DELIMITER ;**.

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

7

Questão 7

A secretaria precisa corrigir e-mails de alunos registrados de forma incorreta no sistema.

Crie uma **procedure** que receba os parâmetros necessários (ID do aluno e e-mail correto) e proceda à alteração na tabela.

(Utilize a tabela dAlunos no código da procedure)

Explicação e Resultado

Após **executar** o código, a procedure estará **criada** no banco de dados:

```
DELIMITER $$

CREATE PROCEDURE sp_AtualizaEmail(alunoID INT, novoEmail VARCHAR(100))
BEGIN
    UPDATE dAlunos
    SET email = novoEmail
    WHERE id_aluno = alunoID;

    SELECT 'E-mail alterado com sucesso!';
END $$

DELIMITER ;
```

Action Output

#	Time	Action	Message
1	22:52:29	CREATE PROCEDURE sp_...	0 row(s) affected

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

7

Questão 7

A secretaria precisa corrigir e-mails de alunos registrados de forma incorreta no sistema.

Crie uma **procedure** que receba os parâmetros necessários (ID do aluno e e-mail correto) e proceda à alteração na tabela.

(Utilize a tabela dAlunos no código da procedure)

Explicação e Resultado

Para **testar** a procedure, podemos efetuar os seguintes passos:

Consultamos a tabela **dAlunos** para verificar o e-mail do aluno **antes da alteração**:

```
SELECT * FROM dAlunos;
```

ID_Aluno	Nome_Aluno	Email
1	Ana	ana123@gmail.com
2	Bruno	bruno_vargas@outlook.com
3	carla	carlinha1999@gmail.com
4	Diego	diicastroneves@gmail.com
NULL	NULL	NULL

Chamamos a procedure sp_AtualizaEmail, passando os valores do alunoID e do novoEmail como argumentos (no exemplo, **ID = 4** e e-mail atualizado como **diego_neves@gmail.com**):

```
CALL sp_AtualizaEmail(4, 'diego_neves@gmail.com');
```

E-mail alterado com sucesso!
E-mail alterado com sucesso!

Consultamos novamente a tabela **dAlunos** para verificar se o e-mail foi atualizado corretamente, e o **resultado** mostrará que **o e-mail do aluno com o ID informado foi corrigido:**

```
SELECT * FROM dAlunos;
```

ID_Aluno	Nome_Aluno	Email
1	Ana	ana123@gmail.com
2	Bruno	bruno_vargas@outlook.com
3	carla	carlinha1999@gmail.com
4	Diego	diego_neves@gmail.com
NULL	NULL	NULL

GABARITO EXPLICATIVO

8

Questão 8

Quando um aluno solicitar a exclusão completa de seus dados, todos os seus dados pessoais, incluindo as matrículas associadas a esse aluno, devem ser excluídos do banco de dados.

Crie uma **procedure** que receba o ID do aluno e exclua todos os seus dados pessoais e suas matrículas.

(Utilize as tabelas fMatriculas e dAlunos no código da procedure)

Explicação e Resultado

```
DELIMITER $$

CREATE PROCEDURE sp_RemoveAluno(alunoID INT)
BEGIN
    DELETE FROM fMatriculas
    WHERE id_aluno = alunoID;

    DELETE FROM dAlunos
    WHERE id_aluno = alunoID;

    SELECT CONCAT('Dados pessoais e matrículas do aluno de ID = ', alunoID, ' excluídos com sucesso!');
END $$

DELIMITER ;
```

Para **criar uma procedure que exclua todos os dados pessoais de um aluno, incluindo suas matrículas associadas**, seguimos os seguintes passos:

Alteração do delimitador padrão:

Alteramos o delimitador padrão do MySQL para **$$** com o comando **DELIMITER.** Isso permite incluir múltiplas instruções na procedure, separadas por ponto e vírgula (;).

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

8

Questão 8

Quando um aluno solicitar a exclusão completa de seus dados, todos os seus dados pessoais, incluindo as matrículas associadas a esse aluno, devem ser excluídos do banco de dados.

Crie uma **procedure** que receba o ID do aluno e exclua todos os seus dados pessoais e suas matrículas.

(Utilize as tabelas fMatriculas e dAlunos no código da procedure)

Explicação e Resultado

Definição da procedure:

Utilizamos o comando **CREATE PROCEDURE**, seguido do nome **sp_RemoveAluno** e do parâmetro de entrada (**alunoID INT**), que identifica o aluno que será removido do banco de dados.

Instruções no bloco:

Abrimos o bloco de instruções com o comando **BEGIN**.

Primeiro, utilizamos o comando **DELETE** na tabela **fMatriculas** para **remover as matrículas** do aluno com o ID especificado no parâmetro **alunoID**.

Em seguida, utilizamos outro comando **DELETE** para **remover os dados pessoais** do aluno na tabela **dAlunos**, usando o mesmo **alunoID** como critério.

Por fim, adicionamos um comando **SELECT** para exibir uma **mensagem de confirmação**, indicando que todos os dados do aluno foram excluídos com sucesso.

Encerramento do bloco e restauração do delimitador:

Encerramos o bloco de instruções com **END $$**.

Restauramos o delimitador padrão para o ponto e vírgula (;) com o comando **DELIMITER ;**.

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

8

Questão 8

Quando um aluno solicitar a exclusão completa de seus dados, todos os seus dados pessoais, incluindo as matrículas associadas a esse aluno, devem ser excluídos do banco de dados.

Crie uma **procedure** que receba o ID do aluno e exclua todos os seus dados pessoais e suas matrículas.

(Utilize as tabelas fMatriculas e dAlunos no código da procedure)

Explicação e Resultado

Após **executar** o código, a procedure será **criada** no banco de dados:

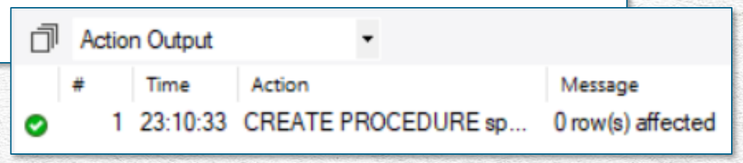
```
DELIMITER $$

CREATE PROCEDURE sp_RemoveAluno(alunoID INT)
BEGIN
    DELETE FROM fMatriculas
    WHERE id_aluno = alunoID;

    DELETE FROM dAlunos
    WHERE id_aluno = alunoID;

    SELECT CONCAT('Dados pessoais e matrículas do aluno de ID = ', alunoID, ' excluídos com sucesso!');
END $$

DELIMITER ;
```



CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

8

Questão 8

Quando um aluno solicitar a exclusão completa de seus dados, todos os seus dados pessoais, incluindo as matrículas associadas a esse aluno, devem ser excluídos do banco de dados.

Crie uma **procedure** que receba o ID do aluno e exclua todos os seus dados pessoais e suas matrículas.

(Utilize as tabelas fMatriculas e dAlunos no código da procedure)

Explicação e Resultado

Para **testar** a procedure, podemos efetuar os seguintes passos:

Consultamos as tabelas **dAlunos** e **fMatriculas** para verificar os registros **antes da exclusão**:

```
SELECT * FROM dAlunos;
```

ID_Aluno	Nome_Aluno	Email
1	Ana	ana123@gmail.com
2	Bruno	bruno_vargas@outlook.com
3	carla	carlinha1999@gmail.com
4	Diego	diego_neves@gmail.com
NULL	NULL	NULL

```
SELECT * FROM fMatriculas;
```

ID_Matricula	ID_Aluno	ID_Curso	Data_Cadastro
1	1	1	2021-03-11
2	1	2	2021-06-21
3	2	3	2021-01-08
4	3	1	2021-04-03
5	4	1	2021-05-10
6	4	3	2021-05-10
NULL	NULL	NULL	NULL

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

8

Questão 8

Quando um aluno solicitar a exclusão completa de seus dados, todos os seus dados pessoais, incluindo as matrículas associadas a esse aluno, devem ser excluídos do banco de dados.

Crie uma **procedure** que receba o ID do aluno e exclua todos os seus dados pessoais e suas matrículas.

(Utilize as tabelas fMatriculas e dAlunos no código da procedure)

Explicação e Resultado

Chamamos a procedure sp_RemoveAluno, passando o ID do aluno que será excluído (no exemplo, **ID = 4**):

```
CALL sp_RemoveAluno(4);
```

	CONCAT('Dados pessoais e matrículas do aluno de ID = ', alunoID, ' excluídos com sucesso!')
▶	Dados pessoais e matrículas do aluno de ID = 4 excluídos com sucesso!

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

8

Questão 8

Quando um aluno solicitar a exclusão completa de seus dados, todos os seus dados pessoais, incluindo as matrículas associadas a esse aluno, devem ser excluídos do banco de dados.

Crie uma **procedure** que receba o ID do aluno e exclua todos os seus dados pessoais e suas matrículas.

(Utilize as tabelas fMatriculas e dAlunos no código da procedure)

Explicação e Resultado

Verificamos novamente as tabelas **dAlunos** e **fMatriculas** para confirmar a exclusão, e os **resultados** mostram que **todos os registros do aluno especificado foram removidos**, tanto as matrículas da tabela fMatriculas quanto os dados pessoais da tabela dAlunos:

```
SELECT * FROM dAlunos;
```

ID_Aluno	Nome_Aluno	Email
1	Ana	ana123@gmail.com
2	Bruno	bruno_vargas@outlook.com
3	carla	carlinha1999@gmail.com
NULL	NULL	NULL

```
SELECT * FROM fMatriculas;
```

ID_Matricula	ID_Aluno	ID_Curso	Data_Cadastro
1	1	1	2021-03-11
2	1	2	2021-06-21
3	2	3	2021-01-08
4	3	1	2021-04-03
NULL	NULL	NULL	NULL

GABARITO EXPLICATIVO

9

Questão 9

A secretaria precisa cadastrar novos cursos no sistema.

Crie uma **procedure** que receba os dados do novo curso (ID, nome e preço) e os insira na tabela.

(Utilize a tabela dCursos no código da procedure)

Explicação e Resultado

```
DELIMITER $$

CREATE PROCEDURE sp_RegistraCurso(idCurso INT, nomeCurso VARCHAR(100), preco DECIMAL(10, 2))
BEGIN
    INSERT INTO dCursos (id_curso, nome_curso, preco_curso) VALUES
    (idCurso, nomeCurso, preco);

    SELECT 'Curso cadastrado com sucesso!';
END $$

DELIMITER ;
```

Para **criar uma procedure que permita cadastrar novos cursos no sistema**, seguimos os seguintes passos:

Alteração do delimitador padrão:

Alteramos o delimitador padrão do MySQL para **$$** com o comando **DELIMITER**. Isso nos permite usar múltiplas instruções no corpo da procedure.

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

9

Questão 9

A secretaria precisa cadastrar novos cursos no sistema.

Crie uma **procedure** que receba os dados do novo curso (ID, nome e preço) e os insira na tabela.

(Utilize a tabela dCursos no código da procedure)

Explicação e Resultado

Definição da procedure:

Utilizamos o comando **CREATE PROCEDURE**, seguido do nome **sp_RegistraCurso** e dos parâmetros de entrada:

idCurso INT: identifica o curso que será adicionado ao sistema.

nomeCurso VARCHAR(100): nome do curso a ser cadastrado.

preco DECIMAL(10, 2): preço do curso.

Instrução de inserção:

Abrimos o bloco de instruções com o comando **BEGIN**.

Utilizamos o comando **INSERT INTO** para inserir os valores recebidos nos parâmetros nas colunas correspondentes da tabela **dCursos**.

Após a inserção, adicionamos um comando **SELECT** para exibir uma **mensagem** confirmando que o curso foi cadastrado com sucesso.

Encerramento do bloco e restauração do delimitador:

Encerramos o bloco com **END $$**.

Restauramos o delimitador padrão para o ponto e vírgula (;) com o comando **DELIMITER ;**.

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

9

Questão 9

A secretaria precisa cadastrar novos cursos no sistema.

Crie uma **procedure** que receba os dados do novo curso (ID, nome e preço) e os insira na tabela.

(Utilize a tabela dCursos no código da procedure)

Explicação e Resultado

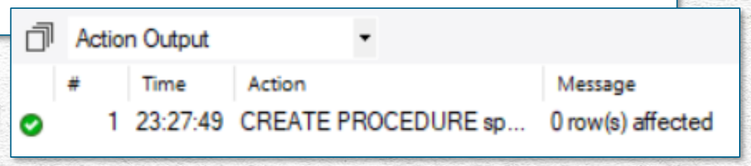
Após **executar** o código, a procedure será **criada** no banco de dados:

```
DELIMITER $$

CREATE PROCEDURE sp_RegistraCurso(idCurso INT, nomeCurso VARCHAR(100), preco DECIMAL(10, 2))
BEGIN
    INSERT INTO dCursos (id_curso, nome_curso, preco_curso) VALUES
    (idCurso, nomeCurso, preco);

    SELECT 'Curso cadastrado com sucesso!';
END $$

DELIMITER ;
```



CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

9

Questão 9

A secretaria precisa cadastrar novos cursos no sistema.

Crie uma **procedure** que receba os dados do novo curso (ID, nome e preço) e os insira na tabela.

(Utilize a tabela dCursos no código da procedure)

Explicação e Resultado

Para **testar** a procedure, podemos efetuar os seguintes passos:

Consultamos a tabela **dCursos** para verificar os registros **antes da inserção**:

```
SELECT * FROM dCursos;
```

→

ID_Curso	Nome_Curso	Preco_Curso
1	Excel	110.00
2	VBA	220.00
3	Power BI	165.00
NULL	NULL	NULL

Chamamos a procedure sp_RegistraCurso, passando os valores do novo curso (no exemplo, **ID = 4**, **Nome = 'Python'**, **Preço = 250.90**):

```
CALL sp_RegistraCurso(4, 'Python', 250.90);
```

→

Curso cadastrado com sucesso!
Curso cadastrado com sucesso!

Verificamos novamente a tabela **dCursos** para confirmar que o curso foi cadastrado e o **resultado** mostra que **o novo curso foi adicionado à tabela** com as informações corretas:

```
SELECT * FROM dCursos;
```

→

ID_Curso	Nome_Curso	Preco_Curso
1	Excel	110.00
2	VBA	220.00
3	Power BI	165.00
4	Python	250.90
NULL	NULL	NULL

GABARITO EXPLICATIVO

10

Questão 10

Um aluno solicitou sua transferência de curso e a secretaria precisa atualizar as informações.

Crie uma **procedure** que receba os parâmetros necessários (ID do aluno, ID do curso atual, ID do novo curso para o qual o aluno será transferido e data do novo cadastro) e efetue a alteração na tabela.

(Utilize a tabela fMatriculas no código da procedure)

Explicação e Resultado

```
DELIMITER $$

CREATE PROCEDURE sp_TransfereAluno(alunoID INT, cursoAtualID INT, novoCursoID INT, dataCadastro DATE)
BEGIN
    UPDATE fMatriculas
    SET id_curso = novoCursoID, data_cadastro = dataCadastro
    WHERE id_aluno = alunoID AND id_curso = cursoAtualID;

    SELECT 'Transferência efetuada com sucesso!';

END $$

DELIMITER ;
```

Para **criar uma procedure que permita transferir um aluno de um curso para outro**, seguimos as etapas a seguir:

Alteração do delimitador padrão:

Alteramos o delimitador para **$$** com o comando **DELIMITER**. Isso permite incluir múltiplas instruções no corpo da procedure.

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

10

Questão 10

Um aluno solicitou sua transferência de curso e a secretaria precisa atualizar as informações.

Crie uma **procedure** que receba os parâmetros necessários (ID do aluno, ID do curso atual, ID do novo curso para o qual o aluno será transferido e data do novo cadastro) e efetue a alteração na tabela.

(Utilize a tabela fMatriculas no código da procedure)

Explicação e Resultado

Definição da procedure:

Usamos o comando **CREATE PROCEDURE**, seguido do nome **sp_TransfereAluno** e dos parâmetros:

alunoID INT: ID do aluno que será transferido.

cursoAtualID INT: ID do curso no qual o aluno está atualmente matriculado.

novoCursoID INT: ID do novo curso para o qual o aluno será transferido.

dataCadastro DATE: data do cadastro do aluno no novo curso.

Instrução de inserção:

Abrimos o bloco de instruções com o comando **BEGIN**.

Utilizamos o comando **UPDATE** para modificar o valor das colunas **id_curso** e **data_cadastro** na tabela **fMatriculas**, substituindo o ID do curso atual pelo ID do novo curso e a data pela data da nova matrícula.

A condição **WHERE** assegura que a atualização ocorra **apenas para o aluno e curso especificados**, garantindo precisão.

Após a atualização, usamos o comando **SELECT** para exibir uma **mensagem** confirmando que a transferência foi realizada com sucesso.

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

10

Questão 10

Um aluno solicitou sua transferência de curso e a secretaria precisa atualizar as informações.

Crie uma **procedure** que receba os parâmetros necessários (ID do aluno, ID do curso atual, ID do novo curso para o qual o aluno será transferido e data do novo cadastro) e efetue a alteração na tabela.

(Utilize a tabela fMatriculas no código da procedure)

Explicação e Resultado

Encerramento do bloco e restauração do delimitador:

Finalizamos a procedure com **END $$**.

Restauramos o delimitador padrão (;) com **DELIMITER ;**.

Após **executar** o código, a procedure será **criada** no banco de dados:

```
DELIMITER $$

CREATE PROCEDURE sp_TransfereAluno(alunoID INT, cursoAtualID INT, novoCursoID INT, dataCadastro DATE)
BEGIN
    UPDATE fMatriculas
    SET id_curso = novoCursoID, data_cadastro = dataCadastro
    WHERE id_aluno = alunoID AND id_curso = cursoAtualID;

    SELECT 'Transferência efetuada com sucesso!';

END $$

DELIMITER ;
```

Action Output

#	Time	Action	Message
1	23:44:23	CREATE PROCEDURE sp...	0 row(s) affected

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

10

Questão 10

Um aluno solicitou sua transferência de curso e a secretaria precisa atualizar as informações.

Crie uma **procedure** que receba os parâmetros necessários (ID do aluno, ID do curso atual, ID do novo curso para o qual o aluno será transferido e data do novo cadastro) e efetue a alteração na tabela.

(Utilize a tabela fMatriculas no código da procedure)

Explicação e Resultado

Para **testar** a procedure, podemos efetuar os seguintes passos:

Verificamos o estado atual da tabela **fMatriculas** para confirmar os dados **antes da transferência**:

```
SELECT * FROM fMatriculas;
```

ID_Matricula	ID_Aluno	ID_Curso	Data_Cadastro
1	1	1	2021-03-11
2	1	2	2021-06-21
3	2	3	2021-01-08
4	3	1	2021-04-03
NULL	NULL	NULL	NULL

Chamamos a procedure sp_TransfereAluno, passando os parâmetros correspondentes. Por exemplo: **Ana (ID = 1)** está matriculada no curso **VBA (ID = 2)** e deseja transferir-se para **Python (ID = 4)**. Como data da nova matrícula, podemos informar a função **current_date()** como parâmetro; assim, inserimos a **data atual**:

```
CALL sp_TransfereAluno(1, 2, 4, current_date());
```

Transferência efetuada com sucesso!
Transferência efetuada com sucesso!

Consultamos novamente a tabela **fMatriculas** e o **resultado** mostra que **os dados foram atualizados corretamente**:

```
SELECT * FROM fMatriculas;
```

ID_Matricula	ID_Aluno	ID_Curso	Data_Cadastro
1	1	1	2021-03-11
2	1	4	2024-12-17
3	2	3	2021-01-08
4	3	1	2021-04-03
NULL	NULL	NULL	NULL

MÓDULO 14

SUBQUERIES

SUBQUERIES

SUBQUERIES

Índice Geral

Índice deste Módulo

O QUE VEREMOS NESTE MÓDULO

MÓDULO 14 SUBQUERIES

01	O que é uma subquery?	453
02	Onde utilizamos subqueries	454
03	Subquery como filtro de uma nova consulta (Escalar) - Exemplo 1	458
04	Subquery como filtro de uma nova consulta (Escalar) - Exemplo 2	459
05	Subquery como filtro de uma nova consulta (Escalar) - Exemplo 3	460
06	Subquery como filtro de uma nova consulta (Escalar) - Exemplo Desafio	461
07	Subquery como filtro de uma nova consulta (Lista) - Exemplo 1	462
08	Subquery como filtro de uma nova consulta (Lista) - Exemplo 2	463
09	Subquery para criar uma nova coluna na consulta	464
10	Subquery para criar uma tabela na consulta	465
11	EXISTS	466
12	EXISTS (Exemplo)	467
13	EXERCÍCIOS	470
14	GABARITO EXPLICATIVO	474

O que é uma subquery?

Uma Subconsulta (ou Subquery ou SubSELECT) nada mais é do que uma consulta dentro de outra consulta. Ou seja, com uma subquery conseguimos utilizar o resultado de uma query (consulta) dentro de outra consulta.

O exemplo abaixo ilustra bem a ideia principal de uma subquery. Para descobrir os produtos que têm o Preco_Unit maior do que a média, podemos adicionar um SELECT dentro de outro SELECT para criar uma consulta mais otimizada.

```
SELECT AVG(Preco_Unit) FROM produtos; -- Média = 1788.125

SELECT
    *
FROM produtos
WHERE Preco_Unit > 1788.125;
```

```
SELECT
    *
FROM produtos
WHERE Preco_Unit > (SELECT AVG(Preco_Unit) FROM produtos);
```

Onde utilizamos subqueries

Subqueries com WHERE (escalar)

Nesta situação, utilizamos o resultado de um SELECT para filtrar um outro SELECT.

Observe no print ao lado. Temos um SELECT mais interno sendo utilizado com o filtro WHERE de uma consulta principal.

Para este caso, utilizamos o termo “escalar” para dizer que o resultado do SELECT mais interno é um valor único, e não uma lista de valores.

```
SELECT
    Coluna1,
    Coluna2
FROM Tabela
WHERE Coluna1 = (SELECT);
```

Onde utilizamos subqueries

Subqueries com WHERE (lista)

Aqui temos uma situação semelhante à anterior, com a diferença de que o resultado da subquery seria uma lista de vários valores que poderiam ser passados no WHERE.

Neste caso, como o filtro seria de mais de 1 valor, utilizamos o IN.

```
SELECT
    Coluna1,
    Coluna2
FROM Tabela
WHERE Coluna1 IN (SELECT);
```

Onde utilizamos subqueries

Subqueries dentro de um SELECT

Outra opção seria utilizar o resultado de um SELECT como uma nova coluna de uma query principal.

```
SELECT
      Coluna1,
      Coluna2,
      (SELECT)
FROM Tabela;
```

Onde utilizamos subqueries

Subqueries após um FROM

Podemos utilizar o resultado de um SELECT como uma tabela de um outro SELECT.

Neste caso, precisamos utilizar o comando AS para dar um nome para essa tabela.

```
SELECT
    Coluna1,
    Coluna2
FROM (SELECT) AS T;
```

Subquery como filtro de uma nova consulta (Escalar) – Exemplo 1

No exemplo ao lado, utilizamos uma subconsulta para descobrir quais pedidos foram feitos no banco de dados Base, onde o nome do Gerente é 'Julia Freitas'.

Como na tabela "pedidos" não temos a informação de nome do gerente (apenas o ID_Loja) usamos uma subquery para descobrir qual é o ID da loja cujo Gerente é a Julia Freitas, para somente então filtrar a tabela Pedidos.

```sql
-- Exercicio 1. Quais foram os pedidos realizados na loja onde o gerente é o Marcelo Castro?

SELECT * FROM pedidos;
SELECT * FROM lojas;

SET @varNomeGerente = 'Julia Freitas';

SELECT
    *
FROM pedidos
WHERE ID_Loja = (
            SELECT ID_Loja
            FROM lojas
            WHERE Gerente = @varNomeGerente
            );
```

Subquery como filtro de uma nova consulta (Escalar) – Exemplo 2

Já no exemplo a seguir, queremos saber quais são os produtos que têm o Preco_unit maior do que a média.

Para isso, em vez de descobrir primeiro qual é a média para só depois filtrar a tabela, podemos usar o SELECT que calcula essa média diretamente no filtro da tabela de produtos, conforme mostrado ao lado.

```sql
-- 1. Subquery como filtro de uma nova consulta
-- Exercício 2. Quais produtos têm o Preco_Unit acima da média?

SELECT AVG(Preco_Unit) FROM produtos; -- Média = 1788.125

SELECT
    *
FROM produtos
WHERE Preco_Unit > (SELECT AVG(Preco_Unit) FROM produtos);
```

Subquery como filtro de uma nova consulta (Escalar) – Exemplo 3

No próximo exemplo, utilizamos uma subquery para filtrar a tabela de Produtos, para mostrar apenas os produtos que são da Categoria Notebook.

```sql
-- 1. Subquery como filtro de uma nova consulta
-- Exercicio 3. Quais produtos são da categoria 'Notebook'?

SELECT * FROM categorias;

SELECT * FROM produtos
WHERE ID_Categoria = (
                SELECT
                    ID_Categoria
                FROM categorias
                WHERE Categoria = 'Notebook'
                );
```

Subquery como filtro de uma nova consulta (Escalar) – Exemplo Desafio

No próximo exemplo, temos um desafio.

Precisamos saber as informações do cliente que gerou mais receita para a empresa. Mas como fazer isso?

```sql
-- 1. Subquery como filtro de uma nova consulta
-- Exercício Desafio. Descubra todas as informações sobre o cliente que gerou mais receita para a empresa.

SELECT * FROM clientes
WHERE ID_Cliente =
        (SELECT
            ID_Cliente
        FROM pedidos
        GROUP BY ID_Cliente
        ORDER BY SUM(Receita_Venda) DESC
        LIMIT 1
        );
```

A consulta mais interna retorna o ID do cliente que, agrupado pela soma de receita de venda, ordenado em ordem crescente e considerando apenas o primeiro (LIMIT 1) nos dá exatamente o ID do cliente que gerou mais receita para a empresa. De posse dessa informação, passamos o resultado dessa consulta mais interna para a consulta mais externa, retornando as informações para o cliente cujo ID é o mesmo retornado pela consulta mais interna (subconsulta).

Subquery como filtro de uma nova consulta (Lista) – Exemplo 1

Agora vamos ver como funcionam as subqueries que retornam listas como resultado.

Ao lado, queremos a soma total de receita, mas apenas para os produtos cuja marca é DELL.

Na tabela Pedidos não temos a informação do nome da marca, apenas o ID do produto. Por isso, usamos uma subquery que retorna todos os Ids da marca DELL, para somente então filtrar a consulta principal.

```sql
-- Exercicio 1. Descubra qual foi a receita total associada aos produtos da marca DELL.

SELECT
    SUM(Receita_Venda) AS 'Receita Marca DELL'
FROM pedidos
WHERE ID_Produto IN (
                SELECT
                    ID_Produto
                FROM produtos
                WHERE Marca_Produto = 'DELL'
                );
```

Subquery como filtro de uma nova consulta (Lista) – Exemplo 2

No exemplo ao lado, queremos saber quais pedidos foram feitos na região 'Sudeste'.

Para isso, utilizamos a solução mostrada ao lado.

Observe que para chegar no resultado final, utilizamos duas subqueries: a mais interna para descobrir as cidades associadas à região Sudeste, e uma intermediária para, a partir das Cidades, descobrir os IDs das lojas.

```sql
-- Exercicio 2. Quais pedidos foram feitos na região 'Sudeste'.

SELECT
    *
FROM pedidos
WHERE ID_Loja IN (
            SELECT
                ID_Loja
            FROM lojas
            WHERE Loja IN (
                    SELECT
                        Cidade
                    FROM locais
                    WHERE Região = 'Sudeste'
                    )
            );
```

Subquery para criar uma nova coluna na consulta

Nessa nova situação, queremos incluir na consulta uma coluna que retorne a média geral de preços.

Para isso, utilizamos uma subquery dentro do SELECT da consulta mais externa, conforme mostrado ao lado.

```
7  SELECT
8      *,
9      (SELECT
10         AVG(Preco_Unit)
11     FROM produtos) AS 'Média Geral de Preço'
12 FROM produtos;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ID_Produto	Nome_Produto	ID_Categoria	Marca_Produto	Num_Serie	Preco_Unit	Custo_Unit	Média Geral de Preço
1	Monitor LED 19,5" Full HD HDMI	1	DELL	MNT-DL-831923	2300	966	1788.1250
2	Monitor Curvo 24" 144Hz HDMI	1	SAMSUNG	MNT-SS-001939	2800	980	1788.1250
3	Webcam Full HD 1080p	1	LOGITECH	WBC-LT-934GG4	450	90	1788.1250
4	Kit Teclado + Mouse sem fio Wireless	2	DELL	KTM-DL-041039	350	129.5	1788.1250
5	Kit Teclado + Mouse Slim Bluetooth	2	DELL	KTM-DL-111924	280	109.2	1788.1250
6	Cadeira Gamer reclinável Azul/Laranja	3	ALTURA	CGM-AL-9N914J	1800	540	1788.1250
7	Cadeira Gamer PC Racer Vermelha	3	ALTURA	CGM-AL-0147FI	3100	1395	1788.1250
8	Headphone Bluetooth 2000	4	SONY	HDP-SN-194821	600	258	1788.1250
9	Fone de Ouvido Tune T5000	4	JBL	HDP-JB-091934	780	327.6	1788.1250
10	Microfone Condensador MC1000	5	AKG	MIC-AK-237591	1100	275	1788.1250

Result 1 | Result 2

Subquery para criar uma tabela na consulta

A última aplicação da subconsulta é como uma tabela dentro da consulta principal.

No exemplo ao lado, queremos saber qual foi a quantidade máxima vendida, a quantidade mínima vendida e a média de vendas.

Para isso, criamos uma subconsulta mais interna, que retorna o agrupamento das vendas por produto. Como essa query retorna uma tabela, alimentamos a tabela mais externa com o resultado dessa query mais interna, e calculamos o máximo, mínimo e média de vendas, conforme mostrado ao lado.

```sql
-- Exercicio 1. Do total de vendas por produto, qual foi a quantidade máxima vendida? E a quantidade mínima? E a média?

SELECT
    MAX(Vendas) AS 'Máximo Vendas',
    MIN(Vendas) AS 'Mínimo Vendas',
    AVG(Vendas) AS 'Média Vendas'
FROM (
        SELECT
            ID_Produto,
            COUNT(*) AS 'Vendas'
        FROM pedidos
        GROUP BY ID_Produto
        ) AS t;
```

Result Grid | Filter Rows:

Máximo Vendas	Mínimo Vendas	Média Vendas
349	6	93.5000

EXISTS

O operador EXISTS é usado para testar a existência de qualquer registro em uma subconsulta. Ele retorna TRUE se a subconsulta retornar um ou mais registros.

A sintaxe é mostrada abaixo:

```
SELECT coluna(s)
FROM tabela
WHERE EXISTS (SELECT);
```

EXISTS (Exemplo)

Na imagem abaixo, temos um exemplo de aplicação do EXISTS.

A ideia é verificar se todas as categorias possuem pelo menos 1 exemplar de produtos.

```
-- Exercicio. Você deverá verificar se todos as categorias possuem pelo menos 1 exemplar de produtos (na tabela de produtos); caso alguma categoria não possua nenhum exemplar você deverá descobrir qual é/quais são.
```

EXISTS (Exemplo)

Para resolver essa questão, usamos o EXISTS como mostrado ao lado.

O retorno do EXISTS será uma tabela com todas as categorias que possuem pelo menos um exemplar de produtos.

E como fazemos para descobrir qual categoria não tem um exemplar?

Podemos usar o NOT EXISTS.

```
19 •  SELECT
20        *
21    FROM categorias
22    WHERE EXISTS (
23                SELECT
24                    *
25                FROM produtos
26                WHERE categorias.ID_Categoria = produtos.ID_Categoria
27                );
28
29
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ID_Categoria	Categoria
1	Monitor
2	Kit Teclado e Mouse
3	Cadeira Gamer
4	Headphone
5	Microfone
6	Notebook

EXISTS (Exemplo)

Com o NOT EXISTS, verificamos a não existência da informação.

Ao fazer essa pequena modificação no código, o resultado é exatamente a categoria que não possui nenhum exemplar de produto: no caso, a categoria Acessórios.

```
19 •  SELECT
20        *
21    FROM categorias
22    WHERE NOT EXISTS (
23                SELECT
24                    *
25                FROM produtos
26                WHERE categorias.ID_Categoria = produtos.ID_Categoria
27                );
28
29
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ID_Categoria	Categoria
7	Acessórios

EXERCÍCIOS

1

Questão 1

A equipe de marketing deseja identificar clientes com renda anual acima da média para futuras campanhas exclusivas.

Liste os clientes cuja Renda_Anual seja maior que a média de renda anual de todos os clientes.

Ordene o resultado pela coluna Renda_Anual. **(Utilize a tabela clientes)**

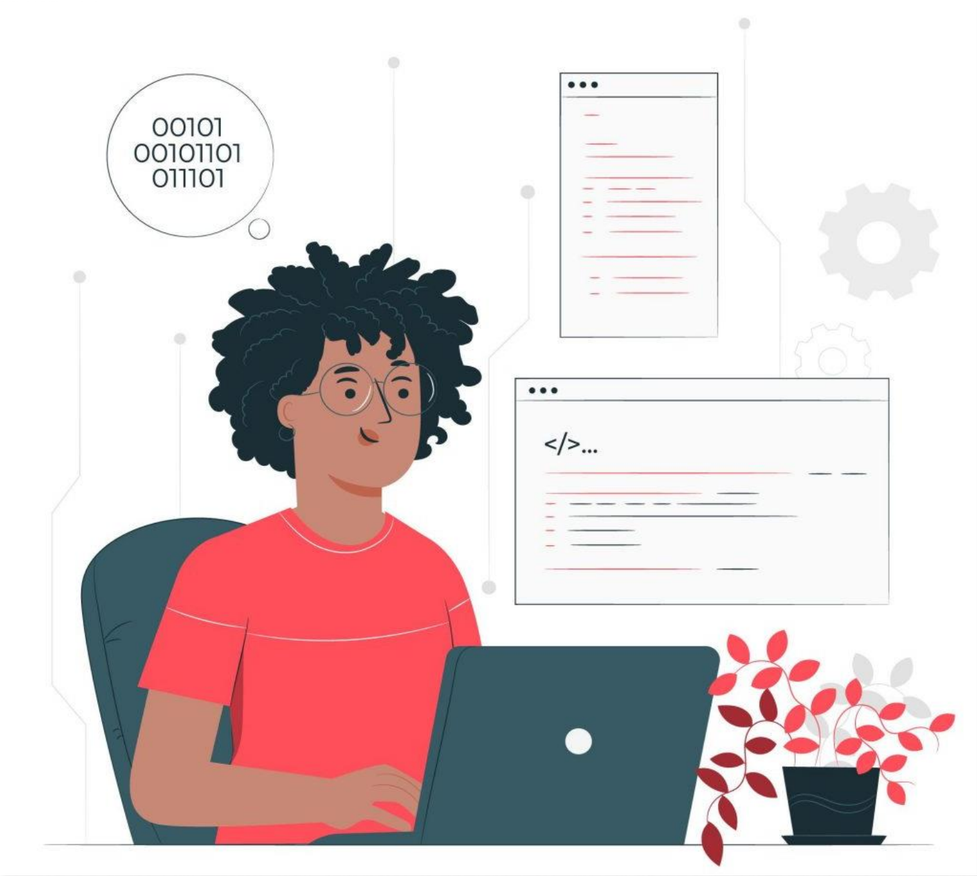
2

Questão 2

A gerência deseja saber quais produtos têm preço abaixo do custo médio de todos os produtos da loja.

Retorne os produtos cujo Preco_Unit seja menor que o custo médio (Custo_Unit) de todos os produtos. **(Utilize a tabela produtos)**

EXERCÍCIOS



3

Questão 3

O RH solicitou que descobríssemos quais lojas possuem um número de funcionários superior à média de todas as lojas cadastradas.

Liste o nome das lojas e o número de funcionários. **(Utilize a tabela lojas)**

4

Questão 4

O setor de produtos deseja saber quantos produtos de cada categoria estão sendo comercializados pela empresa.

Crie um relatório com o nome de cada categoria e a quantidade de produtos cadastrados. **(Utilize as tabelas produtos e categorias)**

EXERCÍCIOS

5

Questão 5

Gere um relatório com o ID, o nome completo e o total de pedidos realizados por cada cliente, para auxiliar a equipe de vendas a identificar os clientes mais ativos.

Ordene o resultado pelo total de pedidos, do maior para o menor total. **(Utilize as tabelas clientes e pedidos)**

6

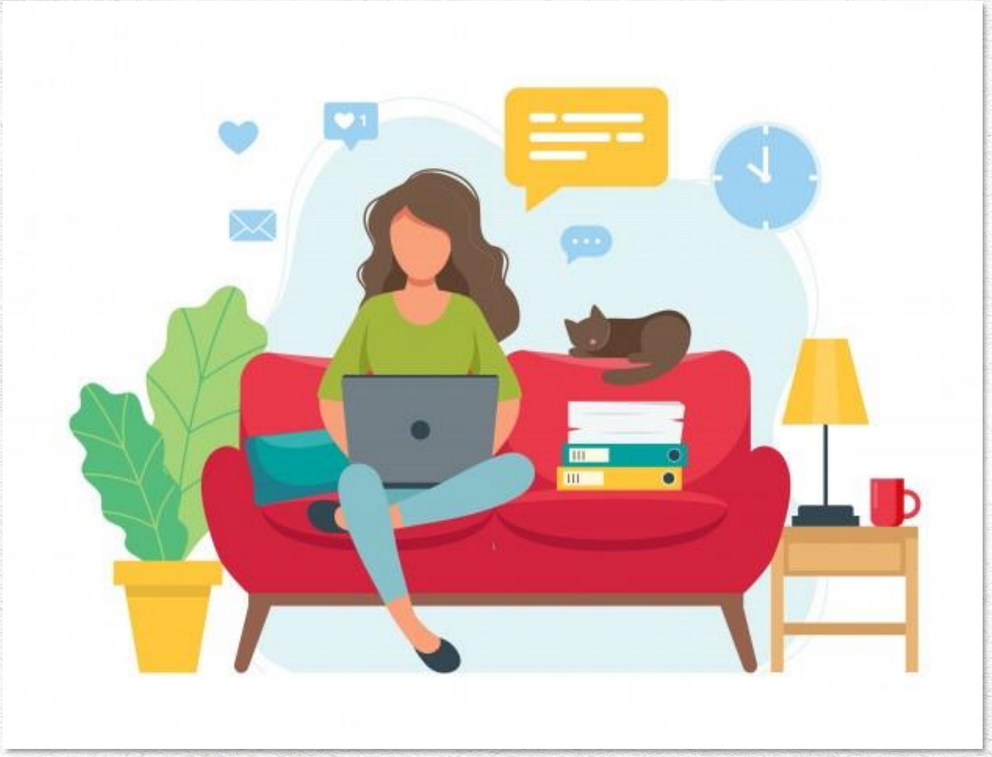
Questão 6

A equipe de estratégia quer saber quais lojas não alcançaram uma receita total maior que R$ 10.000.000.

Crie uma consulta que liste as lojas com receita total inferior a R$ 10.000.000.

Inclua no resultado o nome da loja, o nome do gerente e a receita total de cada loja retornada. **(Utilize as tabelas pedidos e lojas)**

EXERCÍCIOS



7

Questão 7

Liste os nomes completos e os e-mails dos clientes que realizaram compras de produtos da categoria "Headphone", permitindo que o marketing direcione campanhas promocionais a esse público. **(Utilize as tabelas clientes, pedidos, produtos e categorias)**

8

Questão 8

O departamento de produtos precisa descobrir se existe alguma loja que ainda não efetuou vendas dos produtos da marca "DELL".

Liste o nome das lojas que eventualmente preencham essa condição, se houver. **(Utilize as tabelas lojas, pedidos e produtos)**

GABARITO EXPLICATIVO

1

Questão 1

A equipe de marketing deseja identificar clientes com renda anual acima da média para futuras campanhas exclusivas.

Liste os clientes cuja Renda_Anual seja maior que a média de renda anual de todos os clientes.

Ordene o resultado pela coluna Renda_Anual. **(Utilize a tabela clientes)**

Explicação e Resultado

Para **listar os clientes cuja Renda_Anual seja maior que a média da renda anual de todos os clientes**, primeiramente, podemos **calcular essa média** aplicando a função de agregação **AVG** à coluna **Renda_Anual** da tabela **CLIENTES**:

```
SELECT AVG(Renda_Anual)
FROM clientes;
```

→

	AVG(Renda_Anual)
▶	59800.0000

Agora que conhecemos a média, incorporamos esse cálculo em uma **subquery dentro da cláusula WHERE da consulta principal** que fazemos para retornar as **todas as colunas** da tabela **CLIENTES**. Isso permite **comparar a renda de cada cliente com a média calculada**. Além disso, **ordenamos** o resultado pela **coluna Renda_Anual**, conforme solicitado no enunciado:

```
SELECT
    *
FROM clientes
WHERE Renda_Anual > (SELECT AVG(Renda_Anual) FROM clientes)
ORDER BY Renda_Anual;
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

1

Questão 1

A equipe de marketing deseja identificar clientes com renda anual acima da média para futuras campanhas exclusivas.

Liste os clientes cuja Renda_Anual seja maior que a média de renda anual de todos os clientes.

Ordene o resultado pela coluna Renda_Anual. **(Utilize a tabela clientes)**

Explicação e Resultado

Dessa forma, retornamos o **resultado** abaixo, que **mostra apenas os clientes cuja renda anual ultrapassa a média de R$ 59.800,00** (um total de 61 clientes):

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ID_Cliente	Nome	Sobrenome	Data_Nascime	Esta	Sexo	Email	Telefone	Renda_Anual	Qtd_	Escolaridade
1	Ruben	Torres	1985-08-07	C	M	ruben35@hotmail.com	(85) 94132-1176	60000	3	Pós-graduado
6	Marco	Mehta	1984-05-04	C	M	marco14@yahoo.com.br	(85) 99019-3803	60000	3	Pós-graduado
7	Rob	Verhoff	1984-07-02	S	F	rob4@hotmail.com	(11) 98162-4760	60000	4	Pós-graduado
34	Jennifer	Russell	1998-12-13	C	F	jennifer93@hotmail.com	(21) 96714-5068	60000	0	Parcial
10	Curtis	Lu	1983-10-30	C	M	curtis9@yahoo.com.br	(71) 94509-1440	60000	4	Pós-graduado
83	Emily	Johnson	1977-07-14	S	F	emily1@gmail.com	(11) 98332-4796	60000	2	Ensino médio
22	Russell	Xie	1998-09-12	C	M	russell7@hotmail.com	NULL	60000	0	Parcial
39	Amanda	Carter	1997-10-11	C	F	amanda53@gmail.com	(71) 98283-7962	60000	0	Parcial
41	Nathan	Simmons	1996-02-19	C	M	nathan11@hotmail.com	(11) 92589-9135	60000	0	Parcial
51	Ana	Price	2000-08-15	C	F	ana0@yahoo.com.br	(21) 99193-2686	60000	0	Parcial
65	Caleb	Carter	1996-09-20	S	M	caleb40@gmail.com	(31) 97809-1800	60000	0	Parcial
63	Jessica	Henderson	1993-10-04	C	F	jessica29@gmail.com	(21) 93673-6903	60000	0	Parcial
94	Andrés	Anand	1982-08-05	C	M	andrés18@gmail.com	(41) 92114-3295	60000	1	Pós-graduado
95	Edwin	Nara	1981-10-22	C	M	edwin39@yahoo.com.br	(71) 97194-6478	60000	1	Parcial
96	Mallory	Rubio	1981-04-26	S	F	mallory7@hotmail.com	(62) 93152-6617	60000	1	Parcial

clientes 18

Output

Action Output

#	Time	Action	Message
1	21:17:37	SELECT * FROM clientes WHERE Renda_Anual > (SELECT AVG(Renda_Anual) FROM clientes)...	61 row(s) returned

GABARITO EXPLICATIVO

2

Questão 2

A gerência deseja saber quais produtos têm preço abaixo do custo médio de todos os produtos da loja.

Retorne os produtos cujo Preco_Unit seja menor que o custo médio (Custo_Unit) de todos os produtos. **(Utilize a tabela produtos)**

Explicação e Resultado

Para **identificar quais produtos têm preço abaixo do custo médio de todos os produtos da loja**, começamos calculando a **média da coluna Custo_Unit** na tabela **PRODUTOS**, aplicando a função de agregação **AVG** à essa coluna:

```
SELECT AVG(Custo_Unit)
FROM produtos;
```

→

	AVG(Custo_Unit)
▶	626.4875

Após calcularmos o custo médio, podemos utilizá-lo em uma **subquery**. Na **consulta principal** em que retornamos **todas as colunas** da tabela **PRODUTOS**, **comparamos o Preco_Unit de cada produto com o custo médio** retornado pela subquery. Essa comparação é feita na cláusula **WHERE**. Veja como fica o código:

```
SELECT
    *
FROM produtos
WHERE Preco_Unit < (SELECT AVG(Custo_Unit) FROM produtos);
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

2

Questão 2

A gerência deseja saber quais produtos têm preço abaixo do custo médio de todos os produtos da loja.

Retorne os produtos cujo Preco_Unit seja menor que o custo médio (Custo_Unit) de todos os produtos. **(Utilize a tabela produtos)**

Explicação e Resultado

Essa consulta nos traz uma **lista dos produtos cujo preço unitário (Preco_Unit) é inferior ao custo médio (R$ 626,4875)** de todos os produtos. Abaixo, vemos o resultado final, que exibe 4 produtos atendendo a essa condição:

ID_Produto	Nome_Produto	ID_Categoria	Marca_Produto	Num_Serie	Preco_Unit	Custo_Unit
3	Webcam Full HD 1080p	1	LOGITECH	WBC-LT-934GG4	450	90
4	Kit Teclado + Mouse sem fio Wireless	2	DELL	KTM-DL-041039	350	129.5
5	Kit Teclado + Mouse Slim Bluetooth	2	DELL	KTM-DL-111924	280	109.2
8	Headphone Bluetooth 2000	4	SONY	HDP-SN-194821	600	258

GABARITO EXPLICATIVO

3

Questão 3

O RH solicitou que descobríssemos quais lojas possuem um número de funcionários superior à média de todas as lojas cadastradas.

Liste o nome das lojas e o número de funcionários. **(Utilize a tabela lojas)**

Explicação e Resultado

Para **descobrir quais lojas possuem um número de funcionários superior à média de todas as lojas cadastradas**, começamos calculando a **média do número de funcionários** por loja utilizando a função de agregação **AVG** aplicada à coluna **Num_Funcionarios** da tabela **LOJAS**:

```
SELECT AVG(Num_Funcionarios)
FROM lojas;
```

→

	AVG(Num_Funcionarios)
▶	20.0000

Com a média calculada, podemos utilizá-la em uma **subquery**. A **consulta principal** verifica, na cláusula **WHERE**, **quais lojas têm um número de funcionários (Num_Funcionarios) superior à média retornada pela subquery**. Além disso, **selecionamos** apenas as colunas **Loja** e **Num_Funcionarios** da tabela **LOJAS**, como solicitado no enunciado:

```
SELECT
    Loja,
    Num_Funcionarios
FROM lojas
WHERE Num_Funcionarios > (SELECT AVG(Num_Funcionarios) FROM lojas);
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

3

Questão 3

O RH solicitou que descobríssemos quais lojas possuem um número de funcionários superior à média de todas as lojas cadastradas.

Liste o nome das lojas e o número de funcionários. **(Utilize a tabela lojas)**

Explicação e Resultado

O **resultado** dessa consulta é **uma lista que mostra as lojas que possuem um número de funcionários acima da média calculada** (20 funcionários). No total, 3 lojas atendem a esse critério:

Loja	Num_Funcionarios
Rio de Janeiro	23
Curitiba	31
São Paulo	33

GABARITO EXPLICATIVO

4

Questão 4

O setor de produtos deseja saber quantos produtos de cada categoria estão sendo comercializados pela empresa.

Crie um relatório com o nome de cada categoria e a quantidade de produtos cadastrados. **(Utilize as tabelas produtos e categorias)**

	Categoria	Total_Produtos
▶	Monitor	3
	Kit Teclado e Mouse	2
	Cadeira Gamer	2
	Headphone	2
	Microfone	3
	Notebook	4
	Acessórios	0

Explicação e Resultado

Para **descobrir quantos produtos de cada categoria estão sendo comercializados pela empresa**, utilizamos uma **subquery na cláusula SELECT** da consulta principal. Essa subquery retorna uma coluna que chamamos de **Total_Produtos**, que **conta quantos produtos estão relacionados ao ID_Categoria de cada linha da tabela CATEGORIAS**:

```
SELECT
    Categoria,
    (SELECT COUNT(*)
     FROM produtos
     WHERE produtos.ID_Categoria = categorias.ID_Categoria
    ) AS Total_Produtos
FROM categorias;
```

Aqui, **para cada categoria** listada na tabela **CATEGORIAS**, a subquery executa a **contagem de produtos** na tabela **PRODUTOS** que possuem **o mesmo ID_Categoria**. O resultado é retornado na coluna **Total_Produtos**.

Dessa forma, temos o número **total de produtos cadastrados para cada categoria**, conforme mostrado ao lado:

Índice Geral

Índice deste Módulo

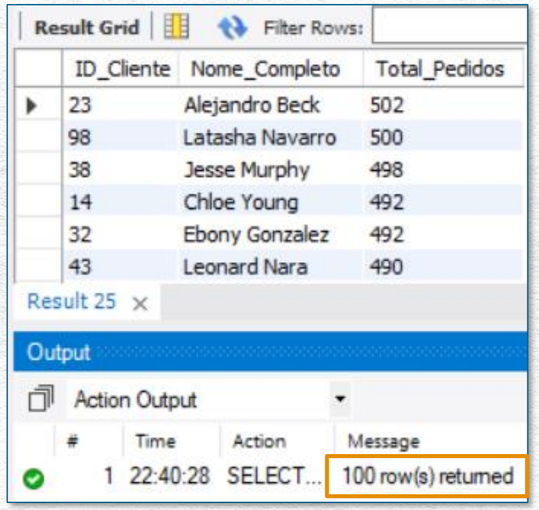
GABARITO EXPLICATIVO

5

Questão 5

Gere um relatório com o ID, o nome completo e o total de pedidos realizados por cada cliente, para auxiliar a equipe de vendas a identificar os clientes mais ativos.

Ordene o resultado pelo total de pedidos, do maior para o menor total. **(Utilize as tabelas clientes e pedidos)**



Explicação e Resultado

Para **gerar um relatório que mostre o ID, o nome completo e o total de pedidos realizados por cada cliente**, utilizamos uma **subquery na cláusula SELECT** da consulta principal. Essa subquery retorna uma coluna chamada **Total_Pedidos**, que conta o número de pedidos registrados na tabela **PEDIDOS** para cada cliente listado na tabela **CLIENTES**:

```sql
SELECT
    ID_Cliente,
    CONCAT(Nome, ' ', Sobrenome) AS Nome_Completo,
    (SELECT COUNT(*)
     FROM pedidos
     WHERE pedidos.ID_Cliente = clientes.ID_Cliente
    ) AS Total_Pedidos
FROM clientes
ORDER BY Total_Pedidos DESC;
```

Na subquery, a **contagem** de pedidos é feita **comparando** o **ID_Cliente** da tabela **CLIENTES** com o **ID_Cliente** na tabela **PEDIDOS**. Além disso, na consulta principal, **retornamos** a coluna **ID_Cliente** e utilizamos a função **CONCAT** para criar a coluna **Nome_Completo**, combinando o nome e o sobrenome de cada cliente em uma única string. O resultado é **ordenado** em ordem **decrescente** pelo **Total_Pedidos**, destacando os clientes mais ativos no topo do relatório.

Dessa forma, o resultado final apresenta as informações ao lado:

Índice Geral
Índice deste Módulo

GABARITO EXPLICATIVO

6

Questão 6

A equipe de estratégia quer saber quais lojas não alcançaram uma receita total maior que R$ 10.000.000.

Crie uma consulta que liste as lojas com receita total inferior a R$ 10.000.000.

Inclua no resultado o nome da loja, o nome do gerente e a receita total de cada loja retornada. **(Utilize as tabelas pedidos e lojas)**

Explicação e Resultado

Para **listar as lojas que não alcançaram uma receita total maior que R$ 10.000.000**, utilizamos uma **subquery tanto na cláusula SELECT quanto na cláusula WHERE** da consulta principal.

Na **subquery da cláusula SELECT**, a função **SUM(Receita_Venda)** é aplicada para somar todas as receitas da tabela **PEDIDOS** para **cada loja**:

```sql
SELECT
    Loja,
    Gerente,
    (SELECT SUM(Receita_Venda)
     FROM pedidos
     WHERE pedidos.ID_Loja = lojas.ID_Loja
    ) AS Receita_Total
FROM lojas
WHERE (
    SELECT SUM(Receita_Venda)
    FROM pedidos
    WHERE pedidos.ID_Loja = lojas.ID_Loja
) < 10000000;
```

Subquery na cláusula SELECT

Subquery na cláusula WHERE

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

6

Questão 6

A equipe de estratégia quer saber quais lojas não alcançaram uma receita total maior que R$ 10.000.000.

Crie uma consulta que liste as lojas com receita total inferior a R$ 10.000.000.

Inclua no resultado o nome da loja, o nome do gerente e a receita total de cada loja retornada. **(Utilize as tabelas pedidos e lojas)**

Explicação e Resultado

Além disso, a **subquery na cláusula WHERE** é responsável por **filtrar as lojas com receita total inferior a R$ 10.000.000**.

O **resultado** final será composto pelo **nome da loja**, o **nome do gerente** e a **receita total de cada loja que não atingiu o valor de R$ 10.000.000**, como mostrado abaixo:

	Loja	Gerente	Receita_Total
▶	Niterói	Matheus Leal	9986950
	Goiânia	Carlos Henrique	9943620

GABARITO EXPLICATIVO

7

Questão 7

Liste os nomes completos e os e-mails dos clientes que realizaram compras de produtos da categoria "Headphone", permitindo que o marketing direcione campanhas promocionais a esse público. **(Utilize as tabelas clientes, pedidos, produtos e categorias)**

Explicação e Resultado

Para **listar os nomes completos e os e-mails dos clientes que realizaram compras de produtos da categoria 'Headphone'**, utilizamos **uma série de subqueries dentro da cláusula WHERE** da consulta principal.

De baixo para cima, primeiro, **a primeira subquery (mais interna)** busca o **ID_Categoria** correspondente à categoria **'Headphone'**, utilizando a tabela **CATEGORIAS**.

Com esse ID_Categoria, passamos para a **segunda subquery**, que vai buscar os **ID_Produto dessa categoria** na tabela **PRODUTOS**. Ou seja, a primeira subquery identifica a categoria 'Headphone', e a segunda subquery usa esse ID para encontrar todos os produtos dessa categoria.

Na sequência, a **terceira subquery** busca os **ID_Cliente que realizaram pedidos desses produtos**, utilizando a tabela **PEDIDOS**, onde comparamos o ID_Produto com os resultados da subquery anterior.

Finalmente, a **consulta principal** utiliza esses **ID_Cliente** para retornar os **nomes completos** e **e-mails** dos clientes cadastrados na tabela **CLIENTES que realizaram compras dos produtos da categoria 'Headphone'**.

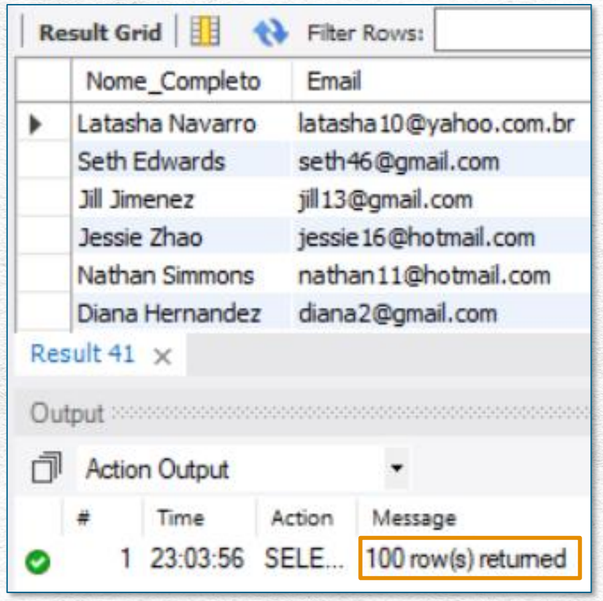
CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

7

Questão 7

Liste os nomes completos e os e-mails dos clientes que realizaram compras de produtos da categoria "Headphone", permitindo que o marketing direcione campanhas promocionais a esse público. **(Utilize as tabelas clientes, pedidos, produtos e categorias)**



Explicação e Resultado

A **consulta completa** fica assim:

```sql
SELECT
    CONCAT(clientes.Nome, ' ', clientes.Sobrenome) AS Nome_Completo,
    Email
FROM clientes
WHERE ID_Cliente IN (
    SELECT ID_Cliente
    FROM pedidos
    WHERE ID_Produto IN (
        SELECT ID_Produto
        FROM produtos
        WHERE ID_Categoria = (
            SELECT ID_Categoria
            FROM categorias
            WHERE Categoria = 'Headphone'
        )
    )
);
```

3ª Subquery

2ª Subquery

1ª Subquery

Dessa forma, **a consulta retorna os dados de todos os clientes que compraram produtos da categoria 'Headphone'**:

GABARITO EXPLICATIVO

8

Questão 8

O departamento de produtos precisa descobrir se existe alguma loja que ainda não efetuou vendas dos produtos da marca "DELL".

Liste o nome das lojas que eventualmente preencham essa condição, se houver. **(Utilize as tabelas lojas, pedidos e produtos)**

Explicação e Resultado

Para **descobrir se existe alguma loja que ainda não efetuou vendas de produtos da marca 'DELL'**, utilizamos a cláusula **NOT EXISTS** para verificar a ausência de vendas de produtos dessa marca em cada loja, assim como **duas subqueries dentro da cláusula WHERE** da consulta principal.

De baixo para cima, a **primeira subquery (mais interna)** é responsável por buscar os **ID_Produto** da marca **'DELL'** na tabela **PRODUTOS**.

Em seguida, a **segunda subquery** utiliza a tabela **PEDIDOS** para verificar se há registros de vendas dos produtos retornados pela primeira subquery em cada loja (**ID_Loja**).

A cláusula **NOT EXISTS** retorna as lojas para as quais não existe nenhum pedido registrado com produtos da marca **'DELL'**. Ou seja, ela **verifica se não há vendas de produtos dessa marca para uma determinada loja e, se não houver, a loja será incluída no resultado.**

CONTINUA NA PRÓXIMA PÁGINA

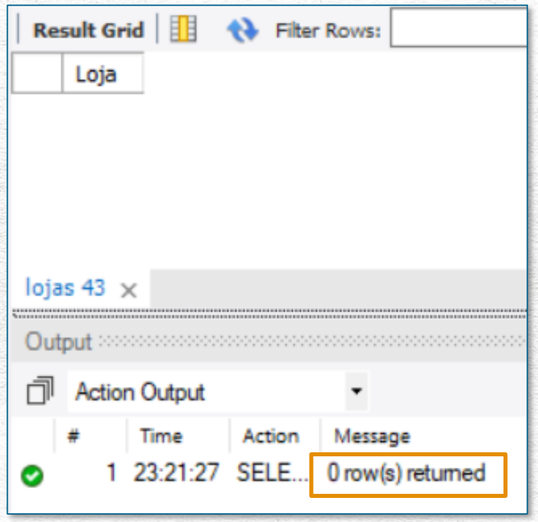
GABARITO EXPLICATIVO

8

Questão 8

O departamento de produtos precisa descobrir se existe alguma loja que ainda não efetuou vendas dos produtos da marca "DELL".

Liste o nome das lojas que eventualmente preencham essa condição, se houver. **(Utilize as tabelas lojas, pedidos e produtos)**



Explicação e Resultado

A **consulta completa** fica assim:

```
SELECT Loja
FROM lojas
WHERE NOT EXISTS (
    SELECT pedidos.ID_Loja
    FROM pedidos
    WHERE pedidos.ID_Produto IN (
        SELECT ID_Produto
        FROM produtos
        WHERE Marca_Produto = 'DELL'
    )
);
```

2ª Subquery

1ª Subquery

E o **resultado** retornado é o mostrado ao lado. Ou seja, **de todas as lojas cadastradas na tabela LOJAS, nenhuma foi retornada no resultado, pois todas elas já venderam ao menos uma vez produtos da marca 'DELL':**

SQL IMPRESSIONADOR

MySQL

Apostila completa